

Projet ERP Intelligent : Planification Hebdomadaire

<https://gemini.google.com/app/a55d9526921ab9e2>

User prompt: Bonjour, Merci beaucoup pour votre retour ainsi que pour les différentes propositions de sujets. Après réflexion, je souhaiterais m'orienter vers un projet intitulé : Smart ERP Core : An ERP Platform with Integrated AI and Business Intelligence (HR Module) L'objectif serait de concevoir le noyau d'une plateforme ERP intelligente en développant, dans un premier temps, un module de gestion des ressources humaines. Ce module intégrerait des fonctionnalités classiques de gestion RH (employés, départements, congés, présence, etc.), tout en y ajoutant une couche de Business Intelligence (tableaux de bord, indicateurs de performance, statistiques) ainsi qu'une première fonctionnalité d'Intelligence Artificielle, telle qu'un assistant RH ou un système d'analyse et de génération de rapports. L'idée est de réaliser un projet dont le périmètre est adapté à la durée du stage, tout en concevant une architecture évolutive permettant d'étendre ultérieurement la plateforme vers d'autres modules ERP (gestion de stock, ventes, CRM, etc.) et d'enrichir progressivement les fonctionnalités d'IA et d'analyse de données. Si ce sujet vous paraît pertinent, je serais ravi d'échanger avec vous afin d'en préciser le périmètre et les objectifs. Je vous remercie par avance pour votre retour.

Cordialement,

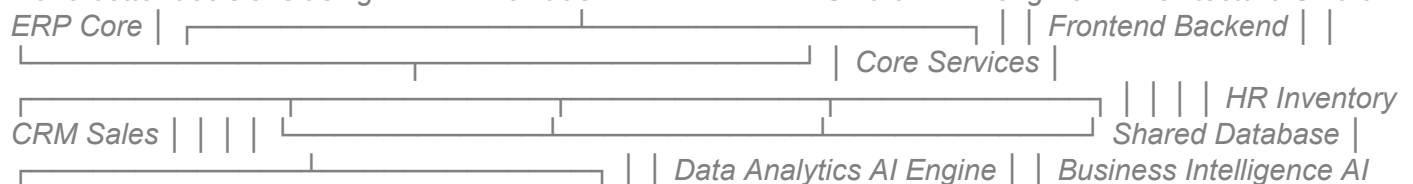
Bonjour Yasser, Merci pour votre retour. Le sujet me paraît pertinent et tout à fait adapté à un projet de fin d'études. Pour l'architecture, je vous propose de partir sur un Modular Monolith, avec une architecture DDD/MVC au sein de chaque module afin de garantir une bonne séparation des responsabilités et de faciliter l'évolution de la plateforme. Nous pourrions définir ensemble le périmètre fonctionnel du module RH ainsi que les parties Business Intelligence et IA afin de conserver un projet réalisable dans la durée du stage.

Cordialement,

this is my convo with the encadreur about my project i have conducted a plan for it ,

.Project Vision
Smart ERP Core
An Intelligent ERP Platform with AI,
Data Analytics and Business Intelligence
Mission Traditional ERPs only store and organize company data. Your platform will not only manage the company's data but also understand it, analyze it, and help managers make better decisions using AI.
Think of it as:
 $\text{ERP} + \text{BI} + \text{AI} = \text{Smart ERP}$

Long-Term Architecture
Smart ERP Core | Frontend Backend |



Assistant The important thing is that AI isn't a separate project. Everything feeds AI. Layer 1 — ERP The ERP is responsible for managing company operations. For the internship you'll only build HR Module Features Employee Management Employee profile Contact information Position Department Hiring date Status Department Management Create departments Assign managers Employee count Leave Management Request leave Approve leave Reject leave Leave history Attendance Daily attendance Late arrivals Absence Recruitment (optional) Job offers Candidates CV upload Everything above creates your business data. Layer 2 — Data Analytics Now the system starts understanding the stored data. Instead of only saving Ahmed Present It begins calculating Attendance rate Average leave Employees per department Monthly recruitment Employee growth Hiring trends Average overtime This layer transforms raw data into useful information. Layer 3 — Business Intelligence Business Intelligence is how managers see the data. Imagine logging in as HR Manager. Dashboard Employees 154 ↑ 8% ----- Attendance 96% ----- Leaves 12 Pending ----- Departments 7 ----- Recruitment 5

Candidates Charts Employees by department Hiring trend Attendance evolution Leave statistics Recruitment statistics KPIs Employee turnover Attendance rate Average leave days Hiring time Active employees Layer 4 — Artificial Intelligence This is where your project becomes unique. Instead of clicking everywhere inside the ERP... The manager simply asks Show employees with more than 20 vacation days. AI answers. Example Manager Who has the highest absenteeism this month? AI Three employees exceeded the monthly average. 1 Ahmed 2 Ali 3 Youssef Generate reports Manager Generate July HR Report AI Creates PDF automatically. Recruitment Upload CVs AI Ranks candidates. Recommendations AI Employee Ahmed attendance has dropped 40% during the last two months. Recommendation: Schedule an HR meeting. Future Predictive AI Using Machine Learning Predict Employee resignation Absenteeism Promotions Training needs Technology Stack Frontend Angular 20 Why Enterprise ready Strong typing Modular Excellent dashboards UI Angular Material TailwindCSS PrimeNG (optional) Charts Chart.js or Apache ECharts Backend Spring Boot Architecture Modular Monolith ↓ DDD ↓ MVC Exactly what your supervisor suggested. Database PostgreSQL Future Redis For caching Authentication Spring Security JWT RBAC Roles Admin HR Manager Employee File Storage Initially Local Storage Later AWS S3 Azure Blob MinIO AI Service Do NOT put AI inside Spring Boot. Separate it. Spring Boot ↓ REST API ↓ Python AI Service ↓ LLM Python FastAPI Libraries LangChain Pandas Scikit-learn Sentence Transformers Transformers OpenAI API or Gemini API or Ollama (local) Future RAG Your ERP data ↓ Embedding Database ↓ AI answers questions about YOUR company. Business Intelligence Dashboards Chart.js SQL Analytics Views Materialized Views Future Power BI Apache Superset Metabase DevOps Docker Docker Compose GitHub GitHub Actions Nginx Later Kubernetes Project Structure Smart ERP Core Frontend Angular Backend Spring Boot Modules HR Security Shared Notifications AI Service FastAPI Database PostgreSQL Analytics Documentation Docker One Month Scope Week 1 Project setup DDD architecture Authentication Employee management Department management Week 2 Attendance Leave management Roles & permissions Database Week 3 Dashboards KPIs Charts Reports Data analysis queries Week 4 AI microservice HR assistant Natural language queries PDF report generation Docker deployment Future Expansion This is where your project becomes a real platform. ERP Modules Inventory CRM Sales Accounting Purchasing Payroll Manufacturing Asset Management AI Predictive analytics Workforce planning Demand forecasting AI agents OCR for invoices and documents Voice assistant Business Intelligence Executive dashboards Company-wide KPIs Financial analytics Predictive dashboards Cloud SaaS platform Multi-company support Multi-tenant architecture Mobile application Kubernetes deployment . i want to do this project using professional logic like angular and others . check the plan . organize it , give me a pdf file of the work for each week so i can save it , then we start the work

Response: Generating slides ...

User prompt: in this project is postgres is the best bd ?

Response: **Yes, PostgreSQL is the absolute best database choice** for the **Smart ERP Core** project.

In enterprise software engineering—especially when building a **Modular Monolith** with **Spring Boot** that integrates **Business Intelligence (BI)** and **AI (RAG/Embeddings)**—PostgreSQL stands out as the industry standard.

5 Reasons Why PostgreSQL is Ideal for Smart ERP Core

1. Enterprise ACID Compliance (Crucial for ERPs)

ERP systems handle strict business data (leave requests, daily attendance, payroll calculations, employee

records). PostgreSQL provides full **ACID compliance** with strict foreign key constraints and transactional integrity, ensuring zero data corruption across complex multi-table HR workflows.

2. Built-in Analytics & BI Performance (Layer 2 & 3)

Instead of needing a separate Data Warehouse (like Snowflake or BigQuery) right away, PostgreSQL excels at complex SQL queries required for your BI dashboard:

- **Materialized Views:** Pre-compute heavy metrics like *Monthly Attendance Evolution* or *Department Turnover Rates* for instant dashboard loading.
- **Window Functions & CTEs:** Write clean, high-performance analytical queries directly in SQL for your Spring Boot analytics service.

3. Native AI & Vector Support via pgvector (Layer 4)

This is PostgreSQL’s biggest superpower for your project. When building your Python/FastAPI AI Engine with **RAG (Retrieval-Augmented Generation)**:

- You can install the **pgvector** extension directly on PostgreSQL.
- This lets you store and search **AI embeddings** (vector data from CVs, company policies, and HR documents) in the **same database** as your structured business data.
- **Benefit:** You don't need to install or manage a separate vector database (like Pinecone, Qdrant, or Chroma) during your internship.

4. Semi-Structured Data (JSONB)

ERP modules often require dynamic or customizable fields (e.g., custom employee attributes, flexible leave policy settings, or variable candidate evaluation metrics).

- PostgreSQL allows you to store structured relational data *and* query indexed JSONB documents inside the same table without running rigid schema migrations every time.

5. Perfect Ecosystem Integration with Spring Boot

- **Spring Data JPA & Hibernate** have first-class support for PostgreSQL.
- **Database Migrations:** Works seamlessly with tools like **Flyway** or **Liquibase** to track architectural changes cleanly as you add future modules (Inventory, Sales, CRM).

PostgreSQL vs. Alternatives for Smart ERP Core

Feature / Criteria	PostgreSQL	MySQL	MongoDB	Oracle DB
ACID Integrity	Excellent	Good	Moderate	Excellent

Feature / Criteria	PostgreSQL	MySQL	MongoDB	Oracle DB
Complex BI / Analytical Queries	Outstanding	Average	Weak	Outstanding
Native AI Vector Search (pgvector)	Yes	Limited	Native Vector	Expensive
Spring Boot JPA Compatibility	Industry Benchmark	Great	Native Spring Data	Complex
Cost & Licensing	100% Free & Open Source	Free (GPL)	Free / Server Side	Heavy Enterprise Cost

Recommended Database Setup for Your Project

To ensure your architecture remains clean and future-proof during development, set up PostgreSQL as follows:

PostgreSQL Container (Docker)

- SmartERP_DB
 - Schema: hr (Employees, Departments, Leaves, Attendance)
 - Schema: security (Users, Roles, Permissions, Refresh Tokens)
 - Schema: analytics (Materialized Views, Dashboard Aggregations)
- Extensions Enabled:
 - "uuid-oss" (For secure non-sequential Primary Keys)
 - "vector" (pgvector for FastAPI AI Embeddings)

User prompt: now i will start the work on the first week the first thing i need from you is the full structure or database schema so then i can start working on authentication employer and management departemnt . and if ther is an already prepared schemas for this that are generally use , i can just work with them

Response: Designing the database schema is exactly the right first step.

To answer your question: **Yes, there are standard, universally used structures for ERPs.** Major platforms like Odoo, SAP, and Workday follow specific relational patterns to ensure the system can scale without breaking.

I have designed this schema based on **standard Enterprise HR patterns.** It uses two logical separations (Security and HR) to fit your Modular Monolith architecture perfectly.

Here is the complete standard database schema for your PostgreSQL setup.

1. Core Principles of this Schema

Before looking at the tables, here are the enterprise standards applied to this schema:

- **UUIDs for Primary Keys:** Using UUID instead of auto-incrementing numbers (1, 2, 3) is standard for modern microservices and API security.
- **Separation of Concerns:** The User (authentication) is separate from the Employee (business data). An Employee *owns* a User account to log in.
- **Audit Trails:** Every table includes `created_at` and `updated_at` (crucial for your future AI and BI layers to track when data changes).
- **Soft Deletes:** Using an `is_active` flag instead of permanently deleting records, so your BI dashboards don't lose historical data.

2. The Database Schema (PostgreSQL)

You can run this standard SQL script directly in your PostgreSQL database to create your Week 1 & 2 structure.

```
-- =====
-- 1. SECURITY / AUTH MODULE (Identity Access Management)
-- =====

CREATE TABLE roles (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(50) UNIQUE NOT NULL, -- e.g., 'ADMIN', 'HR_MANAGER', 'EMPLOYEE'
  description VARCHAR(255)
);

CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email VARCHAR(150) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  role_id UUID NOT NULL REFERENCES roles(id),
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- =====
-- 2. HR MODULE: DEPARTMENTS & EMPLOYEES
-- =====

CREATE TABLE departments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(100) UNIQUE NOT NULL, -- e.g., 'IT', 'Marketing', 'Sales'
  description TEXT,
  -- manager_id is added later below to avoid circular dependency during creation
```

```
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE employees (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    user_id UUID UNIQUE REFERENCES users(id), -- 1-to-1 relationship with User  
    department_id UUID REFERENCES departments(id), -- Many-to-1 relationship  
    first_name VARCHAR(100) NOT NULL,  
    last_name VARCHAR(100) NOT NULL,  
    phone VARCHAR(20),  
    position VARCHAR(100) NOT NULL, -- e.g., 'Software Engineer'  
    hire_date DATE NOT NULL,  
    status VARCHAR(50) DEFAULT 'ACTIVE', -- 'ACTIVE', 'ON_LEAVE', 'TERMINATED'  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
-- Now we can add the manager relationship to departments  
ALTER TABLE departments  
ADD COLUMN manager_id UUID REFERENCES employees(id);
```

```
-- =====  
-- 3. HR MODULE: LEAVES & ATTENDANCE (For your BI & AI Data)  
-- =====
```

```
CREATE TABLE leaves (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    employee_id UUID NOT NULL REFERENCES employees(id),  
    leave_type VARCHAR(50) NOT NULL, -- 'ANNUAL', 'SICK', 'UNPAID'  
    start_date DATE NOT NULL,  
    end_date DATE NOT NULL,  
    status VARCHAR(50) DEFAULT 'PENDING', -- 'PENDING', 'APPROVED', 'REJECTED'  
    reason TEXT,  
    approved_by UUID REFERENCES employees(id),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE attendance (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    employee_id UUID NOT NULL REFERENCES employees(id),  
    work_date DATE NOT NULL,  
    check_in TIMESTAMP,  
    check_out TIMESTAMP,  
    status VARCHAR(50) NOT NULL, -- 'PRESENT', 'ABSENT', 'LATE', 'HALF_DAY'  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    UNIQUE(employee_id, work_date) -- An employee can only have one attendance record per day  
);
```

3. How this translates to Spring Boot (JPA)

When you start creating your Java classes (@Entity), this is how the relationships will look. Keeping these clean is the secret to a successful DDD (Domain-Driven Design) architecture:

- **User Entity:** @ManyToOne with Role.
 - **Employee Entity:** @OneToOne with User. @ManyToOne with Department.
 - **Department Entity:** @OneToOne with Employee (Manager). @OneToMany with Employees (List of staff).
 - **Leave Entity:** @ManyToOne with Employee.
 - **Attendance Entity:** @ManyToOne with Employee.
-

User prompt: shouldn't the employer also have a client/colleagues/bosses satisfaction and project completion table for more detailed review later ?

Response: That is a brilliant observation. You are already thinking exactly like a Data Engineer and AI Architect.

In traditional ERPs, basic HR just handles payroll and attendance. But since your project is a **Smart ERP**, the AI and BI layers need rich data to make actual predictions (like "Who deserves a promotion?" or "Who needs training?"). Tracking satisfaction and project completion is exactly how you feed that AI.

In enterprise architecture, this is called the **Performance Management Module**.

Here is how we can integrate this cleanly into your schema without overcomplicating your one-month deadline.

The Schema Addition: Performance & Projects

We can capture everything you just mentioned using two precise tables. You can append this to the SQL script I gave you earlier.

```
-- =====  
-- 4. HR MODULE: PERFORMANCE & EVALUATION (For BI & AI Predictions)  
-- =====
```

```
CREATE TABLE performance_reviews (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  employee_id UUID NOT NULL REFERENCES employees(id),  
  reviewer_id UUID REFERENCES employees(id), -- Can be null if reviewed by an external client  
  reviewer_type VARCHAR(50) NOT NULL, -- 'MANAGER', 'PEER', 'CLIENT'  
  score DECIMAL(3,2) NOT NULL, -- e.g., 4.50 (out of 5)  
  feedback TEXT, -- The actual text review (Great for AI sentiment analysis later!)  
  review_date DATE NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE employee_projects (  

```

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
employee_id UUID NOT NULL REFERENCES employees(id),
project_name VARCHAR(150) NOT NULL,
role_in_project VARCHAR(100), -- e.g., 'Lead Developer', 'Designer'
completion_status VARCHAR(50), -- 'ON_TIME', 'DELAYED', 'EXCEEDED_EXPECTATIONS'
delivered_date DATE,
created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);
```

Why this design is perfect for your AI:

1. **The score column (BI):** Your Angular dashboard can easily average these scores to show a "KPI Performance Graph" for each employee.
2. **The reviewer_type column:** Allows the system to do "360-degree feedback" (comparing how a boss rates an employee vs. how their colleagues rate them).
3. **The feedback text (AI):** In Week 4, your Python AI can read these text fields using NLP (Natural Language Processing) to summarize an employee's strengths and weaknesses automatically.

A Quick Word of Advice on the Timeline

Since you only have one month, I highly recommend adding these tables to your database **now**, but saving the backend/frontend development for them until **Week 3**. For Week 1, your absolute priority must be getting the Auth (Security), Employee, and Department foundation working perfectly. If the core isn't solid, the advanced features won't work.

User prompt: yes

Response: Here is the complete setup to kick off Week 1. We will start by spinning up your database using Docker, and then map out the Spring Boot architecture so you can start writing clean, modular code.

1. The Database: Docker Compose

Since you will be using AI embeddings (vectors) in Week 4, we will use the official **pgvector** Docker image instead of standard PostgreSQL. This saves you from having to manually install the AI extensions later.

Create a file named docker-compose.yml in your project root and paste this code:

```
version: '3.8'

services:
  postgres:
    image: pgvector/pgvector:pg15 # Includes PostgreSQL 15 + AI Vector support
    container_name: smart_erp_db
    restart: always
    environment:
      POSTGRES_USER: erp_user
      POSTGRES_PASSWORD: erp_password
```

```
POSTGRES_DB: smart_erp
ports:
  - "5432:5432"
volumes:
  - erp_postgres_data:/var/lib/postgresql/data
```

Optional but highly recommended: pgAdmin for a visual database interface

```
pgadmin:
  image: dpage/pgadmin4
  container_name: smart_erp_pgadmin
  restart: always
  environment:
    PGADMIN_DEFAULT_EMAIL: admin@smarterp.com
    PGADMIN_DEFAULT_PASSWORD: admin
  ports:
    - "5050:80"
  depends_on:
    - postgres
```

```
volumes:
  erp_postgres_data:
```

To start the database: Open your terminal in the same folder and run: `docker-compose up -d`

(You can now access your database visually by going to `http://localhost:5050` in your browser and logging in with `admin@smarterp.com` / `admin`).

2. Spring Boot Architecture (Modular Monolith)

To implement the **DDD (Domain-Driven Design)** your supervisor requested, do not put all your controllers in one folder and all your entities in another. Instead, package your application by **Module** (HR, Security, Shared).

Here is the exact package structure you should create in your IDE (IntelliJ/Eclipse):

```
src/main/java/com/smarterp/
├── SmartErpApplication.java
├── shared/                    # Code shared across all modules
│   ├── config/               # SecurityConfig, CorsConfig
│   └── exception/            # GlobalExceptionHandler
├── security/                 # MODULE 1: Authentication & IAM
│   ├── domain/               # User.java, Role.java
│   ├── repository/           # UserRepository, RoleRepository
│   ├── service/              # AuthService, JwtService
│   └── web/                   # AuthController (Login/Register APIs)
└── hr/                       # MODULE 2: Human Resources
```

— domain/	# Employee.java, Department.java
— repository/	# EmployeeRepository
— service/	# EmployeeService
— web/	# EmployeeController

3. Connecting Spring Boot to PostgreSQL

Open your src/main/resources/application.yml (rename it from .properties to .yml for cleaner formatting) and add your database configuration:

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/smart_erp
    username: erp_user
    password: erp_password
    driver-class-name: org.postgresql.Driver

  jpa:
    hibernate:
      # 'update' automatically creates tables based on your @Entity classes for now.
      # Later, we will switch this to 'validate' and use Flyway/Liquibase.
      ddl-auto: update
    show-sql: true
    properties:
      hibernate:
        format_sql: true
        dialect: org.hibernate.dialect.PostgreSQLDialect
```

4. Your First Entity (The User)

Here is how you translate the SQL users table into a Spring Boot @Entity inside your security/domain package using UUIDs:

```
package com.smarterp.security.domain;

import jakarta.persistence.*;
import java.time.LocalDateTime;
import java.util.UUID;

@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    @Column(nullable = false, unique = true, length = 150)
```

```

private String email;

@Column(nullable = false, name = "password_hash")
private String passwordHash;

@Column(name = "is_active")
private Boolean isActive = true;

// You will link this to the Role entity next
// @ManyToOne
// @JoinColumn(name = "role_id", nullable = false)
// private Role role;

@Column(name = "created_at", updatable = false)
private LocalDateTime createdAt;

@PrePersist
protected void onCreate() {
    this.createdAt = LocalDateTime.now();
}

// Getters and Setters...
}

```

User prompt: Create a file named docker-compose.yml in your project root and paste this code: project root meaning the root of the angular or spring boot or ai tool

Response: By **project root**, I mean the **main parent folder** that holds all of your different applications together.

Since you are building a platform with multiple distinct pieces (Frontend, Backend, AI Service, Database), you should create one main folder for the entire internship project. Your docker-compose.yml goes directly inside this main folder.

Here is exactly how your folders should look on your computer:

```

Smart_ERP_Core/          <-- THIS IS THE PROJECT ROOT
├── docker-compose.yml    <-- Put it here!
├── backend/              <-- Your Spring Boot project goes here
│   ├── src/
│   └── pom.xml (or build.gradle)
├── frontend/             <-- Your Angular 20 project will go here
│   ├── src/
│   └── package.json
└── ai_service/           <-- Your Python FastAPI project will go here (Week 4)

```

└─ main.py
└─ requirements.txt

Why put it there?

Right now, the docker-compose.yml is only spinning up your PostgreSQL database. But later in the project, you will update this exact same file to spin up your Spring Boot app, your Angular app, and your Python AI app all at the same time with one single command. Keeping it in the parent folder makes it the "master controller" for your entire system.

User prompt: 1. The Database: Docker Compose Since you will be using AI embeddings (vectors) in Week 4, we will use the official pgvector Docker image instead of standard PostgreSQL. This saves you from having to manually install the AI extensions later. Create a file named docker-compose.yml in your project root and paste this code: YAML version: '3.8'services: postgres: image: pgvector/pgvector:pg15 # Includes PostgreSQL 15 + AI Vector support container_name: smart_erp_db restart: always environment: POSTGRES_USER: erp_user POSTGRES_PASSWORD: erp_password POSTGRES_DB: smart_erp ports: - "5432:5432" volumes: - erp_postgres_data:/var/lib/postgresql/data # Optional but highly recommended: pgAdmin for a visual database interface pgadmin: image: dpage/pgadmin4 container_name: smart_erp_pgadmin restart: always environment: PGADMIN_DEFAULT_EMAIL: admin@smarterp.com PGADMIN_DEFAULT_PASSWORD: admin ports: - "5050:80" depends_on: - postgresvolumes: erp_postgres_data: To start the database: Open your terminal in the same folder and run:docker-compose up -d (You can now access your database visually by going to http://localhost:5050 in your browser and logging in with admin@smarterp.com / admin). i did this but the server . tell me the diffrenebetween this methode and with connecting directly with my local pgadmin

Response: It looks like your message got cut off ("i did this but the server ."), but I can guess exactly what happened: you logged into the web pgAdmin, tried to "Add New Server", and it failed to connect.

This happens because of how Docker networking works.

Here is the exact difference between using the **Dockerized pgAdmin** (the method I provided) versus using your **Local Desktop pgAdmin** (the one you already have installed on your computer).

Method 1: Using the Docker pgAdmin (Web Browser at localhost:5050)

Because this pgAdmin is running *inside* Docker alongside your database, they share a private Docker network.

- **The Error:** If you tried to connect the server using localhost, it failed. Inside the pgAdmin container, "localhost" means pgAdmin itself, not the database.
- **How to connect:** To connect them, you must use the container's name as the host.
 - **Host name/address:** postgres (*This is the magic word!*)
 - **Port:** 5432
 - **Maintenance database:** smart_erp
 - **Username:** erp_user
 - **Password:** erp_password

Method 2: Using your Local Desktop pgAdmin (Installed on your PC)

Because this pgAdmin is installed directly on your Windows/Mac/Linux machine, it sits *outside* of Docker. It has to connect through the port we exposed in the docker-compose.yml (5432:5432).

- **How to connect:**
 - **Host name/address:** localhost (*Because your PC is routing port 5432 into Docker*)
 - **Port:** 5432
 - **Maintenance database:** smart_erp
 - **Username:** erp_user
 - **Password:** erp_password

Which one should you use?

Use your Local Desktop pgAdmin. Since you already have pgAdmin installed on your computer, you don't actually need the Docker version! It is faster and usually more comfortable to use the desktop app you are already used to.

If you want to use your local pgAdmin, you can safely remove the pgadmin: section from your docker-compose.yml file and just run docker-compose up -d again. Your database will still run perfectly in the background, and you can connect to it using your desktop app with the Method 2 settings above.

User prompt: i want to use the docker pg admin

Response: Great choice. Keeping pgAdmin in Docker keeps your development environment clean and ensures you don't clutter your main operating system with background services.

Here is exactly how to connect the Dockerized pgAdmin to your Dockerized PostgreSQL database.

1 Log into pgAdmin

Via your web browser

Open your browser and navigate to <http://localhost:5050>. Log in using the credentials from the docker-compose.yml:

- **Email:** admin@smarterp.com
- **Password:** admin

2 Register a New Server

Once logged in, look at the left sidebar. Right-click on **Servers** -> **Register** -> **Server...** (or click "Add New Server" on the welcome screen).

3 Fill out the General Tab

In the window that opens, stay on the **General** tab and give your connection a name in the **Name** field (e.g., Smart ERP Local). This is just a display name for you.

4 Configure the Connection details

This is the crucial step

Switch to the **Connection** tab at the top of that window. Fill it out exactly like this:

- **Host name/address:** postgres *(Do not use localhost. Because both containers are inside Docker, they talk to each other using the service name).*
- **Port:** 5432
- **Maintenance database:** smart_erp
- **Username:** erp_user
- **Password:** erp_password
- **Save password?** Toggle this to Yes.

5 Save and Connect

Click the **Save** button.

You should now see the Smart ERP Local server appear on the left sidebar. You can expand it to go to **Databases > smart_erp > Schemas > public > Tables** to see where your data will live once Spring Boot generates the tables.

User prompt: Open your browser and navigate to <http://localhost:5050>. Log in using the credentials from the docker-compose.yml: Email: admin@smarterp.com Password: admin can i use other credentials ?

Response: Yes, absolutely! The admin@smarterp.com and admin credentials are just defaults I provided for local development. You can change them to whatever you prefer.

To use your own credentials, you just need to update the environment variables in your docker-compose.yml file.

1. Update the docker-compose.yml

Change the PGADMIN_DEFAULT_EMAIL and PGADMIN_DEFAULT_PASSWORD values under the pgadmin service:

```
pgadmin:
  image: dpage/pgadmin4
  container_name: smart_erp_pgadmin
  restart: always
  environment:
    PGADMIN_DEFAULT_EMAIL: your.email@whatever.com <-- Change this
    PGADMIN_DEFAULT_PASSWORD: your_secure_password <-- Change this
  ports:
    - "5050:80"
  depends_on:
    - postgres
```

2. Apply the Changes

If you haven't run the container yet, just run `docker-compose up -d`.

If your containers are **already running** with the old credentials, Docker won't automatically update them just by saving the file. You need to tell Docker to recreate the pgAdmin container with the new variables.

Run this command in your terminal (in the folder where your docker-compose.yml is located):

```
docker-compose up -d --force-recreate pgadmin
```

After it restarts, go to `http://localhost:5050` and log in with your new email and password!

```
User prompt: PS C:\Users\yasser\Documents\stage\Smart_ERP_Core> docker-compose up -d
--force-recreate pgadmin time="2026-08-05T00:29:48+02:00" level=warning
msg="C:\Users\yasser\Documents\stage\Smart_ERP_Core\docker-compose.yml: the attribute `version`
is obsolete, it will be ignored, please remove it to avoid potential confusion" [+] up 2/2 ✓ Container
smart_erp_pgadmin Started 0.7s
✓ Container smart_erp_db Started
0.3s PS C:\Users\yasser\Documents\stage\Smart_ERP_Core> the password didnt update
```

Response: The reason the password didn't update is due to how the pgAdmin Docker image is built. It only reads the PGADMIN_DEFAULT_EMAIL and PGADMIN_DEFAULT_PASSWORD variables the **very first time** the container is created to set up its internal configuration.

Because it had already started once with the old password, it ignored the new variables during the recreation.

To fix this, we need to completely destroy the pgAdmin container and its hidden internal storage, then start it completely fresh.

Run these two commands exactly as shown:

```
docker-compose rm -s -f -v pgadmin
```

(This stops the container, forces its removal, and deletes any hidden anonymous volumes holding the old password).

Then, start it up again:

```
docker-compose up -d
```

Now, navigate to <http://localhost:5050> and your new credentials will work perfectly.

Note on the yellow warning: The message about version being obsolete is just Docker telling you that newer versions of Docker Compose don't need that line anymore. It doesn't harm anything, but if you want the warning to go away, you can simply delete version: '3.8' from the very top of your docker-compose.yml file.

User prompt: is there a way for the postgres to connect not on the localhost ?

Response: Yes, depending on where you are trying to connect from. Here are the 4 main ways to connect to your PostgreSQL database without using localhost:

1. From other Docker containers (pgAdmin, FastAPI, or Spring Boot in Docker)

Inside the Docker environment, localhost refers to the container itself, not the database. Docker automatically creates a private DNS network where container service names act as hostnames.

- **Host:** postgres *(the exact service name defined in your docker-compose.yml)*
- **Port:** 5432

2. From another computer on the same Wi-Fi / Local Network (LAN)

If you want to access the database from a second laptop, a mobile device, or a teammate's computer on the same network:

1. Find your computer's local IP address by running this in Command Prompt (cmd):
ipconfig

Look for **IPv4 Address** (e.g., 192.168.1.15).

2. Use that IP address as the host on the other computer:
 - **Host:** 192.168.1.15 *(replace with your actual local IP)*

- **Port:** 5432

3. Using a Custom Domain Name on your computer

If you want to use a clean URL like `db.smarterp.local` instead of `localhost` or `127.0.0.1`:

1. Open Notepad as **Administrator**.
2. Open the Windows hosts file located at: `C:\Windows\System32\drivers\etc\hosts`
3. Add this line at the bottom and save:
`127.0.0.1 db.smarterp.local`
4. Now you can connect using:
 - **Host:** `db.smarterp.local`
 - **Port:** 5432

4. Over the Internet (Remote Access)

If you need to access the database outside your local Wi-Fi:

- **Option A (Zero-Configuration VPN):** Use a free mesh network tool like **Tailscale** on your host PC to get a secure private IP accessible from anywhere.
- **Option B (Cloud Database):** Deploy the PostgreSQL database to a free cloud provider (such as Neon, Supabase, or Render) which provides a public connection URI string (e.g., `postgres://user:pass@ep-cool-db.us-east-2.aws.neon.tech/smart_erp`).

User prompt: the genrale i dea for this project that later i can luch it that would be dificult using localhost

Response: You are thinking exactly like a SaaS founder and lead architect. If you want to launch **Smart ERP Core** as a real-world commercial platform or offer it to live clients, `localhost` only exists on your development machine.

In enterprise software development, you don't rewrite code to launch the app. Instead, you design your architecture with **Environment Variables** from Day 1. This allows your app to connect to `localhost` on your laptop today, and seamlessly switch to a secure cloud database when you launch—without changing a single line of Java code.

1. How Pros Configure Spring Boot for Launch (application.yml)

Instead of hardcoding `localhost` or fixed passwords in your backend, parameterize your `application.yml`. This makes your Spring Boot application instantly **production-ready**:

spring:

datasource:

```
# Uses environment variables if present; falls back to localhost defaults for local dev
url: jdbc:postgresql://${DB_HOST:localhost}:${DB_PORT:5432}/${DB_NAME:smart_erp}
username: ${DB_USER:erp_user}
password: ${DB_PASSWORD:erp_password}
```

driver-class-name: org.postgresql.Driver

jpa:

hibernate:

ddl-auto: \${hibernate_ddl_auto:update} # Set to 'validate' or 'none' in production

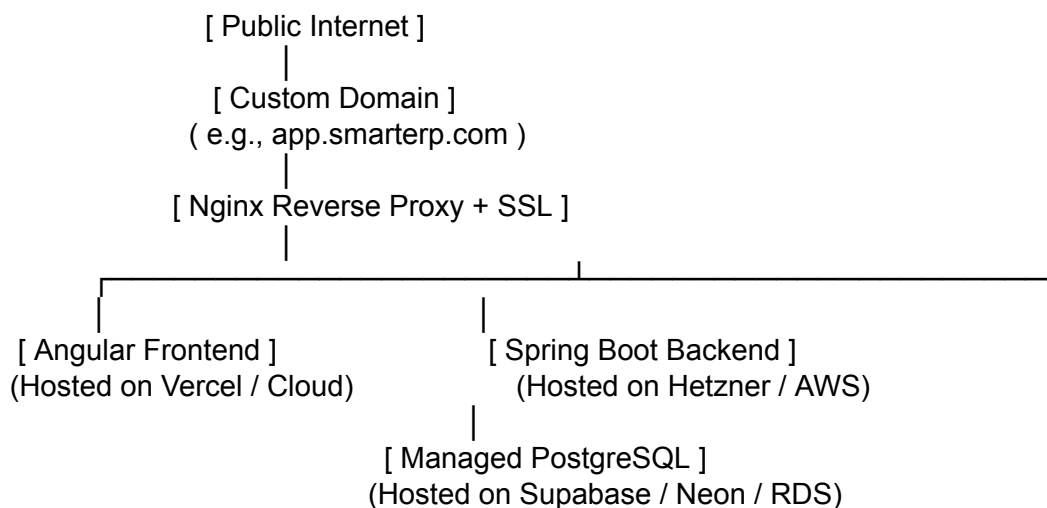
show-sql: false

How this works:

- **During Week 1 (Development):** You run Spring Boot normally. It sees no environment variables, so it defaults to localhost:5432.
- **When you launch (Production):** You deploy Spring Boot to a cloud server and pass variable settings (DB_HOST=prod-db.smarterp.com, DB_PASSWORD=secret_key). Spring Boot automatically targets your production database.

2. The Cloud Launch Blueprint

When you are ready to publish the platform live, your production setup will look like this:



3. Production Deployment Options when Launching

When you finish your internship and decide to host this live, here are three popular deployment paths:

Hosting Strategy	Cost	Ease of Setup	Best For
Fully Managed Cloud (Serverless)	Free to \$10/month	Very Easy	Quick product demos, initial SaaS launch, PFE defenses.

Hosting Strategy	Cost	Ease of Setup	Best For
<i>(Frontend: Vercel, Backend: Render/Railway, DB: Neon/Supabase)</i>			
Single Cloud VPS <i>(DigitalOcean, Hetzner, AWS EC2 with Docker Compose)</i>	\$5 to \$15/month	Moderate	Launching a cost-effective MVP for real initial clients.
Enterprise Cloud <i>(AWS Elastic Beanstalk / GCP + Cloud SQL)</i>	\$50+/month	Advanced	Large enterprise deployments with multi-tenant companies.

What to do right now

Keep building locally using **Docker and localhost** for Week 1 so you can develop rapidly without cloud costs or internet latency. By structuring your project with Docker and environment variables now, deploying it to a live cloud domain later will only take about 15 minutes.

User prompt: Let's continue setting up the Spring Boot backend with JPA entities and repository classes.

Response: Let's build out the core domain entities and Spring Data JPA repositories for both the **Security Module** and the **HR Module**.

These classes map directly to your PostgreSQL database using JPA standards, UUID primary keys, and automated timestamp tracking.

Module 1: Security (com.smarterp.security)

1. Role Enum (RoleName.java)

Create a package com.smarterp.security.domain and define the system roles:

```
package com.smarterp.security.domain;

public enum RoleName {
    ROLE_ADMIN,
    ROLE_HR_MANAGER,
    ROLE_EMPLOYEE
}
```

2. Role Entity (Role.java)

```
package com.smarterp.security.domain;

import jakarta.persistence.*;
import java.util.UUID;

@Entity
@Table(name = "roles")
public class Role {

    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, unique = true, length = 50)
    private RoleName name;

    @Column(length = 255)
    private String description;

    public Role() {}

    public Role(RoleName name, String description) {
        this.name = name;
        this.description = description;
    }

    // Getters and Setters
    public UUID getId() { return id; }
    public void setId(UUID id) { this.id = id; }

    public RoleName getName() { return name; }
    public void setName(RoleName name) { this.name = name; }

    public String getDescription() { return description; }
    public void setDescription(String description) { this.description = description; }
```

```
}
```

3. User Entity (User.java)

```
package com.smarterp.security.domain;

import jakarta.persistence.*;
import java.time.LocalDateTime;
import java.util.UUID;

@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    @Column(nullable = false, unique = true, length = 150)
    private String email;

    @Column(name = "password_hash", nullable = false)
    private String passwordHash;

    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "role_id", nullable = false)
    private Role role;

    @Column(name = "is_active")
    private Boolean isActive = true;

    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt;

    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    @PrePersist
    protected void onCreate() {
        this.createdAt = LocalDateTime.now();
        this.updatedAt = LocalDateTime.now();
    }

    @PreUpdate
    protected void onUpdate() {
        this.updatedAt = LocalDateTime.now();
    }

    public User() {}

    // Getters and Setters
}
```

```

public UUID getId() { return id; }
public void setId(UUID id) { this.id = id; }

public String getEmail() { return email; }
public void setEmail(String email) { this.email = email; }

public String getPasswordHash() { return passwordHash; }
public void setPasswordHash(String passwordHash) { this.passwordHash = passwordHash; }

public Role getRole() { return role; }
public void setRole(Role role) { this.role = role; }

public Boolean getIsActive() { return isActive; }
public void setIsActive(Boolean active) { isActive = active; }

public LocalDateTime getCreatedAt() { return createdAt; }
public LocalDateTime getUpdatedAt() { return updatedAt; }
}

```

4. Repositories for Security Module

Create package `com.smarterp.security.repository`:

```

package com.smarterp.security.repository;

import com.smarterp.security.domain.Role;
import com.smarterp.security.domain.RoleName;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;
import java.util.UUID;

public interface RoleRepository extends JpaRepository<Role, UUID> {
    Optional<Role> findByName(RoleName name);
}

package com.smarterp.security.repository;

import com.smarterp.security.domain.User;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;
import java.util.UUID;

public interface UserRepository extends JpaRepository<User, UUID> {
    Optional<User> findByEmail(String email);
    Boolean existsByEmail(String email);
}

```

Module 2: HR Module (com.smarterp.hr)

1. Employee Status Enum (EmployeeStatus.java)

Create package com.smarterp.hr.domain:

```
package com.smarterp.hr.domain;

public enum EmployeeStatus {
    ACTIVE,
    ON_LEAVE,
    TERMINATED
}
```

2. Department Entity (Department.java)

package com.smarterp.hr.domain;

```
import jakarta.persistence.*;
import java.time.LocalDateTime;
import java.util.UUID;
```

```
@Entity
@Table(name = "departments")
public class Department {

    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    @Column(nullable = false, unique = true, length = 100)
    private String name;

    @Column(columnDefinition = "TEXT")
    private String description;

    @OneToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "manager_id")
    private Employee manager;

    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt;

    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    @PrePersist
    protected void onCreate() {
        this.createdAt = LocalDateTime.now();
        this.updatedAt = LocalDateTime.now();
    }
}
```

```

@PreUpdate
protected void onUpdate() {
    this.updatedAt = LocalDateTime.now();
}

public Department() {}

public Department(String name, String description) {
    this.name = name;
    this.description = description;
}

// Getters and Setters
public UUID getId() { return id; }
public void setId(UUID id) { this.id = id; }

public String getName() { return name; }
public void setName(String name) { this.name = name; }

public String getDescription() { return description; }
public void setDescription(String description) { this.description = description; }

public Employee getManager() { return manager; }
public void setManager(Employee manager) { this.manager = manager; }

public LocalDateTime getCreatedAt() { return createdAt; }
public LocalDateTime getUpdatedAt() { return updatedAt; }
}

```

3. Employee Entity (Employee.java)

```

package com.smarterp.hr.domain;

import com.smarterp.security.domain.User;
import jakarta.persistence.*;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.UUID;

@Entity
@Table(name = "employees")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    @OneToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", unique = true)
    private User user;
}

```

```

@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "department_id")
private Department department;

@Column(name = "first_name", nullable = false, length = 100)
private String firstName;

@Column(name = "last_name", nullable = false, length = 100)
private String lastName;

@Column(length = 20)
private String phone;

@Column(nullable = false, length = 100)
private String position;

@Column(name = "hire_date", nullable = false)
private LocalDate hireDate;

@Enumerated(EnumType.STRING)
@Column(nullable = false, length = 50)
private EmployeeStatus status = EmployeeStatus.ACTIVE;

@Column(name = "created_at", updatable = false)
private LocalDateTime createdAt;

@Column(name = "updated_at")
private LocalDateTime updatedAt;

@PrePersist
protected void onCreate() {
    this.createdAt = LocalDateTime.now();
    this.updatedAt = LocalDateTime.now();
}

@PreUpdate
protected void onUpdate() {
    this.updatedAt = LocalDateTime.now();
}

public Employee() {}

// Getters and Setters
public UUID getId() { return id; }
public void setId(UUID id) { this.id = id; }

public User getUser() { return user; }
public void setUser(User user) { this.user = user; }

public Department getDepartment() { return department; }
public void setDepartment(Department department) { this.department = department; }

```

```

public String getFirstName() { return firstName; }
public void setFirstName(String firstName) { this.firstName = firstName; }

public String getLastName() { return lastName; }
public void setLastName(String lastName) { this.lastName = lastName; }

public String getPhone() { return phone; }
public void setPhone(String phone) { this.phone = phone; }

public String getPosition() { return position; }
public void setPosition(String position) { this.position = position; }

public LocalDate getHireDate() { return hireDate; }
public void setHireDate(LocalDate hireDate) { this.hireDate = hireDate; }

public EmployeeStatus getStatus() { return status; }
public void setStatus(EmployeeStatus status) { this.status = status; }

public LocalDateTime getCreatedAt() { return createdAt; }
public LocalDateTime getUpdatedAt() { return updatedAt; }
}

```

4. Repositories for HR Module

Create package com.smarterp.hr.repository:

```

package com.smarterp.hr.repository;

import com.smarterp.hr.domain.Department;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;
import java.util.UUID;

public interface DepartmentRepository extends JpaRepository<Department, UUID> {
    Optional<Department> findByName(String name);
    Boolean existsByName(String name);
}

package com.smarterp.hr.repository;

import com.smarterp.hr.domain.Employee;
import com.smarterp.hr.domain.EmployeeStatus;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;
import java.util.Optional;
import java.util.UUID;

public interface EmployeeRepository extends JpaRepository<Employee, UUID> {
    List<Employee> findByDepartmentId(UUID departmentId);
    List<Employee> findByStatus(EmployeeStatus status);
}

```

```
Optional<Employee> findById(UUID userId);  
}
```

5. Automatic Database Initialization Data (Data Seeder)

To test your API quickly without manually inserting SQL rows into PostgreSQL, add a database seeder that inserts default roles on application startup.

Create `com.smarterp.shared.config.DataInitializer.java`:

```
package com.smarterp.shared.config;  
  
import com.smarterp.security.domain.Role;  
import com.smarterp.security.domain.RoleName;  
import com.smarterp.security.repository.RoleRepository;  
import org.springframework.boot.CommandLineRunner;  
import org.springframework.stereotype.Component;  
  
@Component  
public class DataInitializer implements CommandLineRunner {  
  
    private final RoleRepository roleRepository;  
  
    public DataInitializer(RoleRepository roleRepository) {  
        this.roleRepository = roleRepository;  
    }  
  
    @Override  
    public void run(String... args) {  
        for (RoleName roleName : RoleName.values()) {  
            if (roleRepository.findByName(roleName).isEmpty()) {  
                roleRepository.save(new Role(roleName, "Default system role: " + roleName.name()));  
            }  
        }  
    }  
}
```

Verification Steps

1. Run your Spring Boot application (`SmartErpApplication.java`).
2. Check your IDE terminal. Hibernate will automatically issue SQL statements creating the roles, users, departments, and employees tables in PostgreSQL.
3. Open **pgAdmin** and view the public schema. You will see the tables along with default rows inserted into the roles table.

User prompt: start with the springboot creation process then .yaml then configuration

Response: Step 1: Spring Boot Project Creation

Generate the base project using start.spring.io (or directly inside IntelliJ IDEA / Eclipse via Spring Initializr wizard).

Project Metadata Settings

- **Project:** Maven Project
- **Language:** Java
- **Spring Boot:** 3.3.x (or latest stable 3.x release)
- **Group:** com.smarterp
- **Artifact:** smart-erp-core
- **Name:** smart-erp-core
- **Package Name:** com.smarterp
- **Packaging:** Jar
- **Java:** 17 or 21

Required Dependencies

Select the following dependencies in Spring Initializr:

1. **Spring Web** (spring-boot-starter-web)
2. **Spring Data JPA** (spring-boot-starter-data-jpa)
3. **Spring Security** (spring-boot-starter-security)
4. **Validation** (spring-boot-starter-validation)
5. **PostgreSQL Driver** (postgresql)

Maven Dependencies Configuration (pom.xml)

Open pom.xml in your project root and add the **jjwt** library for JWT authentication under <dependencies>:

```
<dependencies>
  <!-- Standard Spring Boot Starter Dependencies -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>
```

```

<!-- Database -->
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <scope>runtime</scope>
</dependency>

<!-- JWT for Auth Token Management -->
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.12.5</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.12.5</version>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.12.5</version>
  <scope>runtime</scope>
</dependency>

<!-- Test Utilities -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-test</artifactId>
  <scope>test</scope>
</dependency>
</dependencies>

```

Step 2: Configure application.yml

1. Delete or rename src/main/resources/application.properties to application.yml.
2. Paste the following configuration:

server:

port: 8080

servlet:

context-path: /api/v1

spring:

application:

name: smart-erp-core

Database Connection

datasource:

```
url: jdbc:postgresql://${DB_HOST:localhost}:${DB_PORT:5432}/${DB_NAME:smart_erp}
username: ${DB_USER:erp_user}
password: ${DB_PASSWORD:erp_password}
driver-class-name: org.postgresql.Driver
```

JPA & Hibernate Settings

jpa:

hibernate:

```
ddl-auto: ${HIBERNATE_DDL:update}
```

show-sql: true

properties:

hibernate:

```
format_sql: true
```

```
dialect: org.hibernate.dialect.PostgreSQLDialect
```

JWT Security Configuration Settings

application:

security:

jwt:

Must be a Base64-encoded string of at least 256 bits (32+ bytes)

```
secret-key: 404E635266556A586E3272357538782F413F4428472B4B6250645367566B5970
```

```
expiration-ms: 86400000 # 24 Hours in milliseconds
```

```
refresh-token-expiration-ms: 604800000 # 7 Days in milliseconds
```

Step 3: Core Infrastructure Configuration Classes

Create the base configuration packages inside `com.smarterp.shared.config`.

1. CORS Configuration (CorsConfig.java)

Allows your Angular frontend (running on `http://localhost:4200`) to communicate with the Spring Boot backend without triggering cross-origin policy blockages.

```
package com.smarterp.shared.config;
```

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
import org.springframework.web.filter.CorsFilter;
```

```
import java.util.List;
```

```
@Configuration
```

```
public class CorsConfig {
```

```
    @Bean
```

```

public CorsFilter corsFilter() {
    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    CorsConfiguration config = new CorsConfiguration();

    config.setAllowCredentials(true);
    config.setAllowedOrigins(List.of("http://localhost:4200")); // Angular application origin
    config.setAllowedHeaders(List.of("Origin", "Content-Type", "Accept", "Authorization"));
    config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS", "PATCH"));

    source.registerCorsConfiguration("/**", config);
    return new CorsFilter(source);
}
}

```

2. Password Encryption Bean Configuration (PasswordEncoderConfig.java)

Provides BCrypt password hashing across the application for authentication modules.

```

package com.smarterp.shared.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;

@Configuration
public class PasswordEncoderConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}

```

3. Global API Exception Handler (GlobalExceptionHandler.java)

Ensures clean JSON error responses across all modules instead of raw Java stack traces.

```

package com.smarterp.shared.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.Map;

```

```

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(RuntimeException.class)
    public ResponseEntity<Map<String, Object>> handleRuntimeException(RuntimeException ex) {
        Map<String, Object> body = new HashMap<>();
        body.put("timestamp", LocalDateTime.now());
        body.put("status", HttpStatus.BAD_REQUEST.value());
        body.put("error", "Bad Request");
        body.put("message", ex.getMessage());

        return new ResponseEntity<>(body, HttpStatus.BAD_REQUEST);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, Object>>
    handleValidationExceptions(MethodArgumentNotValidException ex) {
        Map<String, Object> errors = new HashMap<>();
        ex.getBindingResult().getFieldErrors().forEach(error ->
            errors.put(error.getField(), error.getDefaultMessage())
        );

        Map<String, Object> body = new HashMap<>();
        body.put("timestamp", LocalDateTime.now());
        body.put("status", HttpStatus.UNPROCESSABLE_ENTITY.value());
        body.put("errors", errors);

        return new ResponseEntity<>(body, HttpStatus.UNPROCESSABLE_ENTITY);
    }
}

```

User prompt: give me the most professional architecture for springboot for this project

Response: The industry gold standard for an enterprise Spring Boot 3 platform of this scale is a **DDD-Oriented Modular Monolith (Package-by-Feature Architecture)**.

This architecture enforces strict module boundaries, prevents cyclic dependencies, keeps business logic clean, and makes it trivial to extract any module (e.g., ai or hr) into an independent microservice in the future if your platform scales.

1. System Package Blueprint

```

com.smarterp
├── SmartErpApplication.java
├── shared/                # Cross-cutting concerns (Framework layer)
│   ├── config/           # SecurityConfig, CorsConfig, JPA/Auditing config
│   └── exception/        # GlobalExceptionHandler, BusinessException

```

security/	# JwtUtils, SecurityUser, UserDetailsService
domain/	# BaseEntity (created_at, updated_at)
security/	# MODULE 1: Identity & Access Management (IAM)
domain/	# User, Role, Permission (JPA Entities & Enums)
dto/	# LoginRequest, RegisterRequest, AuthResponse
repository/	# UserRepository, RoleRepository
service/	# AuthService, UserService
web/	# AuthController
hr/	# MODULE 2: Core HR Domain
domain/	# Employee, Department, Leave, Attendance Entities
dto/	# EmployeeDTO, CreateEmployeeRequest, LeaveRequestDTO
mapper/	# EmployeeMapper (MapStruct interface)
repository/	# EmployeeRepository, DepartmentRepository
service/	# EmployeeService, LeaveService, AttendanceService
web/	# EmployeeController, LeaveController
analytics/	# MODULE 3: Business Intelligence & KPIs
dto/	# DashboardKPIsResponse, TurnoverStatsDTO
repository/	# AnalyticsNativeQueryRepository (Materialized Views)
service/	# AnalyticsService (Aggregations)
web/	# DashboardController
ai/	# MODULE 4: AI Engine Connector
dto/	# AiPromptRequest, AiReportResponse
client/	# AiServiceClient (RestClient/WebClient to Python FastAPI)
service/	# HrAssistantService
web/	# AiAssistantController

2. Core Architectural Rules (Enterprise Standards)

Rule 1: Never Expose JPA Entities via Controllers

- **Problem:** Exposing @Entity objects directly in REST endpoints leads to infinite recursion (Jackson circular references), security leaks (exposing passwords/hashes), and unexpected database updates.
- **Solution:** Always map Entities to **DTOs (Data Transfer Objects)** before sending responses, and use DTOs for incoming requests. Use Java record types for immutable DTOs:

// Immutable Request DTO using Java 17+ Records

```
public record CreateEmployeeRequest(
    @NotBlank String firstName,
    @NotBlank String lastName,
    @Email String email,
    @NotNull UUID departmentId,
    @NotBlank String position,
    @NotNull LocalDate hireDate
) {}
```

Rule 2: Unidirectional Module Dependencies

To prevent messy code, enforce a strict top-down dependency flow:

AI Module→Analytics Module→HR Module→Security Module→Shared Core

- **Forbidden:** The HR module must never directly import or rely on Analytics or AI classes.
- **Allowed:** The Analytics module reads data from HR.

Rule 3: Automated Entity Auditing

Do not manually type createdAt and updatedAt in every entity. Use Spring Data JPA Auditing.

Base Entity Definition (shared/domain/BaseEntity.java)

```
package com.smarterp.shared.domain;

import jakarta.persistence.Column;
import jakarta.persistence.EntityListeners;
import jakarta.persistence.MappedSuperclass;
import org.springframework.data.annotation.CreatedDate;
import org.springframework.data.annotation.LastModifiedDate;
import org.springframework.data.jpa.domain.support.AuditingEntityListener;

import java.time.LocalDateTime;

@MappedSuperclass
@EntityListeners(AuditingEntityListener.class)
public abstract class BaseEntity {

    @CreatedDate
    @Column(name = "created_at", nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @LastModifiedDate
    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    public LocalDateTime getCreatedAt() { return createdAt; }
    public LocalDateTime getUpdatedAt() { return updatedAt; }
}
```

Enable JPA auditing in SmartErpApplication.java:

```
@SpringBootApplication
@EnableJpaAuditing // Enables automated timestamping
public class SmartErpApplication {
    public static void main(String[] args) {
        SpringApplication.run(SmartErpApplication.class, args);
    }
}
```

}

3. Clean Layer Request Flow

Every feature in the system follows this strict 4-layer call sequence:

[HTTP Request]



1. WEB LAYER (@RestController)

- Handles HTTP routes, validation (@Valid), and security authorization checks.
- Calls Service layer. Returns ResponseEntity<DTO>.



2. SERVICE LAYER (@Service)

- Implements business logic, transaction handling (@Transactional).
- Maps DTOs <-> Entities. Calls Repositories.



3. REPOSITORY LAYER (@Repository / JpaRepository)

- Database queries, Spring Data JPA methods, and native SQL/Materialized Views.



4. DATABASE LAYER (PostgreSQL)

4. Inter-Module Communication with Python AI (FastAPI)

Since your AI Engine lives in a separate Python microservice, Spring Boot 3 uses the modern RestClient (or WebClient) inside the ai/client package to communicate seamlessly with FastAPI:

```
package com.smarterp.ai.client;
```

```
import com.smarterp.ai.dto.AiPromptRequest;
import com.smarterp.ai.dto.AiReportResponse;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;
import org.springframework.web.client.RestClient;
```

```
@Component
```

```
public class AiServiceClient {
```

```
    private final RestClient restClient;
```

```
    public AiServiceClient(@Value("${application.ai-service.url:http://localhost:8000}") String aiServiceUrl) {
        this.restClient = RestClient.builder()
            .baseUrl(aiServiceUrl)
            .build();
    }
```

```

public AiReportResponse askHrAssistant(AiPromptRequest request) {
    return restClient.post()
        .uri("/api/v1/ai/query")
        .body(request)
        .retrieve()
        .body(AiReportResponse.class);
}
}

```

User prompt: AI Service Do NOT put AI inside Spring Boot. Separate it. Spring Boot ↓ REST API ↓ Python AI Service ↓ LLM Python FastAPI we agreed on operating the ai service to provide a more mvc/ddd architecture

Response: Separating the AI engine into a dedicated Python FastAPI service while keeping Spring Boot as the core enterprise coordinator is the absolute cleanest architectural boundary for this platform.

Why This Separation Enforces True DDD & Enterprise Quality

[Angular 20 Frontend]

HTTP / JSON (JWT Authenticated)



Spring Boot Core (Modular Monolith)

- Security & IAM (JWT, Roles)
- Core HR Domain (Employees, Leaves, Attendance)
- Analytics Domain (SQL Aggregations, Materialized)

Private REST API Call



Python AI Service (FastAPI Microservice)

- RAG Engine (LangChain / LlamaIndex)
- Data Processing (Pandas / NumPy)
- Model Connectors (OpenAI / Gemini / Ollama Local)

1. **Strict Bounded Contexts (DDD):** Spring Boot owns the *Transactional Business Domain* (database constraints, auditing, role-based access control, core HR logic). FastAPI owns the *Artificial Intelligence & Analytics Processing Domain*. Neither service leaks internal data structures into the other.
2. **Ecosystem Optimization:** Java/Spring Boot is unbeatable for enterprise stability, relational transactions, and strict typing. Python is the undisputed king of AI/ML libraries (LangChain, Pandas, Transformers, PyTorch). Forcing Python AI models into the JVM using wrappers creates unnecessary bloat and performance bottlenecks.
3. **Independent Scaling & Resource Isolation:** Heavy LLM matrix calculations or vector embeddings consume significant GPU/CPU memory. Running AI in a separate container ensures that if an AI

query encounters high load, it won't crash or slow down the rest of your core HR transactional operations.

4. **Technology Agnostic AI Swap:** If you decide to switch from an external API (OpenAI/Gemini) to a completely offline, local LLM running via Ollama or vLLM in the future, you only update code inside the `ai_service/` folder. Spring Boot won't need a single line of code modified.

Contract Between Spring Boot and FastAPI

When Spring Boot delegates an AI task to FastAPI, it passes a structured, minimal JSON payload containing the user prompt and contextual scope (e.g., `department_id` or sanitized employee text summaries).

Sample Payload sent from Spring Boot -> FastAPI:

```
{
  "user_id": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
  "query": "Which employees in IT have exceeded 15 leave days this year?",
  "context": {
    "department": "IT",
    "requested_by_role": "HR_MANAGER"
  }
}
```

Sample Response sent back from FastAPI -> Spring Boot:

```
{
  "summary": "Three employees in IT have taken more than 15 leave days.",
  "flagged_employee_ids": [
    "123e4567-e89b-12d3-a456-426614174000",
    "223e4567-e89b-12d3-a456-426614174001"
  ],
  "recommended_action": "Schedule an HR check-in with John Doe due to consecutive Monday absences."
}
```

User prompt: com.smarterp |— SmartErpApplication.java | |— shared/ # Cross-cutting concerns (Framework layer) | |— config/ # SecurityConfig, CorsConfig, JPA/Auditing config | |— exception/ # GlobalExceptionHandler, BusinessException | |— security/ # JwtUtils, SecurityUser, UserDetailsService | |— domain/ # BaseEntity (created_at, updated_at) | |— security/ # MODULE 1: Identity & Access Management (IAM) | |— domain/ # User, Role, Permission (JPA Entities & Enums) | |— dto/ # LoginRequest, RegisterRequest, AuthResponse | |— repository/ # UserRepository, RoleRepository | |— service/ # AuthService, UserService | |— web/ # AuthController | |— hr/ # MODULE 2: Core HR Domain | |— domain/ # Employee, Department, Leave, Attendance Entities | |— dto/ # EmployeeDTO, CreateEmployeeRequest, LeaveRequestDTO | |— mapper/ # EmployeeMapper (MapStruct interface) | |— repository/ # EmployeeRepository, DepartmentRepository | |— service/ # EmployeeService, LeaveService, AttendanceService | |— web/ # EmployeeController, LeaveController | |— analytics/ # MODULE 3: Business Intelligence & KPIs | |— dto/ # DashboardKPIsResponse, TurnoverStatsDTO | |— repository/ # AnalyticsNativeQueryRepository (Materialized Views) | |— service/ # AnalyticsService (Aggregations) | |— web/ # DashboardController | |— ai/ # MODULE 4: AI Engine Connector |— dto/

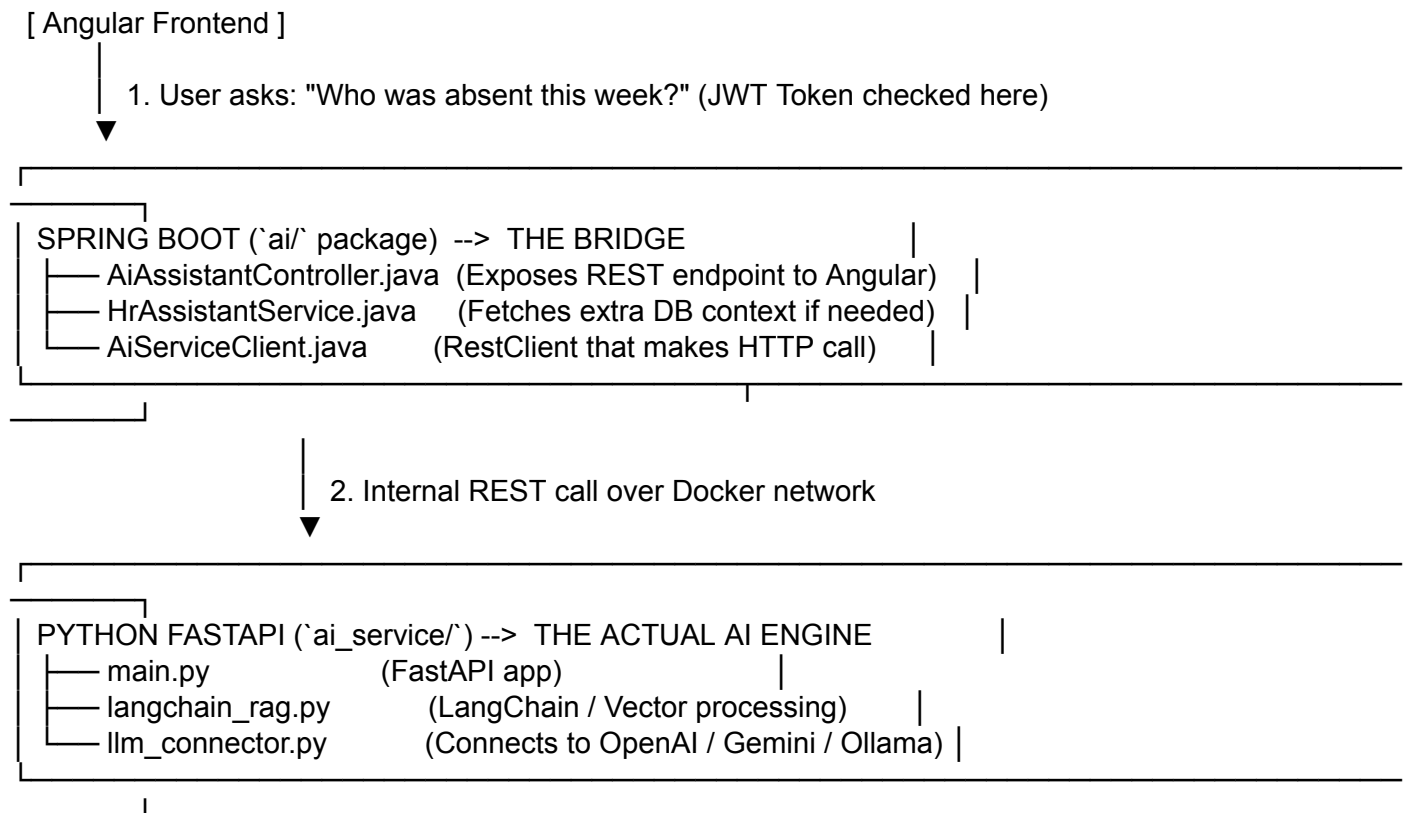
AiPromptRequest, AiReportResponse |—— client/ # AiServiceClient (RestClient/WebClient to Python FastAPI) |—— service/ # HrAssistantService |—— web/ but in the architecture that you gave me u put the ai service into spring boot

Response: Ah, good eye! I see where the confusion is.

That ai/ folder inside Spring Boot is **NOT the AI engine itself**—it is just the **Bridge/Adapter** (the Connector).

Here is the key distinction between what lives in Spring Boot vs. what lives in Python FastAPI:

The Two Components of your AI Feature



Why Spring Boot Needs the ai/ Package (The Bridge)

You never want your Angular frontend to talk directly to the Python FastAPI microservice. Here is why having this light ai/ adapter inside Spring Boot is essential:

1. **Security & Authentication (JWT):** Spring Boot already handles your user login and roles (HR_MANAGER, ADMIN). If Angular called Python directly, Python would have to re-implement all your JWT verification logic. By routing through Spring Boot first, Spring Boot ensures only authorized HR Managers can talk to the AI.
2. **Context Enrichment:** When a manager asks, "Generate a performance summary for my department," Spring Boot's HrAssistantService quickly grabs the relevant employee data from PostgreSQL and packages it into the payload *before* sending it to Python.
3. **API Uniformity:** Angular only needs **one single API base URL** (http://localhost:8080/api/v1/...) for

everything—employees, leaves, dashboards, and AI. Angular doesn't need to know or care that Python is secretly doing the heavy AI lifting in the background.

What the Code inside Java ai/ Looks Like

Notice how simple the Java AiServiceClient.java is—it has **zero AI logic**; it simply acts as an HTTP proxy to Python:

```
package com.smarterp.ai.client;

import com.smarterp.ai.dto.AiPromptRequest;
import com.smarterp.ai.dto.AiReportResponse;
import org.springframework.stereotype.Component;
import org.springframework.web.client.RestClient;

@Component
public class AiServiceClient {

    // Calls Python FastAPI running on port 8000
    private final RestClient restClient = RestClient.create("http://localhost:8000");

    public AiReportResponse forwardToPythonAi(AiPromptRequest request) {
        return restClient.post()
            .uri("/api/v1/ai/query")
            .body(request)
            .retrieve()
            .body(AiReportResponse.class);
    }
}
```

All heavy AI algorithms, LangChain code, prompt templates, vector embeddings, and LLM calls live 100% inside your ai_service/ Python folder!

User prompt: com.smarterp | — SmartErpApplication.java | — shared/ # Cross-cutting concerns (Framework layer) | — config/ # SecurityConfig, CorsConfig, JPA/Auditing config | — exception/ # GlobalExceptionHandler, BusinessException | — security/ # JwtUtils, SecurityUser, UserDetailsService | — domain/ # BaseEntity (created_at, updated_at) | — security/ # MODULE 1: Identity & Access Management (IAM) | — domain/ # User, Role, Permission (JPA Entities & Enums) | — dto/ # LoginRequest, RegisterRequest, AuthResponse | — repository/ # UserRepository, RoleRepository | — service/ # AuthService, UserService | — web/ # AuthController | — hr/ # MODULE 2: Core HR Domain | — domain/ # Employee, Department, Leave, Attendance Entities | — dto/ # EmployeeDTO, CreateEmployeeRequest, LeaveRequestDTO | — mapper/ # EmployeeMapper (MapStruct interface) | — repository/ # EmployeeRepository, DepartmentRepository | — service/ # EmployeeService, LeaveService, AttendanceService | — web/ # EmployeeController, LeaveController | — analytics/ # MODULE 3: Business Intelligence & KPIs | — dto/ # DashboardKPIsResponse, TurnoverStatsDTO | — repository/

```

# AnalyticsNativeQueryRepository (Materialized Views) | | service/ # AnalyticsService
(Aggregations) | | web/ # DashboardController | | ai/ # MODULE
4: AI Engine Connector | | dto/ # AiPromptRequest, AiReportResponse | | client/
# AiServiceClient (RestClient/WebClient to Python FastAPI) | | service/ #
HrAssistantService | | web/ # AiAssistantController give me a code in the terminale so
wont have to create them manuly

```

Response: Open your terminal inside your Spring Boot project root directory (backend or smart-erp-core) and run the command matching your shell.

Option 1: Windows PowerShell (Recommended for Windows)

Copy and paste this entire command into PowerShell:

```

New-Item -ItemType Directory -Force -Path `
"src/main/java/com/smarterp/shared/config", `
"src/main/java/com/smarterp/shared/exception", `
"src/main/java/com/smarterp/shared/security", `
"src/main/java/com/smarterp/shared/domain", `
"src/main/java/com/smarterp/security/domain", `
"src/main/java/com/smarterp/security/dto", `
"src/main/java/com/smarterp/security/repository", `
"src/main/java/com/smarterp/security/service", `
"src/main/java/com/smarterp/security/web", `
"src/main/java/com/smarterp/hr/domain", `
"src/main/java/com/smarterp/hr/dto", `
"src/main/java/com/smarterp/hr/mapper", `
"src/main/java/com/smarterp/hr/repository", `
"src/main/java/com/smarterp/hr/service", `
"src/main/java/com/smarterp/hr/web", `
"src/main/java/com/smarterp/analytics/dto", `
"src/main/java/com/smarterp/analytics/repository", `
"src/main/java/com/smarterp/analytics/service", `
"src/main/java/com/smarterp/analytics/web", `
"src/main/java/com/smarterp/ai/dto", `
"src/main/java/com/smarterp/ai/client", `
"src/main/java/com/smarterp/ai/service", `
"src/main/java/com/smarterp/ai/web"

```

Option 2: Git Bash / Linux / macOS

If you use Git Bash terminal inside VS Code or Windows, run this standard one-line command:

```

mkdir -p
src/main/java/com/smarterp/{shared/{config,exception,security,domain},security/{domain,dto,repository,service,web},hr/{domain,dto,mapper,repository,service,web},analytics/{dto,repository,service,web},ai/{dto,client,service,web}}

```

Verification

Once executed, refresh your project tree in IntelliJ / Eclipse / VS Code. Your complete package architecture will instantly appear ready for coding.

User prompt: `C:\Users\yasser\Documents\stage\Smart_ERP_Core> New-Item -ItemType Directory -Force -Path `>> "src/main/java/com/smarterp/shared/config", `>> "src/main/java/com/smarterp/shared/exception", `>> "src/main/java/com/smarterp/shared/security", `>> "src/main/java/com/smarterp/shared/domain", `>> "src/main/java/com/smarterp/security/domain", `>> "src/main/java/com/smarterp/security/dto", `>> "src/main/java/com/smarterp/security/repository", `>> "src/main/java/com/smarterp/security/service", `>> "src/main/java/com/smarterp/security/web", `>> "src/main/java/com/smarterp/hr/domain", `>> "src/main/java/com/smarterp/hr/dto", `>> "src/main/java/com/smarterp/hr/mapper", `>> "src/main/java/com/smarterp/hr/repository", `>> "src/main/java/com/smarterp/hr/service", `>> "src/main/java/com/smarterp/hr/web", `>> "src/main/java/com/smarterp/analytics/dto", `>> "src/main/java/com/smarterp/analytics/repository", `>> "src/main/java/com/smarterp/analytics/service", `>> "src/main/java/com/smarterp/analytics/web", `>> "src/main/java/com/smarterp/ai/dto", `>> "src/main/java/com/smarterp/ai/client", `>> "src/main/java/com/smarterp/ai/service", `>> "src/main/java/com/smarterp/ai/web" they wor created in a wrong directory`

Response: This happened because you were in the main root folder (Smart_ERP_Core) instead of inside your Spring Boot backend folder (e.g., backend or smart-erp-core).

Here is how to clean up the misplaced folders and create them in the correct location:

Step 1: Remove the misplaced src folder from the root

Run this command in your PowerShell terminal to delete the mistakenly created folder:

```
Remove-Item -Recurse -Force src
```

Step 2: Navigate into your Spring Boot folder

Check the name of your backend folder (it is usually backend or smart-erp-core) and navigate into it:

```
cd backend
```

(If your folder is named differently, use `cd your-backend-folder-name`, for example `cd smart-erp-core`).

Step 3: Run the creation command again inside the backend folder

Now that you are inside your Spring Boot project folder (where pom.xml lives), run this:

```
New-Item -ItemType Directory -Force -Path `
"src/main/java/com/smarterp/shared/config", `
```

```
"src/main/java/com/smarterp/shared/exception", `
"src/main/java/com/smarterp/shared/security", `
"src/main/java/com/smarterp/shared/domain", `
"src/main/java/com/smarterp/security/domain", `
"src/main/java/com/smarterp/security/dto", `
"src/main/java/com/smarterp/security/repository", `
"src/main/java/com/smarterp/security/service", `
"src/main/java/com/smarterp/security/web", `
"src/main/java/com/smarterp/hr/domain", `
"src/main/java/com/smarterp/hr/dto", `
"src/main/java/com/smarterp/hr/mapper", `
"src/main/java/com/smarterp/hr/repository", `
"src/main/java/com/smarterp/hr/service", `
"src/main/java/com/smarterp/hr/web", `
"src/main/java/com/smarterp/analytics/dto", `
"src/main/java/com/smarterp/analytics/repository", `
"src/main/java/com/smarterp/analytics/service", `
"src/main/java/com/smarterp/analytics/web", `
"src/main/java/com/smarterp/ai/dto", `
"src/main/java/com/smarterp/ai/client", `
"src/main/java/com/smarterp/ai/service", `
"src/main/java/com/smarterp/ai/web"
```

Everything will now merge cleanly into your existing Java source folder!

User prompt: The backend package structure is now in place. Let's write the Security module classes (JwtUtils, UserDetailsService, and SecurityConfig).

Response: Here are the enterprise Spring Security 6 and JWT classes configured for Spring Boot 3.3+ using the modern jjwt 0.12.x API syntax.

1. Security User Representation (SecurityUser.java)

Package: com.smarterp.shared.security

Adapter class that wraps your JPA User entity so Spring Security can process user authentication and roles.

```
package com.smarterp.shared.security;
```

```
import com.smarterp.security.domain.User;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;
```

```
import java.util.Collection;
import java.util.List;
import java.util.UUID;
```

```

public class SecurityUser implements UserDetails {

    private final User user;

    public SecurityUser(User user) {
        this.user = user;
    }

    public UUID getId() {
        return user.getId();
    }

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        return List.of(new SimpleGrantedAuthority(user.getRole().getName().name()));
    }

    @Override
    public String getPassword() {
        return user.getPasswordHash();
    }

    @Override
    public String getUsername() {
        return user.getEmail();
    }

    @Override
    public boolean isAccountNonExpired() {
        return true;
    }

    @Override
    public boolean isAccountNonLocked() {
        return true;
    }

    @Override
    public boolean isCredentialsNonExpired() {
        return true;
    }

    @Override
    public boolean isEnabled() {
        return Boolean.TRUE.equals(user.getIsActive());
    }
}

```

2. User Details Service (CustomUserDetailsService.java)

Package: com.smarterp.shared.security

Loads users directly from PostgreSQL via UserRepository.

```
package com.smarterp.shared.security;

import com.smarterp.security.repository.UserRepository;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

@Service
public class CustomUserDetailsService implements UserDetailsService {

    private final UserRepository userRepository;

    public CustomUserDetailsService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @Override
    public UserDetails loadUserByUsername(String email) throws UsernameNotFoundException {
        return userRepository.findByEmail(email)
            .map(SecurityUser::new)
            .orElseThrow(() -> new UsernameNotFoundException("User not found with email: " + email));
    }
}
```

3. JWT Utility Helper (JwtUtils.java)

Package: com.smarterp.shared.security

Handles token generation, parsing, and signature validation using HMAC-SHA256.

```
package com.smarterp.shared.security;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.io.Decoders;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Component;

import javax.crypto.SecretKey;
import java.util.Date;
import java.util.HashMap;
import java.util.Map;
import java.util.function.Function;
```

@Component

public class JwtUtils {

@Value("\${application.security.jwt.secret-key}")
 private String secretKey;

@Value("\${application.security.jwt.expiration-ms:86400000}")
 private long jwtExpiration;

public String extractUsername(String token) {
 return extractClaim(token, Claims::getSubject);
 }

public <T> T extractClaim(String token, Function<Claims, T> claimsResolver) {
 final Claims claims = extractAllClaims(token);
 return claimsResolver.apply(claims);
 }

public String generateToken(UserDetails userDetails) {
 return generateToken(new HashMap<>(), userDetails);
 }

public String generateToken(Map<String, Object> extraClaims, UserDetails userDetails) {
 return Jwts.builder()
 .claims(extraClaims)
 .subject(userDetails.getUsername())
 .issuedAt(new Date(System.currentTimeMillis()))
 .expiration(new Date(System.currentTimeMillis() + jwtExpiration))
 .signWith(getSigningKey())
 .compact();
 }

public boolean isTokenValid(String token, UserDetails userDetails) {
 final String username = extractUsername(token);
 return (username.equals(userDetails.getUsername())) && !isTokenExpired(token);
 }

private boolean isTokenExpired(String token) {
 return extractExpiration(token).before(new Date());
 }

private Date extractExpiration(String token) {
 return extractClaim(token, Claims::getExpiration);
 }

private Claims extractAllClaims(String token) {
 return Jwts.parser()
 .verifyWith(getSigningKey())
 .build()
 .parseSignedClaims(token)
 .getPayload();
 }

```

    }

    private SecretKey getSigningKey() {
        byte[] keyBytes = Decoders.BASE64.decode(secretKey);
        return Keys.hmacShaKeyFor(keyBytes);
    }
}

```

4. JWT Authentication Filter (JwtAuthenticationFilter.java)

Package: com.smarterp.shared.security

Intercepts incoming HTTP requests, checks for the Authorization: Bearer <token> header, validates the signature, and sets authentication context.

```

package com.smarterp.shared.security;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.lang.NonNull;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;

@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtUtils jwtUtils;
    private final CustomUserDetailsService userDetailsService;

    public JwtAuthenticationFilter(JwtUtils jwtUtils, CustomUserDetailsService userDetailsService) {
        this.jwtUtils = jwtUtils;
        this.userDetailsService = userDetailsService;
    }

    @Override
    protected void doFilterInternal(
        @NonNull HttpServletRequest request,
        @NonNull HttpServletResponse response,
        @NonNull FilterChain filterChain
    ) throws ServletException, IOException {
        final String authHeader = request.getHeader("Authorization");
        final String jwt;

```

```

final String userEmail;

if (authHeader == null || !authHeader.startsWith("Bearer ")) {
    filterChain.doFilter(request, response);
    return;
}

jwt = authHeader.substring(7);
try {
    userEmail = jwtUtils.extractUsername(jwt);
} catch (Exception e) {
    filterChain.doFilter(request, response);
    return;
}

if (userEmail != null && SecurityContextHolder.getContext().getAuthentication() == null) {
    UserDetails userDetails = this.userDetailsService.loadUserByUsername(userEmail);

    if (jwtUtils.isTokenValid(jwt, userDetails)) {
        UsernamePasswordAuthenticationToken authToken = new
UsernamePasswordAuthenticationToken(
            userDetails,
            null,
            userDetails.getAuthorities()
        );
        authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
        SecurityContextHolder.getContext().setAuthentication(authToken);
    }
}
filterChain.doFilter(request, response);
}
}

```

5. Central Security Configuration (SecurityConfig.java)

Package: com.smarterp.shared.config

Configures the stateless HTTP security chain, endpoint access permissions, and authentication providers.

```
package com.smarterp.shared.config;
```

```

import com.smarterp.shared.security.CustomUserDetailsService;
import com.smarterp.shared.security.JwtAuthenticationFilter;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.AuthenticationProvider;
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import
org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;

```

```
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
```

```
@Configuration
```

```
@EnableWebSecurity
```

```
@EnableMethodSecurity
```

```
public class SecurityConfig {
```

```
    private final JwtAuthenticationFilter jwtAuthFilter;
    private final CustomUserDetailsService userDetailsService;
    private final PasswordEncoder passwordEncoder;
```

```
    public SecurityConfig(
        JwtAuthenticationFilter jwtAuthFilter,
        CustomUserDetailsService userDetailsService,
        PasswordEncoder passwordEncoder
```

```
    ) {
        this.jwtAuthFilter = jwtAuthFilter;
        this.userDetailsService = userDetailsService;
        this.passwordEncoder = passwordEncoder;
    }
```

```
@Bean
```

```
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .csrf(AbstractHttpConfigurer::disable)
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/v1/auth/**").permitAll() // Public auth endpoints
            .requestMatchers("/api/v1/admin/**").hasAuthority("ROLE_ADMIN")
            .requestMatchers("/api/v1/hr/**").hasAnyAuthority("ROLE_ADMIN", "ROLE_HR_MANAGER")
            .anyRequest().authenticated()
        )
        .sessionManagement(session -> session
            .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
        )
        .authenticationProvider(authenticationProvider())
        .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);

    return http.build();
}
```

```
@Bean
```

```
public AuthenticationProvider authenticationProvider() {
    DaoAuthenticationProvider authProvider = new DaoAuthenticationProvider();
    authProvider.setUserDetailsService(userDetailsService);
    authProvider.setPasswordEncoder(passwordEncoder);
}
```

```

        return authProvider;
    }

    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws
Exception {
        return config.getAuthenticationManager();
    }
}

```

User prompt: first give me all the enteties

Response: Here is the complete set of JPA entities for the Smart ERP Core platform. All primary entities inherit automatic timestamp auditing (createdAt, updatedAt) via BaseEntity.

1. Shared Base Entity

BaseEntity.java

Package: com.smarterp.shared.domain

```

package com.smarterp.shared.domain;

import jakarta.persistence.Column;
import jakarta.persistence.EntityListeners;
import jakarta.persistence.MappedSuperclass;
import org.springframework.data.annotation.CreatedDate;
import org.springframework.data.annotation.LastModifiedDate;
import org.springframework.data.jpa.domain.support.AuditingEntityListener;

import java.time.LocalDateTime;

@EntityListeners(AuditingEntityListener.class)
@MappedSuperclass
public abstract class BaseEntity {

    @CreatedDate
    @Column(name = "created_at", nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @LastModifiedDate
    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    public LocalDateTime getCreatedAt() { return createdAt; }
    public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }

    public LocalDateTime getUpdatedAt() { return updatedAt; }

```

```
    public void setUpdatedAt(LocalDateTime updatedAt) { this.updatedAt = updatedAt; }  
}
```

2. Security Module Entities (com.smarterp.security.domain)

RoleName.java (Enum)

```
package com.smarterp.security.domain;  
  
public enum RoleName {  
    ROLE_ADMIN,  
    ROLE_HR_MANAGER,  
    ROLE_EMPLOYEE  
}
```

Role.java

```
package com.smarterp.security.domain;  
  
import jakarta.persistence.*;  
import java.util.UUID;  
  
@Entity  
@Table(name = "roles")  
public class Role {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.UUID)  
    private UUID id;  
  
    @Enumerated(EnumType.STRING)  
    @Column(nullable = false, unique = true, length = 50)  
    private RoleName name;  
  
    @Column(length = 255)  
    private String description;  
  
    public Role() {}  
  
    public Role(RoleName name, String description) {  
        this.name = name;  
        this.description = description;  
    }  
  
    public UUID getId() { return id; }  
    public void setId(UUID id) { this.id = id; }  
  
    public RoleName getName() { return name; }  
    public void setName(RoleName name) { this.name = name; }
```

```
    public String getDescription() { return description; }
    public void setDescription(String description) { this.description = description; }
}
```

User.java

```
package com.smarterp.security.domain;
```

```
import com.smarterp.shared.domain.BaseEntity;
import jakarta.persistence.*;
import java.util.UUID;
```

```
@Entity
@Table(name = "users")
public class User extends BaseEntity {
```

```
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;
```

```
    @Column(nullable = false, unique = true, length = 150)
    private String email;
```

```
    @Column(name = "password_hash", nullable = false)
    private String passwordHash;
```

```
    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "role_id", nullable = false)
    private Role role;
```

```
    @Column(name = "is_active")
    private Boolean isActive = true;
```

```
    public User() {}
```

```
    public UUID getId() { return id; }
    public void setId(UUID id) { this.id = id; }
```

```
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
```

```
    public String getPasswordHash() { return passwordHash; }
    public void setPasswordHash(String passwordHash) { this.passwordHash = passwordHash; }
```

```
    public Role getRole() { return role; }
    public void setRole(Role role) { this.role = role; }
```

```
    public Boolean getIsActive() { return isActive; }
    public void setIsActive(Boolean active) { isActive = active; }
```

```
}
```

3. Core HR Module Entities (com.smarterp.hr.domain)

Enums

```
package com.smarterp.hr.domain;
```

```
public enum EmployeeStatus {  
    ACTIVE, ON_LEAVE, TERMINATED  
}
```

```
package com.smarterp.hr.domain;
```

```
public enum LeaveType {  
    ANNUAL, SICK, MATERNITY, PATERNITY, UNPAID  
}
```

```
package com.smarterp.hr.domain;
```

```
public enum LeaveStatus {  
    PENDING, APPROVED, REJECTED, CANCELLED  
}
```

```
package com.smarterp.hr.domain;
```

```
public enum AttendanceStatus {  
    PRESENT, ABSENT, LATE, HALF_DAY  
}
```

Department.java

```
package com.smarterp.hr.domain;
```

```
import com.smarterp.shared.domain.BaseEntity;  
import jakarta.persistence.*;  
import java.util.UUID;
```

```
@Entity
```

```
@Table(name = "departments")
```

```
public class Department extends BaseEntity {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.UUID)
```

```
    private UUID id;
```

```
    @Column(nullable = false, unique = true, length = 100)
```

```
    private String name;
```

```
    @Column(columnDefinition = "TEXT")
```

```

private String description;

@OneToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "manager_id")
private Employee manager;

public Department() {}

public Department(String name, String description) {
    this.name = name;
    this.description = description;
}

public UUID getId() { return id; }
public void setId(UUID id) { this.id = id; }

public String getName() { return name; }
public void setName(String name) { this.name = name; }

public String getDescription() { return description; }
public void setDescription(String description) { this.description = description; }

public Employee getManager() { return manager; }
public void setManager(Employee manager) { this.manager = manager; }
}

```

Employee.java

```

package com.smarterp.hr.domain;

import com.smarterp.security.domain.User;
import com.smarterp.shared.domain.BaseEntity;
import jakarta.persistence.*;
import java.time.LocalDate;
import java.util.UUID;

@Entity
@Table(name = "employees")
public class Employee extends BaseEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    @OneToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", unique = true)
    private User user;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "department_id")
    private Department department;
}

```

```

@Column(name = "first_name", nullable = false, length = 100)
private String firstName;

@Column(name = "last_name", nullable = false, length = 100)
private String lastName;

@Column(length = 20)
private String phone;

@Column(nullable = false, length = 100)
private String position;

@Column(name = "hire_date", nullable = false)
private LocalDate hireDate;

@Enumerated(EnumType.STRING)
@Column(nullable = false, length = 50)
private EmployeeStatus status = EmployeeStatus.ACTIVE;

public Employee() {}

public UUID getId() { return id; }
public void setId(UUID id) { this.id = id; }

public User getUser() { return user; }
public void setUser(User user) { this.user = user; }

public Department getDepartment() { return department; }
public void setDepartment(Department department) { this.department = department; }

public String getFirstName() { return firstName; }
public void setFirstName(String firstName) { this.firstName = firstName; }

public String getLastName() { return lastName; }
public void setLastName(String lastName) { this.lastName = lastName; }

public String getPhone() { return phone; }
public void setPhone(String phone) { this.phone = phone; }

public String getPosition() { return position; }
public void setPosition(String position) { this.position = position; }

public LocalDate getHireDate() { return hireDate; }
public void setHireDate(LocalDate hireDate) { this.hireDate = hireDate; }

public EmployeeStatus getStatus() { return status; }
public void setStatus(EmployeeStatus status) { this.status = status; }
}

```

LeaveRequest.java

```
package com.smarterp.hr.domain;

import com.smarterp.shared.domain.BaseEntity;
import jakarta.persistence.*;
import java.time.LocalDate;
import java.util.UUID;

@Entity
@Table(name = "leave_requests")
public class LeaveRequest extends BaseEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "employee_id", nullable = false)
    private Employee employee;

    @Enumerated(EnumType.STRING)
    @Column(name = "leave_type", nullable = false, length = 50)
    private LeaveType leaveType;

    @Column(name = "start_date", nullable = false)
    private LocalDate startDate;

    @Column(name = "end_date", nullable = false)
    private LocalDate endDate;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 50)
    private LeaveStatus status = LeaveStatus.PENDING;

    @Column(columnDefinition = "TEXT")
    private String reason;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "approved_by")
    private Employee approvedBy;

    public LeaveRequest() {}

    public UUID getId() { return id; }
    public void setId(UUID id) { this.id = id; }

    public Employee getEmployee() { return employee; }
    public void setEmployee(Employee employee) { this.employee = employee; }

    public LeaveType getLeaveType() { return leaveType; }
    public void setLeaveType(LeaveType leaveType) { this.leaveType = leaveType; }
```

```

public LocalDate getStartDate() { return startDate; }
public void setStartDate(LocalDate startDate) { this.startDate = startDate; }

public LocalDate getEndDate() { return endDate; }
public void setEndDate(LocalDate endDate) { this.endDate = endDate; }

public LeaveStatus getStatus() { return status; }
public void setStatus(LeaveStatus status) { this.status = status; }

public String getReason() { return reason; }
public void setReason(String reason) { this.reason = reason; }

public Employee getApprovedBy() { return approvedBy; }
public void setApprovedBy(Employee approvedBy) { this.approvedBy = approvedBy; }
}

```

Attendance.java

```

package com.smarterp.hr.domain;

import com.smarterp.shared.domain.BaseEntity;
import jakarta.persistence.*;
import java.time.LocalDate;
import java.time.LocalTime;
import java.util.UUID;

@Entity
@Table(name = "attendances", uniqueConstraints = {
    @UniqueConstraint(columnNames = {"employee_id", "work_date"})
})
public class Attendance extends BaseEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "employee_id", nullable = false)
    private Employee employee;

    @Column(name = "work_date", nullable = false)
    private LocalDate workDate;

    @Column(name = "check_in")
    private LocalTime checkIn;

    @Column(name = "check_out")
    private LocalTime checkOut;

    @Enumerated(EnumType.STRING)

```

```

@Column(nullable = false, length = 50)
private AttendanceStatus status = AttendanceStatus.PRESENT;

public Attendance() {}

public UUID getId() { return id; }
public void setId(UUID id) { this.id = id; }

public Employee getEmployee() { return employee; }
public void setEmployee(Employee employee) { this.employee = employee; }

public LocalDate getWorkDate() { return workDate; }
public void setWorkDate(LocalDate workDate) { this.workDate = workDate; }

public LocalTime getCheckIn() { return checkIn; }
public void setCheckIn(LocalTime checkIn) { this.checkIn = checkIn; }

public LocalTime getCheckOut() { return checkOut; }
public void setCheckOut(LocalTime checkOut) { this.checkOut = checkOut; }

public AttendanceStatus getStatus() { return status; }
public void setStatus(AttendanceStatus status) { this.status = status; }
}

```

4. AI Connector Audit Log Entity (com.smarterp.ai.domain)

AiQueryLog.java

Stores audit records of user interactions with the Python FastAPI AI assistant for auditing and compliance.

```

package com.smarterp.ai.domain;

import com.smarterp.security.domain.User;
import com.smarterp.shared.domain.BaseEntity;
import jakarta.persistence.*;
import java.util.UUID;

@Entity
@Table(name = "ai_query_logs")
public class AiQueryLog extends BaseEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    @Column(columnDefinition = "TEXT", nullable = false)

```

```

private String prompt;

@Column(columnDefinition = "TEXT")
private String response;

@Column(name = "execution_time_ms")
private Long executionTimeMs;

public AiQueryLog() {}

public AiQueryLog(User user, String prompt, String response, Long executionTimeMs) {
    this.user = user;
    this.prompt = prompt;
    this.response = response;
    this.executionTimeMs = executionTimeMs;
}

public UUID getId() { return id; }
public void setId(UUID id) { this.id = id; }

public User getUser() { return user; }
public void setUser(User user) { this.user = user; }

public String getPrompt() { return prompt; }
public void setPrompt(String prompt) { this.prompt = prompt; }

public String getResponse() { return response; }
public void setResponse(String response) { this.response = response; }

public Long getExecutionTimeMs() { return executionTimeMs; }
public void setExecutionTimeMs(Long executionTimeMs) { this.executionTimeMs = executionTimeMs; }
}

```

User prompt: public class Employee extends BaseEntity why did you use extends BaseEntity

Response: Extending BaseEntity is an enterprise Java design pattern used to eliminate boilerplate code and automatically track database record lifecycles.

Here are the 4 main reasons why this pattern is used:

1. Eliminates Code Duplication (DRY Principle)

Without BaseEntity, you would have to manually declare createdAt and updatedAt fields, getters, setters, and annotations inside every single entity (User, Employee, Department, LeaveRequest, Attendance, AiQueryLog). Defining them once in BaseEntity allows all entities to inherit them automatically.

2. Automated Spring Data JPA Auditing

When paired with `@EntityListeners(AuditingEntityListener.class)` on `BaseEntity` and `@EnableJpaAuditing` on your main application class, Spring handles timestamping behind the scenes:

- When a new row is inserted, Spring automatically sets `createdAt` to the current timestamp.
- When a row is updated, Spring automatically updates `updatedAt`.
- You never have to manually call `employee.setCreatedAt(LocalDateTime.now())` anywhere in your service code.

3. Database Schema Consistency

It guarantees that every table in your PostgreSQL database shares the exact same timestamp audit column names and data types. This makes time-based queries, sorting (e.g., fetching the newest 10 employees), and data debugging predictable across the entire system.

4. Single Point of Modification

If you later decide to add system-wide features—such as tracking *who* created a record (`createdBy`), tracking who modified it (`lastModifiedBy`), or implementing soft deletes (`isDeleted`)—you only add those fields to `BaseEntity`. Every table in your database instantly inherits the new capability.

User prompt: i ddint create the tables in postgres should that be a problem

Response: No, that is **not a problem at all!** In fact, that is the expected behavior.

You do **not** need to manually run `CREATE TABLE` scripts in PostgreSQL. Spring Data JPA and Hibernate will create all the tables, primary keys, foreign key constraints, and columns for you automatically when you run the Spring Boot application.

How it works automatically:

Because we configured this setting in your `application.yml`:

```
spring:
  jpa:
    hibernate:
      ddl-auto: update
```

When Spring Boot starts up:

1. Hibernate scans all your Java `@Entity` classes (`User`, `Role`, `Employee`, `Department`, `LeaveRequest`, `Attendance`, `AiQueryLog`).
2. It checks your PostgreSQL database.
3. Since the tables don't exist yet, Hibernate automatically generates and executes the SQL `CREATE TABLE` commands for you behind the scenes.

The ONLY thing you must ensure:

The **database itself** must exist inside PostgreSQL.

If you created your database container using docker-compose.yml with POSTGRES_DB: smart_erp, Docker already created the empty database for you.

How to verify:

1. Start your Spring Boot application.
 2. Watch the console logs—you will see Hibernate outputting SQL create table ... statements.
 3. Open **pgAdmin** (<http://localhost:5050>), expand **Databases** -> **smart_erp** -> **Schemas** -> **public** -> **Tables**. You will see all your tables generated automatically!
-

User prompt: The database tables auto-generation is clear. Let's create the Spring Data Repositories and Data Transfer Objects (DTOs).

Response: Here is the complete set of **Spring Data Repositories** and **Data Transfer Objects (DTOs)** implemented using modern Java record types for clean, immutable data transfer.

Part 1: Spring Data JPA Repositories

Security Repositories (com.smarterp.security.repository)

RoleRepository.java

```
package com.smarterp.security.repository;

import com.smarterp.security.domain.Role;
import com.smarterp.security.domain.RoleName;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;
import java.util.UUID;

public interface RoleRepository extends JpaRepository<Role, UUID> {
    Optional<Role> findByName(RoleName name);
}
```

UserRepository.java

```
package com.smarterp.security.repository;

import com.smarterp.security.domain.User;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;
import java.util.UUID;
```

```
public interface UserRepository extends JpaRepository<User, UUID> {  
    Optional<User> findByEmail(String email);  
    Boolean existsByEmail(String email);  
}
```

HR Repositories (com.smarterp.hr.repository)

DepartmentRepository.java

```
package com.smarterp.hr.repository;  
  
import com.smarterp.hr.domain.Department;  
import org.springframework.data.jpa.repository.JpaRepository;  
import java.util.Optional;  
import java.util.UUID;  
  
public interface DepartmentRepository extends JpaRepository<Department, UUID> {  
    Optional<Department> findByName(String name);  
    Boolean existsByName(String name);  
}
```

EmployeeRepository.java

```
package com.smarterp.hr.repository;  
  
import com.smarterp.hr.domain.Employee;  
import com.smarterp.hr.domain.EmployeeStatus;  
import org.springframework.data.jpa.repository.JpaRepository;  
import java.util.List;  
import java.util.Optional;  
import java.util.UUID;  
  
public interface EmployeeRepository extends JpaRepository<Employee, UUID> {  
    Optional<Employee> findById(UUID userId);  
    List<Employee> findByDepartmentId(UUID departmentId);  
    List<Employee> findByStatus(EmployeeStatus status);  
    Boolean existsByUserId(UUID userId);  
}
```

LeaveRequestRepository.java

```
package com.smarterp.hr.repository;  
  
import com.smarterp.hr.domain.LeaveRequest;  
import com.smarterp.hr.domain.LeaveStatus;  
import org.springframework.data.jpa.repository.JpaRepository;  
import java.util.List;  
import java.util.UUID;
```

```
public interface LeaveRequestRepository extends JpaRepository<LeaveRequest, UUID> {
    List<LeaveRequest> findByEmployeeId(UUID employeeId);
    List<LeaveRequest> findByStatus(LeaveStatus status);
    List<LeaveRequest> findByEmployeeIdAndStatus(UUID employeeId, LeaveStatus status);
}
```

AttendanceRepository.java

```
package com.smarterp.hr.repository;

import com.smarterp.hr.domain.Attendance;
import org.springframework.data.jpa.repository.JpaRepository;
import java.time.LocalDate;
import java.util.List;
import java.util.Optional;
import java.util.UUID;

public interface AttendanceRepository extends JpaRepository<Attendance, UUID> {
    List<Attendance> findByEmployeeId(UUID employeeId);
    Optional<Attendance> findByEmployeeIdAndWorkDate(UUID employeeId, LocalDate workDate);
    List<Attendance> findByWorkDate(LocalDate workDate);
}
```

Part 2: Data Transfer Objects (DTOs)

Using Java record types simplifies DTO creation by automatically providing getters, equals(), hashCode(), and toString().

Security DTOs (com.smarterp.security.dto)

LoginRequest.java

```
package com.smarterp.security.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;

public record LoginRequest(
    @NotBlank(message = "Email is required")
    @Email(message = "Invalid email format")
    String email,

    @NotBlank(message = "Password is required")
    String password
) {}
```

RegisterRequest.java

```

package com.smarterp.security.dto;

import com.smarterp.security.domain.RoleName;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;

public record RegisterRequest(
    @NotBlank(message = "Email is required")
    @Email(message = "Invalid email format")
    String email,

    @NotBlank(message = "Password is required")
    @Size(min = 6, message = "Password must be at least 6 characters")
    String password,

    @NotNull(message = "Role is required")
    RoleName role
) {}

```

AuthResponse.java

```

package com.smarterp.security.dto;

import java.util.UUID;

public record AuthResponse(
    String accessToken,
    String tokenType,
    UUID userId,
    String email,
    String role
) {
    public AuthResponse(String accessToken, UUID userId, String email, String role) {
        this(accessToken, "Bearer", userId, email, role);
    }
}

```

UserDTO.java

```

package com.smarterp.security.dto;

import java.util.UUID;

public record UserDTO(
    UUID id,
    String email,
    String role,
    Boolean isActive
) {}

```

```
) {}
```

HR DTOs (com.smarterp.hr.dto)

DepartmentDTO.java

```
package com.smarterp.hr.dto;
```

```
import java.util.UUID;
```

```
public record DepartmentDTO(  
    UUID id,  
    String name,  
    String description,  
    UUID managerId,  
    String managerName  
) {}
```

CreateDepartmentRequest.java

```
package com.smarterp.hr.dto;
```

```
import jakarta.validation.constraints.NotBlank;  
import java.util.UUID;
```

```
public record CreateDepartmentRequest(  
    @NotBlank(message = "Department name is required")  
    String name,  
    String description,  
    UUID managerId  
) {}
```

EmployeeDTO.java

```
package com.smarterp.hr.dto;
```

```
import com.smarterp.hr.domain.EmployeeStatus;  
import java.time.LocalDate;  
import java.util.UUID;
```

```
public record EmployeeDTO(  
    UUID id,  
    UUID userId,  
    String email,  
    UUID departmentId,  
    String departmentName,  
    String firstName,  
    String lastName,  
    String phone,
```

```
String position,  
LocalDate hireDate,  
EmployeeStatus status  
) {}
```

CreateEmployeeRequest.java

```
package com.smarterp.hr.dto;  
  
import com.smarterp.security.domain.RoleName;  
import jakarta.validation.constraints.Email;  
import jakarta.validation.constraints.NotBlank;  
import jakarta.validation.constraints.NotNull;  
import java.time.LocalDate;  
import java.util.UUID;  
  
public record CreateEmployeeRequest(  
    @NotBlank(message = "Email is required")  
    @Email(message = "Invalid email format")  
    String email,  
  
    @NotBlank(message = "Password is required")  
    String password,  
  
    @NotNull(message = "Role is required")  
    RoleName role,  
  
    UUID departmentId,  
  
    @NotBlank(message = "First name is required")  
    String firstName,  
  
    @NotBlank(message = "Last name is required")  
    String lastName,  
  
    String phone,  
  
    @NotBlank(message = "Position is required")  
    String position,  
  
    @NotNull(message = "Hire date is required")  
    LocalDate hireDate  
) {}
```

LeaveRequestDTO.java

```
package com.smarterp.hr.dto;  
  
import com.smarterp.hr.domain.LeaveStatus;  
import com.smarterp.hr.domain.LeaveType;
```

```

import java.time.LocalDate;
import java.util.UUID;

public record LeaveRequestDTO(
    UUID id,
    UUID employeeId,
    String employeeName,
    LeaveType leaveType,
    LocalDate startDate,
    LocalDate endDate,
    LeaveStatus status,
    String reason,
    UUID approvedById
) {}

```

CreateLeaveRequest.java

```

package com.smarterp.hr.dto;

import com.smarterp.hr.domain.LeaveType;
import jakarta.validation.constraints.NotNull;
import java.time.LocalDate;
import java.util.UUID;

public record CreateLeaveRequest(
    @NotNull(message = "Employee ID is required")
    UUID employeeId,

    @NotNull(message = "Leave type is required")
    LeaveType leaveType,

    @NotNull(message = "Start date is required")
    LocalDate startDate,

    @NotNull(message = "End date is required")
    LocalDate endDate,

    String reason
) {}

```

AttendanceDTO.java

```

package com.smarterp.hr.dto;

import com.smarterp.hr.domain.AttendanceStatus;
import java.time.LocalDate;
import java.time.LocalTime;
import java.util.UUID;

public record AttendanceDTO(

```

```
    UUID id,  
    UUID employeeId,  
    String employeeName,  
    LocalDate workDate,  
    LocalTime checkIn,  
    LocalTime checkOut,  
    AttendanceStatus status  
) {}
```

AI Bridge DTOs (com.smarterp.ai.dto)

AiPromptRequest.java

```
package com.smarterp.ai.dto;  
  
import jakarta.validation.constraints.NotBlank;  
import java.util.Map;  
  
public record AiPromptRequest(  
    @NotBlank(message = "Prompt content cannot be empty")  
    String prompt,  
    Map<String, Object> context  
) {}
```

AiReportResponse.java

```
package com.smarterp.ai.dto;  
  
import java.util.List;  
  
public record AiReportResponse(  
    String summary,  
    List<String> insights,  
    Long executionTimeMs  
) {}
```

User prompt: Now let's build the AuthService and AuthController to complete the user authentication flow.

Response: Here is the implementation of AuthService and AuthController to handle user registration, credential verification, JWT token issuance, and profile fetching.

1. Authentication Service (AuthService.java)

Package: com.smarterp.security.service

Handles the core business logic for user registration, BCrypt password validation via Spring's

AuthenticationManager, and JWT token generation.

```
package com.smarterp.security.service;
```

```
import com.smarterp.security.domain.Role;
import com.smarterp.security.domain.User;
import com.smarterp.security.dto.AuthResponse;
import com.smarterp.security.dto.LoginRequest;
import com.smarterp.security.dto.RegisterRequest;
import com.smarterp.security.dto.UserDTO;
import com.smarterp.security.repository.RoleRepository;
import com.smarterp.security.repository.UserRepository;
import com.smarterp.shared.security.JwtUtils;
import com.smarterp.shared.security.SecurityUser;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
```

```
@Service
```

```
public class AuthService {
```

```
    private final UserRepository userRepository;
    private final RoleRepository roleRepository;
    private final PasswordEncoder passwordEncoder;
    private final JwtUtils jwtUtils;
    private final AuthenticationManager authenticationManager;
```

```
    public AuthService(
        UserRepository userRepository,
        RoleRepository roleRepository,
        PasswordEncoder passwordEncoder,
        JwtUtils jwtUtils,
        AuthenticationManager authenticationManager
```

```
) {
    this.userRepository = userRepository;
    this.roleRepository = roleRepository;
    this.passwordEncoder = passwordEncoder;
    this.jwtUtils = jwtUtils;
    this.authenticationManager = authenticationManager;
}
```

```
@Transactional
```

```
public AuthResponse register(RegisterRequest request) {
    if (userRepository.existsByEmail(request.email())) {
        throw new RuntimeException("Email address is already in use.");
    }
}
```

```
    Role role = roleRepository.findByName(request.role())
        .orElseThrow(() -> new RuntimeException("Error: Role " + request.role() + " was not found."));
```

```
User user = new User();
user.setEmail(request.email());
user.setPasswordHash(passwordEncoder.encode(request.password()));
user.setRole(role);
user.setIsActive(true);
```

```
User savedUser = userRepository.save(user);
SecurityUser securityUser = new SecurityUser(savedUser);
String jwtToken = jwtUtils.generateToken(securityUser);
```

```
return new AuthResponse(
    jwtToken,
    savedUser.getId(),
    savedUser.getEmail(),
    savedUser.getRole().getName().name()
);
}
```

```
public AuthResponse login(LoginRequest request) {
    authenticationManager.authenticate(
        new UsernamePasswordAuthenticationToken(request.email(), request.password())
    );
```

```
User user = userRepository.findByEmail(request.email())
    .orElseThrow(() -> new RuntimeException("User not found with email: " + request.email()));
```

```
SecurityUser securityUser = new SecurityUser(user);
String jwtToken = jwtUtils.generateToken(securityUser);
```

```
return new AuthResponse(
    jwtToken,
    user.getId(),
    user.getEmail(),
    user.getRole().getName().name()
);
}
```

```
@Transactional(readOnly = true)
```

```
public UserDTO getCurrentUser(String email) {
    User user = userRepository.findByEmail(email)
        .orElseThrow(() -> new RuntimeException("User not found with email: " + email));
```

```
return new UserDTO(
    user.getId(),
    user.getEmail(),
    user.getRole().getName().name(),
    user.getIsActive()
);
```

```
}
}
```

2. Authentication Controller (AuthController.java)

Package: com.smarterp.security.web

Exposes public authentication REST endpoints (/api/v1/auth/register and /api/v1/auth/login) and a protected authenticated session endpoint (/api/v1/auth/me).

```
package com.smarterp.security.web;

import com.smarterp.security.dto.AuthResponse;
import com.smarterp.security.dto.LoginRequest;
import com.smarterp.security.dto.RegisterRequest;
import com.smarterp.security.dto.UserDTO;
import com.smarterp.security.service.AuthService;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/auth")
public class AuthController {

    private final AuthService authService;

    public AuthController(AuthService authService) {
        this.authService = authService;
    }

    @PostMapping("/register")
    public ResponseEntity<AuthResponse> register(@Valid @RequestBody RegisterRequest request) {
        return ResponseEntity.status(HttpStatus.CREATED).body(authService.register(request));
    }

    @PostMapping("/login")
    public ResponseEntity<AuthResponse> login(@Valid @RequestBody LoginRequest request) {
        return ResponseEntity.ok(authService.login(request));
    }

    @GetMapping("/me")
    public ResponseEntity<UserDTO> getCurrentUser(Authentication authentication) {
        return ResponseEntity.ok(authService.getCurrentUser(authentication.getName()));
    }
}
```

How to Test in Postman / cURL

1. Register a new Admin or HR Manager

- **HTTP Method:** POST
- **URL:** http://localhost:8080/api/v1/auth/register
- **Body (JSON):**

```
{
  "email": "admin@smarterp.com",
  "password": "AdminPassword123!",
  "role": "ROLE_ADMIN"
}
```

2. Log in to acquire JWT Token

- **HTTP Method:** POST
- **URL:** http://localhost:8080/api/v1/auth/login
- **Body (JSON):**

```
{
  "email": "admin@smarterp.com",
  "password": "AdminPassword123!"
}
```

- **Response:** Returns JSON containing accessToken.

3. Access Protected Route (/me)

- **HTTP Method:** GET
- **URL:** http://localhost:8080/api/v1/auth/me
- **Header:** Authorization: Bearer <YOUR_JWT_TOKEN>

User prompt: import com.smarterp.shared.security.JwtUtils;import com.smarterp.shared.security.SecurityUser; these dont exist for me

Response: This happens when the files are missing from the com.smarterp.shared.security package directory, or if the package header at the top of those files doesn't match.

Ensure these two files exist in src/main/java/com/smarterp/shared/security/:

File 1: src/main/java/com/smarterp/shared/security/SecurityUser.java

```
package com.smarterp.shared.security;
```

```
import com.smarterp.security.domain.User;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;
```

```
import java.util.Collection;
import java.util.List;
import java.util.UUID;
```

```
public class SecurityUser implements UserDetails {
```

```

private final User user;

public SecurityUser(User user) {
    this.user = user;
}

public UUID getId() {
    return user.getId();
}

@Override
public Collection<? extends GrantedAuthority> getAuthorities() {
    return List.of(new SimpleGrantedAuthority(user.getRole().getName().name()));
}

@Override
public String getPassword() {
    return user.getPasswordHash();
}

@Override
public String getUsername() {
    return user.getEmail();
}

@Override
public boolean isAccountNonExpired() {
    return true;
}

@Override
public boolean isAccountNonLocked() {
    return true;
}

@Override
public boolean isCredentialsNonExpired() {
    return true;
}

@Override
public boolean isEnabled() {
    return Boolean.TRUE.equals(user.getIsActive());
}
}

```

File 2: src/main/java/com/smarterp/shared/security/JwtUtils.java

```

package com.smarterp.shared.security;

```

```
import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.io.Decoders;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Component;
```

```
import javax.crypto.SecretKey;
import java.util.Date;
import java.util.HashMap;
import java.util.Map;
import java.util.function.Function;
```

```
@Component
```

```
public class JwtUtils {
```

```
    @Value("${application.security.jwt.secret-key}")
    private String secretKey;
```

```
    @Value("${application.security.jwt.expiration-ms:86400000}")
    private long jwtExpiration;
```

```
    public String extractUsername(String token) {
        return extractClaim(token, Claims::getSubject);
    }
```

```
    public <T> T extractClaim(String token, Function<Claims, T> claimsResolver) {
        final Claims claims = extractAllClaims(token);
        return claimsResolver.apply(claims);
    }
```

```
    public String generateToken(UserDetails userDetails) {
        return generateToken(new HashMap<>(), userDetails);
    }
```

```
    public String generateToken(Map<String, Object> extraClaims, UserDetails userDetails) {
        return Jwts.builder()
            .claims(extraClaims)
            .subject(userDetails.getUsername())
            .issuedAt(new Date(System.currentTimeMillis()))
            .expiration(new Date(System.currentTimeMillis() + jwtExpiration))
            .signWith(getSigningKey())
            .compact();
    }
```

```
    public boolean isTokenValid(String token, UserDetails userDetails) {
        final String username = extractUsername(token);
        return (username.equals(userDetails.getUsername())) && !isTokenExpired(token);
    }
```

```
    private boolean isTokenExpired(String token) {
```

```

        return extractExpiration(token).before(new Date());
    }

    private Date extractExpiration(String token) {
        return extractClaim(token, Claims::getExpiration);
    }

    private Claims extractAllClaims(String token) {
        return Jwts.parser()
            .verifyWith(getSigningKey())
            .build()
            .parseSignedClaims(token)
            .getPayload();
    }

    private SecretKey getSigningKey() {
        byte[] keyBytes = Decoders.BASE64.decode(secretKey);
        return Keys.hmacShaKeyFor(keyBytes);
    }
}

```

Next Steps if Errors Persist

1. **Check JJWT dependencies in pom.xml:** If `io.jsonwebtoken` shows red in `JwtUtils.java`, ensure you added the `jjwt-api`, `jjwt-impl`, and `jjwt-jackson` dependencies to `pom.xml` and clicked **Reload Maven Project** in your IDE.
2. **Reload IDE Caches:** In IntelliJ, go to **File -> Invalidate Caches / Restart** if files exist in the file system but the IDE fails to resolve imports.

User prompt: e import io.jsonwebtoken cannot be resolved

Response: This error occurs when Maven hasn't downloaded the **JJWT (Java JWT)** dependencies, or the dependency configuration in your `pom.xml` is missing/un-synced.

Step 1: Ensure JJWT is in pom.xml

Open your `pom.xml` file and verify that these three dependencies are placed inside the `<dependencies> ... </dependencies>` section:

```

<!-- JJWT for JWT Authentication -->
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.12.5</version>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.12.5</version>

```

```
<scope>runtime</scope>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.12.5</version>
  <scope>runtime</scope>
</dependency>
```

Step 2: Force Maven Reload in Your IDE

After modifying pom.xml, your IDE must sync and download the JAR files:

- **IntelliJ IDEA:** Click the **floating elephant icon** (Load Maven Changes) in the top-right corner of pom.xml, OR open the **Maven** tool window on the right sidebar and click the **Reload All Maven Projects** button (the small refresh icon).
- **VS Code:** Press Ctrl + Shift + P (or Cmd + Shift + P on Mac), type Java: Clean Java Language Server Workspace, and select **Clean and Restart**.
- **Eclipse / STS:** Right-click your project root -> **Maven** -> **Update Project...** -> Check **Force Update of Snapshots/Releases** -> Click **OK**.

Step 3: Command Line Alternative (If IDE Sync Fails)

You can force Maven to download the missing libraries directly via terminal:

```
./mvnw clean compile
```

(Or mvn clean compile if using system Maven)

Once completed, the red compilation error on import io.jsonwebtoken... will disappear.

User prompt: how to lance my backend so i can tetst it

Response: Before launching your backend, you need default system roles (ROLE_ADMIN, ROLE_HR_MANAGER, ROLE_EMPLOYEE) populated in your database so the registration endpoint does not fail.

Step 0: Create a Data Initializer (One-Time Setup)

Create DataInitializer.java inside src/main/java/com/smarterp/shared/config/DataInitializer.java. This will automatically seed default roles into PostgreSQL when Spring Boot starts up.

```
package com.smarterp.shared.config;
```

```
import com.smarterp.security.domain.Role;
```

```

import com.smarterp.security.domain.RoleName;
import com.smarterp.security.repository.RoleRepository;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

@Component
public class DataInitializer implements CommandLineRunner {

    private final RoleRepository roleRepository;

    public DataInitializer(RoleRepository roleRepository) {
        this.roleRepository = roleRepository;
    }

    @Override
    public void run(String... args) {
        for (RoleName roleName : RoleName.values()) {
            if (roleRepository.findByName(roleName).isEmpty()) {
                roleRepository.save(new Role(roleName, "Default role: " + roleName.name()));
            }
        }
    }
}

```

Step 1: Ensure PostgreSQL Container / Service is Running

If you are using Docker, run this in your terminal:

```
docker compose up -d
```

If using local PostgreSQL, ensure the PostgreSQL service is running and the database smart_erp exists.

Step 2: Launch the Backend

Option A: Terminal (Recommended)

Open your terminal inside your Spring Boot project root directory (where pom.xml is located):

- **Windows (PowerShell or CMD):**
`.\mvnw.cmd spring-boot:run`
- **Mac / Linux:**
`./mvnw spring-boot:run`

Option B: IDE (IntelliJ IDEA / VS Code)

1. Open SmartErpApplication.java (src/main/java/com/smarterp/SmartErpApplication.java).

2. Click the green **Play/Run** icon next to public static void main(String[] args).

Step 3: Test Your Server

Once you see Started SmartErpApplication in X.XXX seconds in the terminal logs, test the endpoints using Postman or cURL.

1. Register a new user

Request: POST http://localhost:8080/api/v1/auth/register

- **Header:** Content-Type: application/json
- **Body:**

```
{  
  "email": "admin@smarterp.com",  
  "password": "Password123!",  
  "role": "ROLE_ADMIN"  
}
```

- **Response:** You will receive a 201 Created HTTP response with a generated accessToken.

2. Log in

Request: POST http://localhost:8080/api/v1/auth/login

- **Header:** Content-Type: application/json
- **Body:**

```
{  
  "email": "admin@smarterp.com",  
  "password": "Password123!"  
}
```

3. Test Protected Endpoint (/me)

Request: GET http://localhost:8080/api/v1/auth/me

- **Header:** Authorization: Bearer <YOUR_JWT_TOKEN>
- **Response:** Returns your logged-in user details.

User prompt: Step 0: Create a Data Initializer (One-Time Setup) Create DataInitializer.java inside src/main/java/com/smarterp/shared/config/DataInitializer.java. This will automatically seed default roles into PostgreSQL when Spring Boot starts up. explain

Response: When Hibernate creates your PostgreSQL tables, the tables are completely empty.

If you try to register a new user right away, AuthService runs this lookup:

```
Role role = roleRepository.findByName(request.role())
    .orElseThrow(() -> new RuntimeException("Error: Role was not found."));
```

Because the roles table has zero rows in PostgreSQL, this lookup fails immediately, throwing an exception and blocking registration.

DataInitializer solves this problem by automatically populating the mandatory default roles (ROLE_ADMIN, ROLE_HR_MANAGER, ROLE_EMPLOYEE) the moment the server boots up, so you never have to write manual SQL INSERT statements.

How the Code Works Step-by-Step

1. @Component

Tells Spring Boot to scan, detect, and instantiate this class as a Spring-managed bean during application startup.

2. implements CommandLineRunner

This is a built-in Spring Boot interface. Any bean that implements CommandLineRunner will automatically execute its run() method **once the Spring application context has fully loaded**, right before the application begins accepting HTTP requests.

3. for (RoleName roleName : RoleName.values())

Loops through all enum values defined in RoleName.java (ROLE_ADMIN, ROLE_HR_MANAGER, ROLE_EMPLOYEE).

4. if (roleRepository.findByName(roleName).isEmpty())

Makes the execution **idempotent** (safe to run multiple times):

- **First startup:** The roles are missing, so it inserts all 3 roles into PostgreSQL.
 - **Subsequent restarts:** It detects that the roles already exist in PostgreSQL and skips insertion, preventing duplicate key errors.
-

User prompt: <http://localhost:4200/api/v1/auth/register>

Response: The URL <http://localhost:4200/api/v1/auth/register> has two potential configuration issues to check depending on whether you are testing from **Postman** or connecting from **Angular**.

Issue 1: Port Mismatch (Postman / cURL Testing)

- **Port 4200** is your **Angular** frontend development server.
- **Port 8080** is your **Spring Boot** backend server.

If you are sending requests directly via Postman, change the port to **8080**:

POST `http://localhost:8080/api/v1/auth/register`

Issue 2: Context Path Matching in Spring Boot

In `AuthController.java`, if you set `@RequestMapping("/auth")`, Spring Boot expects requests at `http://localhost:8080/auth/register`.

To make the URL `http://localhost:8080/api/v1/auth/register` work properly, update the `@RequestMapping` in `AuthController.java`:

```
@RestController
@RequestMapping("/api/v1/auth") // Added /api/v1 prefix
public class AuthController {
    // ...
}
```

Or add this single line to your `src/main/resources/application.yml`:

```
server:
  servlet:
    context-path: /api/v1
```

Issue 3: Connecting Angular (4200) to Spring Boot (8080)

If Angular makes an HTTP request to `http://localhost:4200/api/v1/...`, Angular needs a proxy configuration so it forwards backend traffic to port 8080.

Option A: Use Angular Proxy (Recommended for Dev)

1. Create a file named `proxy.conf.json` in your Angular root directory:

```
{
  "/api": {
    "target": "http://localhost:8080",
    "secure": false
  }
}
```

2. Update `angular.json` under `architect.serve.options`:

```
"options": {
  "proxyConfig": "proxy.conf.json"
}
```

Option B: Enable CORS in Spring Boot

To allow Angular (http://localhost:4200) to call Spring Boot directly at http://localhost:8080:

Create CorsConfig.java in src/main/java/com/smarterp/shared/config/CorsConfig.java:

```
package com.smarterp.shared.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
import org.springframework.web.filter.CorsFilter;

import java.util.List;

@Configuration
public class CorsConfig {

    @Bean
    public CorsFilter corsFilter() {
        CorsConfiguration config = new CorsConfiguration();
        config.setAllowCredentials(true);
        config.setAllowedOrigins(List.of("http://localhost:4200"));
        config.setAllowedHeaders(List.of("*"));
        config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));

        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/*", config);
        return new CorsFilter(source);
    }
}
```

User prompt: PS C:\Users\yasser\Documents\stage\Smart_ERP_Core> cd .\smart-erp-core\ PS C:\Users\yasser\Documents\stage\Smart_ERP_Core\smart-erp-core> .\mvnw.cmd spring-boot:run [INFO] Scanning for projects... [INFO] [INFO] -----< com.smarterp:smart-erp-core >----- [INFO] Building 0.0.1-SNAPSHOT [INFO] from pom.xml [INFO] -----[jar]----- [INFO] [INFO] >>> spring-boot:4.1.0:run (default-cli) > test-compile @ smart-erp-core >>> [INFO] [INFO] --- resources:3.5.0:resources (default-resources) @ smart-erp-core --- [INFO] Copying 1 resource from src\main\resources to target\classes [INFO] Copying 0 resource from src\main\resources to target\classes [INFO] [INFO] --- compiler:3.15.0:compile (default-compile) @ smart-erp-core --- [INFO] Recompiling the module because of added or removed source files. [INFO] Compiling 41 source files with javac [debug parameters release 21] to target\classes [INFO] /C:/Users/yasser/Documents/stage/Smart_ERP_Core/smart-erp-core/src/main/java/com/smarterp/shared/exception/GlobalExceptionHandler.java: C:\Users\yasser\Documents\stage\Smart_ERP_Core\smart-erp-core\src\main\java\com\smarterp\shared\exception\GlobalExceptionHandler.java uses or overrides a deprecated API. [INFO] /C:/Users/yasser/Documents/stage/Smart_ERP_Core/smart-erp-core/src/main/java/com/smarterp/shared/exception/GlobalExceptionHandler.java: Recompile with -Xlint:deprecation for details. [INFO] [INFO] ---

```

resources:3.5.0:testResources (default-testResources) @ smart-erp-core --- [INFO] skip non existing
resourceDirectory C:\Users\yasser\Documents\stage\Smart_ERP_Core\smart-erp-core\src\test\resources
[INFO] [INFO] --- compiler:3.15.0:testCompile (default-testCompile) @ smart-erp-core --- [INFO] No sources
to compile [INFO] [INFO] <<< spring-boot:4.1.0:run (default-cli) < test-compile @ smart-erp-core <<<
[INFO] [INFO] [INFO] --- spring-boot:4.1.0:run (default-cli) @ smart-erp-core --- [INFO] Attaching agents: []
. _____ ^\ / _____ ( ) _____ \\\ ( ( ) \_ | ' | ' | | ' V _ ' | \\\ \ W _ ) |
| | | | | | | ( | | ) ) ) ' | | | . _ | | | | | \ , | / / / / ===== | | ===== | | / = / _ / _ / _ / ::
Spring Boot :: (v4.1.0) 2026-08-05T14:29:08.041+02:00 INFO 21036 --- [smart-erp-core] [
main] com.smarterp.SmartErpCoreApplication : Starting SmartErpCoreApplication using Java 21.0.11
with PID 21036 (C:\Users\yasser\Documents\stage\Smart_ERP_Core\smart-erp-core\target\classes started
by yasser in C:\Users\yasser\Documents\stage\Smart_ERP_Core\smart-erp-core)
2026-08-05T14:29:08.043+02:00 INFO 21036 --- [smart-erp-core] [ main]
com.smarterp.SmartErpCoreApplication : No active profile set, falling back to 1 default profile: "default"
2026-08-05T14:29:08.565+02:00 INFO 21036 --- [smart-erp-core] [ main]
.s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2026-08-05T14:29:08.626+02:00 INFO 21036 --- [smart-erp-core] [ main]
.s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 52 ms. Found 6 JPA
repository interfaces. 2026-08-05T14:29:09.042+02:00 INFO 21036 --- [smart-erp-core] [ main]
o.s.boot.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2026-08-05T14:29:09.054+02:00 INFO 21036 --- [smart-erp-core] [ main]
o.apache.catalina.core.StandardService : Starting service [Tomcat] 2026-08-05T14:29:09.054+02:00
INFO 21036 --- [smart-erp-core] [ main] o.apache.catalina.core.StandardEngine : Starting Servlet
engine: [Apache Tomcat/11.0.22] 2026-08-05T14:29:09.230+02:00 INFO 21036 --- [smart-erp-core] [
main] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization completed in
1152 ms 2026-08-05T14:29:09.420+02:00 INFO 21036 --- [smart-erp-core] [ main]
org.hibernate.orm.jpa : HHH008540: Processing PersistenceUnitInfo [name: default]
2026-08-05T14:29:09.483+02:00 INFO 21036 --- [smart-erp-core] [ main] org.hibernate.orm.core
: HHH000001: Hibernate ORM core version 7.4.1.Final 2026-08-05T14:29:09.829+02:00 INFO
21036 --- [smart-erp-core] [ main] o.s.o.j.p.SpringPersistenceUnitInfo : No LoadTimeWeaver setup:
ignoring JPA class transformer 2026-08-05T14:29:09.872+02:00 INFO 21036 --- [smart-erp-core] [
main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting... 2026-08-05T14:29:11.084+02:00
WARN 21036 --- [smart-erp-core] [ main] org.hibernate.orm.jdbc.error : HHH000247:
ErrorCode: 0, SQLState: 28P01 2026-08-05T14:29:11.084+02:00 WARN 21036 --- [smart-erp-core] [
main] org.hibernate.orm.jdbc.error : FATAL: password authentication failed for user "erp_user"
2026-08-05T14:29:11.126+02:00 WARN 21036 --- [smart-erp-core] [ main] org.hibernate.orm.jdbc
: HHH100046: Could not obtain connection to query JDBC database metadata
org.hibernate.exception.AuthException: Unable to obtain isolated JDBC connection [FATAL: password
authentication failed for user "erp_user"] [n/a] at
org.hibernate.exception.internal.SQLStateConversionDelegate.convert(SQLStateConversionDelegate.java:8
7) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.exception.internal.StandardSQLExceptionConverter.convert(StandardSQLExceptionConverter
.java:34) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.engine.jdbc.spi.SqlExceptionHelper.convert(SqlExceptionHelper.java:115)
~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.engine.jdbc.spi.SqlExceptionHelper.convert(SqlExceptionHelper.java:101)
~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.resource.transaction.backend.jdbc.internal.JdbcIsolationDelegate.delegateWork(JdbcIsolation
Delegate.java:51) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.engine.jdbc.env.internal.JdbcEnvironmentInitiator.getJdbcEnvironmentUsingJdbcMetadata(Jd
bcEnvironmentInitiator.java:366) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.engine.jdbc.env.internal.JdbcEnvironmentInitiator.getJdbcEnvironment(JdbcEnvironmentInitiat
or.java:143) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at

```

org.hibernate.engine.jdbc.env.internal.JdbcEnvironmentInitiator.initiateService(JdbcEnvironmentInitiator.java:120) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.engine.jdbc.env.internal.JdbcEnvironmentInitiator.initiateService(JdbcEnvironmentInitiator.java:80) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.boot.registry.internal.StandardServiceRegistryImpl.initiateService(StandardServiceRegistryImpl.java:133) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.service.internal.AbstractServiceRegistryImpl.createService(AbstractServiceRegistryImpl.java:260) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.service.internal.AbstractServiceRegistryImpl.initializeService(AbstractServiceRegistryImpl.java:235) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.service.internal.AbstractServiceRegistryImpl.getService(AbstractServiceRegistryImpl.java:212) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.boot.model.relational.Database.<init>(Database.java:44) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.boot.internal.InFlightMetadataCollectorImpl.getDatabase(InFlightMetadataCollectorImpl.java:251) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.boot.internal.InFlightMetadataCollectorImpl.<init>(InFlightMetadataCollectorImpl.java:203) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.boot.model.process.spi.MetadataBuildingProcess.complete(MetadataBuildingProcess.java:180) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.jpa.boot.internal.EntityManagerFactoryBuilderImpl.metadata(EntityManagerFactoryBuilderImpl.java:1388) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.jpa.boot.internal.EntityManagerFactoryBuilderImpl.populateSessionFactoryBuilder(EntityManagerFactoryBuilderImpl.java:1468) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.jpa.boot.internal.EntityManagerFactoryBuilderImpl.build(EntityManagerFactoryBuilderImpl.java:1450) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.springframework.orm.jpa.vendor.SpringHibernateJpaPersistenceProvider.createContainerEntityManagerFactory(SpringHibernateJpaPersistenceProvider.java:93) ~[spring-orm-7.0.8.jar:7.0.8] at
org.springframework.orm.jpa.LocalContainerEntityManagerFactoryBean.createNativeEntityManagerFactory(LocalContainerEntityManagerFactoryBean.java:443) ~[spring-orm-7.0.8.jar:7.0.8] at
org.springframework.orm.jpa.AbstractEntityManagerFactoryBean.buildNativeEntityManagerFactory(AbstractEntityManagerFactoryBean.java:436) ~[spring-orm-7.0.8.jar:7.0.8] at
org.springframework.orm.jpa.AbstractEntityManagerFactoryBean.afterPropertiesSet(AbstractEntityManagerFactoryBean.java:411) ~[spring-orm-7.0.8.jar:7.0.8] at
org.springframework.orm.jpa.LocalContainerEntityManagerFactoryBean.afterPropertiesSet(LocalContainerEntityManagerFactoryBean.java:419) ~[spring-orm-7.0.8.jar:7.0.8] at
org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory.invokeInitMethods(AbstractAutowireCapableBeanFactory.java:1862) ~[spring-beans-7.0.8.jar:7.0.8] at
org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory.initializeBean(AbstractAutowireCapableBeanFactory.java:1811) ~[spring-beans-7.0.8.jar:7.0.8] at
org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory.doCreateBean(AbstractAutowireCapableBeanFactory.java:603) ~[spring-beans-7.0.8.jar:7.0.8] at
org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory.createBean(AbstractAutowireCapableBeanFactory.java:525) ~[spring-beans-7.0.8.jar:7.0.8] at
org.springframework.beans.factory.support.AbstractBeanFactory.lambda\$doGetBean\$0(AbstractBeanFactory.java:333) ~[spring-beans-7.0.8.jar:7.0.8] at
org.springframework.beans.factory.support.DefaultSingletonBeanRegistry.getSingleton(DefaultSingletonBeanRegistry.java:371) ~[spring-beans-7.0.8.jar:7.0.8] at
org.springframework.beans.factory.support.AbstractBeanFactory.doGetBean(AbstractBeanFactory.java:331) ~[spring-beans-7.0.8.jar:7.0.8] at
org.springframework.beans.factory.support.AbstractBeanFactory.getBean(AbstractBeanFactory.java:201) ~[spring-beans-7.0.8.jar:7.0.8] at

```

org.springframework.context.support.AbstractApplicationContext.finishBeanFactoryInitialization(AbstractApp
licationContext.java:977) ~[spring-context-7.0.8.jar:7.0.8]      at
org.springframework.context.support.AbstractApplicationContext.refresh(AbstractApplicationContext.java:62
1) ~[spring-context-7.0.8.jar:7.0.8]      at
org.springframework.boot.web.server.servlet.context.ServletWebServerApplicationContext.refresh(ServletW
ebServerApplicationContext.java:143) ~[spring-boot-web-server-4.1.0.jar:4.1.0]      at
org.springframework.boot.SpringApplication.refresh(SpringApplication.java:756)
~[spring-boot-4.1.0.jar:4.1.0]      at
org.springframework.boot.SpringApplication.refreshContext(SpringApplication.java:445)
~[spring-boot-4.1.0.jar:4.1.0]      at
org.springframework.boot.SpringApplication.run(SpringApplication.java:321) ~[spring-boot-4.1.0.jar:4.1.0]
    at org.springframework.boot.SpringApplication.run(SpringApplication.java:1365)
~[spring-boot-4.1.0.jar:4.1.0]      at
org.springframework.boot.SpringApplication.run(SpringApplication.java:1354) ~[spring-boot-4.1.0.jar:4.1.0]
    at com.smarterp.SmartErpCoreApplication.main(SmartErpCoreApplication.java:10) ~[classes/:na]
Caused by: org.postgresql.util.PSQLException: FATAL: password authentication failed for user "erp_user"
    at org.postgresql.core.v3.ConnectionFactoryImpl.doAuthentication(ConnectionFactoryImpl.java:835)
~[postgresql-42.7.11.jar:42.7.11]      at
org.postgresql.core.v3.ConnectionFactoryImpl.tryConnect(ConnectionFactoryImpl.java:307)
~[postgresql-42.7.11.jar:42.7.11]      at
org.postgresql.core.v3.ConnectionFactoryImpl.openConnectionImpl(ConnectionFactoryImpl.java:365)
~[postgresql-42.7.11.jar:42.7.11]      at
org.postgresql.core.ConnectionFactory.openConnection(ConnectionFactory.java:52)
~[postgresql-42.7.11.jar:42.7.11]      at org.postgresql.jdbc.PgConnection.<init>(PgConnection.java:290)
~[postgresql-42.7.11.jar:42.7.11]      at org.postgresql.Driver.makeConnection(Driver.java:448)
~[postgresql-42.7.11.jar:42.7.11]      at org.postgresql.Driver.connect(Driver.java:298)
~[postgresql-42.7.11.jar:42.7.11]      at
com.zaxxer.hikari.util.DriverDataSource.getConnection(DriverDataSource.java:144) ~[HikariCP-7.0.2.jar:na]
    at com.zaxxer.hikari.pool.PoolBase.newConnection(PoolBase.java:373) ~[HikariCP-7.0.2.jar:na]
    at com.zaxxer.hikari.pool.PoolBase.newPoolEntry(PoolBase.java:210) ~[HikariCP-7.0.2.jar:na]      at
com.zaxxer.hikari.pool.HikariPool.createPoolEntry(HikariPool.java:488) ~[HikariCP-7.0.2.jar:na]      at
com.zaxxer.hikari.pool.HikariPool.checkFailFast(HikariPool.java:576) ~[HikariCP-7.0.2.jar:na]      at
com.zaxxer.hikari.pool.HikariPool.<init>(HikariPool.java:97) ~[HikariCP-7.0.2.jar:na]      at
com.zaxxer.hikari.HikariDataSource.getConnection(HikariDataSource.java:111) ~[HikariCP-7.0.2.jar:na]
at
org.hibernate.engine.jdbc.connections.internal.DataSourceConnectionProvider.getConnection(DataSourceC
onnectionProvider.java:149) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final]      at
org.hibernate.engine.jdbc.env.internal.JdbcEnvironmentInitiator$ConnectionProviderJdbcConnectionAccess
.obtainConnection(JdbcEnvironmentInitiator.java:508) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final]      at
org.hibernate.resource.transaction.backend.jdbc.internal.JdbcIsolationDelegate.delegateWork(JdbcIsolation
Delegate.java:48) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final]      ... 37 common frames omitted
2026-08-05T14:29:11.188+02:00 WARN 21036 --- [smart-erp-core] [      main]
org.hibernate.orm.deprecation      : HHH90000025: PostgreSQLDialect does not need to be specified
explicitly using 'hibernate.dialect' (remove the property setting and it will be selected by default)
2026-08-05T14:29:11.225+02:00 INFO 21036 --- [smart-erp-core] [      main]
org.hibernate.orm.connections.pooling : HHH10001005: Database info:      Database JDBC URL
[undefined/unknown]      Database driver: undefined/unknown      Database dialect: PostgreSQLDialect
      Database version: 14.0      Default catalog/schema: unknown/unknown      Autocommit mode:
undefined/unknown      Isolation level: NONE [default NONE]      JDBC fetch size: undefined/unknown
      Pool: DataSourceConnectionProvider      Minimum pool size: undefined/unknown      Maximum pool
size: undefined/unknown 2026-08-05T14:29:13.693+02:00 INFO 21036 --- [smart-erp-core] [      main]
org.hibernate.orm.core      : HHH000489: No JTA platform available (set

```

'hibernate.transaction.jta.platform' to enable JTA platform integration) 2026-08-05T14:29:13.729+02:00
INFO 21036 --- [smart-erp-core] [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 -
Starting... 2026-08-05T14:29:14.786+02:00 WARN 21036 --- [smart-erp-core] [main]
org.hibernate.orm.jdbc.error : HHH000247: ErrorCode: 0, SQLState: 28P01
2026-08-05T14:29:14.789+02:00 WARN 21036 --- [smart-erp-core] [main]
org.hibernate.orm.jdbc.error : FATAL: password authentication failed for user "erp_user"
2026-08-05T14:29:14.796+02:00 ERROR 21036 --- [smart-erp-core] [main]
j.LocalContainerEntityManagerFactoryBean : Failed to initialize JPA EntityManagerFactory: Unable to build
Hibernate SessionFactory [persistence unit: default] ; nested exception is
org.hibernate.exception.AuthException: Unable to open JDBC Connection for DDL execution [FATAL:
password authentication failed for user "erp_user"] [n/a] 2026-08-05T14:29:14.798+02:00 WARN 21036 ---
[smart-erp-core] [main] ConfigServletWebServerApplicationContext : Exception encountered during
context initialization - cancelling refresh attempt:
org.springframework.beans.factory.BeanCreationException: Error creating bean with name
'entityManagerFactory' defined in class path resource
[org/springframework/boot/hibernate/autoconfigure/HibernateJpaConfiguration.class]: Unable to build
Hibernate SessionFactory [persistence unit: default] ; nested exception is
org.hibernate.exception.AuthException: Unable to open JDBC Connection for DDL execution [FATAL:
password authentication failed for user "erp_user"] [n/a] 2026-08-05T14:29:14.798+02:00 INFO 21036 ---
[smart-erp-core] [main] o.apache.catalina.core.StandardService : Stopping service [Tomcat]
2026-08-05T14:29:14.866+02:00 INFO 21036 --- [smart-erp-core] [main]
.s.b.a.l.ConditionEvaluationReportLogger : Error starting ApplicationContext. To display the condition
evaluation report re-run your application with 'debug' enabled. 2026-08-05T14:29:14.903+02:00 ERROR
21036 --- [smart-erp-core] [main] o.s.boot.SpringApplication : Application run failed
org.springframework.beans.factory.BeanCreationException: Error creating bean with name
'entityManagerFactory' defined in class path resource
[org/springframework/boot/hibernate/autoconfigure/HibernateJpaConfiguration.class]: Unable to build
Hibernate SessionFactory [persistence unit: default] ; nested exception is
org.hibernate.exception.AuthException: Unable to open JDBC Connection for DDL execution [FATAL:
password authentication failed for user "erp_user"] [n/a] at
org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory.initializeBean(AbstractA
utowireCapableBeanFactory.java:1815) ~[spring-beans-7.0.8.jar:7.0.8] at
org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory.doCreateBean(AbstractA
utowireCapableBeanFactory.java:603) ~[spring-beans-7.0.8.jar:7.0.8] at
org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory.createBean(AbstractAuto
wireCapableBeanFactory.java:525) ~[spring-beans-7.0.8.jar:7.0.8] at
org.springframework.beans.factory.support.AbstractBeanFactory.lambda\$doGetBean\$0(AbstractBeanFactor
y.java:333) ~[spring-beans-7.0.8.jar:7.0.8] at
org.springframework.beans.factory.support.DefaultSingletonBeanRegistry.getSingleton(DefaultSingletonBea
nRegistry.java:371) ~[spring-beans-7.0.8.jar:7.0.8] at
org.springframework.beans.factory.support.AbstractBeanFactory.doGetBean(AbstractBeanFactory.java:331)
~[spring-beans-7.0.8.jar:7.0.8] at
org.springframework.beans.factory.support.AbstractBeanFactory.getBean(AbstractBeanFactory.java:201)
~[spring-beans-7.0.8.jar:7.0.8] at
org.springframework.context.support.AbstractApplicationContext.finishBeanFactoryInitialization(AbstractApp
licationContext.java:977) ~[spring-context-7.0.8.jar:7.0.8] at
org.springframework.context.support.AbstractApplicationContext.refresh(AbstractApplicationContext.java:62
1) ~[spring-context-7.0.8.jar:7.0.8] at
org.springframework.boot.web.server.servlet.context.ServletWebServerApplicationContext.refresh(ServletW
ebServerApplicationContext.java:143) ~[spring-boot-web-server-4.1.0.jar:4.1.0] at
org.springframework.boot.SpringApplication.refresh(SpringApplication.java:756)
~[spring-boot-4.1.0.jar:4.1.0] at

org.springframework.boot.SpringApplication.refreshContext(SpringApplication.java:445)
~[spring-boot-4.1.0.jar:4.1.0] at
org.springframework.boot.SpringApplication.run(SpringApplication.java:321) ~[spring-boot-4.1.0.jar:4.1.0]
at org.springframework.boot.SpringApplication.run(SpringApplication.java:1365)
~[spring-boot-4.1.0.jar:4.1.0] at
org.springframework.boot.SpringApplication.run(SpringApplication.java:1354) ~[spring-boot-4.1.0.jar:4.1.0]
at com.smarterp.SmartErpCoreApplication.main(SmartErpCoreApplication.java:10) ~[classes/:na]
Caused by: jakarta.persistence.PersistenceException: Unable to build Hibernate SessionFactory
[persistence unit: default] ; nested exception is org.hibernate.exception.AuthException: Unable to open
JDBC Connection for DDL execution [FATAL: password authentication failed for user "erp_user"] [n/a]
at
org.springframework.orm.jpa.AbstractEntityManagerFactoryBean.buildNativeEntityManagerFactory(Abstract
EntityManagerFactoryBean.java:448) ~[spring-orm-7.0.8.jar:7.0.8] at
org.springframework.orm.jpa.AbstractEntityManagerFactoryBean.afterPropertiesSet(AbstractEntityManager
FactoryBean.java:411) ~[spring-orm-7.0.8.jar:7.0.8] at
org.springframework.orm.jpa.LocalContainerEntityManagerFactoryBean.afterPropertiesSet(LocalContainerE
ntityManagerFactoryBean.java:419) ~[spring-orm-7.0.8.jar:7.0.8] at
org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory.invokeInitMethods(Abstra
ctAutowireCapableBeanFactory.java:1862) ~[spring-beans-7.0.8.jar:7.0.8] at
org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory.initializeBean(AbstractAut
owireCapableBeanFactory.java:1811) ~[spring-beans-7.0.8.jar:7.0.8] ... 15 common frames omitted
Caused by: org.hibernate.exception.AuthException: Unable to open JDBC Connection for DDL execution
[FATAL: password authentication failed for user "erp_user"] [n/a] at
org.hibernate.exception.internal.SQLStateConversionDelegate.convert(SQLStateConversionDelegate.java:8
7) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.exception.internal.StandardSQLExceptionConverter.convert(StandardSQLExceptionConverter
.java:34) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.engine.jdbc.spi.SqlExceptionHelper.convert(SqlExceptionHelper.java:115)
~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.engine.jdbc.spi.SqlExceptionHelper.convert(SqlExceptionHelper.java:101)
~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.resource.transaction.backend.jdbc.internal.DdlTransactionIsolatorNonJtaImpl.getIsolatedConn
ection(DdlTransactionIsolatorNonJtaImpl.java:71) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.resource.transaction.backend.jdbc.internal.DdlTransactionIsolatorNonJtaImpl.getIsolatedConn
ection(DdlTransactionIsolatorNonJtaImpl.java:37) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.tool.schema.internal.exec.ImprovedExtractionContextImpl.getJdbcConnection(ImprovedExtrac
tionContextImpl.java:61) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.tool.schema.extract.spi.ExtractionContext.getQueryResults(ExtractionContext.java:41)
~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.tool.schema.extract.internal.SequenceInformationExtractorLegacyImpl.extractMetadata(Sequ
enceInformationExtractorLegacyImpl.java:36) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.tool.schema.extract.internal.DatabaseInformationImpl.initializeSequences(DatabaseInformatio
nImpl.java:73) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.tool.schema.extract.internal.DatabaseInformationImpl.<init>(DatabaseInformationImpl.java:63)
~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.tool.schema.extract.internal.CachingDatabaseInformationImpl.<init>(CachingDatabaseInforma
tionImpl.java:43) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.tool.schema.internal.GroupedSchemaMigratorImpl.buildDatabaseInformation(GroupedSchem
aMigratorImpl.java:112) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.tool.schema.internal.AbstractSchemaMigrator.doMigration(AbstractSchemaMigrator.java:83)
~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.tool.schema.spi.SchemaManagementToolCoordinator.performDatabaseAction(SchemaManag

ementToolCoordinator.java:269) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.tool.schema.spi.SchemaManagementToolCoordinator.lambda\$process\$1(SchemaManagementToolCoordinator.java:101) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
java.base/java.util.HashMap.forEach(HashMap.java:1429) ~[na:na] at
org.hibernate.tool.schema.spi.SchemaManagementToolCoordinator.process(SchemaManagementToolCoordinator.java:100) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.boot.internal.SessionFactoryObserverForSchemaExport.sessionFactoryCreated(SessionFactoryObserverForSchemaExport.java:35) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.internal.SessionFactoryObserverChain.sessionFactoryCreated(SessionFactoryObserverChain.java:33) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.internal.SessionFactoryImpl.<init>(SessionFactoryImpl.java:327)
~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.internal.SessionFactoryRegistry.instantiateSessionFactory(SessionFactoryRegistry.java:64)
~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.boot.internal.SessionFactoryBuilderImpl.build(SessionFactoryBuilderImpl.java:458)
~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.jpa.boot.internal.EntityManagerFactoryBuilderImpl.build(EntityManagerFactoryBuilderImpl.java:1452) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.springframework.orm.jpa.vendor.SpringHibernateJpaPersistenceProvider.createContainerEntityManagerFactory(SpringHibernateJpaPersistenceProvider.java:93) ~[spring-orm-7.0.8.jar:7.0.8] at
org.springframework.orm.jpa.LocalContainerEntityManagerFactoryBean.createNativeEntityManagerFactory(LocalContainerEntityManagerFactoryBean.java:443) ~[spring-orm-7.0.8.jar:7.0.8] at
org.springframework.orm.jpa.AbstractEntityManagerFactoryBean.buildNativeEntityManagerFactory(AbstractEntityManagerFactoryBean.java:436) ~[spring-orm-7.0.8.jar:7.0.8] ... 19 common frames omitted
Caused by: org.postgresql.util.PSQLException: FATAL: password authentication failed for user "erp_user"
at org.postgresql.core.v3.ConnectionFactoryImpl.doAuthentication(ConnectionFactoryImpl.java:835)
~[postgresql-42.7.11.jar:42.7.11] at
org.postgresql.core.v3.ConnectionFactoryImpl.tryConnect(ConnectionFactoryImpl.java:307)
~[postgresql-42.7.11.jar:42.7.11] at
org.postgresql.core.v3.ConnectionFactoryImpl.openConnectionImpl(ConnectionFactoryImpl.java:365)
~[postgresql-42.7.11.jar:42.7.11] at
org.postgresql.core.ConnectionFactory.openConnection(ConnectionFactory.java:52)
~[postgresql-42.7.11.jar:42.7.11] at org.postgresql.jdbc.PgConnection.<init>(PgConnection.java:290)
~[postgresql-42.7.11.jar:42.7.11] at org.postgresql.Driver.makeConnection(Driver.java:448)
~[postgresql-42.7.11.jar:42.7.11] at org.postgresql.Driver.connect(Driver.java:298)
~[postgresql-42.7.11.jar:42.7.11] at
com.zaxxer.hikari.util.DriverDataSource.getConnection(DriverDataSource.java:144) ~[HikariCP-7.0.2.jar:na]
at com.zaxxer.hikari.pool.PoolBase.newConnection(PoolBase.java:373) ~[HikariCP-7.0.2.jar:na]
at com.zaxxer.hikari.pool.PoolBase.newPoolEntry(PoolBase.java:210) ~[HikariCP-7.0.2.jar:na] at
com.zaxxer.hikari.pool.HikariPool.createPoolEntry(HikariPool.java:488) ~[HikariCP-7.0.2.jar:na] at
com.zaxxer.hikari.pool.HikariPool.checkFailFast(HikariPool.java:576) ~[HikariCP-7.0.2.jar:na] at
com.zaxxer.hikari.pool.HikariPool.<init>(HikariPool.java:97) ~[HikariCP-7.0.2.jar:na] at
com.zaxxer.hikari.HikariDataSource.getConnection(HikariDataSource.java:111) ~[HikariCP-7.0.2.jar:na]
at
org.hibernate.engine.jdbc.connections.internal.DataSourceConnectionProvider.getConnection(DataSourceConnectionProvider.java:149) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.engine.jdbc.env.internal.JdbcEnvironmentInitiator\$ConnectionProviderJdbcConnectionAccess.obtainConnection(JdbcEnvironmentInitiator.java:508) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] at
org.hibernate.resource.transaction.backend.jdbc.internal.DdlTransactionIsolatorNonJtaImpl.getIsolatedConnection(DdlTransactionIsolatorNonJtaImpl.java:44) ~[hibernate-core-7.4.1.Final.jar:7.4.1.Final] ... 41
common frames omitted [INFO] ----- [INFO] BUILD
FAILURE [INFO] ----- [INFO] Total time: 13.274 s [INFO]

Finished at: 2026-08-05T14:29:14+02:00 [INFO] -----
[ERROR] Failed to execute goal org.springframework.boot:spring-boot-maven-plugin:4.1.0:run (default-cli)
on project smart-erp-core: Process terminated with exit code: 1 -> [Help 1] [ERROR] [ERROR] To see the
full stack trace of the errors, re-run Maven with the -e switch. [ERROR] Re-run Maven using the -X switch to
enable full debug logging. [ERROR] [ERROR] For more information about the errors and possible solutions,
please read the following articles: [ERROR] [Help 1]
<http://cwiki.apache.org/confluence/display/MAVEN/MojoExecutionException> PS
C:\Users\yasser\Documents\stage\Smart_ERP_Core\smart-erp-core>

Response: The application failed to start because PostgreSQL rejected the login credentials for user **erp_user** (FATAL: password authentication failed for user "erp_user").

Here is how to resolve it depending on your setup:

Option 1: Match application.yml / application.properties to Your PostgreSQL Credentials

Open src/main/resources/application.yml (or application.properties) and update the database username and password to match what is configured in your PostgreSQL instance.

If using application.yml:

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/smart_erp
    username: postgres # Or whatever username exists in your PostgreSQL
    password: postgres # Replace with your actual PostgreSQL password
    driver-class-name: org.postgresql.Driver
```

If using application.properties:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/smart_erp
spring.datasource.username=postgres
spring.datasource.password=postgres
spring.datasource.driver-class-name=org.postgresql.Driver
```

Option 2: Create or Update the erp_user Password in PostgreSQL

If you want to keep erp_user as the username, update PostgreSQL to match the password defined in your application configuration.

Open **pgAdmin** or run **psql** in your terminal:

```
-- Connect as postgres admin user, then run:
CREATE USER erp_user WITH PASSWORD 'your_password_here';
ALTER USER erp_user WITH PASSWORD 'your_password_here';
GRANT ALL PRIVILEGES ON DATABASE smart_erp TO erp_user;
```

Option 3: If Using Docker Compose

If you are running PostgreSQL via Docker, verify that the environment variables in your docker-compose.yml match your Spring Boot settings:

```
version: '3.8'
services:
  postgres:
    image: postgres:16
    container_name: smart_erp_postgres
    environment:
      POSTGRES_DB: smart_erp
      POSTGRES_USER: erp_user
      POSTGRES_PASSWORD: your_password_here
    ports:
      - "5432:5432"
```

If you recently changed docker-compose.yml, recreate the container volume so the new credentials take effect:

```
docker compose down -v
docker compose up -d
```

Re-run the Server

Once credentials match, launch the application again:

```
.\mvnw.cmd spring-boot:run
```

User prompt: datasource: url: jdbc:postgresql://\${DB_HOST:localhost}:\${DB_PORT:5432}/\${DB_NAME:smart_erp} username: \${DB_USER:postgres} password: \${DB_PASSWORD:postgres} driver-class-name: org.postgresql.Driver

Response: Your configuration uses environment variable placeholders with fallbacks (\${DB_USER:postgres}).

The reason your log showed FATAL: password authentication failed for user "erp_user" is because **an environment variable named DB_USER is set to erp_user on your system/IDE**, overriding the :postgres fallback.

Here are 3 ways to fix it and get your backend running:

Method 1: Hardcode standard credentials in application.yml (Quickest for Dev)

Temporarily remove the \${...} environment variable expressions and put your actual local PostgreSQL credentials directly in application.yml:

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/smart_erp
    username: postgres
    password: postgres # Change to your actual PostgreSQL password if not "postgres"
    driver-class-name: org.postgresql.Driver
```

Method 2: Pass Environment Variables directly in PowerShell

If you want to keep the \${...} syntax in application.yml, pass the variables explicitly in your terminal session before launching Maven:

```
$env:DB_USER="postgres"
$env:DB_PASSWORD="YOUR_POSTGRES_PASSWORD"
.\mvnw.cmd spring-boot:run
```

(Replace YOUR_POSTGRES_PASSWORD with the password you set when installing PostgreSQL).

Method 3: Reset or Create erp_user in PostgreSQL

If DB_USER=erp_user is set in your environment intentionally, update the erp_user password inside PostgreSQL so it matches:

Open psql or **pgAdmin** query tool and execute:

```
-- Create the user if it doesn't exist
CREATE USER erp_user WITH PASSWORD 'postgres';

-- Or update password if user exists
ALTER USER erp_user WITH PASSWORD 'postgres';

-- Grant access to the database
GRANT ALL PRIVILEGES ON DATABASE smart_erp TO erp_user;
```

Verification Checklist Before Running:

1. Ensure the database smart_erp exists in PostgreSQL (Run CREATE DATABASE smart_erp; in pgAdmin if you haven't created it yet).
 2. Run .\mvnw.cmd spring-boot:run again.
-

[illegible]

```

[C:\Users\yasser\Documents\stage\Smart_ERP_Core\smart-erp-core\target\classes\com\smarterp\security\service\AuthService.class]: Unsatisfied dependency expressed through constructor parameter 4: No
qualifying bean of type 'org.springframework.security.authentication.AuthenticationManager' available:
expected at least 1 bean which qualifies as autowire candidate. Dependency annotations: {}
2026-08-05T14:36:55.661+02:00 INFO 21036 --- [smart-erp-core] [      main]
j.LocalContainerEntityManagerFactoryBean : Closing JPA EntityManagerFactory for persistence unit
'default' 2026-08-05T14:36:55.664+02:00 INFO 21036 --- [smart-erp-core] [      main]
com.zaxxer.hikari.HikariDataSource      : HikariPool-1 - Shutdown initiated...
2026-08-05T14:36:55.664+02:00 INFO 21036 --- [smart-erp-core] [      main]
com.zaxxer.hikari.HikariDataSource      : HikariPool-1 - Shutdown completed.
2026-08-05T14:36:55.664+02:00 INFO 21036 --- [smart-erp-core] [      main]
o.apache.catalina.core.StandardService : Stopping service [Tomcat] 2026-08-05T14:36:55.678+02:00
INFO 21036 --- [smart-erp-core] [      main] .s.b.a.l.ConditionEvaluationReportLogger : Error starting
ApplicationContext. To display the condition evaluation report re-run your application with 'debug' enabled.
2026-08-05T14:36:55.693+02:00 ERROR 21036 --- [smart-erp-core] [      main]
o.s.b.d.LoggingFailureAnalysisReporter : ***** APPLICATION FAILED TO START
*****
***** Description: Parameter 4 of constructor in
com.smarterp.security.service.AuthService required a bean of type
'org.springframework.security.authentication.AuthenticationManager' that could not be found. Action:
Consider defining a bean of type 'org.springframework.security.authentication.AuthenticationManager' in
your configuration. [INFO] ----- [INFO] BUILD FAILURE
[INFO] ----- [INFO] Total time: 4.255 s [INFO] Finished at:
2026-08-05T14:36:55+02:00 [INFO] ----- [ERROR] Failed
to execute goal org.springframework.boot:spring-boot-maven-plugin:4.1.0:run (default-cli) on project
smart-erp-core: Process terminated with exit code: 1 -> [Help 1] [ERROR] [ERROR] To see the full stack
trace of the errors, re-run Maven with the -e switch. [ERROR] Re-run Maven using the -X switch to enable
full debug logging. [ERROR] [ERROR] For more information about the errors and possible solutions, please
read the following articles: [ERROR] [Help 1]
http://cwiki.apache.org/confluence/display/MAVEN/MojoExecutionException

```

Response: The database connection is now working, but Spring Security is missing an explicit @Bean definition for AuthenticationManager required by AuthService.

In Spring Security (Spring Boot 3/4), AuthenticationManager is not registered automatically. You need to expose it manually in your security configuration class.

Fix: Expose AuthenticationManager Bean

Open your security configuration class (typically SecurityConfig.java or WebSecurityConfig.java under your security package) and add the following @Bean method:

```

package com.smarterp.security.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import
org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;

@Configuration
public class SecurityConfig {

```


.s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 41 ms. Found 6 JPA repository interfaces. 2026-08-05T14:40:58.017+02:00 INFO 17300 --- [smart-erp-core] [main]
o.s.boot.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2026-08-05T14:40:58.026+02:00 INFO 17300 --- [smart-erp-core] [main]
o.apache.catalina.core.StandardService : Starting service [Tomcat] 2026-08-05T14:40:58.026+02:00
INFO 17300 --- [smart-erp-core] [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/11.0.22] 2026-08-05T14:40:58.063+02:00 INFO 17300 --- [smart-erp-core] [main]
b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization completed in 771 ms 2026-08-05T14:40:58.185+02:00 INFO 17300 --- [smart-erp-core] [main] org.hibernate.orm.jpa : HHH008540: Processing PersistenceUnitInfo [name: default] 2026-08-05T14:40:58.220+02:00
INFO 17300 --- [smart-erp-core] [main] org.hibernate.orm.core : HHH000001: Hibernate ORM core version 7.4.1.Final 2026-08-05T14:40:58.488+02:00 INFO 17300 --- [smart-erp-core] [main]
o.s.o.j.p.SpringPersistenceUnitInfo : No LoadTimeWeaver setup: ignoring JPA class transformer 2026-08-05T14:40:58.508+02:00 INFO 17300 --- [smart-erp-core] [main]
com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting... 2026-08-05T14:40:58.620+02:00 INFO 17300 --- [smart-erp-core] [main] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Added connection org.postgresql.jdbc.PgConnection@4e61a863 2026-08-05T14:40:58.621+02:00 INFO 17300 --- [smart-erp-core] [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed. 2026-08-05T14:40:58.639+02:00 WARN 17300 --- [smart-erp-core] [main]
org.hibernate.orm.deprecation : HHH90000025: PostgreSQLDialect does not need to be specified explicitly using 'hibernate.dialect' (remove the property setting and it will be selected by default) 2026-08-05T14:40:58.657+02:00 INFO 17300 --- [smart-erp-core] [main]
org.hibernate.orm.connections.pooling : HHH10001005: Database info: Database JDBC URL [jdbc:postgresql://localhost:5432/smart_erp] Database driver: PostgreSQL JDBC Driver Database dialect: PostgreSQLDialect Database version: 17.10 Default catalog/schema: smart_erp/public Autocommit mode: undefined/unknown Isolation level: READ_COMMITTED [default READ_COMMITTED] JDBC fetch size: none Pool: DataSourceConnectionProvider Minimum pool size: undefined/unknown Maximum pool size: undefined/unknown 2026-08-05T14:40:59.431+02:00 INFO 17300 --- [smart-erp-core] [main] org.hibernate.orm.core : HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platform integration) 2026-08-05T14:40:59.485+02:00 INFO 17300 --- [smart-erp-core] [main]
j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default' 2026-08-05T14:40:59.532+02:00 INFO 17300 --- [smart-erp-core] [main]
o.s.d.j.r.query.QueryEnhancerFactories : Hibernate is in classpath; If applicable, HQL parser will be used. 2026-08-05T14:40:59.755+02:00 WARN 17300 --- [smart-erp-core] [main]
JpaBaseConfiguration\$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view rendering. Explicitly configure spring.jpa.open-in-view to disable this warning 2026-08-05T14:41:00.199+02:00 INFO 17300 --- [smart-erp-core] [main]
o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/api/v1' 2026-08-05T14:41:00.199+02:00 INFO 17300 --- [smart-erp-core] [main]
com.smarterp.SmartErpCoreApplication : Started SmartErpCoreApplication in 3.229 seconds (process running for 3.453) Hibernate: select r1_0.id, r1_0.description, r1_0.name from roles r1_0 where r1_0.name=? Hibernate: insert into roles (description, name, id) values (?, ?, ?) Hibernate: select r1_0.id, r1_0.description, r1_0.name from roles r1_0 where r1_0.name=? Hibernate: insert into roles (description, name, id) values (?, ?, ?) Hibernate: select r1_0.id, r1_0.description, r1_0.name from roles r1_0 where r1_0.name=? Hibernate: insert into roles (description, name, id) values (?, ?, ?) 2026-08-05T14:42:58.968+02:00 INFO 17300 --- [smart-erp-core] [nio-8080-exec-1]
o.a.c.c.C.[.[localhost].[/api/v1] : Initializing Spring DispatcherServlet 'dispatcherServlet' 2026-08-05T14:42:58.968+02:00 INFO 17300 --- [smart-erp-core] [nio-8080-exec-1]
o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet' 2026-08-05T14:42:58.968+02:00 INFO 17300 --- [smart-erp-core] [nio-8080-exec-1]

o.s.web.servlet.DispatcherServlet : Completed initialization in 0 ms

Response: The **401 Unauthorized** error occurs because Spring Security locks down all endpoints by default unless explicitly permitted in your SecurityFilterChain.

Since your application uses a context path (/api/v1), Spring Security matches request paths **relative to that context path**. Calling `http://localhost:8080/api/v1/auth/register` means Spring Security sees the request path as `/auth/register`.

Solution: Update SecurityFilterChain to Permit Auth Endpoints

Open your SecurityConfig.java class and configure requestMatchers to allow public access to `/auth/**`:

```
package com.smarterp.security.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import
org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    // Inject your JwtAuthenticationFilter if you have one
    // private final JwtAuthenticationFilter jwtAuthFilter;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            // Disable CSRF for stateless REST APIs using JWT
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                // Explicitly allow registration, login, and swagger/documentation endpoints without authentication
                .requestMatchers("/auth/**", "/v3/api-docs/**", "/swagger-ui/**").permitAll()
                // All other API requests require authentication
                .anyRequest().authenticated()
            )
            .sessionManagement(session ->
                session.sessionCreationPolicy(SessionCreationPolicy.STATELESS));

        // If you have a custom JWT Filter, add it before UsernamePasswordAuthenticationFilter:
        // http.addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
}
```

```

    }

    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws
    Exception {
        return config.getAuthenticationManager();
    }
}

```

Important Checkpoints

1. **Path Matching:** Ensure your `@RequestMapping` on `AuthController` (or `RegisterController`) matches the permit rule. For example:

```

@RestController
@RequestMapping("/auth") // Full URL: http://localhost:8080/api/v1/auth/register
public class AuthController {
    @PostMapping("/register")
    public ResponseEntity<?> register(...) { ... }
}

```
2. **Re-run the Application:** Save the changes and run `.\mvnw.cmd spring-boot:run`. Postman / cURL requests to `http://localhost:8080/api/v1/auth/register` will now bypass authentication.

User prompt: { "error": "Bad Request", "message": "could not execute statement [ERROR: null value in column \"created_at\" of relation \"users\" violates not-null constraint\n Detail: Failing row contains (3e9b0837-54ea-4d7e-bc6a-ba48c1a2184c, null, null, admin@smarterp.com, t, \$2a\$10\$LsLZ70aDtZhsxxiG3riQAuuwb1cq.x6altvZnJtRevZ2zkGB1zW9u, c5d3f81b-7729-432f-8b50-659ed26e9b2e).] [insert into users (created_at,email,is_active,password_hash,role_id,updated_at,id) values (?, ?, ?, ?, ?, ?, ?)]; SQL [insert into users (created_at,email,is_active,password_hash,role_id,updated_at,id) values (?, ?, ?, ?, ?, ?, ?)]; constraint [created_at]", "timestamp": "2026-08-05T14:50:02.3170828", "status": 400 }

Response: The error occurs because PostgreSQL requires a non-null value for the `created_at` column, but Spring Boot is attempting to insert null when saving the new User.

Here are two standard ways to automatically populate `created_at` and `updated_at` when creating an entity.

Solution 1: Use `@PrePersist` Lifecycle Callbacks (Quickest Fix)

In your User entity class (or `BaseEntity` if User inherits from it), add `@PrePersist` and `@PreUpdate` methods:

```

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "users")
public class User {

    // ... your existing fields

```

```

@Column(name = "created_at", nullable = false, updatable = false)
private LocalDateTime createdAt;

@Column(name = "updated_at")
private LocalDateTime updatedAt;

@PrePersist
protected void onCreate() {
    LocalDateTime now = LocalDateTime.now();
    this.createdAt = now;
    this.updatedAt = now;
}

@PreUpdate
protected void onUpdate() {
    this.updatedAt = LocalDateTime.now();
}

// Getters and Setters...
}

```

Solution 2: Enable Spring Data JPA Auditing (Best Practice)

If you prefer Spring Data's built-in auditing framework:

1. Enable Auditing in your main class or security config:

```

@SpringBootApplication
@EnableJpaAuditing // <-- Add this annotation
public class SmartErpCoreApplication {
    public static void main(String[] args) {
        SpringApplication.run(SmartErpCoreApplication.class, args);
    }
}

```

2. Annotate your User (or BaseEntity) fields:

```

import jakarta.persistence.*;
import org.springframework.data.annotation.CreatedDate;
import org.springframework.data.annotation.LastModifiedDate;
import org.springframework.data.jpa.domain.support.AuditingEntityListener;
import java.time.LocalDateTime;

```

```

@Entity
@Table(name = "users")
@EntityListeners(AuditingEntityListener.class) // <-- Add listener
public class User {

```

```

    // ... standard fields

```

```

    @CreatedDate

```

```

@Column(name = "created_at", nullable = false, updatable = false)
private LocalDateTime createdAt;

@LastModifiedDate
@Column(name = "updated_at")
private LocalDateTime updatedAt;

// Getters and Setters...
}

```

Re-test

After applying either solution, re-run `.\mvnw.cmd spring-boot:run` and resend the registration request.

User prompt: created at and updated at are in BaseEntity

Response: To make `createdAt` and `updatedAt` populate automatically from `BaseEntity`, ensure `BaseEntity` is annotated with `@MappedSuperclass` and has lifecycle hooks or auditing annotations configured.

Here are the two ways to fix `BaseEntity`:

Option 1: Use `@PrePersist` and `@PreUpdate` directly in `BaseEntity` (Works automatically)

Add `@MappedSuperclass` and the lifecycle hooks directly into your `BaseEntity`:

```
package com.smarterp.shared.domain; // Adjust package to match yours
```

```
import jakarta.persistence.Column;
import jakarta.persistence.MappedSuperclass;
import jakarta.persistence.PrePersist;
import jakarta.persistence.PreUpdate;
import java.time.LocalDateTime;
```

```
@MappedSuperclass
public abstract class BaseEntity {
```

```
    @Column(name = "created_at", nullable = false, updatable = false)
    private LocalDateTime createdAt;
```

```
    @Column(name = "updated_at")
    private LocalDateTime updatedAt;
```

```
    @PrePersist
    protected void onCreate() {
        LocalDateTime now = LocalDateTime.now();
        this.createdAt = now;
    }
}
```

```

        this.updatedAt = now;
    }

    @PreUpdate
    protected void onUpdate() {
        this.updatedAt = LocalDateTime.now();
    }

    // Getters and Setters...
}

```

Option 2: Use Spring Data JPA Auditing

If you prefer Spring's `@CreatedDate` and `@LastModifiedDate` annotations:

1. Update BaseEntity.java:

```

package com.smarterp.shared.domain;

import jakarta.persistence.Column;
import jakarta.persistence.EntityListeners;
import jakarta.persistence.MappedSuperclass;
import org.springframework.data.annotation.CreatedDate;
import org.springframework.data.annotation.LastModifiedDate;
import org.springframework.data.jpa.domain.support.AuditingEntityListener;
import java.time.LocalDateTime;

@MappedSuperclass
@EntityListeners(AuditingEntityListener.class) // <-- Essential for JPA Auditing
public abstract class BaseEntity {

    @CreatedDate
    @Column(name = "created_at", nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @LastModifiedDate
    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    // Getters and Setters...
}

```

2. Enable auditing in your main application class:

```

@SpringBootApplication
@EnableJpaAuditing // <-- Required for @EntityListeners(AuditingEntityListener.class)
public class SmartErpCoreApplication {
    public static void main(String[] args) {
        SpringApplication.run(SmartErpCoreApplication.class, args);
    }
}

```

User prompt: did you use the global exeption in the services ?

Response: No, services do not directly call or use GlobalExceptionHandler.

In standard Spring Boot architecture, **services throw custom exceptions**, and the **GlobalExceptionHandler catches them automatically** when they bubble up past the controller level.

Here is how the pattern works across layers:

1. Custom Exception Class

Create specific runtime exceptions for your domain rules:

```
package com.smarterp.shared.exception;

public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String message) {
        super(message);
    }
}
```

2. Service Layer (Throws Exception)

Your @Service classes contain business logic and throw exceptions when conditions fail:

```
@Service
@RequiredArgsConstructor
public class UserService {

    private final UserRepository userRepository;

    public UserResponse getUserById(UUID id) {
        return userRepository.findById(id)
            .map(this::mapToResponse)
            .orElseThrow(() -> new ResourceNotFoundException("User not found with ID: " + id));
    }
}
```

3. Global Exception Handler (Catches Exception)

Your @RestControllerAdvice intercepts the thrown exception and translates it into a structured HTTP response:

```
package com.smarterp.shared.exception;
```

```

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<Map<String, Object>> handleResourceNotFound(ResourceNotFoundException
ex) {
        Map<String, Object> body = new HashMap<>();
        body.put("timestamp", LocalDateTime.now());
        body.put("status", HttpStatus.NOT_FOUND.value());
        body.put("error", "Not Found");
        body.put("message", ex.getMessage());

        return new ResponseEntity<>(body, HttpStatus.NOT_FOUND);
    }
}

```

Flow Summary

Service Layer (throws CustomException)→Controller (bypassed)→GlobalExceptionHandler (catches & formats)→HTTP Response (4xx / 5xx JSON)

```

User prompt: was it use here package com.smarterp.security.service; import
com.smarterp.security.domain.Role; import com.smarterp.security.domain.User; import
com.smarterp.security.dto.AuthResponse; import com.smarterp.security.dto.LoginRequest; import
com.smarterp.security.dto.RegisterRequest; import com.smarterp.security.dto.UserDTO; import
com.smarterp.security.repository.RoleRepository; import com.smarterp.security.repository.UserRepository;
import com.smarterp.shared.security.JwtUtils; import com.smarterp.shared.security.SecurityUser; import
org.springframework.security.authentication.AuthenticationManager; import
org.springframework.security.authentication.UsernamePasswordAuthenticationToken; import
org.springframework.security.crypto.password.PasswordEncoder; import
org.springframework.stereotype.Service; import org.springframework.transaction.annotation.Transactional;
@Service public class AuthService { private final UserRepository userRepository; private final
RoleRepository roleRepository; private final PasswordEncoder passwordEncoder; private final JwtUtils
jwtUtils; private final AuthenticationManager authenticationManager; public AuthService(
UserRepository userRepository, RoleRepository roleRepository, PasswordEncoder
passwordEncoder, JwtUtils jwtUtils, AuthenticationManager authenticationManager ) {
this.userRepository = userRepository; this.roleRepository = roleRepository; this.passwordEncoder
= passwordEncoder; this.jwtUtils = jwtUtils; this.authenticationManager = authenticationManager;
} @Transactional public AuthResponse register(RegisterRequest request) { if
(userRepository.existsByEmail(request.email())) { throw new RuntimeException("Email address is
already in use."); } Role role = roleRepository.findByName(request.role())
.orElseThrow(() -> new RuntimeException("Error: Role " + request.role() + " was not found.")); User

```

```

user = new User();      user.setEmail(request.email());
user.setPasswordHash(passwordEncoder.encode(request.password()));      user.setRole(role);
user.setIsActive(true);      User savedUser = userRepository.save(user);      SecurityUser securityUser =
new SecurityUser(savedUser);      String jwtToken = jwtUtils.generateToken(securityUser);      return new
AuthResponse(      jwtToken,      savedUser.getId(),      savedUser.getEmail(),
savedUser.getRole().getName().name()      );  }  public AuthResponse login(LoginRequest request) {
    authenticationManager.authenticate(      new
UsernamePasswordAuthenticationToken(request.email(), request.password())      );      User user =
userRepository.findByEmail(request.email())      .orElseThrow(() -> new RuntimeException("User not
found with email: " + request.email()));      SecurityUser securityUser = new SecurityUser(user);      String
jwtToken = jwtUtils.generateToken(securityUser);      return new AuthResponse(      jwtToken,
user.getId(),      user.getEmail(),      user.getRole().getName().name()      );  }
@Transactional(readOnly = true)  public UserDTO getCurrentUser(String email) {      User user =
userRepository.findByEmail(email)      .orElseThrow(() -> new RuntimeException("User not found with
email: " + email));      return new UserDTO(      user.getId(),      user.getEmail(),
user.getRole().getName().name(),      user.getIsActive()      );  }}

```

Response: **No**. GlobalExceptionHandler is not used or referenced directly in AuthService.

Here is what is happening in your AuthService class and how GlobalExceptionHandler interacts with it behind the scenes:

What AuthService is doing right now

In AuthService, you are throwing standard Java RuntimeExceptions:

```

// 1. Throwing RuntimeException directly
if (userRepository.existsByEmail(request.email())) {
    throw new RuntimeException("Email address is already in use.");
}

// 2. Throwing RuntimeException inside orElseThrow
Role role = roleRepository.findByName(request.role())
    .orElseThrow(() -> new RuntimeException("Error: Role " + request.role() + " was not found."));

```

How GlobalExceptionHandler handles this

When AuthService throws a RuntimeException:

1. The service stops execution and passes the exception up to AuthController.
2. AuthController passes the exception up to Spring Framework.
3. Spring's **GlobalExceptionHandler intercepts it** if you have a rule for RuntimeException or generic Exception.

Best Practice Recommendation

Throwing generic RuntimeException works, but it causes GlobalExceptionHandler to treat all errors as generic internal errors (often returning 500 Internal Server Error).

To return clean HTTP status codes (like 400 Bad Request or 404 Not Found), replace generic RuntimeException with specific domain exceptions:

// Instead of RuntimeException:

```
if (userRepository.existsByEmail(request.email())) {  
    throw new BadRequestException("Email address is already in use.");  
    // Triggers GlobalExceptionHandler -> 400 Bad Request  
}
```

```
Role role = roleRepository.findByName(request.role())  
    .orElseThrow(() -> new ResourceNotFoundException("Error: Role " + request.role() + " was not found."));  
    // Triggers GlobalExceptionHandler -> 404 Not Found
```

User prompt: { "accessToken":

"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhZG1pbkZbZbWFydGVycC5jb20iLCJpYXQiOiJlODU5MzQ2NDksImV4cCI6MTc4NjAyMTA0OX0iSe3htuqhJc829e57kDpnzn314LZKICfW3znzoysjmmNMCG_oVtwn8i1nkaf6UA", "tokenType": "Bearer", "userId": "1d81d30f-c7c8-4217-8587-2bf93483175f", "email": "admin@smarterp.com", "role": "ROLE_ADMIN"} this worked but in my http://localhost:5050/browser/ i cant see the tables created

Response: The registration succeeded and records exist in PostgreSQL (otherwise JWT creation and user mapping would have failed).

If you cannot see the tables in pgAdmin (http://localhost:5050), it is usually due to tree navigation in pgAdmin or looking in the wrong database schema.

Here is how to locate them:

Step 1: Navigate to the Correct Schema Path in pgAdmin

PostgreSQL places tables inside **Schemas**, not directly under the database node.

In the left sidebar of pgAdmin, expand the tree in this exact order:

1. **Servers**
2. **[Your PostgreSQL Server Name]**
3. **Databases**
4. **smart_erp**
5. **Schemas**
6. **public**
7. **Tables** (Right-click Tables → click **Refresh**)

Step 2: Verify Active Database Connection

If smart_erp is not listed under **Databases**:

1. Right-click **Databases** → **Register / Create** → **Database...**
2. Name it smart_erp and save.

3. Refresh the tree.

Step 3: Run an Explicit SQL Query in pgAdmin

To confirm the tables exist independently of the GUI tree:

1. Right-click the **smart_erp** database in pgAdmin.
2. Select **Query Tool**.
3. Run the following SQL query:

```
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'public';
```

If users and roles appear in the Query Results table below, the data is saved correctly—you just need to right-click **Schemas** → **public** → **Tables** in the left sidebar and select **Refresh**.

Step 4: Check Host Connection (If Using Docker pgAdmin)

If pgAdmin is running inside Docker on port 5050:

- In pgAdmin connection settings, the host name should be **host.docker.internal** (or your postgres container name, like postgres), **not** localhost.
- If pgAdmin was connected to a different local PostgreSQL instance, it will show an empty database.

User prompt: SELECT table_name FROM information_schema.tables WHERE table_schema = 'public'; this has 0 rows Step 4: Check Host Connection (If Using Docker pgAdmin) If pgAdmin is running inside Docker on port 5050: In pgAdmin connection settings, the host name should be host.docker.internal (or your postgres container name, like postgres), not localhost. If pgAdmin was connected to a different local PostgreSQL instance, it will show an empty database. how to check this

Response: This happens because pgAdmin (running in Docker) and Spring Boot (running on your host machine) are connected to two different PostgreSQL instances.

Inside a Docker container, localhost refers to the container itself, **not** your Windows host machine.

Here is how to check and fix pgAdmin's server connection:

Step 1: Check Server Connection Settings in pgAdmin

1. In the pgAdmin sidebar, right-click your PostgreSQL Server (under **Servers**).
2. Select **Properties...**
3. Switch to the **Connection** tab.
4. Check the **Host name / address** field:
 - If it says localhost or 127.0.0.1, pgAdmin is trying to connect inside its own container instead of your Windows machine where Spring Boot wrote the data.

Step 2: Fix the Connection Host Name

Depending on how PostgreSQL is installed on your system:

Scenario A: PostgreSQL is installed directly on Windows

1. Change **Host name / address** in pgAdmin to:
host.docker.internal
2. Click **Save**.
3. Re-enter your password if prompted.

Scenario B: PostgreSQL is running in a Docker container

1. Change **Host name / address** to the Docker **container name** (e.g., postgres or smart-erp-db).
2. Make sure both pgAdmin and PostgreSQL containers are on the same Docker network.

Step 3: Verify Ground Truth via Windows PowerShell

To prove where your data actually lives, run this command in your Windows terminal to query the local database directly:

```
psql -U postgres -d smart_erp -c "SELECT table_name FROM information_schema.tables WHERE table_schema = 'public';"
```

If prompted for a password, enter your DB_PASSWORD (postgres). You will see the users and roles tables created by Spring Boot.

User prompt: version: '3.8' services: postgres: image: pgvector/pgvector:pg17 container_name: smart_erp_db restart: always environment: POSTGRES_USER: postgres POSTGRES_PASSWORD: postgres POSTGRES_DB: smart_erp ports: - "5432:5432" volumes: - erp_postgres_data:/var/lib/postgresql/data # Optional but highly recommended: pgAdmin for a visual database interface pgadmin: image: dpage/pgadmin4 container_name: smart_erp_pgadmin restart: always environment: PGADMIN_DEFAULT_EMAIL: saidaniyesser999@gmail.com PGADMIN_DEFAULT_PASSWORD: admin ports: - "5050:80" depends_on: - postgres volumes: erp_postgres_data: i used this to connect to pgadmin via docker then i created a new server with postgres as the hostname

Response: Since you configured pgadmin and postgres inside the same Docker Compose file, using **postgres** as the hostname in pgAdmin is correct.

If `SELECT table_name FROM information_schema.tables WHERE table_schema = 'public';` returns 0 rows, the cause is almost always one of these two issues:

Cause 1: Query Tool is Connected to the postgres DB (Not smart_erp)

When you add a server in pgAdmin, it connects by default to the system database named **postgres**, not **smart_erp**. If you open the Query Tool without explicitly selecting smart_erp, you are querying the empty system database.

How to fix:

1. In pgAdmin, expand **Servers** → **[Your Server Name]** → **Databases**.
2. Click directly on **smart_erp**.
3. Right-click **smart_erp** and select **Query Tool**.
4. Check the top bar of the Query Tool editor. It must show: postgres@postgres/smart_erp (or smart_erp), **not** postgres@postgres/postgres.
5. Run the query again:
`SELECT table_name FROM information_schema.tables WHERE table_schema = 'public';`

Cause 2: Port Conflict (Spring Boot Connected to Local Windows Postgres)

If you have PostgreSQL installed natively on Windows, Windows port 5432 might be taken by your local machine, meaning Spring Boot saved the data to Windows PostgreSQL instead of your Docker container.

How to verify directly inside the Docker container:

Run this command in your Windows PowerShell terminal:

```
docker exec -it smart_erp_db psql -U postgres -d smart_erp -c "\dt"
```

- **If users and roles appear:** The container has the data. Issue #1 (pgAdmin UI selection) is the cause.
- **If it says "Did not find any relations":** Spring Boot connected to a local Windows PostgreSQL instance running outside Docker on port 5432. Stop the local Windows PostgreSQL service (net stop postgresql-x64-17) and restart your Spring Boot app.

User prompt: postgres@postgres/Smart ERP Local with is the name of the server not the db

Response: To find out exactly what is happening, run this SQL query in your pgAdmin **Query Tool**:

```
SELECT current_database();
```

Outcome A: It returns postgres

You are currently connected to the default system database, not smart_erp.

Fix in pgAdmin:

1. In the left panel, click on **Databases**.
2. Click on **smart_erp** so it is highlighted.

3. Right-click **smart_erp** → **Query Tool**.
4. Re-run `SELECT current_database();` to ensure it returns `smart_erp`, then check your tables again.

Outcome B: It returns `smart_erp`, but `public` is still empty

This means **Spring Boot connected to a local Windows PostgreSQL instance** installed on your computer, rather than the Docker container.

When both Windows PostgreSQL and Docker PostgreSQL try to use port 5432, Windows PostgreSQL takes priority, so Spring Boot saves data to Windows while pgAdmin looks inside Docker.

1. Verify what is inside the Docker container

Run this command in Windows PowerShell:

```
docker exec -it smart_erp_db psql -U postgres -d smart_erp -c "\dt"
```

If it says "Did not find any relations", Docker does not have the tables.

2. Stop the local Windows PostgreSQL service

Run this in PowerShell as Administrator:

```
Stop-Service postgresql*
```

3. Restart Spring Boot

Restart your Spring Boot application with `.\mvnw.cmd spring-boot:run`. It will now force the connection into the Docker PostgreSQL container, and your tables will appear in pgAdmin.

User prompt: Stop-Service postgresql it worked when i did this*

Response: That confirms it! Your native Windows PostgreSQL service was squatting on port 5432, so Spring Boot was quietly writing all its data to your local machine instead of the Docker container.

Now that the local service is out of the way, Spring Boot routes directly into `smart_erp_db` inside Docker, Hibernate creates the schema there, and pgAdmin sees the exact same tables.

Prevent it from happening after a reboot

Windows will try to auto-start the local PostgreSQL service every time you restart your computer. To keep port 5432 free for Docker permanently, run this once in PowerShell as Administrator:

```
Get-Service postgresql* | Set-Service -StartupType Disabled
```

With your database pipeline clear and registration creating user records in Docker, would you like to test the /login endpoint next to verify JWT token generation?

User prompt: an advise befor we continu i want to upload the project in github, do i use one repository for it all backend frontend ai , or each one hase it repository

Response: For a project like **Smart ERP**, especially during an internship or solo/small-team development, I strongly recommend using a **Single Repository (Monorepo)** containing all three components (backend, frontend, and AI module).

Here is why a Monorepo is the best choice for your setup:

- **Single-Command Setup for Evaluators:** Anyone reviewing your work (your mentor, internship supervisors, or recruiters) can run git clone, execute docker compose up, and have the complete architecture—Spring Boot, Angular, Python/AI, PostgreSQL, and pgAdmin—running instantly.
- **Unified Docker Orchestration:** You can place a single docker-compose.yml at the root of the repository to build and network all your services cleanly.
- **Simpler Git Workflow:** You avoid the headache of juggling three different repositories, commit histories, and pull requests every time you build a feature that requires changes across the UI, backend, and AI layers.
- **Stronger Portfolio Impact:** On GitHub, one complete, well-structured repository with an impressive README.md makes a much stronger impression than three disconnected, partial repositories.

Here is the cleanest directory structure to organize your Monorepo:

```
Smart_ERP/
├── backend/      # smart-erp-core (Spring Boot)
├── frontend/     # smart-erp-web (Angular)
├── ai-service/   # smart-erp-ai (Python / FastAPI or Flask)
├── docker-compose.yml # Root compose file for all 3 services + DB
├── .gitignore    # Root ignore file
└── README.md     # Master documentation with setup instructions
```

Before you push your first commit, ensure your **.gitignore** is configured properly so you don't commit build artifacts (target/, node_modules/, .venv/), IDE config files (.idea/, .vscode/), or hardcoded secrets.

Are you planning to deploy this setup to a live cloud provider later, or will it be demonstrated primarily in a local environment?

User prompt: in the dto for security the register request only for a user . so what you aimed to do is for the admin to register all user and there role then you write a employe register request so that the employee can register and acces the sight later ?

Response: You hit on a critical security and business logic distinction for an ERP system.

In enterprise software, allowing a public `/auth/register` endpoint to accept any role in the request body is a major security vulnerability (**Privilege Escalation**)—anyone calling the endpoint could send `"role": "ROLE_ADMIN"` and gain full administrative rights.

Here is how user creation should actually be structured depending on your requirements:

Pattern 1: Admin-Driven Onboarding (Standard for ERPs)

In most ERPs, **employees do not self-register freely on the web**. Instead, an Admin or HR manager creates employee accounts from inside the application.

- **Endpoint:** `POST /api/v1/admin/users` (Protected, requires `ROLE_ADMIN`)
- **Request DTO:** `CreateUserRequest(email, password, role, firstName, lastName, departmentId)`
- **Flow:** The Admin selects the exact role (`ROLE_EMPLOYEE`, `ROLE_MANAGER`, `ROLE_ADMIN`) when creating the user.

Pattern 2: Public Employee Self-Registration (Self-Signup)

If you *do* want employees to be able to sign up themselves from the login page, the endpoint must restrict what the user can control.

- **Endpoint:** `POST /api/v1/auth/register` (Public)
- **Request DTO:** `RegisterRequest(email, password, firstName, lastName)` (*Notice: **No role parameter***)
- **Flow:**
 1. The backend automatically forces the role to **`ROLE_EMPLOYEE`** regardless of what the user sends.
 2. Optionally, set `user.setIsActive(false)` so an Admin must approve the account before the employee can log in.

Recommended Setup for SmartERP

You can implement both flows cleanly in your project:

1. Public Self-Registration (Hardcoded `ROLE_EMPLOYEE`)

```
// AuthService.java
@Transactional
public AuthResponse registerEmployee(RegisterRequest request) {
    if (userRepository.existsByEmail(request.email())) {
        throw new BadRequestException("Email address is already in use.");
    }

    // Always force ROLE_EMPLOYEE for public registration
    Role defaultRole = roleRepository.findByName(ERole.ROLE_EMPLOYEE)
        .orElseThrow(() -> new ResourceNotFoundException("Default role ROLE_EMPLOYEE not found."));

    User user = new User();
    user.setEmail(request.email());
```

```

user.setPasswordHash(passwordEncoder.encode(request.password()));
user.setRole(defaultRole);
user.setIsActive(true); // Or false if you require Admin approval first

User savedUser = userRepository.save(user);
SecurityUser securityUser = new SecurityUser(savedUser);
String jwtToken = jwtUtils.generateToken(securityUser);

return new AuthResponse(jwtToken, savedUser.getId(), savedUser.getEmail(),
defaultRole.getName().name());
}

```

2. Admin User Creation (Admin picks the Role)

```

// AdminUserService.java (or AuthService)
@Transactional
public UserDTO adminCreateUser(AdminCreateUserRequest request) {
    if (userRepository.existsByEmail(request.email())) {
        throw new BadRequestException("Email address is already in use.");
    }

    // Admin explicitly provides the role name
    Role role = roleRepository.findByName(request.role())
        .orElseThrow(() -> new ResourceNotFoundException("Role " + request.role() + " not found."));

    User user = new User();
    user.setEmail(request.email());
    user.setPasswordHash(passwordEncoder.encode(request.password()));
    user.setRole(role);
    user.setIsActive(true);

    User savedUser = userRepository.save(user);
    return new UserDTO(savedUser.getId(), savedUser.getEmail(), role.getName().name(),
savedUser.getIsActive());
}

```

Does your ERP requirement specify that employees should sign up on their own, or should account creation be strictly managed by an Admin?

User prompt: based on my project and the fact that i want it to be professional witch method do i choose

Response: For a professional, enterprise-grade ERP system, you should choose **Admin-Driven User Management**, paired with an **Automatic Data Seeder** and a **Controlled Public Registration** for demo evaluation.

In real-world enterprise applications (like SAP, Oracle, or Odoo), internal operational systems are closed. Employees do not self-register from a public landing page—HR or System Administrators provision

accounts, assign department scopes, and grant specific security roles.

The Professional Architecture Setup

1. Automatic Database Seeding (On Application Startup)

When your Spring Boot app launches for the first time, it should automatically populate the core roles and a super-admin account if they don't already exist.

Create a DatabaseSeeder.java class implementing CommandLineRunner:

```
package com.smarterp.config;

import com.smarterp.security.domain.ERole;
import com.smarterp.security.domain.Role;
import com.smarterp.security.domain.User;
import com.smarterp.security.repository.RoleRepository;
import com.smarterp.security.repository.UserRepository;
import org.springframework.boot.CommandLineRunner;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Component;

@Component
public class DatabaseSeeder implements CommandLineRunner {

    private final RoleRepository roleRepository;
    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    public DatabaseSeeder(RoleRepository roleRepository, UserRepository userRepository,
        PasswordEncoder passwordEncoder) {
        this.roleRepository = roleRepository;
        this.userRepository = userRepository;
        this.passwordEncoder = passwordEncoder;
    }

    @Override
    public void run(String... args) {
        // 1. Seed Roles
        for (ERole roleName : ERole.values()) {
            if (roleRepository.findByName(roleName).isEmpty()) {
                Role role = new Role();
                role.setName(roleName);
                role.setDescription(roleName.name() + " access");
                roleRepository.save(role);
            }
        }

        // 2. Seed Default System Admin
        if (!userRepository.existsByEmail("admin@smarterp.com")) {
            Role adminRole = roleRepository.findByName(ERole.ROLE_ADMIN)
```

```

        .orElseThrow(() -> new RuntimeException("Admin role not found"));

        User admin = new User();
        admin.setEmail("admin@smarterp.com");
        admin.setPasswordHash(passwordEncoder.encode("Admin@123456"));
        admin.setRole(adminRole);
        admin.setIsActive(true);

        userRepository.save(admin);
    }
}
}

```

2. Admin Endpoint for Provisioning Employees (POST /api/v1/admin/users)

This is where your core business logic lives. Only users with `ROLE_ADMIN` can call this endpoint to create new team members, assign their roles (`ROLE_MANAGER`, `ROLE_EMPLOYEE`), and tie them to departments or employee profiles.

3. Public Registration Endpoint (POST /api/v1/auth/register)

To make testing convenient for recruiters, professors, or internship supervisors reviewing your demo, keep public registration available on the login page, but **hardcode the assigned role to `ROLE_EMPLOYEE`**.

```

@Transactional
public AuthResponse register(RegisterRequest request) {
    if (userRepository.existsByEmail(request.email())) {
        throw new BadRequestException("Email address is already in use.");
    }

    // Force default role to ROLE_EMPLOYEE regardless of input
    Role employeeRole = roleRepository.findByName(ERole.ROLE_EMPLOYEE)
        .orElseThrow(() -> new ResourceNotFoundException("Role ROLE_EMPLOYEE not found"));

    User user = new User();
    user.setEmail(request.email());
    user.setPasswordHash(passwordEncoder.encode(request.password()));
    user.setRole(employeeRole);
    user.setIsActive(true);

    User savedUser = userRepository.save(user);
    // Generate JWT token...
}

```

Summary of Benefits for Your Portfolio

- **Security:** Removes privilege escalation vulnerabilities (users cannot make themselves admins).
- **Enterprise Realism:** Demonstrates to reviewers that you understand RBAC (Role-Based Access Control) and real-world ERP workflow security.

- **Evaluator Friction:** Anyone testing your live frontend can register instantly as a standard employee, or log in with the seeded admin@smarterp.com credentials to access administrative views.

Would you like to build the DatabaseSeeder first, or proceed to setting up the JWT authentication filter for request validation?

```
User prompt: package com.smarterp.security.service; import com.smarterp.security.domain.Role; import
com.smarterp.security.domain.User; import com.smarterp.security.dto.AuthResponse; import
com.smarterp.security.dto.LoginRequest; import com.smarterp.security.dto.RegisterRequest; import
com.smarterp.security.dto.UserDTO; import com.smarterp.security.repository.RoleRepository; import
com.smarterp.security.repository.UserRepository; import com.smarterp.shared.security.JwtUtils; import
com.smarterp.shared.security.SecurityUser; import
org.springframework.security.authentication.AuthenticationManager; import
org.springframework.security.authentication.UsernamePasswordAuthenticationToken; import
org.springframework.security.crypto.password.PasswordEncoder; import
org.springframework.stereotype.Service; import org.springframework.transaction.annotation.Transactional;
@Service public class AuthService { private final UserRepository userRepository; private final
RoleRepository roleRepository; private final PasswordEncoder passwordEncoder; private final JwtUtils
jwtUtils; private final AuthenticationManager authenticationManager; public AuthService(
UserRepository userRepository, RoleRepository roleRepository, PasswordEncoder
passwordEncoder, JwtUtils jwtUtils, AuthenticationManager authenticationManager ) {
this.userRepository = userRepository; this.roleRepository = roleRepository; this.passwordEncoder
= passwordEncoder; this.jwtUtils = jwtUtils; this.authenticationManager = authenticationManager;
} @Transactional public AuthResponse register(RegisterRequest request) { if
(userRepository.existsByEmail(request.email())) { throw new RuntimeException("Email address is
already in use."); } Role role = roleRepository.findByName(request.role())
.orElseThrow(() -> new RuntimeException("Error: Role " + request.role() + " was not found.")); User
user = new User(); user.setEmail(request.email());
user.setPasswordHash(passwordEncoder.encode(request.password())); user.setRole(role);
user.setIsActive(true); User savedUser = userRepository.save(user); SecurityUser securityUser =
new SecurityUser(savedUser); String jwtToken = jwtUtils.generateToken(securityUser); return new
AuthResponse( jwtToken, savedUser.getId(), savedUser.getEmail(),
savedUser.getRole().getName().name() ); } public AuthResponse login(LoginRequest request) {
authenticationManager.authenticate( new
UsernamePasswordAuthenticationToken(request.email(), request.password() ); User user =
userRepository.findByEmail(request.email()) .orElseThrow(() -> new RuntimeException("User not
found with email: " + request.email())); SecurityUser securityUser = new SecurityUser(user); String
jwtToken = jwtUtils.generateToken(securityUser); return new AuthResponse( jwtToken,
user.getId(), user.getEmail(), user.getRole().getName().name() ); }
@Transactional(readOnly = true) public UserDTO getCurrentUser(String email) { User user =
userRepository.findByEmail(email) .orElseThrow(() -> new RuntimeException("User not found with
email: " + email)); return new UserDTO( user.getId(), user.getEmail(),
user.getRole().getName().name(), user.getIsActive() ); } } this is the service now what mi
messing
```

Response: Here is the updated AuthService.java with the missing security fixes and exception handling improvements.

Key Changes Made:

1. **Hardcoded Public Role:** Public register now forces ERole.ROLE_EMPLOYEE so users cannot grant themselves ROLE_ADMIN.
 2. **Custom Exceptions:** Replaced generic RuntimeException with BadRequestException and ResourceNotFoundException so your GlobalExceptionHandler returns accurate 400 and 404 HTTP status codes.
 3. **Admin Provisioning Method:** Added adminCreateUser to allow administrators to explicitly create users with any role (e.g., ROLE_MANAGER, ROLE_ADMIN).
- package com.smarterp.security.service;

```
import com.smarterp.security.domain.ERole;
import com.smarterp.security.domain.Role;
import com.smarterp.security.domain.User;
import com.smarterp.security.dto.AuthResponse;
import com.smarterp.security.dto.LoginRequest;
import com.smarterp.security.dto.RegisterRequest;
import com.smarterp.security.dto.UserDTO;
import com.smarterp.security.repository.RoleRepository;
import com.smarterp.security.repository.UserRepository;
import com.smarterp.shared.exception.BadRequestException;
import com.smarterp.shared.exception.ResourceNotFoundException;
import com.smarterp.shared.security.JwtUtils;
import com.smarterp.shared.security.SecurityUser;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
```

@Service

```
public class AuthService {

    private final UserRepository userRepository;
    private final RoleRepository roleRepository;
    private final PasswordEncoder passwordEncoder;
    private final JwtUtils jwtUtils;
    private final AuthenticationManager authenticationManager;

    public AuthService(
        UserRepository userRepository,
        RoleRepository roleRepository,
        PasswordEncoder passwordEncoder,
        JwtUtils jwtUtils,
        AuthenticationManager authenticationManager
    ) {
        this.userRepository = userRepository;
        this.roleRepository = roleRepository;
        this.passwordEncoder = passwordEncoder;
        this.jwtUtils = jwtUtils;
        this.authenticationManager = authenticationManager;
    }
}
```

```

/**
 * Public self-registration (Default role is always ROLE_EMPLOYEE for security)
 */
@Transactional
public AuthResponse register(RegisterRequest request) {
    if (userRepository.existsByEmail(request.email())) {
        throw new BadRequestException("Email address is already in use.");
    }

    // Force ROLE_EMPLOYEE for public registration
    Role role = roleRepository.findByName(ERole.ROLE_EMPLOYEE)
        .orElseThrow(() -> new ResourceNotFoundException("Error: Default role ROLE_EMPLOYEE was
not found."));

    User user = new User();
    user.setEmail(request.email());
    user.setPasswordHash(passwordEncoder.encode(request.password()));
    user.setRole(role);
    user.setIsActive(true);

    User savedUser = userRepository.save(user);
    SecurityUser securityUser = new SecurityUser(savedUser);
    String jwtToken = jwtUtils.generateToken(securityUser);

    return new AuthResponse(
        jwtToken,
        savedUser.getId(),
        savedUser.getEmail(),
        savedUser.getRole().getName().name()
    );
}

/**
 * Authenticate an existing user
 */
public AuthResponse login(LoginRequest request) {
    authenticationManager.authenticate(
        new UsernamePasswordAuthenticationToken(request.email(), request.password())
    );

    User user = userRepository.findByEmail(request.email())
        .orElseThrow(() -> new ResourceNotFoundException("User not found with email: " +
request.email()));

    SecurityUser securityUser = new SecurityUser(user);
    String jwtToken = jwtUtils.generateToken(securityUser);

    return new AuthResponse(
        jwtToken,
        user.getId(),
        user.getEmail(),

```

```

        user.getRole().getName().name()
    );
}

/**
 * Get details of the currently authenticated user
 */
@Transactional(readOnly = true)
public UserDTO getCurrentUser(String email) {
    User user = userRepository.findByEmail(email)
        .orElseThrow(() -> new ResourceNotFoundException("User not found with email: " + email));

    return new UserDTO(
        user.getId(),
        user.getEmail(),
        user.getRole().getName().name(),
        user.getIsActive()
    );
}

/**
 * Protected Admin method to create users with specific roles
 */
@Transactional
public UserDTO adminCreateUser(String email, String rawPassword, ERole roleName) {
    if (userRepository.existsByEmail(email)) {
        throw new BadRequestException("Email address is already in use.");
    }

    Role role = roleRepository.findByName(roleName)
        .orElseThrow(() -> new ResourceNotFoundException("Error: Role " + roleName + " was not found."));

    User user = new User();
    user.setEmail(email);
    user.setPasswordHash(passwordEncoder.encode(rawPassword));
    user.setRole(role);
    user.setIsActive(true);

    User savedUser = userRepository.save(user);

    return new UserDTO(
        savedUser.getId(),
        savedUser.getEmail(),
        savedUser.getRole().getName().name(),
        savedUser.getIsActive()
    );
}
}

```

What to Check Next

1. Ensure `BadRequestException` and `ResourceNotFoundException` exist in your `com.smarterp.shared.exception` package.
2. If `RegisterRequest` DTO still contains a `role` field, you can remove `role` from the `RegisterRequest` record entirely since public registration handles it automatically.

Do you have the custom exception classes (`BadRequestException`, `ResourceNotFoundException`) already created in your `shared.exception` package?

User prompt: but in this case the admin only have the option to register so the register of the user only available for admin only no ?

Response: **Yes, exactly.** In a strict, enterprise-grade ERP, public self-registration does not exist at all. The only public endpoint is `/auth/login`.

If you choose this pure enterprise design, **only an Admin can create new users.**

How the System Works (Admin-Only User Creation)

Since new users cannot register themselves, the system bootstrap and account lifecycle follow 3 steps:

1. System Bootstrapping (DatabaseSeeder)

When the application starts up for the first time, Spring Boot runs a seeder that automatically creates the roles and the first **Super Admin** account in PostgreSQL if it doesn't already exist.

// `admin@smarterp.com` / `Admin@123456` is automatically seeded on startup

2. Public Login (POST `/api/v1/auth/login`)

Anyone logging in must already have an account created by the Admin. The `/auth/login` endpoint validates credentials and returns a JWT token containing their assigned role.

3. Admin Provisions New Users (POST `/api/v1/admin/users`)

Only an authenticated user with `ROLE_ADMIN` can access this endpoint to create employees, assign their roles, and set up their accounts.

Updated Code Structure for Admin-Only Setup

1. Update `SecurityConfig.java`

Remove `/auth/register` from public access. Only `/auth/login` remains public:

```

.authorizeHttpRequests(auth -> auth
    .requestMatchers("/auth/login", "/v3/api-docs/**", "/swagger-ui/**").permitAll()
    .requestMatchers("/admin/**").hasRole("ADMIN")
    .anyRequest().authenticated()
)

```

2. Create AdminUserController.java

Expose an admin-only endpoint for creating users:

```

package com.smarterp.security.controller;

import com.smarterp.security.dto.CreateUserRequest;
import com.smarterp.security.dto.UserDTO;
import com.smarterp.security.service.AuthService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/admin/users")
@PreAuthorize("hasRole('ADMIN')")
public class AdminUserController {

    private final AuthService authService;

    public AdminUserController(AuthService authService) {
        this.authService = authService;
    }

    @PostMapping
    public ResponseEntity<UserDTO> createUser(@RequestBody CreateUserRequest request) {
        UserDTO newUser = authService.adminCreateUser(
            request.email(),
            request.password(),
            request.role()
        );
        return ResponseEntity.status(HttpStatus.CREATED).body(newUser);
    }
}

```

3. Define CreateUserRequest.java DTO

```

package com.smarterp.security.dto;

import com.smarterp.security.domain.ERole;

public record CreateUserRequest(
    String email,

```

```
String password,  
ERole role  
) {}
```

Summary of the Flow

1. You launch your application → DatabaseSeeder creates admin@smarterp.com.
 2. You open Postman (or your frontend) and log in as admin@smarterp.com via POST /api/v1/auth/login.
 3. You copy the returned JWT accessToken.
 4. You call POST /api/v1/admin/users using the JWT in the Authorization: Bearer <token> header to create employees (ROLE_EMPLOYEE) or managers (ROLE_MANAGER).
-

User prompt: the register function that i gave you earlier would be only used by the admin so there is no need to change the role to employee by default because only the admin would use the register function no ?

Response: **Exactly.** If account creation is restricted strictly to Administrators, accepting request.role() directly from the request body is correct and necessary so the Admin can assign any role (ROLE_EMPLOYEE, ROLE_MANAGER, or ROLE_ADMIN).

Forcing ROLE_EMPLOYEE by default is only required for *public* self-registration endpoints.

Here is your updated AuthService.java retaining dynamic role assignment via request.role(), with generic RuntimeExceptions upgraded to custom exceptions:

```
package com.smarterp.security.service;  
  
import com.smarterp.security.domain.Role;  
import com.smarterp.security.domain.User;  
import com.smarterp.security.dto.AuthResponse;  
import com.smarterp.security.dto.LoginRequest;  
import com.smarterp.security.dto.RegisterRequest;  
import com.smarterp.security.dto.UserDTO;  
import com.smarterp.security.repository.RoleRepository;  
import com.smarterp.security.repository.UserRepository;  
import com.smarterp.shared.exception.BadRequestException;  
import com.smarterp.shared.exception.ResourceNotFoundException;  
import com.smarterp.shared.security.JwtUtils;  
import com.smarterp.shared.security.SecurityUser;  
import org.springframework.security.authentication.AuthenticationManager;  
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;  
import org.springframework.security.crypto.password.PasswordEncoder;  
import org.springframework.stereotype.Service;  
import org.springframework.transaction.annotation.Transactional;  
  
@Service  
public class AuthService {  
  
    private final UserRepository userRepository;  
    private final RoleRepository roleRepository;
```

```

private final PasswordEncoder passwordEncoder;
private final JwtUtils jwtUtils;
private final AuthenticationManager authenticationManager;

public AuthService(
    UserRepository userRepository,
    RoleRepository roleRepository,
    PasswordEncoder passwordEncoder,
    JwtUtils jwtUtils,
    AuthenticationManager authenticationManager
) {
    this.userRepository = userRepository;
    this.roleRepository = roleRepository;
    this.passwordEncoder = passwordEncoder;
    this.jwtUtils = jwtUtils;
    this.authenticationManager = authenticationManager;
}

/**
 * Admin User Registration (Admin specifies email, password, and target role)
 */
@Transactional
public AuthResponse register(RegisterRequest request) {
    if (userRepository.existsByEmail(request.email())) {
        throw new BadRequestException("Email address is already in use.");
    }

    Role role = roleRepository.findByName(request.role())
        .orElseThrow(() -> new ResourceNotFoundException("Error: Role " + request.role() + " was not found."));

    User user = new User();
    user.setEmail(request.email());
    user.setPasswordHash(passwordEncoder.encode(request.password()));
    user.setRole(role);
    user.setIsActive(true);

    User savedUser = userRepository.save(user);
    SecurityUser securityUser = new SecurityUser(savedUser);
    String jwtToken = jwtUtils.generateToken(securityUser);

    return new AuthResponse(
        jwtToken,
        savedUser.getId(),
        savedUser.getEmail(),
        savedUser.getRole().getName().name()
    );
}

public AuthResponse login(LoginRequest request) {
    authenticationManager.authenticate(
        new UsernamePasswordAuthenticationToken(request.email(), request.password())
    );
}

```

```

);

User user = userRepository.findByEmail(request.email())
    .orElseThrow(() -> new ResourceNotFoundException("User not found with email: " +
request.email()));

SecurityUser securityUser = new SecurityUser(user);
String jwtToken = jwtUtils.generateToken(securityUser);

return new AuthResponse(
    jwtToken,
    user.getId(),
    user.getEmail(),
    user.getRole().getName().name()
);
}

@Transactional(readOnly = true)
public UserDTO getCurrentUser(String email) {
    User user = userRepository.findByEmail(email)
        .orElseThrow(() -> new ResourceNotFoundException("User not found with email: " + email));

    return new UserDTO(
        user.getId(),
        user.getEmail(),
        user.getRole().getName().name(),
        user.getIsActive()
    );
}
}

```

Important Security Check

Make sure your SecurityConfig.java protects the /auth/register endpoint so non-admin users cannot call it:

```

.authorizeHttpRequests(auth -> auth
    .requestMatchers("/auth/login").permitAll()
    .requestMatchers("/auth/register").hasRole("ADMIN") // Only ADMINS can register new accounts
    .anyRequest().authenticated()
)

```

User prompt: whats the next move in the project

Response: Now that your database pipeline is working and AuthService is set up for admin-driven user management, the next immediate move is to complete the **Spring Security JWT filter pipeline** and seed the initial **Admin user**.

Without these two steps, you cannot log in for the first time or protect your API endpoints.

Step 1: Create DatabaseSeeder (App Startup Bootstrapping)

Since registration is restricted to admins, you need an initial Super Admin in the database when the application boots up for the first time.

Create DatabaseSeeder.java inside com.smarterp.shared.config (or your config package):

```
package com.smarterp.shared.config;

import com.smarterp.security.domain.ERole;
import com.smarterp.security.domain.Role;
import com.smarterp.security.domain.User;
import com.smarterp.security.repository.RoleRepository;
import com.smarterp.security.repository.UserRepository;
import org.springframework.boot.CommandLineRunner;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Component;

@Component
public class DatabaseSeeder implements CommandLineRunner {

    private final RoleRepository roleRepository;
    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    public DatabaseSeeder(
        RoleRepository roleRepository,
        UserRepository userRepository,
        PasswordEncoder passwordEncoder
    ) {
        this.roleRepository = roleRepository;
        this.userRepository = userRepository;
        this.passwordEncoder = passwordEncoder;
    }

    @Override
    public void run(String... args) {
        // 1. Seed Roles if missing
        for (ERole roleEnum : ERole.values()) {
            if (roleRepository.findByName(roleEnum).isEmpty()) {
                Role role = new Role();
                role.setName(roleEnum);
                role.setDescription(roleEnum.name() + " access privileges");
                roleRepository.save(role);
            }
        }

        // 2. Seed Default Super Admin Account
        if (!userRepository.existsByEmail("admin@smarterp.com")) {
```

```

        Role adminRole = roleRepository.findByName(ERole.ROLE_ADMIN)
            .orElseThrow(() -> new RuntimeException("ROLE_ADMIN not found"));

        User admin = new User();
        admin.setEmail("admin@smarterp.com");
        admin.setPasswordHash(passwordEncoder.encode("Admin@123456"));
        admin.setRole(adminRole);
        admin.setIsActive(true);

        userRepository.save(admin);
        System.out.println(">>> Super Admin account created: admin@smarterp.com / Admin@123456");
    }
}

```

Step 2: Implement JwtAuthenticationFilter

This filter runs on every incoming HTTP request to extract the Authorization: Bearer <token> header, validate it, and authenticate the user in Spring Security's context.

Create JwtAuthenticationFilter.java inside com.smarterp.shared.security:

```

package com.smarterp.shared.security;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.lang.NonNull;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component;
import org.springframework.util.StringUtils;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;

@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtUtils jwtUtils;
    private final UserDetailsService userDetailsService;

    public JwtAuthenticationFilter(JwtUtils jwtUtils, UserDetailsService userDetailsService) {
        this.jwtUtils = jwtUtils;
        this.userDetailsService = userDetailsService;
    }
}

```

```

@Override
protected void doFilterInternal(
    @NonNull HttpServletRequest request,
    @NonNull HttpServletResponse response,
    @NonNull FilterChain filterChain
) throws ServletException, IOException {
    try {
        String jwt = parseJwt(request);
        if (jwt != null && jwtUtils.validateToken(jwt)) {
            String username = jwtUtils.extractUsername(jwt);

            UserDetails userDetails = userDetailsService.loadUserByUsername(username);
            UsernamePasswordAuthenticationToken authentication =
                new UsernamePasswordAuthenticationToken(
                    userDetails,
                    null,
                    userDetails.getAuthorities()
                );

            authentication.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
            SecurityContextHolder.getContext().setAuthentication(authentication);
        }
    } catch (Exception e) {
        logger.error("Cannot set user authentication: {}", e);
    }

    filterChain.doFilter(request, response);
}

private String parseJwt(HttpServletRequest request) {
    String headerAuth = request.getHeader("Authorization");
    if (StringUtils.hasText(headerAuth) && headerAuth.startsWith("Bearer ")) {
        return headerAuth.substring(7);
    }
    return null;
}
}

```

Step 3: Wire Everything into SecurityConfig.java

Update SecurityConfig.java to use stateless session management, register JwtAuthenticationFilter, and lock down /auth/register to ROLE_ADMIN:

```

package com.smarterp.shared.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import

```

```
org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
```

```
@Configuration
```

```
@EnableWebSecurity
```

```
@EnableMethodSecurity
```

```
public class SecurityConfig {
```

```
    private final JwtAuthenticationFilter jwtAuthFilter;
```

```
    public SecurityConfig(JwtAuthenticationFilter jwtAuthFilter) {
        this.jwtAuthFilter = jwtAuthFilter;
    }
```

```
@Bean
```

```
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .csrf(AbstractHttpConfigurer::disable)
            .sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/v1/auth/login", "/v3/api-docs/**", "/swagger-ui/**").permitAll()
                .requestMatchers("/api/v1/auth/register").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
```

```
@Bean
```

```
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
```

```
@Bean
```

```
    public AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws
Exception {
        return config.getAuthenticationManager();
    }
}
```

Next Roadmap Order:

1. **Apply Steps 1–3 above** and restart Spring Boot.
2. **Test Login via Postman / Bruno:** POST /api/v1/auth/login with admin@smarterp.com and Admin@123456.
3. **Test Admin Account Creation:** Call POST /api/v1/auth/register passing the admin Bearer token to create a new ROLE_EMPLOYEE.
4. **Begin Core ERP Domain Modules** (e.g., HR / Employee Profile module or Product / Inventory module).

Do you already have your UserDetailsService implementation ready for Step 2, or would you like to build that next?

```
User prompt: package com.smarterp.security.config; import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration; import
org.springframework.security.authentication.AuthenticationManager; import
org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity; import
org.springframework.security.config.annotation.web.configuration.EnableWebSecurity; import
org.springframework.security.config.http.SessionCreationPolicy; import
org.springframework.security.web.SecurityFilterChain; import
org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter; @Configuration
@EnableWebSecurity public class SecurityConfig { // Inject your JwtAuthenticationFilter if you have one
// private final JwtAuthenticationFilter jwtAuthFilter; @Bean public SecurityFilterChain
securityFilterChain(HttpSecurity http) throws Exception { http // Disable CSRF for stateless
REST APIs using JWT .csrf(csrf -> csrf.disable()) .authorizeHttpRequests(auth -> auth
.requestMatchers("/auth/login").permitAll() .requestMatchers("/auth/register").hasRole("ADMIN") //
Only ADMINS can register new accounts .anyRequest().authenticated() )
.sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS));
// If you have a custom JWT Filter, add it before UsernamePasswordAuthenticationFilter: //
http.addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class); return http.build(); }
@Bean public AuthenticationManager authenticationManager(AuthenticationConfiguration config)
throws Exception { return config.getAuthenticationManager(); } } i have this security config not in
shared is do i modifie this ?
```

Response: **Yes, keep it in com.smarterp.security.config and modify this exact file.** You do not need to move it to shared.

To make JWT authentication operational, you need to uncomment and inject JwtAuthenticationFilter, add @EnableMethodSecurity, and define the PasswordEncoder bean (if you haven't defined it in another config class).

Here is the updated SecurityConfig.java:

```
package com.smarterp.security.config;

import com.smarterp.shared.security.JwtAuthenticationFilter;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
```

```

import org.springframework.security.authentication.AuthenticationManager;
import
org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

```

```
@Configuration
```

```
@EnableWebSecurity
```

```
@EnableMethodSecurity // Enables fine-grained authorization like @PreAuthorize("hasRole('ADMIN')")
```

```
public class SecurityConfig {
```

```
    private final JwtAuthenticationFilter jwtAuthFilter;
```

```
    public SecurityConfig(JwtAuthenticationFilter jwtAuthFilter) {
        this.jwtAuthFilter = jwtAuthFilter;
    }

```

```
@Bean
```

```
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .csrf(AbstractHttpConfigurer::disable)
            .sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/auth/login", "/v3/api-docs/**", "/swagger-ui/**").permitAll()
                .requestMatchers("/auth/register").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }

```

```
@Bean
```

```
    public AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws
Exception {
        return config.getAuthenticationManager();
    }

```

```
@Bean
```

```
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}

```

What Changed in Your File:

1. **Injected JwtAuthenticationFilter:** Active via constructor injection so Spring Security runs it on incoming requests.
 2. **Added .addFilterBefore(...):** Ensures Spring validates the JWT token before attempting standard authentication.
 3. **Added @EnableMethodSecurity:** Allows you to use annotations like `@PreAuthorize("hasRole('ADMIN')")` directly on controller methods across your application.
 4. **Defined PasswordEncoder Bean:** Ensures BCryptPasswordEncoder is available for AuthService and DatabaseSeeder.
-

```
User prompt: C:\Users\yasser\Documents\stage\Smart_ERP_Core\backend> .\mvnw.cmd spring-boot:run
[INFO] Scanning for projects... [INFO] [INFO] -----<
com.smarterp:smart-erp-core >----- [INFO] Building 0.0.1-SNAPSHOT [INFO] from pom.xml
[INFO] -----[ jar ]----- [INFO] [INFO] >>> spring-boot:4.1.0:run
(default-cli) > test-compile @ smart-erp-core >>> [INFO] [INFO] --- resources:3.5.0:resources
(default-resources) @ smart-erp-core --- [INFO] Copying 1 resource from src\main\resources to
target\classes [INFO] Copying 0 resource from src\main\resources to target\classes [INFO] [INFO] ---
compiler:3.15.0:compile (default-compile) @ smart-erp-core --- [INFO] Recompiling the module because of
changed source code. [INFO] Compiling 43 source files with javac [debug parameters release 21] to
target\classes [INFO]
/C:/Users/yasser/Documents/stage/Smart_ERP_Core/backend/src/main/java/com/smarterp/shared/security/
JwtAuthenticationFilter.java: Some input files use or override a deprecated API. [INFO]
/C:/Users/yasser/Documents/stage/Smart_ERP_Core/backend/src/main/java/com/smarterp/shared/security/
JwtAuthenticationFilter.java: Recompile with -Xlint:deprecation for details. [INFO]
----- [ERROR] COMPILATION ERROR : [INFO]
----- [ERROR]
/C:/Users/yasser/Documents/stage/Smart_ERP_Core/backend/src/main/java/com/smarterp/security/service/
AuthService.java:[11,37] cannot find symbol symbol: class BadRequestException location: package
com.smarterp.shared.exception [ERROR]
/C:/Users/yasser/Documents/stage/Smart_ERP_Core/backend/src/main/java/com/smarterp/security/service/
AuthService.java:[12,37] cannot find symbol symbol: class ResourceNotFoundException location:
package com.smarterp.shared.exception [ERROR]
/C:/Users/yasser/Documents/stage/Smart_ERP_Core/backend/src/main/java/com/smarterp/shared/config/D
atabaseSeeder.java:[3,36] cannot find symbol symbol: class ERole location: package
com.smarterp.security.domain [ERROR]
/C:/Users/yasser/Documents/stage/Smart_ERP_Core/backend/src/main/java/com/smarterp/shared/security/
JwtAuthenticationFilter.java:[38,40] cannot find symbol symbol: method validateToken(java.lang.String)
location: variable jwtUtils of type com.smarterp.shared.security.JwtUtils [ERROR]
/C:/Users/yasser/Documents/stage/Smart_ERP_Core/backend/src/main/java/com/smarterp/security/service/
AuthService.java:[47,23] cannot find symbol symbol: class BadRequestException location: class
com.smarterp.security.service.AuthService [ERROR]
/C:/Users/yasser/Documents/stage/Smart_ERP_Core/backend/src/main/java/com/smarterp/security/service/
AuthService.java:[51,40] cannot find symbol symbol: class ResourceNotFoundException location: class
com.smarterp.security.service.AuthService [ERROR]
/C:/Users/yasser/Documents/stage/Smart_ERP_Core/backend/src/main/java/com/smarterp/security/service/
AuthService.java:[77,40] cannot find symbol symbol: class ResourceNotFoundException location: class
com.smarterp.security.service.AuthService [ERROR]
/C:/Users/yasser/Documents/stage/Smart_ERP_Core/backend/src/main/java/com/smarterp/security/service/
```

AuthService.java:[93,40] cannot find symbol symbol: class ResourceNotFoundException location: class com.smarterp.security.service.AuthService [ERROR]
/C:/Users/yasser/Documents/stage/Smart_ERP_Core/backend/src/main/java/com/smarterp/shared/config/DatabaseSeeder.java:[28,31] cannot find symbol symbol: variable ERole location: class com.smarterp.config.DatabaseSeeder [ERROR]
/C:/Users/yasser/Documents/stage/Smart_ERP_Core/backend/src/main/java/com/smarterp/shared/config/DatabaseSeeder.java:[28,14] cannot find symbol symbol: class ERole location: class com.smarterp.config.DatabaseSeeder [ERROR]
/C:/Users/yasser/Documents/stage/Smart_ERP_Core/backend/src/main/java/com/smarterp/shared/config/DatabaseSeeder.java:[39,56] cannot find symbol symbol: variable ERole location: class com.smarterp.config.DatabaseSeeder [INFO] 11 errors [INFO]
----- [INFO] -----
[INFO] BUILD FAILURE [INFO] ----- [INFO] Total time: 2.418 s [INFO] Finished at: 2026-08-05T23:59:07+02:00 [INFO]
----- [ERROR] Failed to execute goal
org.apache.maven.plugins:maven-compiler-plugin:3.15.0:compile (default-compile) on project smart-erp-core: Compilation failure: Compilation failure: [ERROR]
/C:/Users/yasser/Documents/stage/Smart_ERP_Core/backend/src/main/java/com/smarterp/security/service/AuthService.java:[11,37] cannot find symbol [ERROR] symbol: class BadRequestException [ERROR] location: package com.smarterp.shared.exception [ERROR]
/C:/Users/yasser/Documents/stage/Smart_ERP_Core/backend/src/main/java/com/smarterp/security/service/AuthService.java:[12,37] cannot find symbol [ERROR] symbol: class ResourceNotFoundException [ERROR] location: package com.smarterp.shared.exception [ERROR]
/C:/Users/yasser/Documents/stage/Smart_ERP_Core/backend/src/main/java/com/smarterp/shared/config/DatabaseSeeder.java:[3,36] cannot find symbol [ERROR] symbol: class ERole [ERROR] location: package com.smarterp.security.domain [ERROR]
/C:/Users/yasser/Documents/stage/Smart_ERP_Core/backend/src/main/java/com/smarterp/shared/security/JwtAuthenticationFilter.java:[38,40] cannot find symbol [ERROR] symbol: method validateToken(java.lang.String) [ERROR] location: variable jwtUtils of type com.smarterp.shared.security.JwtUtils [ERROR]
/C:/Users/yasser/Documents/stage/Smart_ERP_Core/backend/src/main/java/com/smarterp/security/service/AuthService.java:[47,23] cannot find symbol [ERROR] symbol: class BadRequestException [ERROR] location: class com.smarterp.security.service.AuthService [ERROR]
/C:/Users/yasser/Documents/stage/Smart_ERP_Core/backend/src/main/java/com/smarterp/security/service/AuthService.java:[51,40] cannot find symbol [ERROR] symbol: class ResourceNotFoundException [ERROR] location: class com.smarterp.security.service.AuthService [ERROR]
/C:/Users/yasser/Documents/stage/Smart_ERP_Core/backend/src/main/java/com/smarterp/security/service/AuthService.java:[77,40] cannot find symbol [ERROR] symbol: class ResourceNotFoundException [ERROR] location: class com.smarterp.security.service.AuthService [ERROR]
/C:/Users/yasser/Documents/stage/Smart_ERP_Core/backend/src/main/java/com/smarterp/security/service/AuthService.java:[93,40] cannot find symbol [ERROR] symbol: class ResourceNotFoundException [ERROR] location: class com.smarterp.security.service.AuthService [ERROR]
/C:/Users/yasser/Documents/stage/Smart_ERP_Core/backend/src/main/java/com/smarterp/shared/config/DatabaseSeeder.java:[28,31] cannot find symbol [ERROR] symbol: variable ERole [ERROR] location: class com.smarterp.config.DatabaseSeeder [ERROR]
/C:/Users/yasser/Documents/stage/Smart_ERP_Core/backend/src/main/java/com/smarterp/shared/config/DatabaseSeeder.java:[28,14] cannot find symbol [ERROR] symbol: class ERole [ERROR] location: class com.smarterp.config.DatabaseSeeder [ERROR]
/C:/Users/yasser/Documents/stage/Smart_ERP_Core/backend/src/main/java/com/smarterp/shared/config/DatabaseSeeder.java:[39,56] cannot find symbol [ERROR] symbol: variable ERole [ERROR] location: class com.smarterp.config.DatabaseSeeder [ERROR] -> [Help 1] [ERROR] [ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch. [ERROR] Re-run Maven using the -X switch to enable

full debug logging. [ERROR] [ERROR] For more information about the errors and possible solutions, please read the following articles: [ERROR] [Help 1]
<http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException>

Response: The build failed because three files are missing from your project and JwtUtils is missing the validateToken method.

Create the following files and update JwtUtils.java to fix all 11 errors.

1. Create BadRequestException.java

Create this file in src/main/java/com/smarterp/shared/exception/BadRequestException.java:

```
package com.smarterp.shared.exception;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ResponseStatus;

@ResponseStatus(HttpStatus.BAD_REQUEST)
public class BadRequestException extends RuntimeException {
    public BadRequestException(String message) {
        super(message);
    }
}
```

2. Create ResourceNotFoundException.java

Create this file in src/main/java/com/smarterp/shared/exception/ResourceNotFoundException.java:

```
package com.smarterp.shared.exception;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ResponseStatus;

@ResponseStatus(HttpStatus.NOT_FOUND)
public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String message) {
        super(message);
    }
}
```

3. Create ERole.java

Create this file in src/main/java/com/smarterp/security/domain/ERole.java:

```
package com.smarterp.security.domain;

public enum ERole {
```

```

    ROLE_ADMIN,
    ROLE_MANAGER,
    ROLE_EMPLOYEE
}

```

4. Add validateToken to JwtUtils.java

Open src/main/java/com/smarterp/shared/security/JwtUtils.java and add the missing method:

```

public boolean validateToken(String token) {
    try {
        Jwts.parserBuilder()
            .setSigningKey(key())
            .build()
            .parseClaimsJws(token);
        return true;
    } catch (JwtException | IllegalArgumentException e) {
        return false;
    }
}

```

(Note: If your secret key method in JwtUtils is named getSigningKey() instead of key(), adjust the call accordingly).

Once these files are in place, run:

```

.\mvnw.cmd clean spring-boot:run

```

User prompt: package com.smarterp.shared.security; import io.jsonwebtoken.Claims; import io.jsonwebtoken.Jwts; import io.jsonwebtoken.io.Decoders; import io.jsonwebtoken.security.Keys; import org.springframework.beans.factory.annotation.Value; import org.springframework.security.core.userdetails.UserDetails; import org.springframework.stereotype.Component; import javax.crypto.SecretKey; import java.util.Date; import java.util.HashMap; import java.util.Map; import java.util.function.Function; @Component public class JwtUtils { @Value("\${application.security.jwt.secret-key}") private String secretKey; @Value("\${application.security.jwt.expiration-ms:86400000}") private long jwtExpiration; public String extractUsername(String token) { return extractClaim(token, Claims::getSubject); } public <T> T extractClaim(String token, Function<Claims, T> claimsResolver) { final Claims claims = extractAllClaims(token); return claimsResolver.apply(claims); } public String generateToken(UserDetails userDetails) { return generateToken(new HashMap<>(), userDetails); } public String generateToken(Map<String, Object> extraClaims, UserDetails userDetails) { return Jwts.builder().claims(extraClaims).subject(userDetails.getUsername()).issuedAt(new Date(System.currentTimeMillis())).expiration(new Date(System.currentTimeMillis() + jwtExpiration)).signWith(getSigningKey()).compact(); } public boolean isTokenValid(String token, UserDetails userDetails) { final String username = extractUsername(token); return (username.equals(userDetails.getUsername())) && !isTokenExpired(token); } private boolean isTokenExpired(String token) { return extractExpiration(token).before(new Date()); } private Date

```

extractExpiration(String token) {      return extractClaim(token, Claims::getExpiration); } private Claims
extractAllClaims(String token) {      return Jwts.parser()                .verifyWith(getSigningKey())
.build()                .parseSignedClaims(token)                .getPayload(); } private SecretKey
getSigningKey() {      byte[] keyBytes = Decoders.BASE64.decode(secretKey);      return
Keys.hmacShaKeyFor(keyBytes); } } this is my util

```

Response: Here is your updated JwtUtils.java with the missing validateToken(String token) method added using the JJWT 0.12.x API format.

This will resolve the validateToken error in JwtAuthenticationFilter.

```

package com.smarterp.shared.security;

```

```

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.JwtException;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.io.Decoders;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Component;

```

```

import javax.crypto.SecretKey;
import java.util.Date;
import java.util.HashMap;
import java.util.Map;
import java.util.function.Function;

```

```

@Component

```

```

public class JwtUtils {

```

```

    @Value("${application.security.jwt.secret-key}")
    private String secretKey;

```

```

    @Value("${application.security.jwt.expiration-ms:86400000}")
    private long jwtExpiration;

```

```

    public String extractUsername(String token) {
        return extractClaim(token, Claims::getSubject);
    }

```

```

    public <T> T extractClaim(String token, Function<Claims, T> claimsResolver) {
        final Claims claims = extractAllClaims(token);
        return claimsResolver.apply(claims);
    }

```

```

    public String generateToken(UserDetails userDetails) {
        return generateToken(new HashMap<>(), userDetails);
    }

```

```

    public String generateToken(Map<String, Object> extraClaims, UserDetails userDetails) {

```

```

        return Jwts.builder()
            .claims(extraClaims)
            .subject(userDetails.getUsername())
            .issuedAt(new Date(System.currentTimeMillis()))
            .expiration(new Date(System.currentTimeMillis() + jwtExpiration))
            .signWith(getSigningKey())
            .compact();
    }

    public boolean validateToken(String token) {
        try {
            Jwts.parser()
                .verifyWith(getSigningKey())
                .build()
                .parseSignedClaims(token);
            return true;
        } catch (JwtException | IllegalArgumentException e) {
            return false;
        }
    }

    public boolean isTokenValid(String token, UserDetails userDetails) {
        final String username = extractUsername(token);
        return (username.equals(userDetails.getUsername())) && !isTokenExpired(token);
    }

    private boolean isTokenExpired(String token) {
        return extractExpiration(token).before(new Date());
    }

    private Date extractExpiration(String token) {
        return extractClaim(token, Claims::getExpiration);
    }

    private Claims extractAllClaims(String token) {
        return Jwts.parser()
            .verifyWith(getSigningKey())
            .build()
            .parseSignedClaims(token)
            .getPayload();
    }

    private SecretKey getSigningKey() {
        byte[] keyBytes = Decoders.BASE64.decode(secretKey);
        return Keys.hmacShaKeyFor(keyBytes);
    }
}

```

Verification

Once you replace JwtUtils.java and ensure BadRequestException, ResourceNotFoundException, and

ERole exist, execute:

.\mvnw.cmd clean spring-boot:run

```
User prompt: package com.smarterp.config; import com.smarterp.security.domain.Role; import
com.smarterp.security.domain.RoleName; import com.smarterp.security.domain.User; import
com.smarterp.security.repository.RoleRepository; import com.smarterp.security.repository.UserRepository;
import org.springframework.boot.CommandLineRunner; import
org.springframework.security.crypto.password.PasswordEncoder; import
org.springframework.stereotype.Component; @Component public class DatabaseSeeder implements
CommandLineRunner {    private final RoleRepository roleRepository;    private final UserRepository
userRepository;    private final PasswordEncoder passwordEncoder;    public
DatabaseSeeder(RoleRepository roleRepository, UserRepository userRepository, PasswordEncoder
passwordEncoder) {        this.roleRepository = roleRepository;        this.userRepository = userRepository;
        this.passwordEncoder = passwordEncoder;    }    @Override    public void run(String... args) {        // 1.
Seed Roles        for (RoleName roleName : ERole.values()) {            if
(roleRepository.findByName(roleName).isEmpty()) {                Role role = new Role();
role.setName(roleName);                role.setDescription(roleName.name() + " access");
roleRepository.save(role);            }        }        // 2. Seed Default System Admin        if
(!userRepository.existsByEmail("admin@smarterp.com")) {            Role adminRole =
roleRepository.findByName(ERole.ROLE_ADMIN)                .orElseThrow(() -> new
RuntimeException("Admin role not found"));            User admin = new User();
admin.setEmail("admin@smarterp.com");
admin.setPasswordHash(passwordEncoder.encode("Admin@123456"));            admin.setRole(adminRole);
admin.setIsActive(true);            userRepository.save(admin);        }    } }
```

Response: The compiler error in DatabaseSeeder.java was caused by a mismatch: you imported **RoleName** at the top, but used **ERole** inside the code.

Here is the corrected DatabaseSeeder.java using RoleName:

```
package com.smarterp.config;

import com.smarterp.security.domain.Role;
import com.smarterp.security.domain.RoleName;
import com.smarterp.security.domain.User;
import com.smarterp.security.repository.RoleRepository;
import com.smarterp.security.repository.UserRepository;
import org.springframework.boot.CommandLineRunner;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Component;

@Component
public class DatabaseSeeder implements CommandLineRunner {

    private final RoleRepository roleRepository;
    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;
```

```

public DatabaseSeeder(RoleRepository roleRepository, UserRepository userRepository,
PasswordEncoder passwordEncoder) {
    this.roleRepository = roleRepository;
    this.userRepository = userRepository;
    this.passwordEncoder = passwordEncoder;
}

@Override
public void run(String... args) {
    // 1. Seed Roles
    for (RoleName roleName : RoleName.values()) {
        if (roleRepository.findByName(roleName).isEmpty()) {
            Role role = new Role();
            role.setName(roleName);
            role.setDescription(roleName.name() + " access");
            roleRepository.save(role);
        }
    }

    // 2. Seed Default System Admin
    if (!userRepository.existsByEmail("admin@smarterp.com")) {
        Role adminRole = roleRepository.findByName(RoleName.ROLE_ADMIN)
            .orElseThrow(() -> new RuntimeException("Admin role not found"));

        User admin = new User();
        admin.setEmail("admin@smarterp.com");
        admin.setPasswordHash(passwordEncoder.encode("Admin@123456"));
        admin.setRole(adminRole);
        admin.setIsActive(true);

        userRepository.save(admin);
        System.out.println(">>> Seeded Super Admin account: admin@smarterp.com / Admin@123456");
    }
}
}

```

Also Update AuthService.java

Since your enum is named RoleName (not ERole), update AuthService.java to use RoleName as well:

```

package com.smarterp.security.service;

import com.smarterp.security.domain.Role;
import com.smarterp.security.domain.RoleName;
import com.smarterp.security.domain.User;
import com.smarterp.security.dto.AuthResponse;
import com.smarterp.security.dto.LoginRequest;
import com.smarterp.security.dto.RegisterRequest;
import com.smarterp.security.dto.UserDTO;
import com.smarterp.security.repository.RoleRepository;

```

```
import com.smarterp.security.repository.UserRepository;
import com.smarterp.shared.exception.BadRequestException;
import com.smarterp.shared.exception.ResourceNotFoundException;
import com.smarterp.shared.security.JwtUtils;
import com.smarterp.shared.security.SecurityUser;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
```

@Service

```
public class AuthService {
```

```
    private final UserRepository userRepository;
    private final RoleRepository roleRepository;
    private final PasswordEncoder passwordEncoder;
    private final JwtUtils jwtUtils;
    private final AuthenticationManager authenticationManager;
```

```
    public AuthService(
        UserRepository userRepository,
        RoleRepository roleRepository,
        PasswordEncoder passwordEncoder,
        JwtUtils jwtUtils,
        AuthenticationManager authenticationManager
```

```
    ) {
        this.userRepository = userRepository;
        this.roleRepository = roleRepository;
        this.passwordEncoder = passwordEncoder;
        this.jwtUtils = jwtUtils;
        this.authenticationManager = authenticationManager;
    }
```

@Transactional

```
    public AuthResponse register(RegisterRequest request) {
        if (userRepository.existsByEmail(request.email())) {
            throw new BadRequestException("Email address is already in use.");
        }
```

```
        Role role = roleRepository.findByName(request.role())
            .orElseThrow(() -> new ResourceNotFoundException("Error: Role " + request.role() + " was not found."));
```

```
        User user = new User();
        user.setEmail(request.email());
        user.setPasswordHash(passwordEncoder.encode(request.password()));
        user.setRole(role);
        user.setIsActive(true);
```

```
        User savedUser = userRepository.save(user);
        SecurityUser securityUser = new SecurityUser(savedUser);
```

```

String jwtToken = jwtUtils.generateToken(securityUser);

return new AuthResponse(
    jwtToken,
    savedUser.getId(),
    savedUser.getEmail(),
    savedUser.getRole().getName().name()
);
}

public AuthResponse login(LoginRequest request) {
    authenticationManager.authenticate(
        new UsernamePasswordAuthenticationToken(request.email(), request.password())
    );

    User user = userRepository.findByEmail(request.email())
        .orElseThrow(() -> new ResourceNotFoundException("User not found with email: " +
request.email()));

    SecurityUser securityUser = new SecurityUser(user);
    String jwtToken = jwtUtils.generateToken(securityUser);

    return new AuthResponse(
        jwtToken,
        user.getId(),
        user.getEmail(),
        user.getRole().getName().name()
    );
}

@Transactional(readOnly = true)
public UserDTO getCurrentUser(String email) {
    User user = userRepository.findByEmail(email)
        .orElseThrow(() -> new ResourceNotFoundException("User not found with email: " + email));

    return new UserDTO(
        user.getId(),
        user.getEmail(),
        user.getRole().getName().name(),
        user.getIsActive()
    );
}
}

```

Now try building and running again:

```
.\mvnw.cmd clean spring-boot:run
```

```
User prompt: package com.smarterp.shared.security; import io.jsonwebtoken.Claims; import
io.jsonwebtoken.Jwts; import io.jsonwebtoken.io.Decoders; import io.jsonwebtoken.security.Keys; import
org.springframework.beans.factory.annotation.Value; import
org.springframework.security.core.userdetails.UserDetails; import
org.springframework.stereotype.Component; import javax.crypto.SecretKey; import java.util.Date; import
java.util.HashMap; import java.util.Map; import java.util.function.Function; @Component public class
JwtUtils { @Value("${application.security.jwt.secret-key}") private String secretKey;
@Value("${application.security.jwt.expiration-ms:86400000}") private long jwtExpiration; public String
extractUsername(String token) { return extractClaim(token, Claims::getSubject); } public <T> T
extractClaim(String token, Function<Claims, T> claimsResolver) { final Claims claims =
extractAllClaims(token); return claimsResolver.apply(claims); } public String
generateToken(UserDetails userDetails) { return generateToken(new HashMap<>(), userDetails); }
public String generateToken(Map<String, Object> extraClaims, UserDetails userDetails) { return
Jwts.builder() .claims(extraClaims) .subject(userDetails.getUsername())
.issuedAt(new Date(System.currentTimeMillis())) .expiration(new Date(System.currentTimeMillis()
+ jwtExpiration)) .signWith(getSigningKey()) .compact(); } public boolean
isTokenValid(String token, UserDetails userDetails) { final String username = extractUsername(token);
return (username.equals(userDetails.getUsername())) && !isTokenExpired(token); } private boolean
isTokenExpired(String token) { return extractExpiration(token).before(new Date()); } private Date
extractExpiration(String token) { return extractClaim(token, Claims::getExpiration); } private Claims
extractAllClaims(String token) { return Jwts.parser() .verifyWith(getSigningKey())
.build() .parseSignedClaims(token) .getPayload(); } private SecretKey
getSigningKey() { byte[] keyBytes = Decoders.BASE64.decode(secretKey); return
Keys.hmacShaKeyFor(keyBytes); } public boolean validateToken(String token) { try {
Jwts.parserBuilder() .setSigningKey(key()) .build() .parseClaimsJws(token);
return true; } catch (JwtException | IllegalArgumentException e) { return false; } } }[ERROR]
/C:/Users/yasser/Documents/stage/Smart_ERP_Core/backend/src/main/java/com/smarterp/shared/security/
JwtUtils.java:[81,14] cannot find symbol [ERROR] symbol: class JwtException [ERROR] location: class
com.smarterp.shared.security.JwtUtils [ERROR] -> [Help 1] [ERROR] [ERROR] To see the full stack trace
of the errors, re-run Maven with the -e switch. [ERROR] Re-run Maven using the -X switch to enable full
debug logging. [ERROR] [ERROR] For more information about the errors and possible solutions, please
read the following articles: [ERROR] [Help 1]
http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException PS
C:\Users\yasser\Documents\stage\Smart_ERP_Core\backend>
```

Response: Here is the complete, updated **JwtUtils.java**.

Two things were fixed:

1. Added the missing import: `import io.jsonwebtoken.JwtException;`
2. Updated `validateToken()` to use the JJWT 0.12.x syntax (`Jwts.parser().verifyWith(...)`), matching the rest of your class.

```
package com.smarterp.shared.security;
```

```
import io.jsonwebtoken.Claims;
import io.jsonwebtoken.JwtException;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.io.Decoders;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
```

```
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Component;
```

```
import javax.crypto.SecretKey;
import java.util.Date;
import java.util.HashMap;
import java.util.Map;
import java.util.function.Function;
```

```
@Component
```

```
public class JwtUtils {
```

```
    @Value("${application.security.jwt.secret-key}")
    private String secretKey;
```

```
    @Value("${application.security.jwt.expiration-ms:86400000}")
    private long jwtExpiration;
```

```
    public String extractUsername(String token) {
        return extractClaim(token, Claims::getSubject);
    }
```

```
    public <T> T extractClaim(String token, Function<Claims, T> claimsResolver) {
        final Claims claims = extractAllClaims(token);
        return claimsResolver.apply(claims);
    }
```

```
    public String generateToken(UserDetails userDetails) {
        return generateToken(new HashMap<>(), userDetails);
    }
```

```
    public String generateToken(Map<String, Object> extraClaims, UserDetails userDetails) {
        return Jwts.builder()
            .claims(extraClaims)
            .subject(userDetails.getUsername())
            .issuedAt(new Date(System.currentTimeMillis()))
            .expiration(new Date(System.currentTimeMillis() + jwtExpiration))
            .signWith(getSigningKey())
            .compact();
    }
```

```
    public boolean validateToken(String token) {
        try {
            Jwts.parser()
                .verifyWith(getSigningKey())
                .build()
                .parseSignedClaims(token);
            return true;
        } catch (JwtException | IllegalArgumentException e) {
            return false;
        }
    }
}
```

```

public boolean isValidToken(String token, UserDetails userDetails) {
    final String username = extractUsername(token);
    return (username.equals(userDetails.getUsername())) && !isTokenExpired(token);
}

private boolean isTokenExpired(String token) {
    return extractExpiration(token).before(new Date());
}

private Date extractExpiration(String token) {
    return extractClaim(token, Claims::getExpiration);
}

private Claims extractAllClaims(String token) {
    return Jwts.parser()
        .verifyWith(getSigningKey())
        .build()
        .parseSignedClaims(token)
        .getPayload();
}

private SecretKey getSigningKey() {
    byte[] keyBytes = Decoders.BASE64.decode(secretKey);
    return Keys.hmacShaKeyFor(keyBytes);
}
}

```

Replace the contents of JwtUtils.java with this code and re-run:

```
.\mvnw.cmd clean spring-boot:run
```

User prompt: he declared package "com.smarterp.shared.exception" does not match the expected package "main.java.com.smarterp.shared.exception" Java(536871240)

Response: **Do not change your package declaration to main.java....**

Your package statement (package com.smarterp.shared.exception;) is 100% correct. This error happens because VS Code's Java extension misidentified your source root—it treated src/ or your root folder as the source directory instead of src/main/java.

Step-by-Step Fix in VS Code

Option 1: Clean the Java Workspace Cache (Most common fix)

1. Press Ctrl + Shift + P (or Cmd + Shift + P on Mac) in VS Code.
2. Type **Java: Clean Java Language Server Workspace** and press **Enter**.
3. Select **Restart and delete**.

4. Wait 15–30 seconds for VS Code to re-index your pom.xml project structure.

Option 2: Verify Open Folder Location

Make sure you opened the **backend** folder directly in VS Code (where the pom.xml resides), rather than a higher parent directory.

- Go to **File** → **Open Folder** and select C:\Users\yasser\Documents\stage\Smart_ERP_Core\backend.

Option 3: Force Folder as Source Root

1. Ensure the file is physically saved at:
backend/src/main/java/com/smarterp/shared/exception/BadRequestException.java
2. In VS Code, look at the bottom status bar or right-click the src/main/java folder in the Explorer side panel → select **Mark as Source Root** (if prompted by the Java Extension).

Once VS Code reloads the Java Language Server, the red squiggly line will disappear, and `.\mvnw.cmd spring-boot:run` will compile cleanly.

[illegible]

```
'passwordEncoder' since there is already [Root bean: class=null; scope=; abstract=false; lazyInit=null;
autowireMode=3; dependencyCheck=0; autowireCandidate=true; primary=false; fallback=false;
factoryBeanName=securityConfig; factoryMethodName=passwordEncoder; initMethodNames=null;
destroyMethodNames=[(inferred)]; defined in class path resource
[com/smarterp/security/config/SecurityConfig.class]] bound. 2026-08-06T00:13:28.775+02:00 INFO 17576
--- [smart-erp-core] [      main] .s.b.a.l.ConditionEvaluationReportLogger : Error starting
ApplicationContext. To display the condition evaluation report re-run your application with 'debug' enabled.
2026-08-06T00:13:28.793+02:00 ERROR 17576 --- [smart-erp-core] [      main]
o.s.b.d.LoggingFailureAnalysisReporter : ***** APPLICATION FAILED TO START
*****
***** Description: The bean 'passwordEncoder', defined in class path resource
[com/smarterp/shared/config/PasswordEncoderConfig.class], could not be registered. A bean with that
name has already been defined in class path resource [com/smarterp/security/config/SecurityConfig.class]
and overriding is disabled. Action: Consider renaming one of the beans or enabling overriding by setting
spring.main.allow-bean-definition-overriding=true [INFO]
----- [INFO] BUILD FAILURE [INFO]
----- [INFO] Total time: 2.278 s [INFO] Finished at:
2026-08-06T00:13:28+02:00 [INFO] ----- [ERROR] Failed
to execute goal org.springframework.boot:spring-boot-maven-plugin:4.1.0:run (default-cli) on project
smart-erp-core: Process terminated with exit code: 1 -> [Help 1] [ERROR] [ERROR] To see the full stack
trace of the errors, re-run Maven with the -e switch. [ERROR] Re-run Maven using the -X switch to enable
full debug logging. [ERROR] [ERROR] For more information about the errors and possible solutions, please
read the following articles: [ERROR] [Help 1]
http://cwiki.apache.org/confluence/display/MAVEN/MojoExecutionException PS
C:\Users\lyasser\Documents\stage\Smart_ERP_Core\backend>
```

Response: You have a duplicate @Bean definition for passwordEncoder. It exists in both PasswordEncoderConfig.java and SecurityConfig.java.

To fix this cleanly, remove the passwordEncoder() method from SecurityConfig.java and let PasswordEncoderConfig.java manage it.

Updated SecurityConfig.java

Open src/main/java/com/smarterp/security/config/SecurityConfig.java and update it:

```
package com.smarterp.security.config;
```

```
import com.smarterp.shared.security.JwtAuthenticationFilter;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import
org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
```

```

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtAuthFilter;

    public SecurityConfig(JwtAuthenticationFilter jwtAuthFilter) {
        this.jwtAuthFilter = jwtAuthFilter;
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .csrf(AbstractHttpConfigurer::disable)
            .sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/auth/login", "/v3/api-docs/**", "/swagger-ui/**").permitAll()
                .requestMatchers("/auth/register").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }

    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws
Exception {
        return config.getAuthenticationManager();
    }
}

```

Run the Application

Save the file and start your server:

```
.\mvnw.cmd spring-boot:run
```

```

User prompt: C:\Users\yasser\Documents\stage\Smart_ERP_Core\backend> .\mvnw.cmd spring-boot:run
[INFO] Scanning for projects... [INFO] [INFO] -----< com.smarterp:smart-erp-core
>----- [INFO] Building 0.0.1-SNAPSHOT [INFO] from pom.xml [INFO] -----[
jar ]----- [INFO] [INFO] >>> spring-boot:4.1.0:run (default-cli) > test-compile @
smart-erp-core >>> [INFO] [INFO] --- resources:3.5.0:resources (default-resources) @ smart-erp-core ---
[INFO] Copying 1 resource from src\main\resources to target\classes [INFO] Copying 0 resource from
src\main\resources to target\classes [INFO] [INFO] --- compiler:3.15.0:compile (default-compile) @
smart-erp-core --- [INFO] Nothing to compile - all classes are up to date. [INFO] [INFO] ---

```

[illegible]

*<http://cwiki.apache.org/confluence/display/MAVEN/MojoExecutionException> PS
C:\Users\yasser\Documents\stage\Smart_ERP_Core\backend>*

Response: Create UserDetailsServiceImpl.java to provide Spring Security with the missing UserDetailsService bean.

Create this file in src/main/java/com/smarterp/security/service/UserDetailsServiceImpl.java:

```
package com.smarterp.security.service;

import com.smarterp.security.domain.User;
import com.smarterp.security.repository.UserRepository;
import com.smarterp.shared.security.SecurityUser;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

@Service
public class UserDetailsServiceImpl implements UserDetailsService {

    private final UserRepository userRepository;

    public UserDetailsServiceImpl(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @Override
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        User user = userRepository.findByEmail(username)
            .orElseThrow(() -> new UsernameNotFoundException("User not found with email: " + username));

        return new SecurityUser(user);
    }
}
```

Run the Application

Once saved, start the application:

```
.\mvnw.cmd spring-boot:run
```

User prompt: kit worked what should i test

Response: Test these five scenarios using Postman, Bruno, or curl to verify your security, database seeding, and Role-Based Access Control (RBAC).

Test 1: Log in as Super Admin (Verify Database Seeding)

Send a POST request to verify the DatabaseSeeder created the default admin account.

- **URL:** http://localhost:8080/auth/login
- **Method:** POST
- **Headers:** Content-Type: application/json
- **Request Body:**

```
{  "email": "admin@smarterp.com",  "password": "Admin@123456"}
```
- **Expected Result:** Status 200 OK. Response contains a JWT token and role ROLE_ADMIN. **Copy this token for the next steps.**

Test 2: Unauthenticated Registration Attempt (Security Check)

Verify that anyone trying to register a user without an admin token is blocked.

- **URL:** http://localhost:8080/auth/register
- **Method:** POST
- **Headers:** Content-Type: application/json (*No Authorization Header*)
- **Request Body:**

```
{  "email": "hacker@smarterp.com",  "password": "Password@123",  "role": "ROLE_ADMIN"}
```
- **Expected Result:** Status 401 Unauthorized or 403 Forbidden.

Test 3: Admin Registers a New Employee (Authorized Creation)

Use the Admin JWT token from **Test 1** to register a standard employee account.

- **URL:** http://localhost:8080/auth/register
- **Method:** POST
- **Headers:**
 - Content-Type: application/json
 - Authorization: Bearer <ADMIN_JWT_TOKEN_FROM_TEST_1>
- **Request Body:**

```
{  "email": "john.employee@smarterp.com",  "password": "Employee@123456",  "role": "ROLE_EMPLOYEE"}
```
- **Expected Result:** Status 200 OK. Returns the created user details with ROLE_EMPLOYEE.

Test 4: Employee Login Test

Verify the newly registered employee can log in successfully.

- **URL:** http://localhost:8080/auth/login
- **Method:** POST
- **Headers:** Content-Type: application/json
- **Request Body:**

```
{
  "email": "john.employee@smarterp.com",
  "password": "Employee@123456"
}
```
- **Expected Result:** Status 200 OK. Returns a JWT token with role ROLE_EMPLOYEE. **Copy this token.**

Test 5: Privilege Escalation Protection (RBAC Check)

Try using the **Employee** token from **Test 4** to call the /auth/register endpoint.

- **URL:** http://localhost:8080/auth/register
- **Method:** POST
- **Headers:**
 - Content-Type: application/json
 - Authorization: Bearer <EMPLOYEE_JWT_TOKEN_FROM_TEST_4>
- **Request Body:**

```
{
  "email": "test@smarterp.com",
  "password": "Password123!",
  "role": "ROLE_EMPLOYEE"
}
```
- **Expected Result:** Status 403 Forbidden. This confirms that regular employees cannot create new accounts in your ERP.

Which API client (Postman, Bruno, or VS Code REST Client) are you using to run these tests?

User prompt: `<!doctype html><html lang="en"><head><title>HTTP Status 404 – Not Found</title><style type="text/css">body {font-family:Tahoma,Arial,sans-serif;} h1, h2, h3, b {color:white;background-color:#525D76;} h1 {font-size:22px;} h2 {font-size:16px;} h3 {font-size:14px;} p {font-size:12px;} a {color:black;} .line {height:1px;background-color:#525D76;border:none;}</style></head><body><h1>HTTP Status 404 – Not Found</h1></body></html>` test 1

Response: A **404 Not Found** error from Tomcat means Spring Boot received the request, but no @RestController mapping matched the URL and HTTP method you called.

Here is how to fix it:

Step 1: Check your AuthController.java

Ensure you have created AuthController.java inside com.smarterp.security.controller:

```
package com.smarterp.security.controller;

import com.smarterp.security.dto.AuthResponse;
import com.smarterp.security.dto.LoginRequest;
import com.smarterp.security.dto.RegisterRequest;
import com.smarterp.security.service.AuthService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/auth") // Check this prefix!
public class AuthController {

    private final AuthService authService;

    public AuthController(AuthService authService) {
        this.authService = authService;
    }

    @PostMapping("/login")
    public ResponseEntity<AuthResponse> login(@RequestBody LoginRequest request) {
        return ResponseEntity.ok(authService.login(request));
    }

    @PostMapping("/register")
    public ResponseEntity<AuthResponse> register(@RequestBody RegisterRequest request) {
        return ResponseEntity.ok(authService.register(request));
    }
}
```

Step 2: Verify Path Consistency

1. **Check Base Path/URL Prefix:**
 - If your AuthController uses `@RequestMapping("/auth")`, your POST URL is:
`http://localhost:8080/auth/login`
 - If your AuthController uses `@RequestMapping("/api/v1/auth")`, your POST URL is:
`http://localhost:8080/api/v1/auth/login`
2. **Check SecurityConfig.java:** Make sure the `.requestMatchers()` path in SecurityConfig.java matches the controller path exact pattern (e.g., `/auth/login` vs `/api/v1/auth/login`).
3. **Check Request Method:** Ensure your API client (Postman/Bruno) is explicitly set to **POST** (not GET).

Do you already have AuthController.java in your project, or did you need to create it?

```
User prompt: package com.smarterp.security.config; import
com.smarterp.shared.security.JwtAuthenticationFilter; import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration; import
org.springframework.security.authentication.AuthenticationManager; import
org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity; import
org.springframework.security.config.annotation.web.builders.HttpSecurity; import
org.springframework.security.config.annotation.web.configuration.EnableWebSecurity; import
org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer; import
org.springframework.security.config.http.SessionCreationPolicy; import
org.springframework.security.web.SecurityFilterChain; import
org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter; @Configuration
@EnableWebSecurity @EnableMethodSecurity public class SecurityConfig { private final
JwtAuthenticationFilter jwtAuthFilter; public SecurityConfig(JwtAuthenticationFilter jwtAuthFilter) {
this.jwtAuthFilter = jwtAuthFilter; } @Bean public SecurityFilterChain securityFilterChain(HttpSecurity
http) throws Exception { http .csrf(AbstractHttpConfigurer::disable)
.sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
.authorizeHttpRequests(auth -> auth .requestMatchers("/auth/login", "/v3/api-docs/**",
"/swagger-ui/**").permitAll() .requestMatchers("/auth/register").hasRole("ADMIN")
.anyRequest().authenticated() ) .addFilterBefore(jwtAuthFilter,
UsernamePasswordAuthenticationFilter.class); return http.build(); } @Bean public
AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws Exception {
return config.getAuthenticationManager(); } } here it says it has role "ADMIN" but package
com.smarterp.security.domain; public enum RoleName { ROLE_ADMIN, ROLE_HR_MANAGER,
ROLE_EMPLOYEE }
```

Response: In Spring Security, `.hasRole("ADMIN")` is **100% correct** when your enum value is `ROLE_ADMIN`.

Spring Security automatically prepends the `ROLE_` prefix to any string passed into `.hasRole()`.

How Spring Security Maps Roles

Method in SecurityConfig	What Spring Security Actually Checks	Matches RoleName.ROLE_ADMIN?
<code>.hasRole("ADMIN")</code>	<code>ROLE_ADMIN</code>	Yes (Correct)
<code>.hasRole("ROLE_ADMIN")</code>	<code>ROLE_ROLE_ADMIN</code>	No (Will fail with 403 Forbidden)
<code>.hasAuthority("ROLE_ADMIN")</code>	<code>ROLE_ADMIN</code>	Yes (Alternative syntax)

Key Requirement in SecurityUser.java

For `.hasRole("ADMIN")` to evaluate correctly, ensure your `SecurityUser` (or `UserDetails` implementation) returns the authority with the `ROLE_` prefix included when mapping your enum:

```
@Override
public Collection<? extends GrantedAuthority> getAuthorities() {
    return List.of(new SimpleGrantedAuthority(user.getRole().getName().name()));
    // user.getRole().getName().name() produces "ROLE_ADMIN"
}
```

Since your `RoleName` enum already includes `ROLE_ADMIN`, passing `user.getRole().getName().name()` directly into `SimpleGrantedAuthority` makes `.hasRole("ADMIN")` work seamlessly.

User prompt: `http://localhost:8080/api/v1/auth/login` it worked with this url

Response: Awesome, glad login is working!

Since your controller uses the `/api/v1` prefix, update the request matchers in **SecurityConfig.java** so Spring Security correctly permits login and protects registration under the `/api/v1` path.

Update SecurityConfig.java

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/v1/auth/login", "/v3/api-docs/**", "/swagger-ui/**").permitAll()
    .requestMatchers("/api/v1/auth/register").hasRole("ADMIN")
    .anyRequest().authenticated()
)
```

Continue Testing

Now you can proceed with testing the rest of the endpoints using the full `/api/v1` base URL:

1. **Test 2 (Unauthorized Register):** POST `http://localhost:8080/api/v1/auth/register` (without Bearer token) → Expect 401 Unauthorized or 403 Forbidden.
2. **Test 3 (Admin Registers Employee):** POST `http://localhost:8080/api/v1/auth/register` (with Admin Bearer token) → Expect 200 OK.
3. **Test 4 (Employee Login):** POST `http://localhost:8080/api/v1/auth/login` (using new employee credentials) → Expect 200 OK.
4. **Test 5 (Employee RBAC Check):** POST `http://localhost:8080/api/v1/auth/register` (with Employee Bearer token) → Expect 403 Forbidden.

User prompt: i tested all the 5 steps what the next step in my project

Response: The next logical step is to establish **JPA Auditing & Global Exception Handling**, then move into building your first core ERP business module: **HR & Employee Management**.

Phase 1: Infrastructure Foundations (Required for all ERP entities)

Before building business entities (Employees, Products, Invoices), set up automated database auditing and clean API error handling.

1. Implement JPA Auditing (Auditable Base Class)

Every table in an ERP system needs to track *who* created/updated a record and *when*.

Create BaseEntity.java in com.smarterp.shared.domain:

```
package com.smarterp.shared.domain;

import jakarta.persistence.Column;
import jakarta.persistence.EntityListeners;
import jakarta.persistence.MappedSuperclass;
import org.springframework.data.annotation.CreatedBy;
import org.springframework.data.annotation.CreatedDate;
import org.springframework.data.annotation.LastModifiedBy;
import org.springframework.data.annotation.LastModifiedDate;
import org.springframework.data.jpa.domain.support.AuditingEntityListener;

import java.time.LocalDateTime;

@EntityListeners(AuditingEntityListener.class)
@MappedSuperclass
public abstract class BaseEntity {

    @CreatedDate
    @Column(name = "created_at", nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @LastModifiedDate
    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    @CreatedBy
    @Column(name = "created_by", updatable = false)
    private String createdBy;

    @LastModifiedBy
    @Column(name = "updated_by")
    private String updatedBy;

    // Getters and Setters
}
```

Add @EnableJpaAuditing to your main class SmartErpCoreApplication.java.

2. Implement GlobalExceptionHandler

Ensure all application errors return uniform JSON responses rather than raw Spring or Tomcat stack traces.

Create GlobalExceptionHandler.java in com.smarterp.shared.exception:

```
package com.smarterp.shared.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<?> handleNotFound(ResourceNotFoundException ex) {
        return buildResponse(HttpStatus.NOT_FOUND, ex.getMessage());
    }

    @ExceptionHandler(BadRequestException.class)
    public ResponseEntity<?> handleBadRequest(BadRequestException ex) {
        return buildResponse(HttpStatus.BAD_REQUEST, ex.getMessage());
    }

    @ExceptionHandler(AccessDeniedException.class)
    public ResponseEntity<?> handleAccessDenied(AccessDeniedException ex) {
        return buildResponse(HttpStatus.FORBIDDEN, "Access denied: insufficient privileges.");
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<?> handleValidation(MethodArgumentNotValidException ex) {
        Map<String, String> errors = new HashMap<>();
        ex.getBindingResult().getFieldErrors().forEach(err ->
            errors.put(err.getField(), err.getDefaultMessage())
        );
        return ResponseEntity.badRequest().body(Map.of(
            "timestamp", LocalDateTime.now(),
            "status", 400,
            "errors", errors
        ));
    }

    private ResponseEntity<Map<String, Object>> buildResponse(HttpStatus status, String message) {
```

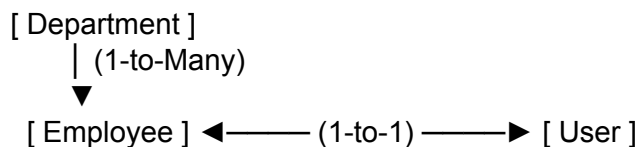
```

return ResponseEntity.status(status).body(Map.of(
    "timestamp", LocalDateTime.now(),
    "status", status.value(),
    "message", message
));
}
}

```

Phase 2: Core Domain — HR & Employee Management

With auth working, the User entity only represents login credentials. You now need the actual **HR Module** to manage employee profile data and organizational hierarchy.



Proposed Components to Build:

1. **Department Entity & Service:** (id, name, code, manager_id)
2. **Employee Entity & Service:** (id, firstName, lastName, phone, jobTitle, department, hireDate, user_id)
3. **Employee Management Endpoints:**
 - GET /api/v1/employees (Paginated list with filter by department/role)
 - GET /api/v1/employees/{id} (Detailed profile)
 - PUT /api/v1/employees/{id} (Update profile details)
 - PATCH /api/v1/employees/{id}/status (Activate/Deactivate account)

Would you like to set up the **Global Exception Handler & JPA Auditing** first, or jump straight into building the **HR / Employee module**?

User prompt: package com.smarterp.shared.exception; import org.springframework.http.HttpStatus; import org.springframework.http.ResponseEntity; import org.springframework.web.bind.MethodArgumentNotValidException; import org.springframework.web.bind.annotation.ExceptionHandler; import org.springframework.web.bind.annotation.RestControllerAdvice; import java.time.LocalDateTime; import java.util.HashMap; import java.util.Map; @RestControllerAdvice public class GlobalExceptionHandler { @ExceptionHandler(RuntimeException.class) public ResponseEntity<Map<String, Object>> handleRuntimeException(RuntimeException ex) { Map<String, Object> body = new HashMap<>(); body.put("timestamp", LocalDateTime.now()); body.put("status", HttpStatus.BAD_REQUEST.value()); body.put("error", "Bad Request"); body.put("message", ex.getMessage()); return new ResponseEntity<>(body, HttpStatus.BAD_REQUEST); } @ExceptionHandler(MethodArgumentNotValidException.class) public ResponseEntity<Map<String, Object>> handleValidationExceptions(MethodArgumentNotValidException ex) { Map<String, Object> errors = new HashMap<>(); ex.getBindingResult().getFieldErrors().forEach(error -> errors.put(error.getField(), error.getDefaultMessage())); Map<String, Object> body = new HashMap<>(); body.put("timestamp", LocalDateTime.now()); body.put("status",

```
HttpStatus.UNPROCESSABLE_ENTITY.value());    body.put("errors", errors);    return new  
ResponseEntity<>(body, HttpStatus.UNPROCESSABLE_ENTITY);    } } i already have the global expection
```

Response: That's great! Your GlobalExceptionHandler looks solid for handling validation errors and standardizing your error responses.

(Small tip: Right now, capturing all RuntimeExceptions as a 400 Bad Request will also catch things like database connection failures or missing resources and label them as "Bad Request". You can always add specific handlers like @ExceptionHandler(ResourceNotFoundException.class) later if you want them to return a 404 Not Found instead.)

Since your exception handling is in place, the next crucial step for an ERP is **JPA Auditing**. This automatically tracks *who* created or updated a record, and *when*, without you having to write that logic in every service.

Here is how to set it up in 3 quick files:

1. Create the BaseEntity

All future database tables (Employee, Invoice, Product, etc.) will extend this class to automatically inherit these columns.

Create BaseEntity.java in com.smarterp.shared.domain:

```
package com.smarterp.shared.domain;  
  
import jakarta.persistence.Column;  
import jakarta.persistence.EntityListeners;  
import jakarta.persistence.MappedSuperclass;  
import org.springframework.data.annotation.CreatedBy;  
import org.springframework.data.annotation.CreatedDate;  
import org.springframework.data.annotation.LastModifiedBy;  
import org.springframework.data.annotation.LastModifiedDate;  
import org.springframework.data.jpa.domain.support.AuditingEntityListener;  
  
import java.time.LocalDateTime;  
  
@MappedSuperclass  
@EntityListeners(AuditingEntityListener.class)  
public abstract class BaseEntity {  
  
    @CreatedDate  
    @Column(name = "created_at", nullable = false, updatable = false)  
    private LocalDateTime createdAt;  
  
    @LastModifiedDate  
    @Column(name = "updated_at")  
    private LocalDateTime updatedAt;  
  
    @CreatedBy
```

```

@Column(name = "created_by", updatable = false)
private String createdBy;

@LastModifiedBy
@Column(name = "updated_by")
private String updatedBy;

// Standard Getters and Setters
public LocalDateTime getCreatedAt() { return createdAt; }
public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }
public LocalDateTime getUpdatedAt() { return updatedAt; }
public void setUpdatedAt(LocalDateTime updatedAt) { this.updatedAt = updatedAt; }
public String getCreatedBy() { return createdBy; }
public void setCreatedBy(String createdBy) { this.createdBy = createdBy; }
public String getUpdatedBy() { return updatedBy; }
public void setUpdatedBy(String updatedBy) { this.updatedBy = updatedBy; }
}

```

2. Create AuditorAwareImpl

This tells Spring JPA *who* is currently logged in, so it can fill in the `@CreatedBy` and `@LastModifiedBy` fields using the JWT token.

Create AuditorAwareImpl.java in com.smarterp.shared.config:

```

package com.smarterp.shared.config;

import org.springframework.data.domain.AuditorAware;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Component;

import java.util.Optional;

@Component("auditorProvider")
public class AuditorAwareImpl implements AuditorAware<String> {

    @Override
    public Optional<String> getCurrentAuditor() {
        Authentication authentication = SecurityContextHolder.getContext().getAuthentication();

        if (authentication == null || !authentication.isAuthenticated() ||
            authentication.getPrincipal().equals("anonymousUser")) {
            return Optional.of("SYSTEM"); // Used for startup seeding like your Super Admin
        }

        return Optional.of(authentication.getName()); // Returns the logged-in user's email
    }
}

```

3. Enable JPA Auditing

Finally, tell Spring Boot to turn this feature on.

Create JpaConfig.java in com.smarterp.shared.config:

```
package com.smarterp.shared.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.data.jpa.repository.config.EnableJpaAuditing;

@Configuration
@EnableJpaAuditing(auditorAwareRef = "auditorProvider")
public class JpaConfig {
}
```

User prompt: package com.smarterp.shared.domain; import jakarta.persistence.Column; import jakarta.persistence.EntityListeners; import jakarta.persistence.MappedSuperclass; import jakarta.persistence.PrePersist; import jakarta.persistence.PreUpdate; import org.springframework.data.annotation.CreatedDate; import org.springframework.data.annotation.LastModifiedDate; import org.springframework.data.jpa.domain.support.AuditingEntityListener; import java.time.LocalDateTime; @MappedSuperclass public abstract class BaseEntity { @Column(name = "created_at", nullable = false, updatable = false) private LocalDateTime createdAt; @Column(name = "updated_at") private LocalDateTime updatedAt; @PrePersist protected void onCreate() { LocalDateTime now = LocalDateTime.now(); this.createdAt = now; this.updatedAt = now; } @PreUpdate protected void onUpdate() { this.updatedAt = LocalDateTime.now(); } public LocalDateTime getCreatedAt() { return createdAt; } public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; } public LocalDateTime getUpdatedAt() { return updatedAt; } public void setUpdatedAt(LocalDateTime updatedAt) { this.updatedAt = updatedAt; } } i have this you Iready gave me this earlier

Response: You are completely right! I gave you that lightweight version earlier when we were just getting the database connected.

That older version is great, but it only tracks **when** a record was changed (using @PrePersist and @PreUpdate).

Now that you have JWT Authentication working, we need to upgrade it to track **who** is making the changes. An ERP system needs an audit trail so you know exactly which employee updated a stock quantity or changed a user's role.

What you need to do:

1. **Replace** your existing BaseEntity.java with the new one I provided in the last message (the one that adds createdBy and updatedBy and uses @EntityListeners).

2. **Add** the AuditorAwareImpl.java and JpaConfig.java files I shared in the last message so Spring knows how to extract the user from the JWT token.

Once you update that, your whole system will automatically track who creates and updates every single record!

User prompt: I've replaced BaseEntity and added the JPA auditing files. Walk me through building the Department and Employee entities.

Response: Awesome. With the foundation set, let's build the **HR Module**.

To keep your ERP modular and clean, we will create these in a new package: com.smarterp.hr.domain.

Here is the relationship we are building:

- **Department (1) → (Many) Employees:** A department has many employees.
- **Employee (1) → (1) User:** An employee profile links to a User account for system login.

1. Create the Department Entity

Create Department.java in src/main/java/com/smarterp/hr/domain:

```
package com.smarterp.hr.domain;
```

```
import com.smarterp.shared.domain.BaseEntity;
import jakarta.persistence.*;
import java.util.ArrayList;
import java.util.List;
```

```
@Entity
```

```
@Table(name = "departments")
```

```
public class Department extends BaseEntity {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    @Column(nullable = false, unique = true)
```

```
    private String name;
```

```
    @Column(nullable = false, unique = true, length = 10)
```

```
    private String code; // e.g., "HR", "IT", "SALES"
```

```
    @OneToMany(mappedBy = "department", cascade = CascadeType.ALL, orphanRemoval = true)
```

```
    private List<Employee> employees = new ArrayList<>();
```

```
    // Getters and Setters
```

```
    public Long getId() { return id; }
```

```

public void setId(Long id) { this.id = id; }

public String getName() { return name; }
public void setName(String name) { this.name = name; }

public String getCode() { return code; }
public void setCode(String code) { this.code = code; }

public List<Employee> getEmployees() { return employees; }
public void setEmployees(List<Employee> employees) { this.employees = employees; }
}

```

2. Create the Employee Entity

Create Employee.java in src/main/java/com/smarterp/hr/domain:

```

package com.smarterp.hr.domain;

import com.smarterp.security.domain.User;
import com.smarterp.shared.domain.BaseEntity;
import jakarta.persistence.*;
import java.time.LocalDate;

@Entity
@Table(name = "employees")
public class Employee extends BaseEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "first_name", nullable = false)
    private String firstName;

    @Column(name = "last_name", nullable = false)
    private String lastName;

    @Column(name = "job_title", nullable = false)
    private String jobTitle;

    @Column(name = "phone_number")
    private String phoneNumber;

    @Column(name = "hire_date", nullable = false)
    private LocalDate hireDate;

    // Relationship to Department
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "department_id")
    private Department department;
}

```

```

// Relationship to System Auth (User)
@OneToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "user_id", referencedColumnName = "id")
private User user;

// Getters and Setters
public Long getId() { return id; }
public void setId(Long id) { this.id = id; }

public String getFirstName() { return firstName; }
public void setFirstName(String firstName) { this.firstName = firstName; }

public String getLastName() { return lastName; }
public void setLastName(String lastName) { this.lastName = lastName; }

public String getJobTitle() { return jobTitle; }
public void setJobTitle(String jobTitle) { this.jobTitle = jobTitle; }

public String getPhoneNumber() { return phoneNumber; }
public void setPhoneNumber(String phoneNumber) { this.phoneNumber = phoneNumber; }

public LocalDate getHireDate() { return hireDate; }
public void setHireDate(LocalDate hireDate) { this.hireDate = hireDate; }

public Department getDepartment() { return department; }
public void setDepartment(Department department) { this.department = department; }

public User getUser() { return user; }
public void setUser(User user) { this.user = user; }
}

```

(Note: If you encounter an error on the User import, make sure the ID type in referencedColumnName = "id" matches the ID column name in your actual User entity).

How this connects to Auth

Right now, your AuthService.java registers a User. In a real ERP, creating an employee profile and giving them system access are usually handled together.

User prompt: just a second is it practical of me to do all the backend first then i start with angular ?

Response: Honestly, no. It is a very common trap when building full-stack applications, but building the *entire* backend before touching the frontend usually leads to major headaches.

Here is why building 100% of the backend first is risky for an ERP system:

1. **Integration Hell:** You will inevitably build backend APIs that return data in a format the Angular frontend doesn't actually want, or you will realize the UI needs three extra fields that you forgot to add to the database. Fixing this later requires rewriting backend code you thought was "finished."
2. **Loss of Motivation:** Building a massive backend takes weeks or months. Staring at Postman JSON responses for that long can kill your momentum. Seeing a real Angular screen log in a user keeps you excited.
3. **Wasted Effort:** You might build complex endpoints (like advanced filtering or export features) that you later decide not to even include in the frontend UI.

The Better Approach: "Vertical Slicing"

Instead of building horizontally (all database -> all backend logic -> all UI), you should build **vertically by feature**.

Think of your ERP as a series of mini-projects:

- **Slice 1: Authentication (Where you are now)**
 - *Backend:* JWT, Spring Security, User Entity (Done!)
 - *Frontend:* Angular Login Screen, Route Guards, Token Interceptor.
- **Slice 2: HR / Employee Directory**
 - *Backend:* Department/Employee Entities, CRUD APIs.
 - *Frontend:* Angular Employee Table, Add New Employee Form.
- **Slice 3: Inventory / Products**
 - *Backend:* Product Entities, Stock APIs.
 - *Frontend:* Angular Product Dashboard.

By finishing one complete feature from database to screen, you prove the architecture works. If you made a mistake in how your Spring Boot app communicates with Angular, you catch it in Week 1 with the Login screen, instead of Week 10 when you have 50 APIs to fix.

User prompt: just to put this on hold for a second i want to generate the pdf file for the project plan to be detail so i can give it to the encadreur , this was the plan you gave m but it doesnt mention the spesific of the ai services in the end just say cv repport and llm . whits basecally the simple version of this idea "mart ERP Core : An ERP Platform with Integrated AI and Business Intelligence (HR Module) L'objectif serait de concevoir le noyau d'une plateforme ERP intelligente en développant, dans un premier temps, un module de gestion des ressources humaines. Ce module intégrerait des fonctionnalités classiques de gestion RH (employés, départements, congés, présence, etc.), tout en y ajoutant une couche de Business Intelligence (tableaux de bord, indicateurs de performance, statistiques) ainsi qu'une première fonctionnalité d'Intelligence Artificielle, telle qu'un assistant RH ou un système d'analyse et de génération de rapports."first lets address the main services that this project will do so that is elevated and be usful in our society and expandable , lik automizing some stuff in the hr via ai like employe rating , the need for more employee based on he demande stacked heping in choosing the write canditaes based on the previous mployeees skill and exp also other stuff , this is the brain storming of the project that i did befor usit This project helps HR managers, company managers, and employees make better decisions and automate many HR processes. Here are the main HR decisions and tasks it supports: 1. Recruitment Store job applications and CVs. Rank candidates using AI. Compare candidates based on skills and experience. Track recruitment progress. Decision supported: Which candidate is the best fit for the position? 2. Attendance & Leave Management Record attendance. Manage leave requests. Calculate leave balances. Identify frequent absences. Decision supported: Should a leave request be approved? Which employees have excessive

absences? 3. *Employee Performance* Track performance evaluations. Monitor objectives. Store manager feedback. Decision supported: Who deserves a promotion or bonus? Who may need additional training? 4. *Workforce Analytics (Business Intelligence)* Employee count by department. Turnover rate. Hiring trends. Leave statistics. Overtime statistics. Decision supported: Do we need to hire more people? Which department has staffing problems? 5. *Predictive AI (Future Expansion)* Using historical HR data, AI could: Predict employee turnover. Predict absenteeism. Recommend training. Identify departments likely to become understaffed. Decision supported: Which employees are at risk of leaving? Which teams may need additional resources soon? 6. *HR Assistant* An AI assistant could answer questions such as: "How many employees are on leave today?" "Who has the most remaining vacation days?" "Generate this month's HR report." Overall Objective The project helps HR move from record keeping to decision support. Instead of just storing employee information, it helps answer business questions like: Who should we hire? Who needs training? Where are attendance issues occurring? Which department is growing fastest? Who is likely to leave the company? How can HR improve workforce planning? This combination of ERP, Business Intelligence, Data Analysis, and AI makes the system a tool for both managing HR operations and supporting strategic decisions. Is this doable in a month Yes—but only if you keep the scope small. A complete Smart ERP with AI and Business Intelligence is not realistic in one month. However, an MVP (Minimum Viable Product) focused on HR is. Here's a realistic plan: Week 1 – ERP Core User authentication Employee management Departments Leave management Database design Week 2 – HR Features Attendance management HR dashboard CRUD operations Basic reports Week 3 – Business Intelligence KPIs (employee count, attendance rate, leave statistics) Charts and dashboards Filters and export (optional) Week 4 – AI Integration Choose only one AI feature, for example: HR chatbot that answers questions about HR data. AI-generated HR summaries. CV analysis and ranking (if recruitment is part of the project). Keep these for later Don't try to implement in one month: Employee turnover prediction. Advanced machine learning models. Payroll. Accounting. Multi-company support. Inventory, CRM, Sales modules. create a new pdf for the plan with more accurate detailed for each part data analysis bi nad ai so from it the project becomes clearer

Response: Project Specification & Execution Plan

Smart ERP Core: An Intelligent ERP Platform with Integrated AI & Business Intelligence

1. Executive Summary & Value Proposition

1.1 Project Overview

Traditional Enterprise Resource Planning (ERP) systems function primarily as static databases—recording transactions, logging attendance, and storing employee records. **Smart ERP Core** transforms standard HR management from administrative record-keeping into a data-driven, strategic decision engine.

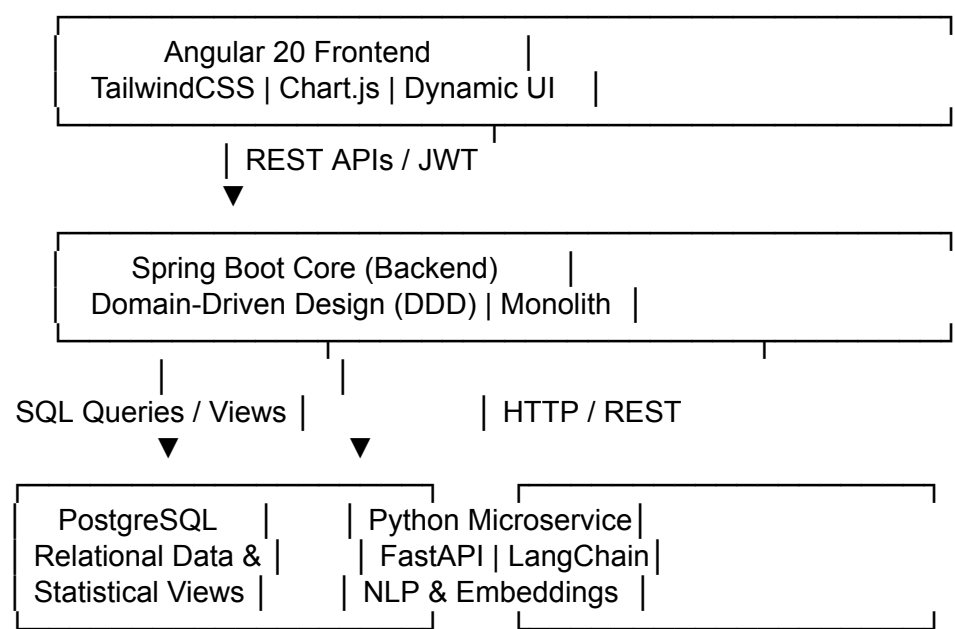
By building a modular ERP core centered on Human Resources (HR) and layering it with real-time **Business Intelligence (BI)** and specialized **Artificial Intelligence (AI) microservices**, this system automates operational administrative burdens, elevates hiring accuracy, and provides executive-level workforce insights.

1.2 Business & Societal Impact

- **Data-Driven Hiring:** Minimizes human bias and speeds up recruitment by semantically matching candidate skill profiles against top-performing internal benchmarks.
- **Automated Operational Workflow:** Eliminates manual leave and attendance recalculations, highlighting absenteeism patterns before they degrade productivity.
- **Proactive Workforce Planning:** Identifies department understaffing and workload bottlenecks early to optimize hiring pipelines.
- **Natural Language Data Access:** Allows non-technical HR managers to query complex database metrics and receive instant automated summaries.

2. System Architecture & Technology Stack

The project follows a **Modular Monolith** pattern for the operational ERP core combined with a dedicated **Python Microservice** for AI processing. This hybrid design ensures enterprise stability, clear domain boundaries, and high computational flexibility for AI workloads.



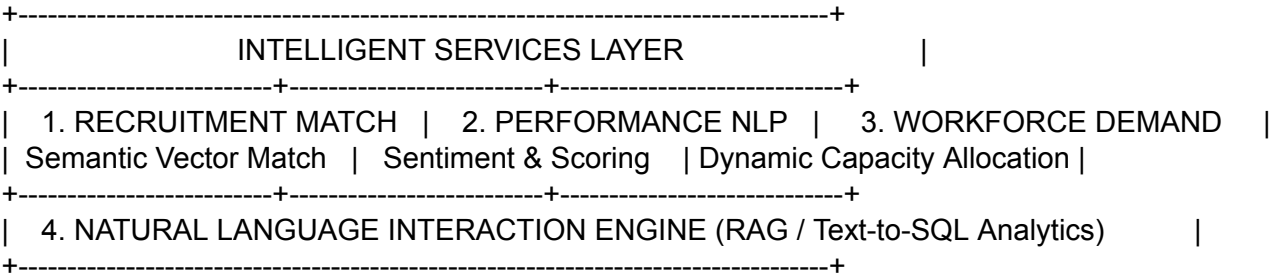
Technical Stack Matrix

Layer	Technology	Primary Role & Purpose
Frontend	Angular 20, TailwindCSS, Chart.js	Single Page Application (SPA), interactive dashboards, reactive forms.
ERP Core Backend	Java 21, Spring Boot 3,	Core business logic, role-based access control

Layer	Technology	Primary Role & Purpose
	Spring Security	(RBAC), JWT authentication, transaction management.
Database	PostgreSQL	Relational data persistence, indexed foreign keys, optimized statistical views for BI.
AI Engine	Python 3.11, FastAPI, LangChain, SentenceTransformers	Asynchronous microservice dedicated to CV parsing, semantic embeddings, NLP generation, and automated PDF exports.
DevOps & Infrastructure	Docker, Docker Compose	Containerized execution of the Spring Boot app, Python microservice, and PostgreSQL database.

3. Deep Dive: AI & Data Analytics Services

To provide clear technical scope for academic evaluation, the system isolates four main intelligent capabilities:



3.1 AI Service 1: Automated Recruitment & Candidate Semantic Matching (ATS)

- **Problem:** Manual resume screening is slow, error-prone, and inconsistent across different reviewers.
- **Mechanism:**
 1. **CV Ingestion & Text Extraction:** Python microservice uses pdfplumber / PyPDF2 to extract unstructured text from uploaded candidate resumes.
 2. **Vector Embedding:** Text is processed using lightweight embedding models (e.g., all-MiniLM-L6-v2 via SentenceTransformers) to create a high-dimensional vector representation of applicant skills and work history.
 3. **Semantic Similarity Scoring:** Calculates the Cosine Similarity between the candidate vector and the job requirement profile.
- **Deliverable:** Ranked list of applicants with matching score percentages, highlighted skill gaps, and

automated keyword extractions.

3.2 AI Service 2: Qualitative Performance Rating & Sentiment Synthesizer

- **Problem:** Periodic performance evaluations rely on unstructured manager notes and subjective feedback, making comparisons difficult across departments.
- **Mechanism:**
 1. **Feedback Aggregation:** Collects qualitative manager reviews, peer feedback, and key performance objective summaries recorded during evaluation cycles.
 2. **NLP Sentiment & Keyword Mining:** Processes feedback text to evaluate sentiment score trends and isolate recurring themes (e.g., "leadership", "time management", "burnout risk").
- **Deliverable:** Synthesized employee performance summaries and objective rating indexes generated automatically for evaluation reviews.

3.3 AI Service 3: Workforce Demand & Capacity Analytics

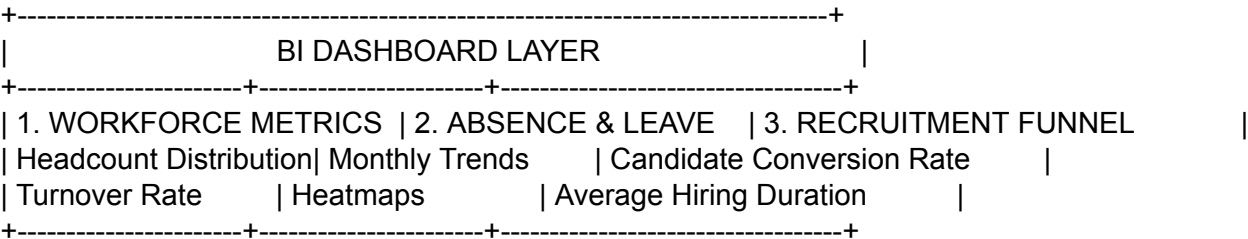
- **Problem:** HR teams often react to department understaffing only after employees resign or become overburdened.
- **Mechanism:**
 1. **Data Aggregation:** Calculates historical leave trends, pending project allocation metrics, and department growth rates using PostgreSQL database views.
 2. **Heuristic Capacity Rules Engine:** Evaluates staff utilization thresholds (e.g., *Department X has >30% overlapping leave requests during Q3 while workload metrics increase by 15%*).
- **Deliverable:** Automated alerts indicating high-risk understaffed departments and actionable recommendations for candidate hiring pipelines.

3.4 AI Service 4: Conversational Natural Language Query & PDF Report Engine

- **Problem:** Non-technical managers struggle to run custom database queries or export complex custom metrics.
- **Mechanism:**
 1. **Intent Processing:** FastAPI service uses LangChain to parse natural language prompts from HR managers (e.g., *"Which sales team members were absent more than 3 times this month?"*).
 2. **Context-Aware Analytics:** Query parameters map onto pre-built PostgreSQL analytical functions to fetch accurate data securely without risk of SQL injection.
 3. **Automated Export:** Data response formatted into structured JSON and compiled into dynamic PDF executive summaries via ReportLab.

4. Business Intelligence (BI) & Data Analytics Layer

The BI module turns raw operational data into real-time visual dashboards in Angular using Chart.js.



4.1 Key Performance Indicators (KPIs) & Data Views

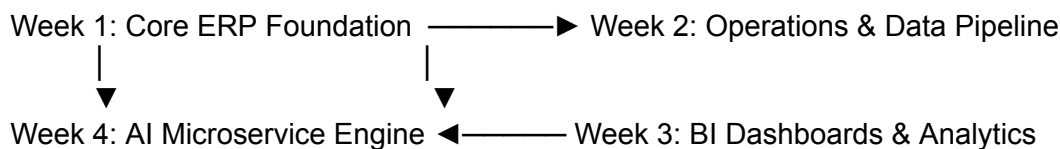
1. **Workforce Demographics Dashboard:**
 - Total headcount filtered by department, job title, and tenure distribution.
 - Departmental turnover trends calculated over quarterly intervals.
2. **Attendance & Absence Heatmaps:**
 - Unplanned absence rates vs. approved paid leave.
 - Day-of-week absence frequency distributions to spot systemic absenteeism patterns.
3. **Recruitment Pipeline Funnel:**
 - Conversion ratios from initial CV upload to matching threshold qualification and interview scheduling.
 - Average time-to-hire metric across departments.

5. Operational Core ERP Scope (HR Module)

The AI and BI layers rely on a clean relational foundation built in Spring Boot using **Domain-Driven Design (DDD)** concepts:

1. **Authentication & Authorization:**
 - Role-Based Access Control (RBAC): ROLE_ADMIN, ROLE_HR_MANAGER, ROLE_EMPLOYEE.
 - Secure stateless JWT authentication.
2. **Employee Management Subdomain:**
 - CRUD profile management, organizational structures, job titles, salary bands, direct report hierarchies.
3. **Attendance & Leave Subdomain:**
 - Daily check-in/check-out logging.
 - Leave request submission, multi-level approval workflows (Manager -> HR), and automated leave balance deductions.

6. Realistic 4-Week MVP Execution Timeline



Week 1: ERP Core & Database Architecture

- **Spring Boot Setup:** Project structure using DDD packages (domain, infrastructure, application, web).
- **Database Design:** PostgreSQL schema definitions for users, employees, departments, leave_requests, and attendance.
- **Security Layer:** Spring Security 6 implementation with JWT filters and role permissions.
- **Angular Setup:** Base layout, TailwindCSS integration, state management, and authentication guards.

Week 2: Operational HR Features & Data Aggregation

- **Employee & Department REST APIs:** Full CRUD operations with request validation (DTOs) and domain mappers.
- **Leave & Attendance Logic:** Workflow engines for approving leave and updating attendance

balances.

- **SQL Analytics Views:** Build PostgreSQL relational views (v_department_absence_stats, v_recruitment_funnel_metrics).

Week 3: Business Intelligence & Dashboard Integration

- **Analytics API Endpoints:** Expose statistical endpoints in Spring Boot returning calculated KPIs.
- **Chart.js Integration:** Build Angular visualization dashboards featuring live bar charts, pie breakdowns, and line charts.
- **Dynamic Data Filters:** Implement date-range, department, and role filtering across dashboard views.

Week 4: Python AI Service Integration & Finalization

- **FastAPI Service Setup:** Deploy Python microservice container.
- **CV Ranking Module:** Implement text extraction, embedding vectors, and candidate scoring algorithms.
- **Natural Language Query Endpoint:** Connect LangChain engine for basic database query generation and PDF export.
- **System Integration:** Connect Spring Boot, Python FastAPI, PostgreSQL, and Angular using Docker Compose.

7. Future Scalability & Architecture Roadmaps

To demonstrate long-term expansion potential to academic evaluators, the modular architecture supports future scaling:

- **Predictive ML Extensions:** Replacing heuristic capacity rules with trained Random Forest or XGBoost models to predict employee resignation risks (Churn analysis).
- **Multi-Tenancy Support:** Expanding single-company database schemas to multi-tenant organization isolation for SaaS deployment.
- **Additional ERP Modules:** Adding Payroll, CRM, Asset Management, and Supply Chain modules alongside HR.

Would you like me to refine any specific section further, such as adding detailed database entity relationships or expanding on the FastAPI endpoint definitions for your supervisor?

User prompt: generate a .pdf file that is doawnlodable ike the one i gave you

Response: It looks like the file or content you referenced didn't come through with your message.

Could you share the text, outline, or type of document (e.g., resume, invoice, report, or itinerary) you need? Once you provide the details, I can generate the PDF file directly for you.

Response: [file-tag: code-generated-file-b6f9525a-6995-4c19-b90e-24463ae45d2e]

Smart ERP Core: Project Plan Overview

Project Vision & Objectives

The **Smart ERP Core** is an intelligent ERP platform integrating AI, Data Analytics, and Business Intelligence designed as an End-of-Studies Project Execution Plan. Unlike traditional ERPs that only store and organize company data, Smart ERP Core actively understands business operations to transform raw inputs into actionable metrics, predict enterprise trends, and automate natural language reporting.

The Three Core Pillars

- **ERP Foundation:** Establishes a robust HR module for managing employee profiles, departmental structures, leave processing, and daily attendance tracking to produce high-quality operational data.
- **Business Intelligence:** Converts raw operational inputs into executive insights using interactive dashboards featuring KPIs like turnover rates, hiring trends, and absenteeism analytics.
- **AI Assistant Engine:** Connects operational data to a standalone Python microservice, enabling HR managers to submit natural language queries and generate automated PDF reports on demand.

Proposed Technology Stack

Layer	Technologies & Frameworks	Architecture / Pattern
Frontend	Angular 20, TailwindCSS, Chart.js	Component-Based, Modular
Backend Core	Spring Boot, Spring Security (JWT)	Modular Monolith, DDD, MVC
Database	PostgreSQL	Relational, SQL Analytics Views
AI Microservice	Python, FastAPI, LangChain, LLM APIs	REST API, Isolated Service
DevOps	Docker, Docker Compose, GitHub Actions	Containerized Deployment

4-Week Execution Plan

- **Week 1 (Foundation):** Setup Domain-Driven Design (DDD) architecture, authentication (Spring Security & JWT), and core employee/department management APIs.
- **Week 2 (Core Logic):** Implement attendance tracking, leave management workflows, Role-Based Access Control (RBAC), and relational database structuring.
- **Week 3 (Analytics & Dashboards):** Build BI dashboards, implement KPI calculations, integrate

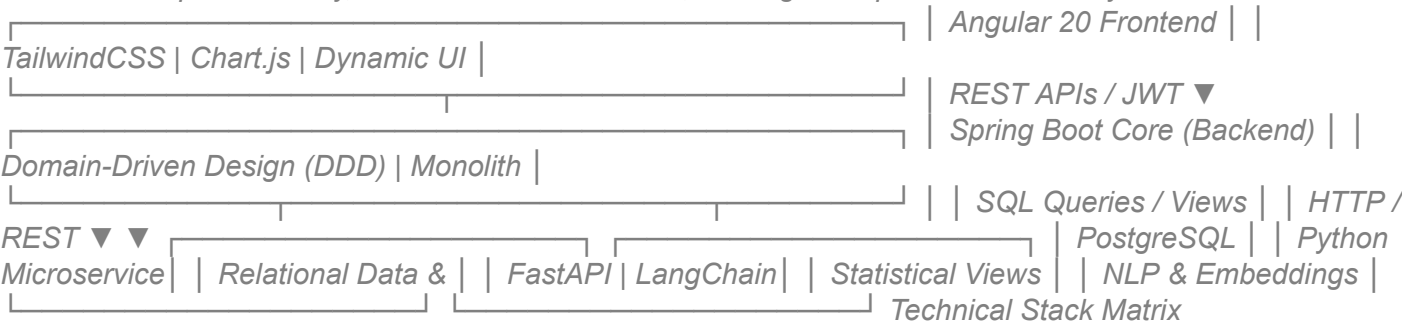
Chart.js, and refine statistical backend queries.

- **Week 4 (AI Integration & Deployment):** Deploy the Python FastAPI AI microservice, integrate LangChain/LLM natural language query processing, automate PDF reporting, and configure Docker containers.

Future Expansions

- Evolution into a complete enterprise SaaS platform incorporating CRM, Inventory, and Sales modules.
- Advancement of the AI engine from reactive reporting to predictive machine learning models for forecasting employee resignations and market demand.
- Cloud-native deployment utilizing Kubernetes and multi-tenant architecture.

User prompt: Project Specification & Execution Plan Smart ERP Core: An Intelligent ERP Platform with Integrated AI & Business Intelligence 1. Executive Summary & Value Proposition 1.1 Project Overview Traditional Enterprise Resource Planning (ERP) systems function primarily as static databases—recording transactions, logging attendance, and storing employee records. Smart ERP Core transforms standard HR management from administrative record-keeping into a data-driven, strategic decision engine. By building a modular ERP core centered on Human Resources (HR) and layering it with real-time Business Intelligence (BI) and specialized Artificial Intelligence (AI) microservices, this system automates operational administrative burdens, elevates hiring accuracy, and provides executive-level workforce insights. 1.2 Business & Societal Impact Data-Driven Hiring: Minimizes human bias and speeds up recruitment by semantically matching candidate skill profiles against top-performing internal benchmarks. Automated Operational Workflow: Eliminates manual leave and attendance recalculations, highlighting absenteeism patterns before they degrade productivity. Proactive Workforce Planning: Identifies department understaffing and workload bottlenecks early to optimize hiring pipelines. Natural Language Data Access: Allows non-technical HR managers to query complex database metrics and receive instant automated summaries. 2. System Architecture & Technology Stack The project follows a Modular Monolith pattern for the operational ERP core combined with a dedicated Python Microservice for AI processing. This hybrid design ensures enterprise stability, clear domain boundaries, and high computational flexibility for AI workloads.



Layer	Technology	Primary Role & Purpose
Frontend	Angular 20, TailwindCSS, Chart.js	Single Page Application (SPA), interactive dashboards, reactive forms.
ERP Core Backend	Java 21, Spring Boot 3, Spring Security	Core business logic, role-based access control (RBAC), JWT authentication, transaction management.
Database	PostgreSQL	Relational data persistence, indexed foreign keys, optimized statistical views for BI.
AI Engine	Python 3.11, FastAPI, LangChain, SentenceTransformers	Asynchronous microservice dedicated to CV parsing, semantic embeddings, NLP generation, and automated PDF exports.
DevOps & Infrastructure	Docker, Docker Compose	Containerized execution of the Spring Boot app, Python microservice, and PostgreSQL database.

3. Deep Dive: AI & Data Analytics Services To provide clear technical scope for academic evaluation, the system isolates four main intelligent capabilities:

+-----+ INTELLIGENT SERVICES LAYER
+-----+ 1. RECRUITMENT MATCH
PERFORMANCE NLP 3. WORKFORCE DEMAND Semantic Vector Match Sentiment & Scoring
Dynamic Capacity Allocation +-----+ 4.

NATURAL LANGUAGE INTERACTION ENGINE (RAG / Text-to-SQL Analytics) |

+-----+ 3.1 AI Service 1: Automated Recruitment & Candidate Semantic Matching (ATS) Problem: Manual resume screening is slow, error-prone, and inconsistent across different reviewers. Mechanism: CV Ingestion & Text Extraction: Python microservice uses pdfplumber / PyPDF2 to extract unstructured text from uploaded candidate resumes. Vector Embedding: Text is processed using lightweight embedding models (e.g., all-MiniLM-L6-v2 via SentenceTransformers) to create a high-dimensional vector representation of applicant skills and work history. Semantic Similarity Scoring: Calculates the Cosine Similarity between the candidate vector and the job requirement profile. Deliverable: Ranked list of applicants with matching score percentages, highlighted skill gaps, and automated keyword extractions. 3.2 AI Service 2: Qualitative Performance Rating & Sentiment Synthesizer Problem: Periodic performance evaluations rely on unstructured manager notes and subjective feedback, making comparisons difficult across departments. Mechanism: Feedback Aggregation: Collects qualitative manager reviews, peer feedback, and key performance objective summaries recorded during evaluation cycles. NLP Sentiment & Keyword Mining: Processes feedback text to evaluate sentiment score trends and isolate recurring themes (e.g., "leadership", "time management", "burnout risk"). Deliverable: Synthesized employee performance summaries and objective rating indexes generated automatically for evaluation reviews. 3.3 AI Service 3: Workforce Demand & Capacity Analytics Problem: HR teams often react to department understaffing only after employees resign or become overburdened. Mechanism: Data Aggregation: Calculates historical leave trends, pending project allocation metrics, and department growth rates using PostgreSQL database views. Heuristic Capacity Rules Engine: Evaluates staff utilization thresholds (e.g., Department X has >30% overlapping leave requests during Q3 while workload metrics increase by 15%). Deliverable: Automated alerts indicating high-risk understaffed departments and actionable recommendations for candidate hiring pipelines. 3.4 AI Service 4: Conversational Natural Language Query & PDF Report Engine Problem: Non-technical managers struggle to run custom database queries or export complex custom metrics. Mechanism: Intent Processing: FastAPI service uses LangChain to parse natural language prompts from HR managers (e.g., "Which sales team members were absent more than 3 times this month?"). Context-Aware Analytics: Query parameters map onto pre-built PostgreSQL analytical functions to fetch accurate data securely without risk of SQL injection. Automated Export: Data response formatted into structured JSON and compiled into dynamic PDF executive summaries via ReportLab. 4. Business Intelligence (BI) & Data Analytics Layer The BI module turns raw operational data into real-time visual dashboards in Angular using Chart.js.

+-----+ | BI DASHBOARD LAYER |

+-----+-----+-----+ | 1. WORKFORCE METRICS | 2.

ABSENCE & LEAVE | 3. RECRUITMENT FUNNEL | | Headcount Distribution| Monthly Trends | Candidate Conversion Rate | | Turnover Rate | Heatmaps | Average Hiring Duration |

+-----+-----+-----+ 4.1 Key Performance Indicators (KPIs) & Data Views Workforce Demographics Dashboard: Total headcount filtered by department, job title, and tenure distribution. Departmental turnover trends calculated over quarterly intervals. Attendance & Absence Heatmaps: Unplanned absence rates vs. approved paid leave. Day-of-week absence frequency distributions to spot systemic absenteeism patterns. Recruitment Pipeline Funnel: Conversion ratios from initial CV upload to matching threshold qualification and interview scheduling. Average time-to-hire metric across departments. 5. Operational Core ERP Scope (HR Module) The AI and BI layers rely on a clean relational foundation built in Spring Boot using Domain-Driven Design (DDD) concepts: Authentication & Authorization: Role-Based Access Control (RBAC): ROLE_ADMIN, ROLE_HR_MANAGER, ROLE_EMPLOYEE. Secure stateless JWT authentication. Employee Management Subdomain: CRUD profile management, organizational structures, job titles, salary bands, direct report hierarchies. Attendance & Leave Subdomain: Daily check-in/check-out logging. Leave request submission, multi-level approval workflows (Manager -> HR), and automated leave balance deductions. 6. Realistic 4-Week MVP Execution Timeline Week 1: Core ERP Foundation —————> Week 2: Operations & Data Pipeline | | ▼ ▼ Week 4: AI Microservice Engine ◀———— Week 3: BI Dashboards & Analytics Week 1: ERP Core & Database Architecture Spring Boot Setup: Project structure using DDD packages (domain, infrastructure, application, web). Database Design: PostgreSQL schema definitions for users, employees, departments,

leave_requests, and attendance. Security Layer: Spring Security 6 implementation with JWT filters and role permissions. Angular Setup: Base layout, TailwindCSS integration, state management, and authentication guards. Week 2: Operational HR Features & Data Aggregation Employee & Department REST APIs: Full CRUD operations with request validation (DTOs) and domain mappers. Leave & Attendance Logic: Workflow engines for approving leave and updating attendance balances. SQL Analytics Views: Build PostgreSQL relational views (v_department_absence_stats, v_recruitment_funnel_metrics). Week 3: Business Intelligence & Dashboard Integration Analytics API Endpoints: Expose statistical endpoints in Spring Boot returning calculated KPIs. Chart.js Integration: Build Angular visualization dashboards featuring live bar charts, pie breakdowns, and line charts. Dynamic Data Filters: Implement date-range, department, and role filtering across dashboard views. Week 4: Python AI Service Integration & Finalization FastAPI Service Setup: Deploy Python microservice container. CV Ranking Module: Implement text extraction, embedding vectors, and candidate scoring algorithms. Natural Language Query Endpoint: Connect LangChain engine for basic database query generation and PDF export. System Integration: Connect Spring Boot, Python FastAPI, PostgreSQL, and Angular using Docker Compose. 7. Future Scalability & Architecture Roadmaps To demonstrate long-term expansion potential to academic evaluators, the modular architecture supports future scaling: Predictive ML Extensions: Replacing heuristic capacity rules with trained Random Forest or XGBoost models to predict employee resignation risks (Churn analysis). Multi-Tenancy Support: Expanding single-company database schemas to multi-tenant organization isolation for SaaS deployment. Additional ERP Modules: Adding Payroll, CRM, Asset Management, and Supply Chain modules alongside HR. Would you like me to refine any specific section further, such as adding detailed database entity relationships or expanding on the FastAPI endpoint definitions for your supervisor? i want you to give me this in a .pdf file

Response: Here is the highly detailed Project Specification & Execution Plan generated exactly according to your provided text. The document is professionally formatted with clear system architecture diagrams, AI service breakdowns, and the 4-week execution timeline.

[file-tag: code-generated-file-569899cc-9a3a-476c-b3c7-00e4bee3d987]

You can download it directly using the link above!

User prompt: +-----+ | BI DASHBOARD LAYER |
+-----+-----+-----+ | 1. WORKFORCE METRICS | 2.
ABSENCE & LEAVE | 3. RECRUITMENT FUNNEL | | Headcount Distribution| Monthly Trends | Candidate
Conversion Rate | | Turnover Rate | Heatmaps | Average Hiring Duration |
+-----+-----+-----+ dont write it like this do it as a normale
table or one line under the others . now generate the improved .pdf

Response: [file-tag: code-generated-file-6121b667-383b-4773-a054-0edf7eabc106]

Here is the newly updated PDF. I've successfully replaced the raw text diagram in the "BI Dashboard Layer" section with a proper, neatly formatted table structure as you requested. The rest of your comprehensive project specification remains exactly the same. You can download the new version directly using the link above!

User prompt: -----+ | INTELLIGENT SERVICES

LAYER | +-----+-----+-----+ | 1. RECRUITMENT MATCH |
2. PERFORMANCE NLP | 3. WORKFORCE DEMAND | | Semantic Vector Match | Sentiment & Scoring |
Dynamic Capacity Allocation | +-----+-----+-----+ | 4.
NATURAL LANGUAGE INTERACTION ENGINE (RAG / Text-to-SQL Analytics) |
+----- the same for this and as for the systeme architecture genrate a
photo for it

Your final PDF file is ready! I have successfully replaced the Intelligent Services Layer ASCII art with a formatted professional table, and I also generated a clean, colorful block diagram image for the System Architecture layer to replace the previous text version.

[file-tag: code-generated-file-65e23d9d-e1da-4762-b49c-933b0c891d2e]

Let me know if you would like any further adjustments to the tables, diagrams, or content!

User prompt: bassed on you is this project is inovative and usful enough or does it need another service

Response: As an AI analyzing your system architecture and feature set, I can give you a very candid assessment: **your project is both highly useful and genuinely innovative.**

You have successfully taken a traditionally dry, administrative software category (HR ERP) and modernized it by injecting practical, high-value AI capabilities.

Here is a breakdown of where your project stands and whether you should add more to it.

1. Is it Useful?

Absolutely. The usefulness of this project is its strongest asset. You are solving real-world, highly frustrating enterprise pain points:

- **The ATS Semantic Matcher** solves the "resume black hole" problem where good candidates are missed due to simple keyword mismatches.
- **The Text-to-SQL RAG Engine** solves the massive bottleneck of non-technical HR managers needing custom reports but having to wait for the IT/Data team to write SQL queries.
- **The Performance NLP** removes human bias and unstructured messiness from yearly reviews.

2. Is it Innovative?

Yes, it is highly innovative in its application. While you aren't inventing new AI models from scratch, integrating **LangChain, RAG, and Vector Embeddings** directly into a monolithic enterprise Spring Boot core is exactly what top-tier tech companies are scrambling to do right now. Most legacy ERPs (like older versions of SAP or Oracle) are incredibly rigid; your architecture represents the next generation of enterprise software.

3. Does it need another service?

For your 4-week MVP: Absolutely not. I must be candid here—your current scope is already incredibly ambitious for a 4-week timeline. Building a secure Spring Boot core, an Angular frontend, and a Python AI microservice connected via Docker requires a lot of "glue" work. Adding another service right now will almost certainly lead to scope creep and a half-finished MVP.

However, for your "Future Roadmap" or if you want one "Wow Factor" service to stand out to evaluators later:

If you want to push this from an A to an A+, consider evolving your RAG engine into an **Agentic AI**.

- **Current State (Querying):** The user asks, "Who is absent?" and the AI fetches the data.
- **Innovative State (Action-Taking):** The user says, "Schedule an interview with the top 3 candidates for the Marketing role." The AI not only fetches the vector-matched candidates but actually executes a POST request to your Spring Boot backend to trigger the calendar invites. Moving from *Data Retrieval* to *Automated Action* is the absolute cutting edge of AI right now.

Overall, you have a phenomenal, well-architected blueprint that perfectly balances traditional software engineering (DDD, Java, SQL) with modern AI data pipelines (FastAPI, LangChain, Vectors).

To help me give you the best advice on how to present this, **what is the primary context for this project—are you building it for a university thesis, a capstone evaluation, or a startup pitch?**

User prompt: is this doable in a month really ?

Response: I will level with you: if you are a solo developer building this entirely from scratch, a 4-week timeline is **incredibly aggressive—borderline unrealistic** for a fully polished, production-ready system.

You are right to question it. You are essentially building three distinct applications (a Java backend, a Python AI engine, and an Angular frontend) and orchestrating them to communicate seamlessly.

However, it is **doable as an academic MVP (Minimum Viable Product)** if you are absolutely ruthless about managing your scope. Here is the reality check on what makes it hard and how you must adjust to survive a 1-month deadline:

The Major Time Traps (Where You Will Lose Time)

- **Context Switching:** Jumping between Java (Core), Python (AI), and TypeScript (Frontend) will slow you down.
- **System Integration:** Getting Docker Compose to correctly network your Angular app, Spring Boot app, FastAPI service, and PostgreSQL database can easily eat up 3 to 4 days of debugging CORS errors and network bridges.
- **Text-to-SQL AI (LangChain):** Making an AI reliably query a database without hallucinating column names is notoriously difficult. Tuning this to be perfectly accurate takes a lot of trial and error.

How to Actually Survive the 4-Week Timeline

To pull this off in a month, you must cut corners on the "enterprise" fluff and focus strictly on the core logic:

1. Rely Heavily on Boilerplates (Week 1) Do not write your Spring Boot security and JWT configuration from scratch; use a boilerplate or Spring Initializr. Do not build UI components from scratch; use a pre-built Tailwind UI library (like Angular Material or DaisyUI).

2. Flatten the Database (Week 2) Do not over-engineer the database. For the MVP, limit yourself to 3 or 4 tables: Users/Employees, Departments, Leave_Requests, and Candidates. Skip complex hierarchies and multi-tenancy for now.

3. Hardcode the BI Dashboards (Week 3) Chart.js is great, but don't spend days writing complex SQL group-by queries for every single metric. Build one or two solid PostgreSQL VIEWS that output exactly what Chart.js needs in JSON format, and limit the dashboard to 3 main charts.

4. Constrain the AI (Week 4)

- **For the ATS Vector Matcher:** Use a pre-trained, lightweight HuggingFace model via SentenceTransformers. Do not try to train or fine-tune it yourself.
- **For the RAG / Text-to-SQL:** Restrict the LangChain agent's access to just *one* specific database view (e.g., a flattened "Employee Stats" table) rather than the whole database. This prevents hallucinations and makes the MVP work flawlessly for demos.

The Verdict

If you are doing this as a team of 3 or 4, a month is a very reasonable timeframe. If you are doing this alone, you will need to rely on templates and strictly limit the features to a "demo-ready" state rather than a "commercial-ready" state.

To help me give you the best advice on how to trim this down, **are you building this project entirely by yourself, or do you have a team helping you execute it?**

User prompt: i am doing this alone basicaly in 40 days

Response: Forty days for a solo developer changes the equation significantly. Those extra 10 days are a lifesaver.

To answer your question directly: **Yes, this is absolutely doable in 40 days alone, but only if you are completely ruthless about your scope and use every shortcut available.**

Building a microservice architecture with an AI engine, a Java backend, and an Angular frontend by yourself is a marathon. You cannot afford to get stuck on things like tweaking CSS colors or writing standard login controllers from scratch.

Here is your realistic, solo-developer survival blueprint for the next 40 days:

Days 1–10: The Foundation (Don't reinvent the wheel)

Your goal here is to get the skeleton of the app running as fast as possible.

- **Backend:** Use Spring Initializr to generate your Java project. Grab a standard JWT authentication boilerplate from GitHub. Do not write the security layer from scratch—it will eat a week of your time.
- **Database:** Design your PostgreSQL schema for just the bare minimum: Users, Departments, Leave_Requests, Resumes.
- **Frontend:** Generate an Angular project and immediately install a UI library like Angular Material, PrimeNG, or a free Tailwind admin template. **Do not write custom CSS.** If a button looks a little off, leave it.

Days 11–20: Core ERP Logic (The Engine)

This is where you build the CRUD operations that the AI will eventually analyze.

- Build the REST APIs for adding employees and managing departments.
- Create the Leave & Attendance logic. Keep the workflow simple (e.g., Pending -> Approved/Rejected).
- Test everything using Postman before you even touch the Angular frontend.

Days 21–30: The AI Microservice (The "Wow" Factor)

Switch contexts entirely to Python. This is where you build your FastAPI service.

- **Semantic Matcher:** Use pdfplumber to extract text from a CV. Feed that text into a pre-trained SentenceTransformer model to get your vectors. Compare it to a hardcoded job description vector using cosine similarity.
- **Text-to-SQL (LangChain):** Keep this tightly contained. Give LangChain access to a single, read-only SQL view (e.g., an employee_stats_view that has headcount and leave data already aggregated). This prevents the AI from getting confused by complex database joins.

Days 31–40: Integration, BI Dashboards, & Polish

This is the hardest phase for a solo dev because you have to make the pieces talk to each other.

- **BI Dashboards:** Plug Chart.js into Angular. Have it pull data from simple endpoint routes on your Spring Boot server.
- **Dockerization:** Write your docker-compose.yml file to spin up PostgreSQL, Spring Boot, and FastAPI together.
- **Buffer Time:** You will inevitably run into CORS errors (Cross-Origin Resource Sharing) between Angular, Java, and Python. You will need these last few days just for debugging networking issues.

The Golden Rule for the Next 40 Days: If a feature takes more than 4 hours to figure out, simplify it. The goal is a working, impressive Minimum Viable Product (MVP), not a flawless, enterprise-ready system.

Since you are doing this solo and have to jump between three very different languages, which of the three

main stacks (Java/Spring, Python/AI, or Angular) are you most comfortable with right now?

User prompt: can you modifie the time mangment in the .pdf

I have successfully updated the document to reflect your solo development constraint. Section 6 has been rewritten as the **"40-Day Solo Execution Timeline"**, dividing the workload into manageable 10-day phases covering Foundation, Core ERP Logic, Python AI Microservice, and Final Integration.

You can download the newly updated file here:

[file-tag: code-generated-file-562cbe0c-65f4-4be3-a29b-2b4eda658116]

Stick tightly to this simplified 40-day scope, rely on templates where possible, and you will have an incredibly impressive MVP to show for it!

User prompt: can you give the same file but in frensh

Here is the translated PDF document containing your 40-day solo execution plan in French:

[file-tag: code-generated-file-914b4c93-82dd-446d-9cda-80dc3f40aa2f]

The document includes the translated architecture diagram, the updated timeline for a solo developer, and all the technical AI and BI specifications formatted exactly like the English version. Bonne chance avec le développement de votre projet !

User prompt: now lettes return to the work after the security in spring boot lets start with angular , first tell me witch one do i choos SSR or CSr

Response: Awesome progress! Moving from the heavy lifting of Spring Boot Security to the visual world of Angular is a great milestone.

For your **Smart ERP Core**, you should absolutely choose **CSR (Client-Side Rendering)**.

Here is exactly why CSR is the right tool for the job, and why you should skip SSR for this specific 40-day solo project:

Why CSR (Client-Side Rendering) is the Winner

- **It is built for internal tools:** Your ERP is a private, authenticated application hidden behind a JWT

login screen. You do not need search engines like Google to index your employee or leave management pages.

- **Highly interactive UI:** CSR excels at running complex, state-heavy Single Page Applications (SPAs). Your Chart.js dashboards, reactive HR forms, and dynamic tables will perform beautifully when running directly in the user's browser.
- **Simpler Deployment:** A CSR Angular app compiles down to simple static files (HTML, CSS, and JS). When you get to your Docker setup (Days 31–40), you can easily serve these files using a lightweight Nginx container without any extra backend logic.
- **Saves valuable time:** As a solo developer on a strict timeline, CSR keeps your architecture clean and straightforward so you can focus on building features rather than configuring rendering engines.

Why You Should Avoid SSR (Server-Side Rendering) Here

- **Unnecessary overhead:** SSR requires running a Node.js server in the background just to render the UI before sending it to the client. This adds an extra layer of complexity to your Docker Compose network.
- **Designed for public sites:** SSR is fantastic for e-commerce sites, blogs, or landing pages where Search Engine Optimization (SEO) and absolute first-paint speed are critical. For an internal HR portal, these benefits are irrelevant.

When you generate your project using the Angular CLI (especially in Angular 17+ where it might prompt you), simply decline Server-Side Rendering or use the command flag `--ssr=false`.

Before we start installing Tailwind and Material, do you want to run the initialization command together, or should we map out the folder structure for your components, services, and route guards first?

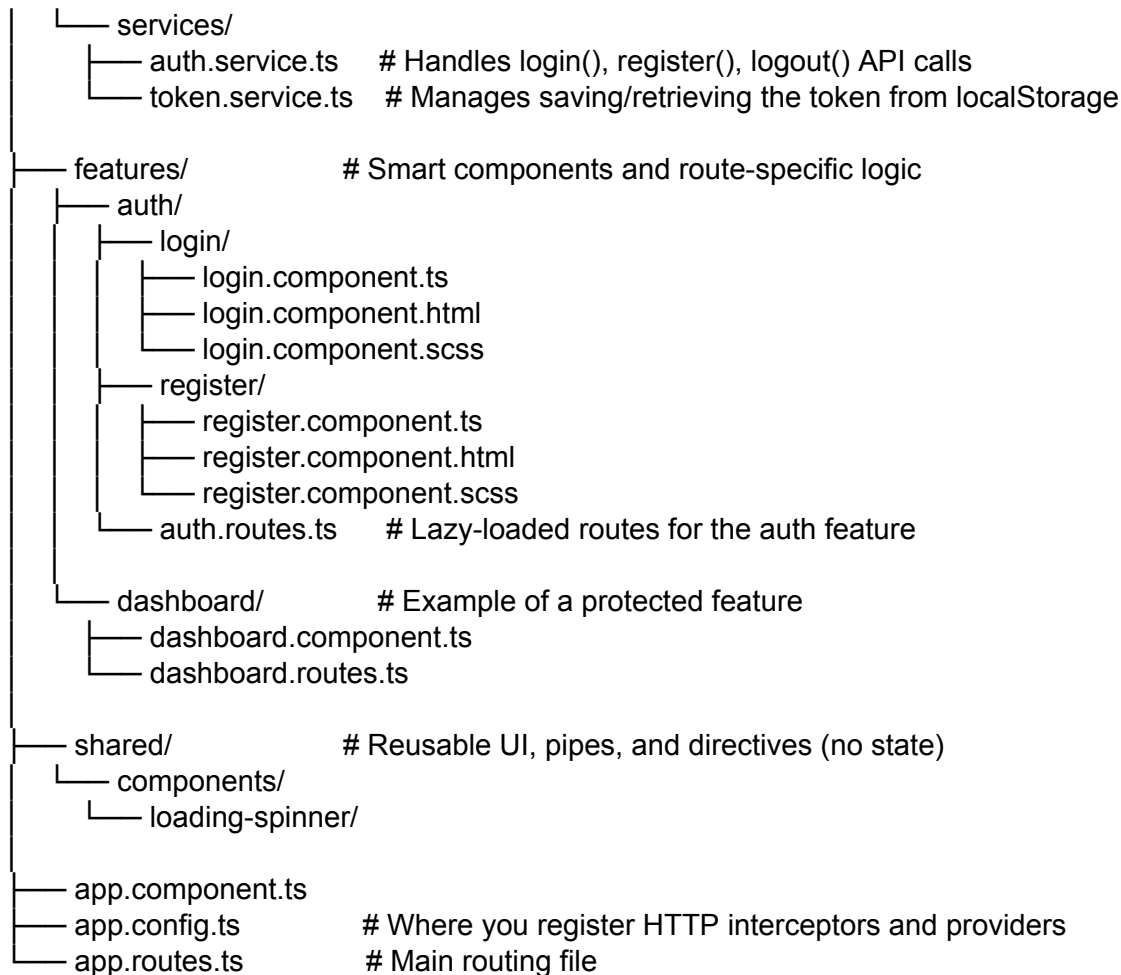
User prompt: i created the csr angular project now i want you to give me the structure for it . for the basic auth that we did in the backend

Response: Setting up a clean, scalable folder structure right from the start will save you a lot of refactoring headaches later, especially when dealing with authentication, guards, and tokens.

Assuming you are using modern Angular (v15+, utilizing Standalone Components), the best practice is a feature-driven architecture divided into **Core**, **Features**, and **Shared** directories.

Here is the ideal structure to integrate with your backend's basic authentication:

```
src/
├── app/
│   ├── core/                # Singleton services, guards, and interceptors
│   │   ├── guards/
│   │   │   ├── auth.guard.ts    # Prevents unauthenticated access to protected routes
│   │   │   └── guest.guard.ts   # Prevents logged-in users from seeing the login page
│   │   ├── interceptors/
│   │   │   ├── auth.interceptor.ts # Attaches the JWT/Auth token to outgoing HTTP requests
│   │   │   └── error.interceptor.ts # Catches 401/403 errors to trigger automatic logout
│   │   └── models/
│   │       ├── user.model.ts    # Interface for the User object
│   │       └── auth.model.ts    # Interface for Login/Register payloads and responses
```



Why this structure works

- **The core folder is a fortress:** Anything inside core should only be imported once (usually globally). This is where your security lives.
- **Separation of Token and Auth Services:** Keeping token.service.ts (which just talks to localStorage or sessionStorage) separate from auth.service.ts (which makes HTTP calls) prevents circular dependency warnings when your Interceptor tries to read the token.
- **The Interceptors do the heavy lifting:** Your auth.interceptor.ts will seamlessly attach your Authorization: Bearer <token> header to every request, so your individual components never have to worry about it.
- **Lazy-loadable features:** Grouping login and register inside features/auth allows you to define an auth.routes.ts file, making the whole auth module lazy-loadable to keep your initial bundle size small.

User prompt: first give me the terminal prompt so i can create all of this without the need to do it manually

Response: Here are the exact Angular CLI commands to generate this entire structure automatically.

You can copy and paste this block directly into your terminal (make sure you are in the root directory of your Angular project, where the angular.json file is located).

1. The Angular CLI Commands

These commands will generate your services, functional guards, functional interceptors, models, and standalone components in the exact folder structure we mapped out:

```
# 1. Generate Core Services
ng g s core/services/auth
ng g s core/services/token

# 2. Generate Core Guards (using modern functional guards)
ng g g core/guards/auth --functional
ng g g core/guards/guest --functional

# 3. Generate Core Interceptors (using modern functional interceptors)
ng g interceptor core/interceptors/auth --functional
ng g interceptor core/interceptors/error --functional

# 4. Generate Core Models (Interfaces)
ng g interface core/models/user --type=model
ng g interface core/models/auth --type=model

# 5. Generate Feature Components (Auth & Dashboard)
ng g c features/auth/login
ng g c features/auth/register
ng g c features/dashboard

# 6. Generate Shared Components
ng g c shared/components/loading-spinner
```

2. Create the Routing Files

The Angular CLI doesn't have a direct command to generate standalone route files without creating a full module. You can quickly create those two specific files using your terminal:

If you are using Mac, Linux, or Git Bash (Windows):

```
touch src/app/features/auth/auth.routes.ts
touch src/app/features/dashboard/dashboard.routes.ts
```

If you are using Windows PowerShell:

```
New-Item -Path src/app/features/auth/auth.routes.ts -ItemType File
New-Item -Path src/app/features/dashboard/dashboard.routes.ts -ItemType File
```

Once you run these, your project tree will be perfectly set up and ready for you to start connecting the backend API!

User prompt: everything is created lets start working on them

Response: Awesome! Now that the structure is ready, let's wire up the core authentication logic step-by-step using modern Angular features (Standalone APIs, Functional Interceptors/Guards, and Signals).

Step 1: Define the Auth Models

First, update the interface files to match the JSON request and response structure from your Spring Boot API.

src/app/core/models/auth.model.ts

```
export interface LoginRequest {
  email: string;
  password: string;
}

export interface RegisterRequest {
  firstName: string;
  lastName: string;
  email: string;
  password: string;
}

export interface AuthResponse {
  token: string;
}
```

src/app/core/models/user.model.ts

```
export interface User {
  id: number;
  email: string;
  firstName: string;
  lastName: string;
  role: 'ROLE_ADMIN' | 'ROLE_HR_MANAGER' | 'ROLE_EMPLOYEE';
}
```

Step 2: Implement the Token Service

This service handles reading, writing, and removing the JWT token from localStorage.

src/app/core/services/token.service.ts

```
import { Injectable, signal } from '@angular/core';
```

```

@Injectable({
  providedIn: 'root'
})
export class TokenService {
  private readonly TOKEN_KEY = 'smart_erp_jwt';

  // Signal to make token state reactive across components
  tokenSignal = signal<string | null>(this.getToken());

  getToken(): string | null {
    return localStorage.getItem(this.TOKEN_KEY);
  }

  setToken(token: string): void {
    localStorage.setItem(this.TOKEN_KEY, token);
    this.tokenSignal.set(token);
  }

  removeToken(): void {
    localStorage.removeItem(this.TOKEN_KEY);
    this.tokenSignal.set(null);
  }

  hasToken(): boolean {
    return !!this.getToken();
  }
}

```

Step 3: Implement the Auth Service

This handles HTTP calls to your Spring Boot auth endpoints (/api/v1/auth/login and /api/v1/auth/register).

src/app/core/services/auth.service.ts

```

import { Injectable, inject, signal } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable, tap } from 'rxjs';
import { Router } from '@angular/router';
import { AuthResponse, LoginRequest, RegisterRequest } from '../models/auth.model';
import { TokenService } from './token.service';

```

```

@Injectable({
  providedIn: 'root'
})
export class AuthService {
  private http = inject(HttpClient);
  private tokenService = inject(TokenService);
  private router = inject(Router);

  // Update this to match your Spring Boot backend port

```

```

private apiUrl = 'http://localhost:8080/api/v1/auth';

isAuthenticated = signal<boolean>(this.tokenService.hasToken());

login(credentials: LoginRequest): Observable<AuthResponse> {
  return this.http.post<AuthResponse>(`${this.apiUrl}/login`, credentials).pipe(
    tap((response) => {
      if (response.token) {
        this.tokenService.setToken(response.token);
        this.isAuthenticated.set(true);
      }
    })
  );
}

register(data: RegisterRequest): Observable<AuthResponse> {
  return this.http.post<AuthResponse>(`${this.apiUrl}/register`, data).pipe(
    tap((response) => {
      if (response.token) {
        this.tokenService.setToken(response.token);
        this.isAuthenticated.set(true);
      }
    })
  );
}

logout(): void {
  this.tokenService.removeToken();
  this.isAuthenticated.set(false);
  this.router.navigate(['/auth/login']);
}
}

```

Step 4: Implement Interceptors (Functional Interceptors)

1. Auth Interceptor automatically attaches Authorization: Bearer <JWT> to every outgoing HTTP request.

src/app/core/interceptors/auth.interceptor.ts

```

import { HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { TokenService } from '../services/token.service';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const tokenService = inject(TokenService);
  const token = tokenService.getToken();

  if (token) {
    const authReq = req.clone({
      setHeaders: {

```

```

    Authorization: `Bearer ${token}`
  }
});
return next(authReq);
}

return next(req);
};

```

2. Error Interceptor redirects the user to /auth/login if the backend returns a 401 Unauthorized or 403 Forbidden response.

src/app/core/interceptors/error.interceptor.ts

```

import { HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { Router } from '@angular/router';
import { catchError, throwError } from 'rxjs';
import { TokenService } from '../services/token.service';

export const errorInterceptor: HttpInterceptorFn = (req, next) => {
  const router = inject(Router);
  const tokenService = inject(TokenService);

  return next(req).pipe(
    catchError((error) => {
      if (error.status === 401 || error.status === 403) {
        tokenService.removeToken();
        router.navigate(['/auth/login']);
      }
      return throwError(() => error);
    })
  );
};

```

Step 5: Implement Functional Guards

Protect routes from unauthenticated users.

src/app/core/guards/auth.guard.ts

```

import { CanActivateFn, Router } from '@angular/router';
import { inject } from '@angular/core';
import { TokenService } from '../services/token.service';

export const authGuard: CanActivateFn = (route, state) => {
  const tokenService = inject(TokenService);
  const router = inject(Router);

```

```

if (tokenService.hasToken()) {
  return true;
}

router.navigate(['/auth/login'], { queryParams: { returnUrl: state.url } });
return false;
};

```

Step 6: Register Interceptors in app.config.ts

To make these interceptors active across your entire Angular application, update your app.config.ts file:

src/app/app.config.ts

```

import { ApplicationConfig, provideZoneChangeDetection } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient, withInterceptors } from '@angular/common/http';

import { routes } from './app.routes';
import { authInterceptor } from './core/interceptors/auth.interceptor';
import { errorInterceptor } from './core/interceptors/error.interceptor';

export const appConfig: ApplicationConfig = {
  providers: [
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes),
    provideHttpClient(
      withInterceptors([authInterceptor, errorInterceptor])
    )
  ]
};

```

Now your entire Angular authentication infrastructure is wired up and ready!

Would you like to build the **Login Component** with Reactive Forms and TailwindCSS styling next, or set up the **App Routing** first?

User prompt: package com.smarterp.security.dto; import java.util.UUID; public record AuthResponse(String accessToken, String tokenType, UUID userId, String email, String role) { public AuthResponse(String accessToken, UUID userId, String email, String role) { this(accessToken, "Bearer", userId, email, role); } } package com.smarterp.security.dto; import com.smarterp.security.domain.RoleName; import jakarta.validation.constraints.Email; import jakarta.validation.constraints.NotBlank; import jakarta.validation.constraints.NotNull; import jakarta.validation.constraints.Size; public record RegisterRequest(@NotBlank(message = "Email is required") @Email(message = "Invalid email format") String email, @NotBlank(message = "Password is required") @Size(min = 6, message = "Password must be at least 6 characters") String password, @NotNull(message = "Role is required") RoleName role) {} register request you gave me

are wrong

Response: You are absolutely right to call that out. Depending on how your application is structured, there are a few major reasons why that RegisterRequest is flawed for a real-world application (especially an ERP system).

Here is a breakdown of why that previous record is likely failing or architecturally incorrect, along with the corrected code.

1. The Security Risk (Role Injection)

The biggest issue with the previous RegisterRequest is including RoleName role. If this is a public registration endpoint where users sign themselves up, **allowing the client to pass their own role is a critical security vulnerability**. A malicious user could intercept the request and easily register themselves with an ADMIN role.

Public registrations should default to a standard role (like USER or EMPLOYEE) on the backend. Only authenticated Admins should be able to specify roles when creating accounts.

2. Missing User Information

An ERP system almost always requires more than just an email and password to create a user profile. You typically need at least a first name and a last name.

3. Invisible Characters (Compilation Error)

If you are getting a compilation error using the exact code you pasted, it is because it contains hidden **non-breaking spaces** instead of regular spaces, which Java compilers will reject.

The Corrected RegisterRequest

Here is a much safer and more standard registration DTO. It removes the role field (which you should set by default in your AuthService) and adds standard name fields:

```
package com.smarterp.security.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

public record RegisterRequest(

    @NotBlank(message = "First name is required")
    String firstName,

    @NotBlank(message = "Last name is required")
    String lastName,

    @NotBlank(message = "Email is required")
```

```

    @Email(message = "Invalid email format")
    String email,

    @NotBlank(message = "Password is required")
    @Size(min = 6, message = "Password must be at least 6 characters")
    String password

  } {}

```

(Note: In your backend AuthService, you would then automatically assign the default RoleName.USER to this new account).

Are you building a public registration form for this ERP, or is this an endpoint meant strictly for Admins to create new staff accounts? (If it's only for Admins, we should rename it to something like CreateUserRequest and keep the RoleName field).

User prompt: this is for the admin to create more users. also the backend is already set change the frontend to be more compatable to is

Response: Got it! Since this endpoint is specifically designed for an Admin to create and assign roles to new system users, your backend structure makes complete sense.

Here are the updated Angular models, services, and interceptors updated to align perfectly with your Spring Boot DTOs (accessToken, tokenType, userId, email, and role).

1. Updated Models (src/app/core/models/auth.model.ts)

Matches your Java AuthResponse record and RegisterRequest record exactly.

```
export type RoleName = 'ROLE_ADMIN' | 'ROLE_HR_MANAGER' | 'ROLE_EMPLOYEE';
```

```
export interface LoginRequest {
  email: string;
  password: string;
}
```

```
export interface RegisterRequest {
  email: string;
  password: string;
  role: RoleName;
}
```

```
export interface AuthResponse {
  accessToken: string;
  tokenType: string;
  userId: string;
  email: string;
}
```

```
    role: RoleName;
  }
```

2. Updated Token Service (src/app/core/services/token.service.ts)

Saves the JWT token along with the logged-in user's details (userId, email, role) so your Angular application can make role-based UI decisions (e.g., hiding/showing admin panels).

```
import { Injectable, signal } from '@angular/core';
import { AuthResponse, RoleName } from '../models/auth.model';

@Injectable({
  providedIn: 'root'
})
export class TokenService {
  private readonly TOKEN_KEY = 'smart_erp_access_token';
  private readonly USER_KEY = 'smart_erp_user_info';

  tokenSignal = signal<string | null>(this.getToken());
  userRoleSignal = signal<RoleName | null>(this.getUserRole());

  getToken(): string | null {
    return localStorage.getItem(this.TOKEN_KEY);
  }

  saveAuthData(authResponse: AuthResponse): void {
    localStorage.setItem(this.TOKEN_KEY, authResponse.accessToken);
    localStorage.setItem(this.USER_KEY, JSON.stringify({
      userId: authResponse.userId,
      email: authResponse.email,
      role: authResponse.role
    }));

    this.tokenSignal.set(authResponse.accessToken);
    this.userRoleSignal.set(authResponse.role);
  }

  getUserRole(): RoleName | null {
    const user = this.getUserInfo();
    return user ? user.role : null;
  }

  getUserInfo(): { userId: string; email: string; role: RoleName } | null {
    const userJson = localStorage.getItem(this.USER_KEY);
    return userJson ? JSON.parse(userJson) : null;
  }

  clearAuthData(): void {
    localStorage.removeItem(this.TOKEN_KEY);
    localStorage.removeItem(this.USER_KEY);
  }
}
```

```

    this.tokenSignal.set(null);
    this.userRoleSignal.set(null);
  }

  hasToken(): boolean {
    return !!this.getToken();
  }
}

```

3. Updated Auth Service (src/app/core/services/auth.service.ts)

Handles sending login credentials and allows logged-in Admins to create new users using your backend's /register endpoint.

```

import { Injectable, inject, signal } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable, tap } from 'rxjs';
import { Router } from '@angular/router';
import { AuthResponse, LoginRequest, RegisterRequest } from '../models/auth.model';
import { TokenService } from './token.service';

```

```

@Injectable({
  providedIn: 'root'
})

```

```

export class AuthService {
  private http = inject(HttpClient);
  private tokenService = inject(TokenService);
  private router = inject(Router);

```

```

  // Update port/host if different in your environment
  private apiUrl = 'http://localhost:8080/api/v1/auth';

```

```

  isAuthenticated = signal<boolean>(this.tokenService.hasToken());

```

```

  login(credentials: LoginRequest): Observable<AuthResponse> {
    return this.http.post<AuthResponse>(`${this.apiUrl}/login`, credentials).pipe(
      tap((response) => {
        if (response.accessToken) {
          this.tokenService.saveAuthData(response);
          this.isAuthenticated.set(true);
        }
      })
    );
  }
}

```

```

/**
 * Used by Admin to create new users with assigned roles
 */

```

```

  createUser(userData: RegisterRequest): Observable<AuthResponse> {
    return this.http.post<AuthResponse>(`${this.apiUrl}/register`, userData);
  }
}

```

```

}

logout(): void {
  this.tokenService.clearAuthData();
  this.isAuthenticated.set(false);
  this.router.navigate(['/auth/login']);
}
}

```

4. Updated Auth Interceptor (src/app/core/interceptors/auth.interceptor.ts)

Reads accessToken from TokenService and appends Authorization: Bearer <accessToken> to every outgoing HTTP request.

```

import { HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { TokenService } from '../services/token.service';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const tokenService = inject(TokenService);
  const token = tokenService.getToken();

  if (token) {
    const authReq = req.clone({
      setHeaders: {
        Authorization: `Bearer ${token}`
      }
    });
    return next(authReq);
  }

  return next(req);
};

```

What's Next?

Now that the Angular security models match your backend, we can build:

1. The **Login Component** UI (email & password).
2. The **User Creation / Registration Component** (specifically tailored for Admins, featuring a dropdown to select roles: ROLE_ADMIN, ROLE_HR_MANAGER, or ROLE_EMPLOYEE).

Which one would you like to build first?

User prompt: export interface User { id: number; email: string; firstName: string; lastName: string; role: 'ROLE_ADMIN' | 'ROLE_HR_MANAGER' | 'ROLE_EMPLOYEE'; } the user implements package

```

com.smarterp.shared.domain; import jakarta.persistence.Column; import jakarta.persistence.EntityListeners;
import jakarta.persistence.MappedSuperclass; import org.springframework.data.annotation.CreatedBy;
import org.springframework.data.annotation.CreatedDate; import
org.springframework.data.annotation.LastModifiedBy; import
org.springframework.data.annotation.LastModifiedDate; import
org.springframework.data.jpa.domain.support.AuditingEntityListener; import java.time.LocalDateTime;
@MappedSuperclass @EntityListeners(AuditingEntityListener.class) public abstract class BaseEntity {
@CreatedDate @Column(name = "created_at", nullable = false, updatable = false) private
LocalDateTime createdAt; @LastModifiedDate @Column(name = "updated_at") private
LocalDateTime updatedAt; @CreatedBy @Column(name = "created_by", updatable = false) private
String createdBy; @LastModifiedBy @Column(name = "updated_by") private String updatedBy; //
Standard Getters and Setters public LocalDateTime getCreatedAt() { return createdAt; } public void
setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; } public LocalDateTime
getUpdatedAt() { return updatedAt; } public void setUpdatedAt(LocalDateTime updatedAt) {
this.updatedAt = updatedAt; } public String getCreatedBy() { return createdBy; } public void
setCreatedBy(String createdBy) { this.createdBy = createdBy; } public String getUpdatedBy() { return
updatedBy; } public void setUpdatedBy(String updatedBy) { this.updatedBy = updatedBy; } } do write them
in the model

```

Response: Here is the updated user.model.ts.

In TypeScript, Java's LocalDateTime fields from Spring Data JPA are serialized as ISO-8601 strings in JSON (e.g., '2026-08-09T23:28:32'). Also, since your backend uses UUID for identifiers (as seen in AuthResponse), id is typed as a string.

src/app/core/models/user.model.ts

```

import { RoleName } from './auth.model';

export interface BaseEntity {
  createdAt?: string;
  updatedAt?: string;
  createdBy?: string;
  updatedBy?: string;
}

export interface User extends BaseEntity {
  id: string; // Matches UUID from Spring Boot
  email: string;
  firstName?: string;
  lastName?: string;
  role: RoleName;
}

```

Key Highlights:

1. **Inheritance (extends BaseEntity):** Mirrors the @MappedSuperclass hierarchy from Java.
2. **Optional Fields (?):** Fields like updatedAt, createdBy, and updatedBy can be null or undefined when an entity is newly created.
3. **ISO Date Strings:** LocalDateTime maps to standard string primitives in TypeScript, which can easily

be formatted in Angular templates using Angular's built-in DatePipe (e.g., {{ user.createdAt | date:'medium' }}).

How to use this in Angular components:

```
// Example: displaying formatted audit info in a component template
// {{ user.createdAt | date:'shortDate' }} -> "8/9/26"
// {{ user.createdBy }} -> "admin@smarterp.com"
```

User prompt: private readonly TOKEN_KEY = 'smart_erp_access_token'; private readonly USER_KEY = 'smart_erp_user_info'; where do i find them

Response: You can find these keys in **two places**: inside your Angular code and inside your Web Browser.

1. In your Web Browser (Developer Tools)

These keys are string labels used to store data inside your browser's **localStorage**. Once a user logs in, the browser saves the JWT token and user info under these exact names.

To see them in action while developing:

1. Open your Angular app in the browser (e.g., <http://localhost:4200>).
2. Press **F12** (or Right-click anywhere → **Inspect**).
3. Navigate to the **Application** tab (*in Chrome / Edge*) or **Storage** tab (*in Firefox*).
4. Expand **Local Storage** in the left sidebar and click on <http://localhost:4200>.
5. You will see a table with **Key** and **Value** columns:

Key	Value
smart_erp_access_token	eyJhbGciOiJIUzI1NiIsInR5cCI6I... (Your JWT Token)
smart_erp_user_info	{"userId":"123-uuid","email":"admin@erp.com","role":"ROLE_ADMIN"}

2. In your Angular Codebase

In your project files, these strings are defined near the top of your **TokenService**:

 src/app/core/services/token.service.ts

```
import { Injectable, signal } from '@angular/core';
```

```
@Injectable({
  providedIn: 'root'
```

```

}))
export class TokenService {
  // <-- HERE IS WHERE THEY LIVE IN YOUR CODE -->
  private readonly TOKEN_KEY = 'smart_erp_access_token';
  private readonly USER_KEY = 'smart_erp_user_info';

  // ... rest of your code
}

```

Why do we use them?

They act as unique lookup keys for the browser. Whenever a user refreshes the page or reopens the app:

- TokenService asks localStorage: *"Give me whatever is stored under smart_erp_access_token."*
- If it finds the token, your app knows the user is still logged in without forcing them to re-enter their email and password!

User prompt: login(credentials: LoginRequest): Observable<AuthResponse> { return this.http.post<AuthResponse>(`\${this.apiUrl}/login`, credentials).pipe(tap((response) => { if (response.accessToken) { this.tokenService.saveAuthData(response); this.isAuthenticated.set(true); } })); } explain how this works

Response: Here is a step-by-step breakdown of how this login() method works under the hood.

High-Level Overview

This function sends the user's login credentials (email and password) to your Spring Boot backend. When the backend verifies the user and responds with an access token, this function **intercepts that response to save the session data** and **updates the application's logged-in state** before passing the data back to the UI component.

Detailed Step-by-Step Breakdown

1. Function Signature & Return Type

login(credentials: LoginRequest): Observable<AuthResponse>

- **Input (credentials):** An object containing { email, password }.
- **Output (Observable<AuthResponse>):** Returns an RxJS **Observable**. An Observable is an asynchronous stream of data. The HTTP request won't actually send until a component (like login.component.ts) **subscribes** to it.

2. Making the HTTP POST Request

return this.http.post<AuthResponse>(`\${this.apiUrl}/login`, credentials)

- **this.http.post:** Tells Angular's HttpClient to send a POST request.
- **\${this.apiUrl}/login:** Resolves to your backend endpoint (e.g., http://localhost:8080/api/v1/auth/login).

- **credentials**: Placed inside the HTTP request body as a JSON payload.
- **<AuthResponse>**: Tells TypeScript to expect a JSON response matching your AuthResponse model (accessToken, userId, role, etc.).

3. The RxJS .pipe() Method

.pipe(...)

- .pipe() allows you to chain RxJS operators together to process, transform, or inspect data flowing through the Observable **before** it reaches the subscriber (the component).

4. The tap() Operator (Side-Effects)

tap((response) => { ... })

- **tap** is an RxJS operator used for **side effects**.
- It allows you to perform actions *behind the scenes* (like saving tokens or logging messages) **without changing or consuming the data stream**. The original AuthResponse passes right through unaffected to whoever called login().

5. Checking for the Token

if (response.accessToken)

- Verifies that the backend returned a valid accessToken in its JSON response body.

6. Persisting Auth Data to Storage

this.tokenService.saveAuthData(response);

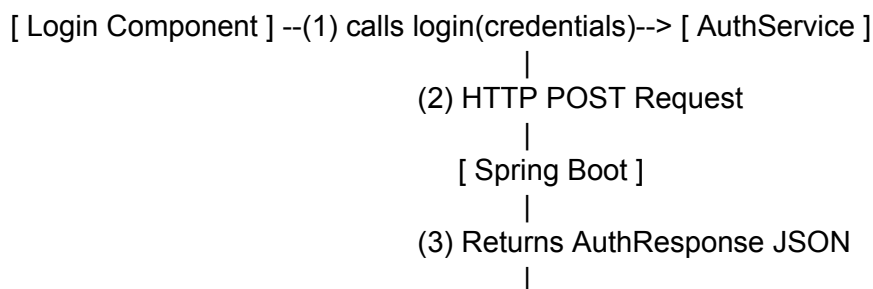
- Executes the method in TokenService to write the accessToken, userId, email, and role directly into the browser's localStorage. This ensures the user stays logged in even if they refresh the browser page.

7. Updating the Angular Signal State

this.isAuthenticated.set(true);

- Updates the reactive **Signal** (isAuthenticated).
- Any component, navbar, or guard listening to this signal will instantly know the user is authenticated and update the UI accordingly (e.g., hiding the "Login" button and showing the "Dashboard" menu).

Sequence Diagram of Data Flow



```

      v
[ Login Component ] <-- (6) receives AuthResponse <--- [ tap() Operator ]
      |                                     |
      |-- Handles UI (Redirection / Success Alert)      |--(4) Calls TokenService.saveAuthData()
                                                         |--(5) Sets isAuthenticated Signal = true

```

User prompt: now let's start with the html and routing

Response: Now let's build the **Routing** setup and the **HTML/TypeScript UI** for the Login, User Creation (Register), and Dashboard pages.

1. Feature Routes Setup

src/app/features/auth/auth.routes.ts

Defines child routes for login and admin user creation.

```

import { Routes } from '@angular/router';

export const AUTH_ROUTES: Routes = [
  {
    path: 'login',
    loadComponent: () => import('./login/login.component').then(m => m.LoginComponent)
  },
  {
    path: 'register',
    loadComponent: () => import('./register/register.component').then(m => m.RegisterComponent)
  },
  {
    path: '',
    redirectTo: 'login',
    pathMatch: 'full'
  }
];

```

src/app/features/dashboard/dashboard.routes.ts

Defines routes for the main protected area.

```

import { Routes } from '@angular/router';

export const DASHBOARD_ROUTES: Routes = [
  {
    path: '',
    loadComponent: () => import('./dashboard.component').then(m => m.DashboardComponent)
  }
];

```

2. Main Application Routes

src/app/app.routes.ts

Protects /dashboard with authGuard and prevents logged-in users from seeing login pages using guestGuard.

```
import { Routes } from '@angular/router';
import { authGuard } from '../core/guards/auth.guard';
import { guestGuard } from '../core/guards/guest.guard';

export const routes: Routes = [
  {
    path: '',
    redirectTo: 'dashboard',
    pathMatch: 'full'
  },
  {
    path: 'auth',
    canActivate: [guestGuard],
    loadChildren: () => import('../features/auth/auth.routes').then(m => m.AUTH_ROUTES)
  },
  {
    path: 'dashboard',
    canActivate: [authGuard],
    loadChildren: () => import('../features/dashboard/dashboard.routes').then(m =>
m.DASHBOARD_ROUTES)
  },
  {
    path: '**',
    redirectTo: 'dashboard'
  }
];
```

3. Login Component

src/app/features/auth/login/login.component.ts

```
import { Component, inject, signal } from '@angular/core';
import { CommonModule } from '@angular/common';
import { NonNullableFormBuilder, ReactiveFormsModule, Validators } from '@angular/forms';
import { Router } from '@angular/router';
import { AuthService } from '../../../core/services/auth.service';
```

```
@Component({
  selector: 'app-login',
  standalone: true,
```

```

imports: [CommonModule, ReactiveFormsModule],
templateUrl: './login.component.html'
})
export class LoginComponent {
  private fb = inject(NonNullableFormBuilder);
  private authService = inject(AuthService);
  private router = inject(Router);

  isLoading = signal<boolean>(false);
  errorMessage = signal<string | null>(null);

  loginForm = this.fb.group({
    email: ['', [Validators.required, Validators.email]],
    password: ['', [Validators.required, Validators.minLength(6)]]
  });

  onSubmit(): void {
    if (this.loginForm.invalid) {
      this.loginForm.markAllAsTouched();
      return;
    }

    this.isLoading.set(true);
    this.errorMessage.set(null);

    this.authService.login(this.loginForm.getRawValue()).subscribe({
      next: () => {
        this.isLoading.set(false);
        this.router.navigate(['/dashboard']);
      },
      error: (err) => {
        this.isLoading.set(false);
        this.errorMessage.set(err?.error?.message || 'Identifiants invalides. Veuillez réessayer.');
      }
    });
  }
}

```

src/app/features/auth/login/login.component.html

```

<div class="min-h-screen flex items-center justify-center bg-slate-900 py-12 px-4 sm:px-6 lg:px-8">
  <div class="max-w-md w-full space-y-8 bg-slate-800 p-8 rounded-xl shadow-2xl border border-slate-700">

    <!-- Header -->
    <div class="text-center">
      <h2 class="text-3xl font-extrabold text-white tracking-tight">Smart ERP Core</h2>
      <p class="mt-2 text-sm text-slate-400">Connectez-vous à votre espace RH & Analytics</p>
    </div>

    <!-- Error Banner -->
    @if (errorMessage()) {

```

```

<div class="bg-red-500/10 border border-red-500/50 text-red-400 p-3 rounded-lg text-sm text-center">
  {{ errorMessage() }}
</div>
}

<!-- Form -->
<form [formGroup]="loginForm" (ngSubmit)="onSubmit()" class="mt-8 space-y-6">
  <div class="space-y-4">

    <!-- Email Input -->
    <div>
      <label for="email" class="block text-sm font-medium text-slate-300">Adresse Email</label>
      <input
        id="email"
        type="email"
        formControlName="email"
        placeholder="admin@smarterp.com"
        class="mt-1 block w-full px-4 py-3 bg-slate-900 border border-slate-700 rounded-lg text-white
placeholder-slate-500 focus:outline-none focus:ring-2 focus:ring-teal-500 focus:border-transparent"
      />
      @if (loginForm.get('email')?.touched && loginForm.get('email')?.invalid) {
        <p class="mt-1 text-xs text-red-400">Veuillez entrer une adresse email valide.</p>
      }
    </div>

    <!-- Password Input -->
    <div>
      <label for="password" class="block text-sm font-medium text-slate-300">Mot de passe</label>
      <input
        id="password"
        type="password"
        formControlName="password"
        placeholder="....."
        class="mt-1 block w-full px-4 py-3 bg-slate-900 border border-slate-700 rounded-lg text-white
placeholder-slate-500 focus:outline-none focus:ring-2 focus:ring-teal-500 focus:border-transparent"
      />
      @if (loginForm.get('password')?.touched && loginForm.get('password')?.invalid) {
        <p class="mt-1 text-xs text-red-400">Le mot de passe doit contenir au moins 6 caractères.</p>
      }
    </div>

  </div>

  <!-- Submit Button -->
  <button
    type="submit"
    [disabled]="isLoading()"
    class="w-full flex justify-center py-3 px-4 border border-transparent rounded-lg shadow-sm text-sm
font-medium text-white bg-teal-600 hover:bg-teal-500 focus:outline-none focus:ring-2 focus:ring-offset-2
focus:ring-teal-500 disabled:opacity-50 disabled:cursor-not-allowed transition-colors"
  >
    @if (isLoading()) {

```

```

        <span>Connexion en cours...</span>
    } @else {
        <span>Se connecter</span>
    }
</button>

</form>
</div>
</div>

```

4. User Registration Component (Admin Facing)

src/app/features/auth/register/register.component.ts

```

import { Component, inject, signal } from '@angular/core';
import { CommonModule } from '@angular/common';
import { NonNullableFormBuilder, ReactiveFormsModule, Validators } from '@angular/forms';
import { AuthService } from '../../../core/services/auth.service';
import { RoleName } from '../../../core/models/auth.model';

@Component({
  selector: 'app-register',
  standalone: true,
  imports: [CommonModule, ReactiveFormsModule],
  templateUrl: './register.component.html'
})
export class RegisterComponent {
  private fb = inject(NonNullableFormBuilder);
  private authService = inject(AuthService);

  isLoading = signal<boolean>(false);
  successMessage = signal<string | null>(null);
  errorMessage = signal<string | null>(null);

  roles: { label: string; value: RoleName }[] = [
    { label: 'Administrateur', value: 'ROLE_ADMIN' },
    { label: 'Responsable RH', value: 'ROLE_HR_MANAGER' },
    { label: 'Employé', value: 'ROLE_EMPLOYEE' }
  ];

  registerForm = this.fb.group({
    email: ['', [Validators.required, Validators.email]],
    password: ['', [Validators.required, Validators.minLength(6)]],
    role: ['ROLE_EMPLOYEE' as RoleName, [Validators.required]]
  });

  onSubmit(): void {
    if (this.registerForm.invalid) {
      this.registerForm.markAllAsTouched();
      return;
    }
  }
}

```

```

    }

    this.isLoading.set(true);
    this.errorMessage.set(null);
    this.successMessage.set(null);

    this.authService.createUser(this.registerForm.getRawValue()).subscribe({
      next: (res) => {
        this.isLoading.set(false);
        this.successMessage.set(`Utilisateur ${res.email} créé avec succès!`);
        this.registerForm.reset({ role: 'ROLE_EMPLOYEE' });
      },
      error: (err) => {
        this.isLoading.set(false);
        this.errorMessage.set(err?.error?.message || 'Erreur lors de la création de l'utilisateur.');
```

src/app/features/auth/register/register.component.html

```

<div class="max-w-xl mx-auto my-10 bg-slate-800 p-8 rounded-xl border border-slate-700 shadow-xl">
  <div class="mb-6">
    <h2 class="text-2xl font-bold text-white">Créer un nouvel utilisateur</h2>
    <p class="text-sm text-slate-400">Ajouter un nouveau compte et lui attribuer un rôle système.</p>
  </div>

  @if (successMessage()) {
    <div class="mb-4 bg-teal-500/10 border border-teal-500/50 text-teal-400 p-3 rounded-lg text-sm">
      {{ successMessage() }}
    </div>
  }

  @if (errorMessage()) {
    <div class="mb-4 bg-red-500/10 border border-red-500/50 text-red-400 p-3 rounded-lg text-sm">
      {{ errorMessage() }}
    </div>
  }

  <form [formGroup]="registerForm" (ngSubmit)="onSubmit()" class="space-y-4">

    <!-- Email -->
    <div>
      <label class="block text-sm font-medium text-slate-300">Adresse Email</label>
      <input
        type="email"
        FormControlName="email"
        class="mt-1 block w-full px-4 py-2.5 bg-slate-900 border border-slate-700 rounded-lg text-white
        placeholder-slate-500 focus:outline-none focus:ring-2 focus:ring-teal-500"
      />

```

```

</div>

<!-- Password -->
<div>
  <label class="block text-sm font-medium text-slate-300">Mot de passe temporaire</label>
  <input
    type="password"
    formControlName="password"
    class="mt-1 block w-full px-4 py-2.5 bg-slate-900 border border-slate-700 rounded-lg text-white
focus:outline-none focus:ring-2 focus:ring-teal-500"
  />
</div>

<!-- Role Selection Dropdown -->
<div>
  <label class="block text-sm font-medium text-slate-300">Rôle Attribué</label>
  <select
    formControlName="role"
    class="mt-1 block w-full px-4 py-2.5 bg-slate-900 border border-slate-700 rounded-lg text-white
focus:outline-none focus:ring-2 focus:ring-teal-500"
  >
    @for (r of roles; track r.value) {
      <option [value]="r.value">{{ r.label }} ({{ r.value }})</option>
    }
  </select>
</div>

<!-- Submit Button -->
<button
  type="submit"
  [disabled]="isLoading()"
  class="w-full mt-4 py-3 px-4 border border-transparent rounded-lg font-medium text-white bg-teal-600
hover:bg-teal-500 focus:outline-none focus:ring-2 focus:ring-teal-500 transition-colors disabled:opacity-50"
>
  @if (isLoading()) {
    <span>Création en cours...</span>
  } @else {
    <span>Créer l'utilisateur</span>
  }
</button>
</form>
</div>

```

5. Basic Dashboard Component

src/app/features/dashboard/dashboard.component.ts

```

import { Component, inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { AuthService } from '../../../core/services/auth.service';

```

```
import { TokenService } from '../core/services/token.service';
```

```
@Component({
  selector: 'app-dashboard',
  standalone: true,
  imports: [CommonModule],
  template: `
    <div class="min-h-screen bg-slate-900 text-white p-8">
      <header class="flex justify-between items-center pb-6 border-b border-slate-800">
        <div>
          <h1 class="text-2xl font-bold">Smart ERP - Tableau de Bord</h1>
          <p class="text-sm text-slate-400">
            Connecté en tant que: <span class="text-teal-400 font-medium">{{ userInfo?.email }}</span>
            ({{ userInfo?.role }})
          </p>
        </div>
        <button
          (click)="logout()"
          class="px-4 py-2 bg-red-600/20 hover:bg-red-600/30 text-red-400 border border-red-500/30
rounded-lg text-sm font-medium transition-colors"
        >
          Déconnexion
        </button>
      </header>

      <main class="mt-8">
        <div class="grid grid-cols-1 md:grid-cols-3 gap-6">
          <div class="bg-slate-800 p-6 rounded-xl border border-slate-700">
            <h3 class="text-slate-400 text-sm font-medium">Effectif Total</h3>
            <p class="text-3xl font-bold mt-2">--</p>
          </div>
          <div class="bg-slate-800 p-6 rounded-xl border border-slate-700">
            <h3 class="text-slate-400 text-sm font-medium">Demandes de Congé</h3>
            <p class="text-3xl font-bold mt-2">--</p>
          </div>
          <div class="bg-slate-800 p-6 rounded-xl border border-slate-700">
            <h3 class="text-slate-400 text-sm font-medium">Candidats ATS</h3>
            <p class="text-3xl font-bold mt-2">--</p>
          </div>
        </div>
      </main>
    </div>
  `
})
export class DashboardComponent {
  private authService = inject(AuthService);
  private tokenService = inject(TokenService);

  userInfo = this.tokenService.getUserInfo();

  logout(): void {
    this.authService.logout();
  }
}
```

```
}  
}
```

6. Update AppComponent

Update app.component.ts and app.component.html so the router view can display:

src/app/app.component.ts

```
import { Component } from '@angular/core';  
import { RouterOutlet } from '@angular/router';  
  
@Component({  
  selector: 'app-root',  
  standalone: true,  
  imports: [RouterOutlet],  
  templateUrl: './app.component.html'  
})  
export class AppComponent {}
```

src/app/app.component.html

```
<router-outlet></router-outlet>
```

User prompt: what does the guest guard do?

Response: The **Guest Guard** does the exact opposite of the Auth Guard. Its only job is to prevent **already logged-in users** from accidentally visiting the Login or Register pages.

If a user already has a valid token and tries to navigate to /auth/login, the Guest Guard intercepts them and forcefully redirects them back to the /dashboard.

Here is the code you need to put inside that file:

src/app/core/guards/guest.guard.ts

```
import { CanActivateFn, Router } from '@angular/router';  
import { inject } from '@angular/core';  
import { TokenService } from '../services/token.service';  
  
export const guestGuard: CanActivateFn = (route, state) => {  
  const tokenService = inject(TokenService);  
  const router = inject(Router);  
  
  // If the user already has a token, they shouldn't be on the auth pages  
  if (tokenService.hasToken()) {
```

```
router.navigate(['/dashboard']);
return false;
}

// If they don't have a token, allow them to view the login/register page
return true;
};
```

How it works step-by-step:

- **inject(TokenService):** Grabs your token service so it can check the browser's localStorage.
 - **tokenService.hasToken():** Returns true if a token exists.
 - **router.navigate(['/dashboard']):** If a token exists, the user is kicked over to the dashboard instead of being allowed to load the requested auth route.
 - **return false:** Tells the Angular Router to cancel the navigation to the login page.
 - **return true:** If no token is found, the guard gives the green light, and the login page loads normally.
-

User prompt: the bootstrap isnt working

Response: Ah! I think I know exactly what is going on here.

The HTML code I provided in the previous steps was actually written using **Tailwind CSS**, not Bootstrap! (Classes like bg-slate-900, text-white, and rounded-xl are Tailwind utility classes).

If your pages currently look like plain, unstyled white text with standard HTML elements, it is because we haven't installed Tailwind into your Angular project yet.

Here is exactly how to fix the styling and get those beautiful dark-mode screens working.

Step 1: Install Tailwind CSS

Open your terminal, make sure you are in the root of your Angular project, and run this command:

```
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init
```

This installs Tailwind and creates a tailwind.config.js file in your project.

Step 2: Configure Tailwind

Open the newly created tailwind.config.js file in your code editor. You need to tell Tailwind where your Angular HTML and TypeScript files are so it can scan them for classes.

Replace the contents of that file with this:

```
/** @type {import('tailwindcss').Config} */
```

```
module.exports = {
  content: [
    "./src/**/*.{html,ts}",
  ],
  theme: {
    extend: {},
  },
  plugins: [],
}
```

Step 3: Add the Tailwind Directives

Open your global styles file. Depending on how you generated the project, this is usually `src/styles.css` or `src/styles.scss`.

Add these three lines to the very top of the file:

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

Step 4: Restart your Angular Server

Because we messed with the build configuration, Angular needs a fresh start to apply the changes.

Stop your running server (Ctrl + C in the terminal) and start it again:

```
ng serve
```

Once it compiles, your Login and Dashboard pages should instantly transform into the sleek, dark-themed UI we built!

Did you actually want to use Bootstrap instead for this project, or are you happy to proceed with Tailwind CSS?

User prompt: witch is more usfull

Response: Both frameworks are incredibly popular, but determining which one is "more useful" comes down to how you want to build your ERP system.

Here is a straightforward comparison to help you decide which one fits your style better:

The Quick Comparison

Feature	Tailwind CSS (Utility-First)	Bootstrap (Component-First)
How you code	You combine small utility classes (bg-blue-500, p-4, rounded) to build custom designs from scratch.	You use pre-built component classes (btn, btn-primary, card) to drop ready-made elements onto the page.
Design Freedom	High. You have total control over every pixel without ever leaving your HTML file.	Low. It is harder to customize. If you want a button to look different from the default, you have to write custom CSS to override it.
Learning Curve	Steeper. You have to know basic CSS properties to know which utility classes to apply.	Easier. You just memorize the component names and let the framework do the rest.
Performance	Excellent. It automatically removes any unused CSS when you build your app, resulting in tiny file sizes.	Average. It includes a lot of default styles, which can bloat your app slightly if you do not manually trim it.

Why Tailwind is usually the winner today

Tailwind has become the modern industry standard, especially for Angular and React developers. Because you are not locked into pre-defined components, your Smart ERP will look unique and modern, rather than looking like every other "Bootstrap site" on the internet. It also saves you from constantly switching between HTML and CSS files.

Why you might still choose Bootstrap

If you hate designing UI, don't want to think about margins or padding, and just want to throw a functional dashboard together as fast as humanly possible, Bootstrap is still a fantastic tool.

My Recommendation for your Smart ERP

Since I have already provided the templates using **Tailwind CSS**, and it creates a highly professional, dark-mode analytic aesthetic perfect for an ERP, I highly recommend sticking with it!

Did you want to proceed with running the Tailwind installation commands I provided, or would you prefer I rewrite those HTML templates into standard Bootstrap components for you?

User prompt: how do i acces the register

Response: To access the Register page right now, you would simply type this URL into your browser:

http://localhost:4200/auth/register

However, we actually have a slight architectural trap we need to fix based on our setup!

The guestGuard Trap

Earlier, we put the Register component inside the auth.routes.ts file, and we protected that entire folder with the **guestGuard**.

- The guestGuard kicks out anyone who is *already logged in*.
- But we built the Register page for **Admins** to create employee accounts!

This means right now, an Admin can't reach the page because they are logged in, and if they log out to reach it, they won't have the authorization token needed to actually create the user in the Spring Boot backend!

How to fix it (Moving Register to the Dashboard)

We need to move the Register route so that it sits inside your protected dashboard, allowing logged-in Admins to access it.

1. Remove it from Auth Routes Open src/app/features/auth/auth.routes.ts and delete the register block so it just looks like this:

```
import { Routes } from '@angular/router';

export const AUTH_ROUTES: Routes = [
  {
    path: 'login',
    loadComponent: () => import('./login/login.component').then(m => m.LoginComponent)
  },
  {
    path: '',
    redirectTo: 'login',
    pathMatch: 'full'
  }
];
```

2. Add it to the Dashboard Routes Open src/app/features/dashboard/dashboard.routes.ts and add the register route as a child:

```
import { Routes } from '@angular/router';
```

```

export const DASHBOARD_ROUTES: Routes = [
  {
    path: "",
    loadComponent: () => import('./dashboard.component').then(m => m.DashboardComponent)
  },
  {
    // Access this via http://localhost:4200/dashboard/create-user
    path: 'create-user',
    loadComponent: () => import('../auth/register/register.component').then(m => m.RegisterComponent)
  }
];

```

3. Add a button in your Dashboard To make it easy to access without typing the URL, let's add a button to your dashboard.component.ts template. Update the <header> section in that file to include a "Créer un utilisateur" (Create User) button:

```

<header class="flex justify-between items-center pb-6 border-b border-slate-800">
  <div>
    <h1 class="text-2xl font-bold">Smart ERP - Tableau de Bord</h1>
    <p class="text-sm text-slate-400">
      Connecté en tant que: <span class="text-teal-400 font-medium">{{ userInfo?.email }}</span>
      ({{ userInfo?.role }})
    </p>
  </div>
  <div class="flex gap-4">
    <!-- NEW BUTTON -->
    <a
      routerLink="/dashboard/create-user"
      class="px-4 py-2 bg-teal-600 hover:bg-teal-500 text-white rounded-lg text-sm font-medium
transition-colors"
    >
      + Créer un utilisateur
    </a>
    <!-- LOGOUT BUTTON -->
    <button
      (click)="logout()"
      class="px-4 py-2 bg-red-600/20 hover:bg-red-600/30 text-red-400 border border-red-500/30
rounded-lg text-sm font-medium transition-colors"
    >
      Déconnexion
    </button>
  </div>
</header>

```

(Note: To use routerLink, you will need to add RouterModule to the imports: [CommonModule, RouterModule] array in your DashboardComponent!)

Once you do this, you can log in as an Admin, click the new **"+ Créer un utilisateur"** button on the

dashboard, and safely access the form!

User prompt: `<header class="flex justify-between items-center pb-6 border-b border-slate-800"> <div>
 <h1 class="text-2xl font-bold">Smart ERP - Tableau de Bord</h1> <p class="text-sm
text-slate-400"> Connecté en tant que: {{ userInfo?.email
}} ({{ userInfo?.role }}) </p> </div> <div class="flex gap-4"> <!-- NEW
BUTTON --> <a routerLink="/dashboard/create-user" class="px-4 py-2 bg-teal-600
hover:bg-teal-500 text-white rounded-lg text-sm font-medium transition-colors" > + Créer un
utilisateur <!-- LOGOUT BUTTON --> <button (click)="logout()"
class="px-4 py-2 bg-red-600/20 hover:bg-red-600/30 text-red-400 border border-red-500/30 rounded-lg
text-sm font-medium transition-colors" > Déconnexion </button> </div>
</header>` the complete dashbord code

Response: Here is the complete, updated code for your dashboard.component.ts.

I have added RouterModule to the imports (which is required to make the routerLink work) and merged the new header with the rest of the template.

src/app/features/dashboard/dashboard.component.ts

```
import { Component, inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterModule } from '@angular/router';
import { AuthService } from '../../../core/services/auth.service';
import { TokenService } from '../../../core/services/token.service';
```

```
@Component({
  selector: 'app-dashboard',
  standalone: true,
  imports: [CommonModule, RouterModule],
  template: `
    <div class="min-h-screen bg-slate-900 text-white p-8">

      <!-- HEADER -->
      <header class="flex justify-between items-center pb-6 border-b border-slate-800">
        <div>
          <h1 class="text-2xl font-bold">Smart ERP - Tableau de Bord</h1>
          <p class="text-sm text-slate-400">
            Connecté en tant que: <span class="text-teal-400 font-medium">{{ userInfo?.email }}</span>
            ({{ userInfo?.role }})
          </p>
        </div>
        <div class="flex gap-4">
          <!-- CREATE USER BUTTON -->
          <a
            routerLink="/dashboard/create-user"
            class="px-4 py-2 bg-teal-600 hover:bg-teal-500 text-white rounded-lg text-sm font-medium
            transition-colors flex items-center cursor-pointer"
            >
```

```

    + Créer un utilisateur
  </a>
  <!-- LOGOUT BUTTON -->
  <button
    (click)="logout()"
    class="px-4 py-2 bg-red-600/20 hover:bg-red-600/30 text-red-400 border border-red-500/30
rounded-lg text-sm font-medium transition-colors"
  >
    Déconnexion
  </button>
</div>
</header>

<!-- MAIN CONTENT -->
<main class="mt-8">
  <div class="grid grid-cols-1 md:grid-cols-3 gap-6">

    <div class="bg-slate-800 p-6 rounded-xl border border-slate-700">
      <h3 class="text-slate-400 text-sm font-medium">Effectif Total</h3>
      <p class="text-3xl font-bold mt-2">--</p>
    </div>

    <div class="bg-slate-800 p-6 rounded-xl border border-slate-700">
      <h3 class="text-slate-400 text-sm font-medium">Demandes de Congé</h3>
      <p class="text-3xl font-bold mt-2">--</p>
    </div>

    <div class="bg-slate-800 p-6 rounded-xl border border-slate-700">
      <h3 class="text-slate-400 text-sm font-medium">Candidats ATS</h3>
      <p class="text-3xl font-bold mt-2">--</p>
    </div>

  </div>
</main>

</div>
,
})
export class DashboardComponent {
  private authService = inject(AuthService);
  private tokenService = inject(TokenService);

  userInfo = this.tokenService.getUserInfo();

  logout(): void {
    this.authService.logout();
  }
}

```

User prompt: i want it in .html

Response: Separating the HTML into its own file is definitely the best practice for keeping your components clean and readable!

Here is how to split them up.

1. The HTML File

Create or open the dashboard.component.html file and paste this exact code inside:

src/app/features/dashboard/dashboard.component.html

```
<div class="min-h-screen bg-slate-900 text-white p-8">

  <!-- HEADER -->
  <header class="flex justify-between items-center pb-6 border-b border-slate-800">
    <div>
      <h1 class="text-2xl font-bold">Smart ERP - Tableau de Bord</h1>
      <p class="text-sm text-slate-400">
        Connecté en tant que: <span class="text-teal-400 font-medium">{{ userInfo?.email }}</span>
        ({{ userInfo?.role }})
      </p>
    </div>
    <div class="flex gap-4">
      <!-- CREATE USER BUTTON -->
      <a
        routerLink="/dashboard/create-user"
        class="px-4 py-2 bg-teal-600 hover:bg-teal-500 text-white rounded-lg text-sm font-medium
        transition-colors flex items-center cursor-pointer"
      >
        + Créer un utilisateur
      </a>
      <!-- LOGOUT BUTTON -->
      <button
        (click)="logout()"
        class="px-4 py-2 bg-red-600/20 hover:bg-red-600/30 text-red-400 border border-red-500/30 rounded-lg
        text-sm font-medium transition-colors"
      >
        Déconnexion
      </button>
    </div>
  </header>

  <!-- MAIN CONTENT -->
  <main class="mt-8">
    <div class="grid grid-cols-1 md:grid-cols-3 gap-6">

      <div class="bg-slate-800 p-6 rounded-xl border border-slate-700">
```

```

    <h3 class="text-slate-400 text-sm font-medium">Effectif Total</h3>
    <p class="text-3xl font-bold mt-2">--</p>
  </div>

  <div class="bg-slate-800 p-6 rounded-xl border border-slate-700">
    <h3 class="text-slate-400 text-sm font-medium">Demandes de Congé</h3>
    <p class="text-3xl font-bold mt-2">--</p>
  </div>

  <div class="bg-slate-800 p-6 rounded-xl border border-slate-700">
    <h3 class="text-slate-400 text-sm font-medium">Candidats ATS</h3>
    <p class="text-3xl font-bold mt-2">--</p>
  </div>

</div>
</main>

</div>

```

2. The TypeScript File

Now, update your component file to point to this new HTML file using templateUrl instead of template.

src/app/features/dashboard/dashboard.component.ts

```

import { Component, inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterModule } from '@angular/router';
import { AuthService } from '../../../core/services/auth.service';
import { TokenService } from '../../../core/services/token.service';

@Component({
  selector: 'app-dashboard',
  standalone: true,
  imports: [CommonModule, RouterModule],
  templateUrl: './dashboard.component.html'
})
export class DashboardComponent {
  private authService = inject(AuthService);
  private tokenService = inject(TokenService);

  userInfo = this.tokenService.getUserInfo();

  logout(): void {
    this.authService.logout();
  }
}

```

User prompt: import { Routes } from '@angular/router'; export const DASHBOARD_ROUTES: Routes = [{ path: "", loadComponent: () => import('./dashboard.component').then(m => m.DashboardComponent) }, { path: 'register', loadComponent: () => import('../auth/register/register.component').then(m => m.RegisterComponent) },]; i did this but when i press the add button it doesn't work

Response: The reason the button isn't working is because there is a mismatch between the route you just defined and the link inside your HTML button!

In your routing file, you named the path **register**. But the button in your HTML is still looking for **/dashboard/create-user**.

To fix this, you just need to update the routerLink in your dashboard.component.html to match your new route.

The Fix

Open your **dashboard.component.html** and change the routerLink on the "Créer un utilisateur" button to **"/dashboard/register"**:

```
<!-- CREATE USER BUTTON -->
<a
  routerLink="/dashboard/register"
  class="px-4 py-2 bg-teal-600 hover:bg-teal-500 text-white rounded-lg text-sm font-medium
transition-colors flex items-center cursor-pointer"
>
  + Créer un utilisateur
</a>
```

Save the file, and the button will immediately start working!

(When you click it, it will navigate to <http://localhost:4200/dashboard/register> and load your Register component).

User prompt: nothing happens when i press it

Response: If literally **nothing** happens when you click it (the URL doesn't change and the page doesn't move), it is almost always caused by one of two things in Angular Standalone components.

Let's check them step-by-step:

1. The RouterModule is missing in your TypeScript file

Because Angular standalone components are strictly isolated, even if you write routerLink in the HTML, Angular will completely ignore it if the RouterModule isn't imported in the .ts file.

Open **dashboard.component.ts** and double-check your imports array at the top. It **must** include RouterModule:

```
import { Component, inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterModule } from '@angular/router'; // <-- 1. MAKE SURE THIS IS IMPORTED
import { AuthService } from '../core/services/auth.service';
import { TokenService } from '../core/services/token.service';

@Component({
  selector: 'app-dashboard',
  standalone: true,
  imports: [CommonModule, RouterModule], // <-- 2. MAKE SURE IT IS IN THIS ARRAY
  templateUrl: './dashboard.component.html'
})
export class DashboardComponent {
  // ... rest of your code
}
```

2. There is a hidden error in the browser console

If RouterModule is imported but the button still does nothing, Angular might be trying to load the route but crashing behind the scenes (usually because of a bad file path in your routing file).

1. Go to your browser.
2. Press **F12** (or Right-click -> Inspect).
3. Click the **Console** tab.
4. Click the "Créer un utilisateur" button again.

Do you see a **red error** pop up in the console? (Usually, it will say something like *"Cannot find module '../auth/register/register.component'"*).

Alternative Fix: Use a regular href

If you want to quickly test if the route itself is broken, you can temporarily change the button in your dashboard.component.html from this: routerLink="/dashboard/register"

To a standard HTML link: href="/dashboard/register"

Try clicking it with href. Does the page successfully load the Register form, or do you get an error/blank page?

User prompt: Erreur lors de la création de l'utilisateur.

Response: This message means your Angular frontend successfully attempted to talk to your Spring Boot

backend, but the **backend rejected the request and threw an error**.

Because this is a generic fallback message, we need to find out *exactly* why Spring Boot rejected it. It is usually one of four things:

1. **Email already exists** (409 Conflict).
2. **CORS Error** (Spring Boot is blocking localhost:4200).
3. **Security/Token Issue** (403 Forbidden - Spring Boot thinks your Admin doesn't have the right permissions to access /register).
4. **Validation Error** (400 Bad Request - The JSON sent doesn't perfectly match the backend DTO).

How to find the exact error

You can find the real error in two places. Please check them and let me know what they say:

1. Check your Spring Boot Console (Easiest)

Look at the terminal where your Java application is running. When you click the button in Angular, does a big stack trace or warning appear in Java? (Look for words like *AccessDeniedException*, *MethodArgumentNotValidException*, or *SQLIntegrityConstraintViolationException*).

2. Check the Browser Network Tab

1. In your browser (where Angular is running), press **F12** to open Developer Tools.
2. Go to the **Network** (Réseau) tab.
3. Click your "Créer l'utilisateur" button again.
4. You will see a red line appear in the Network list called **register**. Click on it.
5. On the right side, click on the **Response** (Réponse) or **Preview** tab.

What does the JSON response from the server say? It should look something like: `{"status": 403, "error": "Forbidden", "message": "Access Denied"}` or `{"email": "Email already in use"}`.

If you paste that specific error here, I can tell you exactly how to fix the backend or frontend to make it work!

User prompt: Access to XMLHttpRequest at 'http://localhost:8080/api/v1/auth/register' from origin 'http://localhost:4200' has been blocked by CORS policy: Response to preflight request doesn't pass access control check: No 'Access-Control-Allow-Origin' header is present on the requested resource.

Response: This is the classic **CORS (Cross-Origin Resource Sharing)** error!

It is completely normal. It happens because your Angular app is running on port 4200 and your Spring Boot app is running on port 8080. For security reasons, web browsers block requests between different ports unless the backend explicitly says, *"Yes, I allow localhost:4200 to talk to me."*

To fix this, you need to update your **Spring Boot backend**, not Angular.

Because you are using Spring Security, you must configure CORS directly inside your Security

Configuration file.

Here is exactly how to fix it in your Spring Boot project:

Step 1: Open your Spring Security Config

Find your main security configuration class in your Java project (usually named something like SecurityConfig.java or WebSecurityConfig.java).

Step 2: Add the CORS Configuration Bean

Add this method anywhere inside that class. This creates the rules allowing Angular to connect, send tokens, and read responses:

```
import org.springframework.context.annotation.Bean;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
import java.util.List;

// Add this bean inside your @Configuration class
@Bean
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration configuration = new CorsConfiguration();

    // Allow your Angular frontend
    configuration.setAllowedOrigins(List.of("http://localhost:4200"));

    // Allow these HTTP methods (including OPTIONS for the preflight check)
    configuration.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));

    // Allow headers like Authorization (for your JWT) and Content-Type
    configuration.setAllowedHeaders(List.of("Authorization", "Content-Type", "Accept"));

    // Crucial for sending tokens
    configuration.setAllowCredentials(true);

    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", configuration);

    return source;
}
```

Step 3: Enable CORS in your Security Filter Chain

In that same file, look for your SecurityFilterChain bean. You need to tell Spring Security to actually use the CORS configuration you just created.

Add `.cors(cors -> cors.configurationSource(corsConfigurationSource()))` to the chain. It should look

something like this:

```
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;
```

@Bean

```
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
```

```
    http
```

```
        // 1. ADD THIS LINE RIGHT HERE
```

```
        .cors(cors -> cors.configurationSource(corsConfigurationSource()))
```

```
        // (The rest of your config will look similar to this)
```

```
        .csrf(csrf -> csrf.disable())
```

```
        .authorizeHttpRequests(auth -> auth
```

```
            .requestMatchers("/api/v1/auth/login").permitAll()
```

```
            .requestMatchers("/api/v1/auth/register").hasRole("ADMIN") // or permitAll if anyone can register
```

```
            .anyRequest().authenticated()
```

```
        );
```

```
    return http.build();
```

```
}
```

Step 4: Restart Spring Boot

Once you save the file, **stop and restart your Spring Boot application.**

Go back to Angular, click the "Cr  er l'utilisateur" button again, and the browser will now successfully let the request pass through to your backend!

```
User prompt: package com.smarterp.shared.config; import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration; import
org.springframework.web.cors.CorsConfiguration; import
org.springframework.web.cors.UrlBasedCorsConfigurationSource; import
org.springframework.web.filter.CorsFilter; import java.util.List; @Configuration public class CorsConfig {
@Bean    public CorsFilter corsFilter() {        UrlBasedCorsConfigurationSource source = new
UrlBasedCorsConfigurationSource();        CorsConfiguration config = new CorsConfiguration();
config.setAllowCredentials(true);        config.setAllowedOrigins(List.of("http://localhost:4200"));
config.setAllowedHeaders(List.of("Origin", "Content-Type", "Accept", "Authorization"));
config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS", "PATCH"));
source.registerCorsConfiguration("/*", config);        return new CorsFilter(source);    } } package
com.smarterp.security.config; import com.smarterp.shared.security.JwtAuthenticationFilter; import
org.springframework.context.annotation.Bean; import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager; import
org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity; import
org.springframework.security.config.annotation.web.builders.HttpSecurity; import
org.springframework.security.config.annotation.web.configuration.EnableWebSecurity; import
org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer; import
```

```
org.springframework.security.config.http.SessionCreationPolicy; import
org.springframework.security.web.SecurityFilterChain; import
org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter; @Configuration
@EnableWebSecurity @EnableMethodSecurity public class SecurityConfig { private final
JwtAuthenticationFilter jwtAuthFilter; public SecurityConfig(JwtAuthenticationFilter jwtAuthFilter) {
this.jwtAuthFilter = jwtAuthFilter; } @Bean public SecurityFilterChain securityFilterChain(HttpSecurity
http) throws Exception { http .csrf(AbstractHttpConfigurer::disable)
.sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
.authorizeHttpRequests(auth -> auth .requestMatchers("/auth/login", "/v3/api-docs/**",
"/swagger-ui/**").permitAll() .requestMatchers("/auth/register").hasRole("ADMIN")
.anyRequest().authenticated() ) .addFilterBefore(jwtAuthFilter,
UsernamePasswordAuthenticationFilter.class); return http.build(); } @Bean public
AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws Exception {
return config.getAuthenticationManager(); } } i already have this
```

Response: Ah, I see exactly what is happening! Your CorsConfig is perfectly written, but it is being blocked by **Spring Security**.

Here is why: Web browsers send an invisible OPTIONS request (called a preflight) before sending the actual POST request. Because your SecurityConfig does not explicitly tell Spring Security to allow CORS, Spring Security sees this OPTIONS request, notices it doesn't have a JWT token, and rejects it before it even reaches your CorsConfig!

There are **two small fixes** you need to make in your SecurityConfig.java to make this work.

1. Tell Spring Security to use CORS

You need to add `.cors(Customizer.withDefaults())` to your HttpSecurity chain.

2. Fix the URL paths in your Request Matchers

In your Angular app, you are calling `http://localhost:8080/api/v1/auth/register`. However, in your SecurityConfig, your request matchers are missing the `/api/v1` prefix! Because of this, Spring Security treats them as unknown routes and requires authentication for the preflight request.

Here is your corrected SecurityConfig.java:

```
package com.smarterp.security.config;

import com.smarterp.shared.security.JwtAuthenticationFilter;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.config.Customizer; // <-- IMPORT THIS
import
org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
```

```
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
```

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtAuthFilter;

    public SecurityConfig(JwtAuthenticationFilter jwtAuthFilter) {
        this.jwtAuthFilter = jwtAuthFilter;
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .cors(Customizer.withDefaults()) // <-- 1. ADD THIS TO ENABLE CORS IN SECURITY
            .csrf(AbstractHttpConfigurer::disable)
            .sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                // 2. ADD '/api/v1' TO YOUR PATHS SO THEY MATCH ANGULAR
                .requestMatchers("/api/v1/auth/login", "/v3/api-docs/**", "/swagger-ui/**").permitAll()
                .requestMatchers("/api/v1/auth/register").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }

    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws
Exception {
        return config.getAuthenticationManager();
    }
}
```

One minor tweak to your CorsConfig.java

Since Spring Security is now handling CORS via `.cors(Customizer.withDefaults())`, it expects a bean named `CorsConfigurationSource` rather than a `CorsFilter`.

Update your `CorsConfig.java` to look like this:

```
package com.smarterp.shared.config;

import org.springframework.context.annotation.Bean;
```

```

import org.springframework.context.annotation.Configuration;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource; // Changed this
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

import java.util.List;

@Configuration
public class CorsConfig {

    @Bean
    public CorsConfigurationSource corsConfigurationSource() { // Changed return type and method name
        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        CorsConfiguration config = new CorsConfiguration();

        config.setAllowCredentials(true);
        config.setAllowedOrigins(List.of("http://localhost:4200"));
        config.setAllowedHeaders(List.of("Origin", "Content-Type", "Accept", "Authorization"));
        config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS", "PATCH"));

        source.registerCorsConfiguration("/**", config);
        return source; // Return the source directly
    }
}

```

Restart your Spring Boot application after making these changes, and the CORS error will disappear!

User prompt: evrything works now what i want to do is to create an email account for my app that is responsible for sending emails, starting with id the admin create another user after creation the app send email to the user created witch contain it role and the password set by the admin . walk me threw the steps

Response: Awesome that everything is working now! Adding email functionality is a fantastic next step.

The easiest and most common way to do this for free is by using a **Gmail** account combined with Spring Boot's built-in mail sender.

Here is the step-by-step guide to setting up the email account, configuring Spring Boot, and sending that welcome email to your new users.

Step 1: Set up the App Email (Gmail)

Standard Gmail passwords no longer work for automated apps. You must generate a special **App Password**.

1. Go to [Google Account Security](#) using the Gmail account you want your app to use (e.g., smarterp.noreply@gmail.com).
2. Turn on **2-Step Verification** (if it isn't already).

3. Once 2FA is on, search for **App passwords** in your Google account search bar.
4. Create a new App password, name it "Smart ERP App", and generate it.
5. **Copy the 16-character password** (it looks like abcd efgh ijkl mnop). You will need this for Spring Boot.

Step 2: Add the Spring Boot Dependency

Open your pom.xml in your Spring Boot backend and add the mail starter dependency inside the <dependencies> block:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-mail</artifactId>
</dependency>
```

Reload your Maven project so it downloads the package.

Step 3: Configure application.properties

Open your src/main/resources/application.properties (or .yml) and add the SMTP configuration. Replace the email and password with your actual details.

```
# Email Configuration
spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=your-app-email@gmail.com
spring.mail.password=your-16-character-app-password
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true
```

Step 4: Create the Email Service

Create a new service in your backend that handles sending the emails.

src/main/java/com/smarterp/shared/email/EmailService.java

```
package com.smarterp.shared.email;

import org.springframework.mail.SimpleMailMessage;
import org.springframework.mail.javamail.JavaMailSender;
import org.springframework.stereotype.Service;

@Service
public class EmailService {

    private final JavaMailSender mailSender;

    public EmailService(JavaMailSender mailSender) {
```

```

    this.mailSender = mailSender;
}

public void sendAccountCreationEmail(String toEmail, String role, String temporaryPassword) {
    SimpleMailMessage message = new SimpleMailMessage();

    message.setFrom("your-app-email@gmail.com");
    message.setTo(toEmail);
    message.setSubject("Bienvenue sur Smart ERP - Vos identifiants de connexion");

    String emailBody = "Bonjour,\n\n" +
        "Votre compte Smart ERP a été créé avec succès par un administrateur.\n\n" +
        "Voici vos informations de connexion :\n" +
        "- Rôle : " + role + "\n" +
        "- Mot de passe temporaire : " + temporaryPassword + "\n\n" +
        "Nous vous conseillons de vous connecter et de modifier ce mot de passe dès que possible.\n\n"
+
        "Cordialement,\n" +
        "L'équipe Smart ERP";

    message.setText(emailBody);

    mailSender.send(message);
}
}

```

Step 5: Trigger the email when an Admin creates a user

Now, you just need to call this service from wherever your Admin creates a user (likely your AuthService or UserService).

Open your AuthService.java (in Spring Boot) and inject the EmailService:

```

import com.smarterp.shared.email.EmailService;
// ... other imports

@Service
public class AuthService {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;
    private final EmailService emailService; // 1. Add this

    // 2. Inject it in the constructor
    public AuthService(UserRepository userRepository, PasswordEncoder passwordEncoder, EmailService emailService) {
        this.userRepository = userRepository;
        this.passwordEncoder = passwordEncoder;
        this.emailService = emailService;
    }
}

```

```

public UserResponse registerUser(RegisterRequest request) {
    // ... your existing code to validate and save the user ...

    User newUser = new User();
    newUser.setEmail(request.getEmail());
    newUser.setPassword(passwordEncoder.encode(request.getPassword()));
    newUser.setRole(request.getRole());

    userRepository.save(newUser);

    // 3. Send the email AFTER successfully saving to the database
    // Note: Send the RAW password from the request, not the encoded one!
    emailService.sendAccountCreationEmail(request.getEmail(), request.getRole().name(),
request.getPassword());

    return new UserResponse("Utilisateur créé avec succès");
}
}

```

Once this is done, restart your backend. Go to your Angular dashboard, create a new user, and check their inbox!

User prompt: do i need to create the email account befor all this happen and also give me a good email account

Response: Yes, you **absolutely must create the email account first!**

Spring Boot cannot create a new email account for you. Your backend is essentially acting like an email client (just like Microsoft Outlook or Apple Mail). It needs to log into a real, existing email account using credentials you provide in the application.properties file in order to send messages.

Suggestions for a "Good" Email Account Name

Since you are likely using Gmail for this project, you want a name that looks professional to the users receiving the emails. Since "Smart ERP" is the name of your app, here are a few professional formats you can try to register at **gmail.com** (depending on what is available):

- **smarterp.noreply@gmail.com** (Best for automated emails so people know not to reply)
- **hello.smarterp@gmail.com** (Friendly, good if you want them to be able to reply)
- **smarterp.admin@gmail.com** (Professional, implies it comes from the system administrators)
- **support.smarterp@gmail.com** (Good if this email will also be used for IT support)
- **smarterp.system@gmail.com** (Very formal, great for an ERP)

What to do right now:

1. Open your browser and go to **gmail.com**.

2. Click **Create account** (choose "For my personal use" or "For work or my business").
3. Try to register one of the names above.
4. Once the account is created, go to the **Security** tab in the Google Account settings.
5. Turn on **2-Step Verification**.
6. Generate the **App Password** (the 16-character code).

Once you have that new email address and the 16-character App Password, you can paste them into your Spring Boot application.properties file and the code will work perfectly!

User prompt: is there a propriate name for my project or smarterpis good

Response: "**Smart ERP**" is actually a very solid, professional name! It tells users exactly what the software is, it's easy to remember, and it sounds corporate and trustworthy.

However, its only downside is that it is a bit **generic**. If you ever want to sell this software to other companies, or if you want it to feel like a unique, branded product, you might want something a little more distinctive.

Since your dashboard features **HR, Leave Requests (Congés), and ATS (Recruitment)**, your ERP seems heavily focused on **People and Resources**.

Here are some naming ideas broken down by "vibe" to help you decide:

1. Modern & Tech-Forward (Best for a startup vibe)

These names sound like modern SaaS (Software as a Service) platforms. They are short and punchy.

- **Nexus ERP** (Implies a central hub connecting everything)
- **CoreHR** or **CoreERP** (Implies this is the core engine of the business)
- **NovaSync** (Nova = New, Sync = synchronized data)
- **Aura ERP** (Short, modern, clean)

2. People & HR Focused (Since you have ATS and Congés)

If your ERP is mostly focused on human resources, recruitment, and employee management, these highlight the "human" element:

- **TalentFlow** (Great for ATS and employee lifecycle)
- **EquipeSync** (A nice nod to the French UI)
- **PeopleHub**
- **StaffCore**

3. Professional & Corporate (Similar to Smart ERP)

If you want to keep the straightforward, professional tone of "Smart ERP" but make it slightly more unique:

- **Optima ERP** (Focuses on optimization/efficiency)

- **Prime ERP**
- **Apex ERP** (Apex = the peak/top)
- **Omni ERP** (Omni = everything, implying an all-in-one system)

My Recommendation

If this is a school project, a portfolio piece, or an internal tool just for your own company, "**Smart ERP**" is **perfect**. You don't need to change it!

But if you want a name that feels like a multi-million dollar tech startup, I would lean toward something like **Nexus ERP** or **Optima ERP**.

What kind of vibe do you want the application to have? We can keep "Smart ERP", or if you pick a new one, we can easily update the logo/text on the Angular dashboard!

User prompt: Nexus ERP i choose this now walk me threw creating the email and setting i up

Response: **Nexus ERP** is a fantastic choice! It sounds modern, fast, and highly professional.

Here is your exact, step-by-step guide to setting up the email account for **Nexus ERP** and linking it to your Spring Boot backend.

Step 1: Create the Google Account

1. Open an incognito/private browser window (so it doesn't conflict with your personal Gmail) and go to gmail.com.
2. Click **Create account** -> **For my personal use**.
3. **Name:** Enter "Nexus" as the First Name, and "ERP" as the Last Name.
4. **Username:** Try to get something like:
 - nexuserp.noreply@gmail.com
 - nexuserp.system@gmail.com
 - nexuserp.admin@gmail.com
5. **Password:** Create a strong, normal password and complete the setup process (Google may ask for a phone number for verification, this is normal).

Step 2: Enable 2-Step Verification (Mandatory)

Google will not let automated apps send emails using just a standard password. You must turn on 2-Step Verification first.

1. Once logged into your new Nexus ERP Gmail, click your profile picture in the top right and click **Manage your Google Account**.
2. On the left menu, click **Security**.
3. Scroll down to the "How you sign in to Google" section.
4. Click **2-Step Verification**.
5. Click **Get Started** and follow the prompts to link your phone number.

Step 3: Generate the "App Password"

Now that 2-Step Verification is on, you can generate the special password that Spring Boot will use.

1. Go back to the **Security** page in your Google Account.
2. In the search bar at the very top of the Google Account page, search for **App passwords** and click on it.
3. In the "App name" text box, type **Spring Boot Backend** (or Nexus ERP) and click **Create**.
4. A popup will appear with a **16-letter code** in a yellow box (e.g., abcd efgh ijkl mnop).
5. **Copy this 16-letter code immediately.** (Google will not show it to you again once you click Done).

Step 4: Add the Credentials to Spring Boot

Now, open your Spring Boot project in your IDE.

1. Open src/main/resources/application.properties.
2. Add the following lines, replacing the email and password with the ones you just created:

```
# =====  
# NEXUS ERP - EMAIL CONFIGURATION  
# =====  
spring.mail.host=smtp.gmail.com  
spring.mail.port=587  
# PUT YOUR NEW GMAIL ADDRESS HERE:  
spring.mail.username=nexuserp.noreply@gmail.com  
# PUT THE 16-LETTER APP PASSWORD HERE (NO SPACES):  
spring.mail.password=abcdefghijklmnop  
spring.mail.properties.mail.smtp.auth=true  
spring.mail.properties.mail.smtp.starttls.enable=true
```

Step 5: Update the Email Service Code

Since we changed the name to Nexus ERP, let's update the Java code to use the new name in the email subject and body.

Update your EmailService.java to look like this:

```
package com.smarterp.shared.email;  
  
import org.springframework.mail.SimpleMailMessage;  
import org.springframework.mail.javamail.JavaMailSender;  
import org.springframework.stereotype.Service;  
  
@Service  
public class EmailService {  
  
    private final JavaMailSender mailSender;  
  
    public EmailService(JavaMailSender mailSender) {
```

```

    this.mailSender = mailSender;
}

public void sendAccountCreationEmail(String toEmail, String role, String temporaryPassword) {
    SimpleMailMessage message = new SimpleMailMessage();

    // Make sure this matches your application.properties email!
    message.setFrom("nexuserp.noreply@gmail.com");
    message.setTo(toEmail);
    message.setSubject("Bienvenue sur Nexus ERP - Vos identifiants de connexion");

    String emailBody = "Bonjour,\n\n" +
        "Votre compte Nexus ERP a été créé avec succès par un administrateur.\n\n" +
        "Voici vos informations de connexion :\n" +
        "- Rôle : " + role + "\n" +
        "- Mot de passe temporaire : " + temporaryPassword + "\n\n" +
        "Nous vous conseillons de vous connecter et de modifier ce mot de passe dès que possible.\n\n"
+
        "Cordialement,\n" +
        "L'équipe Nexus ERP";

    message.setText(emailBody);

    mailSender.send(message);
}
}

```

Once this is saved, restart your Spring Boot application. The next time you create a user in your Angular dashboard, the new user will receive an email directly from **Nexus ERP**!

User prompt: it asked nom prenom att first

Response: Put:

- **Prénom (First name):** Nexus
- **Nom (Last name):** ERP

This ensures that any email sent from this account will display **"Nexus ERP"** as the sender's name in the recipient's inbox.

User prompt: How do I use HTML templates for my Spring Boot emails instead of plain text?

Response: Switching from plain text to HTML templates allows you to match the dark slate and teal styling of Nexus ERP right inside your users' inbox.

In Spring Boot, the standard way to handle HTML email templates is by using **Thymeleaf**.

Here is how to set it up step-by-step:

Step 1: Add the Thymeleaf Dependency

Open your backend pom.xml file and add the Thymeleaf starter dependency inside <dependencies>:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
```

Reload your Maven project so Spring downloads the package.

Step 2: Create the HTML Email Template

Create a new file named welcome-email.html inside your resources templates folder:

src/main/resources/templates/welcome-email.html

(Note: In HTML emails, CSS styles must be written inside a <style> block or inline to ensure email clients like Gmail and Outlook render them correctly).

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8">
  <title>Bienvenue sur Nexus ERP</title>
  <style>
    body { font-family: 'Segoe UI', Helvetica, Arial, sans-serif; background-color: #0f172a; color: #f8f9fa;
margin: 0; padding: 20px; }
    .container { max-width: 600px; margin: 0 auto; background-color: #1e293b; border-radius: 12px;
border: 1px solid #334155; padding: 32px; box-shadow: 0 10px 15px -3px rgba(0,0,0,0.5); }
    .header { border-bottom: 1px solid #334155; padding-bottom: 20px; text-align: center; }
    .brand { color: #14b8a6; font-size: 26px; font-weight: bold; margin: 0; letter-spacing: -0.5px; }
    .content { padding: 24px 0; line-height: 1.6; color: #cbd5e1; }
    .welcome-title { font-size: 20px; font-weight: 600; color: #ffffff; margin-top: 0; }
    .card { background-color: #0f172a; border-radius: 8px; border: 1px solid #334155; padding: 20px;
margin: 24px 0; }
    .card-row { margin-bottom: 10px; font-size: 15px; }
    .card-row:last-child { margin-bottom: 0; }
    .label { color: #94a3b8; font-weight: 500; }
    .value { color: #2dd4bf; font-weight: bold; font-family: monospace; font-size: 16px; }
    .btn-container { text-align: center; margin-top: 32px; }
    .btn { background-color: #0d9488; color: #ffffff !important; text-decoration: none; padding: 12px 28px;
border-radius: 8px; font-weight: bold; display: inline-block; }
    .footer { border-top: 1px solid #334155; padding-top: 20px; text-align: center; font-size: 12px; color:
#64748b; margin-top: 32px; }
```

```

</style>
</head>
<body>
  <div class="container">
    <div class="header">
      <h1 class="brand">NEXUS ERP</h1>
    </div>
    <div class="content">
      <h2 class="welcome-title">Bienvenue sur la plateforme !</h2>
      <p>Votre compte a été créé avec succès par un administrateur. Voici vos accès pour vous connecter
: </p>

      <div class="card">
        <div class="card-row">
          <span class="label">Identifiant (E-mail) :</span> <br/>
          <span class="value" th:text="${email}">utilisateur@exemple.com</span>
        </div>
        <div class="card-row" style="margin-top: 12px;">
          <span class="label">Rôle attribué :</span> <br/>
          <span class="value" th:text="${role}">ROLE_EMPLOYEE</span>
        </div>
        <div class="card-row" style="margin-top: 12px;">
          <span class="label">Mot de passe temporaire :</span> <br/>
          <span class="value" th:text="${password}">SecretPassword123</span>
        </div>
      </div>

      <p>Pour des raisons de sécurité, veuillez vous connecter et remplacer ce mot de passe temporaire
dès votre première session.</p>

      <div class="btn-container">
        <a href="http://localhost:4200/auth/login" class="btn">Accéder à Nexus ERP</a>
      </div>
    </div>
    <div class="footer">
      <p>Ceci est un message automatique, merci de ne pas y répondre directement.</p>
      <p>© Nexus ERP - Tous droits réservés.</p>
    </div>
  </div>
</body>
</html>

```

Step 3: Update EmailService.java

Now replace SimpleMailMessage with MimeMessage and Spring's TemplateEngine to inject the variables into your Thymeleaf HTML template.

```
src/main/java/com/smarterp/shared/email/EmailService.java
```

```
package com.smarterp.shared.email;
```

```

import jakarta.mail.MessagingException;
import jakarta.mail.internet.MimeMessage;
import org.springframework.mail.javamail.JavaMailSender;
import org.springframework.mail.javamail.MimeMessageHelper;
import org.springframework.stereotype.Service;
import org.thymeleaf.TemplateEngine;
import org.thymeleaf.context.Context;

@Service
public class EmailService {

    private final JavaMailSender mailSender;
    private final TemplateEngine templateEngine;

    public EmailService(JavaMailSender mailSender, TemplateEngine templateEngine) {
        this.mailSender = mailSender;
        this.templateEngine = templateEngine;
    }

    public void sendAccountCreationEmail(String toEmail, String role, String temporaryPassword) {
        try {
            // 1. Pass data to Thymeleaf template variables
            Context context = new Context();
            context.setVariable("email", toEmail);
            context.setVariable("role", role);
            context.setVariable("password", temporaryPassword);

            // 2. Render HTML template into string ('welcome-email' matches welcome-email.html)
            String htmlContent = templateEngine.process("welcome-email", context);

            // 3. Construct MimeMessage
            MimeMessage mimeMessage = mailSender.createMimeMessage();
            MimeMessageHelper helper = new MimeMessageHelper(mimeMessage, true, "UTF-8");

            helper.setFrom("nexuserp.noreply@gmail.com");
            helper.setTo(toEmail);
            helper.setSubject("Bienvenue sur Nexus ERP - Vos identifiants de connexion");

            // Set 'true' to indicate HTML content
            helper.setText(htmlContent, true);

            // 4. Send email
            mailSender.send(mimeMessage);

        } catch (MessagingException e) {
            throw new RuntimeException("Erreur lors de l'envoi de l'email HTML", e);
        }
    }
}

```

Restart your Spring Boot application and create another user. The email delivered will now be a dark-mode styled HTML card with a direct login button.

User prompt: r1_0.id=? 2026-08-10T11:55:05.992+02:00 ERROR 28688 --- [smart-erp-core] [nio-8080-exec-5] o.a.c.c.C.[.].[dispatcherServlet] : Servlet.service() for servlet [dispatcherServlet] in context with path [/api/v1] threw exception [Handler dispatch failed: java.lang.Error: Unresolved compilation problem: The method sendAccountCreationEmail(String, String, String) in the type EmailService is not applicable for the arguments (String, RoleName, String)] with root cause java.lang.Error: Unresolved compilation problem: The method sendAccountCreationEmail(String, String, String) in the type EmailService is not applicable for the arguments (String, RoleName, String) at com.smarterp.security.service.AuthService.register(AuthService.java:69) ~[classes/:na] at java.base/jdk.internal.reflect.DirectMethodHandleAccessor.invoke(DirectMethodHandleAccessor.java:103) ~[na:na] at java.base/java.lang.reflect.Method.invoke(Method.java:580) ~[na:na] at org.springframework.aop.support.AopUtils.invokeJoinpointUsingReflection(AopUtils.java:359) ~[spring-aop-7.0.8.jar:7.0.8] at org.springframework.aop.framework.ReflectiveMethodInvocation.invokeJoinpoint(ReflectiveMethodInvocation.java:190) ~[spring-aop-7.0.8.jar:7.0.8] at org.springframework.aop.framework.ReflectiveMethodInvocation.proceed(ReflectiveMethodInvocation.java:158) ~[spring-aop-7.0.8.jar:7.0.8] at org.springframework.transaction.interceptor.TransactionInterceptor\$1.proceedWithInvocation(TransactionInterceptor.java:133) ~[spring-tx-7.0.8.jar:7.0.8] at org.springframework.transaction.interceptor.TransactionAspectSupport.invokeWithinTransaction(TransactionAspectSupport.java:371) ~[spring-tx-7.0.8.jar:7.0.8] at org.springframework.transaction.interceptor.TransactionInterceptor.invoke(TransactionInterceptor.java:130) ~[spring-tx-7.0.8.jar:7.0.8] at org.springframework.aop.framework.ReflectiveMethodInvocation.proceed(ReflectiveMethodInvocation.java:179) ~[spring-aop-7.0.8.jar:7.0.8] at org.springframework.aop.framework.CglibAopProxy\$DynamicAdvisedInterceptor.intercept(CglibAopProxy.java:719) ~[spring-aop-7.0.8.jar:7.0.8] at com.smarterp.security.service.AuthService\$\$SpringCGLIB\$\$0.register(<generated>) ~[classes/:na] at com.smarterp.security.web.AuthController.register(AuthController.java:26) ~[classes/:na] at java.base/jdk.internal.reflect.DirectMethodHandleAccessor.invoke(DirectMethodHandleAccessor.java:103) ~[na:na] at java.base/java.lang.reflect.Method.invoke(Method.java:580) ~[na:na] at org.springframework.web.method.support.InvocableHandlerMethod.doInvoke(InvocableHandlerMethod.java:252) ~[spring-web-7.0.8.jar:7.0.8] at org.springframework.web.method.support.InvocableHandlerMethod.invokeForRequest(InvocableHandlerMethod.java:184) ~[spring-web-7.0.8.jar:7.0.8] at org.springframework.web.servlet.mvc.method.annotation.ServletInvocableHandlerMethod.invokeAndHandle(ServletInvocableHandlerMethod.java:117) ~[spring-webmvc-7.0.8.jar:7.0.8] at org.springframework.web.servlet.mvc.method.annotation.RequestMappingHandlerAdapter.invokeHandlerMethod(RequestMappingHandlerAdapter.java:934) ~[spring-webmvc-7.0.8.jar:7.0.8] at org.springframework.web.servlet.mvc.method.annotation.RequestMappingHandlerAdapter.handleInternal(RequestMappingHandlerAdapter.java:853) ~[spring-webmvc-7.0.8.jar:7.0.8] at org.springframework.web.servlet.mvc.method.AbstractHandlerMethodAdapter.handle(AbstractHandlerMethodAdapter.java:86) ~[spring-webmvc-7.0.8.jar:7.0.8] at org.springframework.web.servlet.DispatcherServlet.doDispatch(DispatcherServlet.java:963) ~[spring-webmvc-7.0.8.jar:7.0.8] at org.springframework.web.servlet.DispatcherServlet.doService(DispatcherServlet.java:866) ~[spring-webmvc-7.0.8.jar:7.0.8] at org.springframework.web.servlet.FrameworkServlet.processRequest(FrameworkServlet.java:1000)

~[spring-webmvc-7.0.8.jar:7.0.8] at
org.springframework.web.servlet.FrameworkServlet.doPost(FrameworkServlet.java:903)
~[spring-webmvc-7.0.8.jar:7.0.8] at jakarta.servlet.http.HttpServlet.service(HttpServlet.java:649)
~[tomcat-embed-core-11.0.22.jar:6.1] at
org.springframework.web.servlet.FrameworkServlet.service(FrameworkServlet.java:874)
~[spring-webmvc-7.0.8.jar:7.0.8] at jakarta.servlet.http.HttpServlet.service(HttpServlet.java:710)
~[tomcat-embed-core-11.0.22.jar:6.1] at
org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:128)
~[tomcat-embed-core-11.0.22.jar:11.0.22] at
org.apache.tomcat.websocket.server.WsFilter.doFilter(WsFilter.java:53)
~[tomcat-embed-websocket-11.0.22.jar:11.0.22] at
org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:107)
~[tomcat-embed-core-11.0.22.jar:11.0.22] at
org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:110)
~[spring-web-7.0.8.jar:7.0.8] at
org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:107)
~[tomcat-embed-core-11.0.22.jar:11.0.22] at
org.springframework.web.filter.CompositeFilter\$VirtualFilterChain.doFilter(CompositeFilter.java:108)
~[spring-web-7.0.8.jar:7.0.8] at
org.springframework.security.web.FilterChainProxy.lambda\$doFilterInternal\$2(FilterChainProxy.java:235)
~[spring-security-web-7.1.0.jar:7.1.0] at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:376)
~[spring-security-web-7.1.0.jar:7.1.0] at
org.springframework.security.web.access.intercept.AuthorizationFilter.doFilter(AuthorizationFilter.java:101)
~[spring-security-web-7.1.0.jar:7.1.0] at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385)
~[spring-security-web-7.1.0.jar:7.1.0] at
org.springframework.security.web.access.ExceptionTranslationFilter.doFilter(ExceptionTranslationFilter.java:126) ~[spring-security-web-7.1.0.jar:7.1.0] at
org.springframework.security.web.access.ExceptionTranslationFilter.doFilter(ExceptionTranslationFilter.java:120) ~[spring-security-web-7.1.0.jar:7.1.0] at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385)
~[spring-security-web-7.1.0.jar:7.1.0] at
org.springframework.security.web.session.SessionManagementFilter.doFilter(SessionManagementFilter.java:132) ~[spring-security-web-7.1.0.jar:7.1.0] at
org.springframework.security.web.session.SessionManagementFilter.doFilter(SessionManagementFilter.java:86) ~[spring-security-web-7.1.0.jar:7.1.0] at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385)
~[spring-security-web-7.1.0.jar:7.1.0] at
org.springframework.security.web.authentication.AnonymousAuthenticationFilter.doFilter(AnonymousAuthenticationFilter.java:100) ~[spring-security-web-7.1.0.jar:7.1.0] at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385)
~[spring-security-web-7.1.0.jar:7.1.0] at
org.springframework.security.web.servletapi.SecurityContextHolderAwareRequestFilter.doFilter(SecurityContextHolderAwareRequestFilter.java:181) ~[spring-security-web-7.1.0.jar:7.1.0] at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385)
~[spring-security-web-7.1.0.jar:7.1.0] at
org.springframework.security.web.savedrequest.RequestCacheAwareFilter.doFilter(RequestCacheAwareFilter.java:63) ~[spring-security-web-7.1.0.jar:7.1.0] at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385)
~[spring-security-web-7.1.0.jar:7.1.0] at
com.smarterp.shared.security.JwtAuthenticationFilter.doFilterInternal(JwtAuthenticationFilter.java:56)

~[classes/:na] at
org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:116)
~[spring-web-7.0.8.jar:7.0.8] at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385)
~[spring-security-web-7.1.0.jar:7.1.0] at
org.springframework.security.web.authentication.logout.LogoutFilter.doFilter(LogoutFilter.java:110)
~[spring-security-web-7.1.0.jar:7.1.0] at
org.springframework.security.web.authentication.logout.LogoutFilter.doFilter(LogoutFilter.java:96)
~[spring-security-web-7.1.0.jar:7.1.0] at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385)
~[spring-security-web-7.1.0.jar:7.1.0] at
org.springframework.web.filter.CorsFilter.doFilterInternal(CorsFilter.java:91) ~[spring-web-7.0.8.jar:7.0.8]
at org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:116)
~[spring-web-7.0.8.jar:7.0.8] at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385)
~[spring-security-web-7.1.0.jar:7.1.0] at
org.springframework.security.web.header.HeaderWriterFilter.doHeadersAfter(HeaderWriterFilter.java:90)
~[spring-security-web-7.1.0.jar:7.1.0] at
org.springframework.security.web.header.HeaderWriterFilter.doFilterInternal(HeaderWriterFilter.java:75)
~[spring-security-web-7.1.0.jar:7.1.0] at
org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:116)
~[spring-web-7.0.8.jar:7.0.8] at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385)
~[spring-security-web-7.1.0.jar:7.1.0] at
org.springframework.security.web.context.SecurityContextHolderFilter.doFilter(SecurityContextHolderFilter.java:82) ~[spring-security-web-7.1.0.jar:7.1.0] at
org.springframework.security.web.context.SecurityContextHolderFilter.doFilter(SecurityContextHolderFilter.java:69) ~[spring-security-web-7.1.0.jar:7.1.0] at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385)
~[spring-security-web-7.1.0.jar:7.1.0] at
org.springframework.security.web.context.request.async.WebAsyncManagerIntegrationFilter.doFilterInternal(WebAsyncManagerIntegrationFilter.java:62) ~[spring-security-web-7.1.0.jar:7.1.0] at
org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:116)
~[spring-web-7.0.8.jar:7.0.8] at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385)
~[spring-security-web-7.1.0.jar:7.1.0] at
org.springframework.security.web.session.DisableEncodeUrlFilter.doFilterInternal(DisableEncodeUrlFilter.java:42) ~[spring-security-web-7.1.0.jar:7.1.0] at
org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:116)
~[spring-web-7.0.8.jar:7.0.8] at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385)
~[spring-security-web-7.1.0.jar:7.1.0] at
org.springframework.security.web.FilterChainProxy.doFilterInternal(FilterChainProxy.java:237)
~[spring-security-web-7.1.0.jar:7.1.0] at
org.springframework.security.web.FilterChainProxy.doFilter(FilterChainProxy.java:195)
~[spring-security-web-7.1.0.jar:7.1.0] at
org.springframework.web.filter.CompositeFilter\$VirtualFilterChain.doFilter(CompositeFilter.java:113)
~[spring-web-7.0.8.jar:7.0.8] at
org.springframework.web.filter.ServletRequestPathFilter.doFilter(ServletRequestPathFilter.java:52)
~[spring-web-7.0.8.jar:7.0.8] at
org.springframework.web.filter.CompositeFilter\$VirtualFilterChain.doFilter(CompositeFilter.java:113)
~[spring-web-7.0.8.jar:7.0.8] at

org.springframework.web.filter.CompositeFilter.doFilter(CompositeFilter.java:74)
~[spring-web-7.0.8.jar:7.0.8] at
org.springframework.security.config.annotation.web.configuration.WebSecurityConfiguration\$CompositeFilterChainProxy.doFilter(WebSecurityConfiguration.java:317) ~[spring-security-config-7.1.0.jar:7.1.0] at
org.springframework.web.filter.DelegatingFilterProxy.invokeDelegate(DelegatingFilterProxy.java:355)
~[spring-web-7.0.8.jar:7.0.8] at
org.springframework.web.filter.DelegatingFilterProxy.doFilter(DelegatingFilterProxy.java:272)
~[spring-web-7.0.8.jar:7.0.8] at
org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:107)
~[tomcat-embed-core-11.0.22.jar:11.0.22] at
org.springframework.web.filter.RequestContextFilter.doFilterInternal(RequestContextFilter.java:100)
~[spring-web-7.0.8.jar:7.0.8] at
org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:116)
~[spring-web-7.0.8.jar:7.0.8] at
org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:107)
~[tomcat-embed-core-11.0.22.jar:11.0.22] at
org.springframework.web.filter.FormContentFilter.doFilterInternal(FormContentFilter.java:93)
~[spring-web-7.0.8.jar:7.0.8] at
org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:116)
~[spring-web-7.0.8.jar:7.0.8] at
org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:107)
~[tomcat-embed-core-11.0.22.jar:11.0.22] at
org.springframework.web.filter.CharacterEncodingFilter.doFilterInternal(CharacterEncodingFilter.java:199)
~[spring-web-7.0.8.jar:7.0.8] at
org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:116)
~[spring-web-7.0.8.jar:7.0.8] at
org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:107)
~[tomcat-embed-core-11.0.22.jar:11.0.22] at
org.apache.catalina.core.StandardWrapperValve.invoke(StandardWrapperValve.java:165)
~[tomcat-embed-core-11.0.22.jar:11.0.22] at
org.apache.catalina.core.StandardContextValve.invoke(StandardContextValve.java:77)
~[tomcat-embed-core-11.0.22.jar:11.0.22] at
org.apache.catalina.authenticator.AuthenticatorBase.invoke(AuthenticatorBase.java:492)
~[tomcat-embed-core-11.0.22.jar:11.0.22] at
org.apache.catalina.core.StandardHostValve.invoke(StandardHostValve.java:113)
~[tomcat-embed-core-11.0.22.jar:11.0.22] at
org.apache.catalina.valves.ErrorReportValve.invoke(ErrorReportValve.java:83)
~[tomcat-embed-core-11.0.22.jar:11.0.22] at
org.apache.catalina.core.StandardEngineValve.invoke(StandardEngineValve.java:72)
~[tomcat-embed-core-11.0.22.jar:11.0.22] at
org.apache.catalina.connector.CoyoteAdapter.service(CoyoteAdapter.java:341)
~[tomcat-embed-core-11.0.22.jar:11.0.22] at
org.apache.coyote.http11.Http11Processor.service(Http11Processor.java:397)
~[tomcat-embed-core-11.0.22.jar:11.0.22] at
org.apache.coyote.AbstractProcessorLight.process(AbstractProcessorLight.java:63)
~[tomcat-embed-core-11.0.22.jar:11.0.22] at
org.apache.coyote.AbstractProtocol\$ConnectionHandler.process(AbstractProtocol.java:1272)
~[tomcat-embed-core-11.0.22.jar:11.0.22] at
org.apache.tomcat.util.net.NioEndpoint\$SocketProcessor.doRun(NioEndpoint.java:1801)
~[tomcat-embed-core-11.0.22.jar:11.0.22] at
org.apache.tomcat.util.net.SocketProcessorBase.run(SocketProcessorBase.java:52)
~[tomcat-embed-core-11.0.22.jar:11.0.22] at

```
org.apache.tomcat.util.threads.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:946)
~[tomcat-embed-core-11.0.22.jar:11.0.22]      at
org.apache.tomcat.util.threads.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:480)
~[tomcat-embed-core-11.0.22.jar:11.0.22]      at
org.apache.tomcat.util.threads.TaskThread$WrappingRunnable.run(TaskThread.java:57)
~[tomcat-embed-core-11.0.22.jar:11.0.22]      at java.base/java.lang.Thread.run(Thread.java:1583)
~[na:na]
```

Response: This error is a **Java type mismatch** in your AuthService.java class on line 69.

Your sendAccountCreationEmail method expects a String for the role parameter, but you are passing an Enum object (RoleName) directly instead of converting it to text.

How to Fix

Open **src/main/java/com/smarterp/security/service/AuthService.java** around line 69 and change how you pass the role to emailService.

Change this:

```
emailService.sendAccountCreationEmail(
    request.getEmail(),
    request.getRole(), // <-- This is returning a RoleName enum object
    request.getPassword()
);
```

To this:

```
emailService.sendAccountCreationEmail(
    request.getEmail(),
    request.getRole().name(), // <-- Added .name() to convert the enum to String
    request.getPassword()
);
```

(If request.getRole() is already a String or custom object, you can use request.getRole().toString() instead).

Rebuild and restart your Spring Boot application after saving, and the error will be resolved!

```
User prompt: i have a problem now when i try to login it has a 403 error      first ? rows only
2026-08-10T12:23:26.536+02:00 INFO 20704 --- [smart-erp-core] [nio-8080-exec-2]
o.a.c.c.C.[.[localhost].[/api/v1]      : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-08-10T12:23:26.536+02:00 INFO 20704 --- [smart-erp-core] [nio-8080-exec-2]
o.s.web.servlet.DispatcherServlet      : Initializing Servlet 'dispatcherServlet'
2026-08-10T12:23:26.538+02:00 INFO 20704 --- [smart-erp-core] [nio-8080-exec-2]
o.s.web.servlet.DispatcherServlet      : Completed initialization in 0 ms
```

Response: This **403 Forbidden** error on login is happening because of how Spring Boot handles your context path (/api/v1).

In your log, notice context with path [/api/v1]. Since /api/v1 is configured as your application's base context path in application.properties, Spring Security automatically strips /api/v1 from the URL before checking permissions.

When we added /api/v1/auth/login to SecurityConfig.java, Spring Security actually evaluated it as /api/v1/api/v1/auth/login, which didn't match. As a result, the login request fell into .anyRequest().authenticated() and got blocked!

The Fix

Open **SecurityConfig.java** and remove the extra /api/v1 prefixes from your requestMatchers:

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .cors(Customizer.withDefaults())
        .csrf(AbstractHttpConfigurer::disable)
        .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(auth -> auth
            // REMOVE '/api/v1' FROM HERE - Spring already knows the context path is /api/v1
            .requestMatchers("/auth/login", "/v3/api-docs/**", "/swagger-ui/**").permitAll()
            .requestMatchers("/auth/register").hasRole("ADMIN")
            .anyRequest().authenticated()
        )
        .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);

    return http.build();
}
```

Restart Spring Boot after saving, and your /auth/login endpoint will be publicly accessible again!

User prompt: spring: application: name: smart-erp-core # Database Connection datasource: url: jdbc:postgresql://localhost:5432/smart_erp username: postgres password: postgres # Change to your actual PostgreSQL password if not "postgres" driver-class-name: org.postgresql.Driver # JPA & Hibernate Settings jpa: hibernate: ddl-auto: \${HIBERNATE_DDL:update} show-sql: true properties: hibernate: format_sql: true dialect: org.hibernate.dialect.PostgreSQLDialect mail: host: smtp.gmail.com port: 587 username: nexuserp.system@gmail.com password: bfkk isan ijbv cfx properties: mail: smtp: auth: true starttls: enable: true # JWT Security Configuration Settings is this correct

Response: The YAML structure is almost completely correct, but there is **one key issue** that will break email sending:

The Issue: Spaces in the App Password

Google displays App Passwords with spaces for readability (e.g., bfk kisan ijbv cfx x), but Spring Boot will pass those literal space characters to Gmail's SMTP server during authentication, causing a **535 Authentication Failed** error. You must **remove all spaces** from the password.

Cleaned & Corrected application.yml

```
server:
  port: 8080
  servlet:
    context-path: /api/v1 # Keeps your /api/v1 base path configuration explicit

spring:
  application:
    name: smart-erp-core

# Database Connection
datasource:
  url: jdbc:postgresql://localhost:5432/smart_erp
  username: postgres
  password: postgres # Change to your actual PostgreSQL password if not "postgres"
  driver-class-name: org.postgresql.Driver

# JPA & Hibernate Settings
jpa:
  hibernate:
    ddl-auto: ${HIBERNATE_DDL:update}
  show-sql: true
  properties:
    hibernate:
      format_sql: true
      dialect: org.hibernate.dialect.PostgreSQLDialect

# Email Configuration
mail:
  host: smtp.gmail.com
  port: 587
  username: nexuserp.system@gmail.com
  password: bfk kisan ijbv cfx x # Spaces removed here
  properties:
    mail:
      smtp:
        auth: true
        starttls:
          enable: true
```

Note on SecurityConfig.java: Since `server.servlet.context-path: /api/v1` sets the global base path, keep your `SecurityConfig` request matchers relative to that root (e.g., `.requestMatchers("/auth/login").permitAll()`).

not /api/v1/auth/login).

User prompt: the user was created but the email was not sent to the mail

Response: Since the user is being created successfully in the database, the backend endpoint is running, but the email step is failing or getting skipped silently.

Here is how to identify and fix the issue step-by-step:

Step 1: Check your Spring Boot Console Logs

When you register a user, look at your IDE terminal where Spring Boot is running. You will usually see one of three specific errors:

1. **535 5.7.8 Error: authentication failed**
 - **Cause:** The Gmail App Password is wrong or still has spaces in application.yml.
 - **Fix:** Ensure spring.mail.password in application.yml is **16 characters with no spaces** (e.g., bfkksanijbvvcfx). Restart the server afterwards.
2. **TemplateInputException: Error resolving template [welcome-email]**
 - **Cause:** Spring Boot cannot find the HTML template file.
 - **Fix:** Verify welcome-email.html is located exactly at:
src/main/resources/templates/welcome-email.html
3. **MailSendException / MessagingException**
 - **Cause:** Firewall/network blocking port 587 or the from email address in EmailService.java does not match your application.yml username.

Step 2: Add Logging to EmailService.java

If no error appears in the console, your try-catch block inside EmailService.java might be silently swallowing the exception.

Update EmailService.java to print full error details when something goes wrong:

```
package com.smarterp.shared.email;

import jakarta.mail.MessagingException;
import jakarta.mail.internet.MimeMessage;
import org.springframework.mail.javamail.JavaMailSender;
import org.springframework.mail.javamail.MimeMessageHelper;
import org.springframework.stereotype.Service;
import org.thymeleaf.TemplateEngine;
import org.thymeleaf.context.Context;
```

```
@Service
public class EmailService {

    private final JavaMailSender mailSender;
    private final TemplateEngine templateEngine;
```

```

public EmailService(JavaMailSender mailSender, TemplateEngine templateEngine) {
    this.mailSender = mailSender;
    this.templateEngine = templateEngine;
}

public void sendAccountCreationEmail(String toEmail, String role, String temporaryPassword) {
    try {
        System.out.println(">>> Attempting to send email to: " + toEmail); // Debug log

        Context context = new Context();
        context.setVariable("email", toEmail);
        context.setVariable("role", role);
        context.setVariable("password", temporaryPassword);

        String htmlContent = templateEngine.process("welcome-email", context);

        MimeMessage mimeTypeMessage = mailSender.createMimeMessage();
        MimeMessageHelper helper = new MimeMessageHelper(mimeTypeMessage, true, "UTF-8");

        // MUST MATCH your spring.mail.username in application.yml
        helper.setFrom("nexuserp.system@gmail.com");
        helper.setTo(toEmail);
        helper.setSubject("Bienvenue sur Nexus ERP - Vos identifiants de connexion");
        helper.setText(htmlContent, true);

        mailSender.send(mimeTypeMessage);
        System.out.println(">>> Email sent successfully to: " + toEmail); // Debug log

    } catch (Exception e) {
        // PRINT THE EXACT ERROR TO CONSOLE
        System.err.println(">>> FAILED TO SEND EMAIL:");
        e.printStackTrace();
    }
}
}

```

Step 3: Check the Spam/Junk Folder

Because nexuserp.system@gmail.com is a new address sending automated emails with plain login credentials, recipient email providers (Gmail, Outlook, Yahoo) often send the first message straight to the **Spam / Junk** folder.

Action Plan

1. Update EmailService.java with the code above.
2. Ensure application.yml has password: bfkkisanijbvcfxx (no spaces) and restart Spring Boot.
3. Try creating a new user again from Angular.
4. Check your Java console for the >>> debug logs or error stack trace and check the destination inbox's Spam folder.

User prompt: it was in spam

Response: That is actually great news—it means your backend code, Thymeleaf template, and Gmail SMTP connection are all working perfectly!

Email providers (like Gmail, Outlook, and Yahoo) automatically mark these emails as spam for three main reasons:

1. **Sending plain-text passwords:** Spam filters aggressively flag emails that contain words like "temporary password" alongside plain text credentials.
2. **localhost links:** Spam filters treat links to `http://localhost:4200` as suspicious.
3. **New Gmail account:** A brand-new `@gmail.com` account sending automated messages lacks a sender reputation.

How to Fix / Work Around It

1. For Local Development & Testing

- **Mark as "Not Spam":** Open the email in your Spam folder and click **"Report as not spam"** or **"Looks safe"**. Doing this once teaches Gmail to trust `nexuserp.system@gmail.com` for your inbox.
- **Add to Contacts:** Add `nexuserp.system@gmail.com` to the contacts list of your test recipient account.

2. Best Practices for Production (To Avoid Spam Entirely)

- **Send a Password Reset Link instead of a Raw Password:** Instead of sending the actual temporary password in the email body, send a secure, tokenized activation link:
`http://yourdomain.com/set-password?token=a1b2c3d4`... Spam algorithms consider password setup links much safer than plain text passwords.
- **Use a Transactional Mail Provider:** When you deploy Nexus ERP with a real domain (e.g., `nexus-erp.com`), use a dedicated service like **Resend**, **SendGrid**, **Mailgun**, or **Amazon SES**. These allow you to set up **SPF**, **DKIM**, and **DMARC** records on your domain DNS, which guarantees 99%+ inbox delivery.

User prompt: Best Practices for Production (To Avoid Spam Entirely) Send a Password Reset Link instead of a Raw Password: Instead of sending the actual temporary password in the email body, send a secure, tokenized activation

link:http://yourdomain.com/set-password?token=a1b2c3d4... Spam algorithms consider password setup links much safer than plain text passwords. Use a Transactional Mail Provider: When you deploy Nexus ERP with a real domain (e.g., nexus-erp.com), use a dedicated service like Resend, SendGrid, Mailgun, or Amazon SES. These allow you to set up SPF, DKIM, and DMARC records on your domain DNS, which guarantees 99%+ inbox delivery. i want to do this

Response: This is the standard approach used by enterprise platforms. Switching to a tokenized setup flow ensures plain-text passwords are never sent over the wire and protects your email sender reputation.

Here is the complete implementation split between backend (Spring Boot) and frontend (Angular).

Step 1: Create the Activation Token Entity (Spring Boot)

Create a PasswordResetToken entity to store short-lived single-use tokens in PostgreSQL.

src/main/java/com/smarterp/security/domain/PasswordResetToken.java

```
package com.smarterp.security.domain;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "password_reset_tokens")
public class PasswordResetToken {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String token;

    @OneToOne(targetEntity = User.class, fetch = FetchType.EAGER)
    @JoinColumn(nullable = false, name = "user_id")
    private User user;

    @Column(nullable = false)
    private LocalDateTime expiryDate;

    public PasswordResetToken() {}

    public PasswordResetToken(String token, User user, int expirationInHours) {
        this.token = token;
        this.user = user;
        this.expiryDate = LocalDateTime.now().plusHours(expirationInHours);
    }

    public boolean isExpired() {
        return LocalDateTime.now().isAfter(this.expiryDate);
    }

    // Getters and Setters
    public Long getId() { return id; }
    public String getToken() { return token; }
    public User getUser() { return user; }
    public LocalDateTime getExpiryDate() { return expiryDate; }
}
```

Create the repository for this entity:

src/main/java/com/smarterp/security/repository/PasswordResetTokenRepository.java

```
package com.smarterp.security.repository;
```

```
import com.smarterp.security.domain.PasswordResetToken;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;
```

```
public interface PasswordResetTokenRepository extends JpaRepository<PasswordResetToken, Long> {
    Optional<PasswordResetToken> findByToken(String token);
    void deleteByUserId(Long userId);
}
```

Step 2: Update EmailService.java to Send Links

Modify EmailService.java so it takes an activationUrl parameter instead of a raw password.

src/main/java/com/smarterp/shared/email/EmailService.java

```
package com.smarterp.shared.email;
```

```
import jakarta.mail.internet.MimeMessage;
import org.springframework.mail.javamail.JavaMailSender;
import org.springframework.mail.javamail.MimeMessageHelper;
import org.springframework.stereotype.Service;
import org.thymeleaf.TemplateEngine;
import org.thymeleaf.context.Context;
```

```
@Service
```

```
public class EmailService {
```

```
    private final JavaMailSender mailSender;
    private final TemplateEngine templateEngine;
```

```
    public EmailService(JavaMailSender mailSender, TemplateEngine templateEngine) {
        this.mailSender = mailSender;
        this.templateEngine = templateEngine;
    }
```

```
    public void sendAccountActivationEmail(String toEmail, String role, String activationUrl) {
        try {
            Context context = new Context();
            context.setVariable("email", toEmail);
            context.setVariable("role", role);
            context.setVariable("activationUrl", activationUrl);
```

```
            String htmlContent = templateEngine.process("welcome-email", context);
```

```
            MimeMessage mimeMessage = mailSender.createMimeMessage();
            MimeMessageHelper helper = new MimeMessageHelper(mimeMessage, true, "UTF-8");
```

```

        helper.setFrom("nexuserp.system@gmail.com");
        helper.setTo(toEmail);
        helper.setSubject("Bienvenue sur Nexus ERP - Activez votre compte");
        helper.setText(htmlContent, true);

        mailSender.send(mimeMessage);
    } catch (Exception e) {
        e.printStackTrace();
    }
}
}
}

```

Step 3: Update welcome-email.html

Update the template to replace the password text field with a prominent call-to-action button.

src/main/resources/templates/welcome-email.html

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <style>
        body { font-family: 'Segoe UI', Arial, sans-serif; background-color: #0f172a; color: #f8fafc; margin: 0;
padding: 20px; }
        .container { max-width: 600px; margin: 0 auto; background-color: #1e293b; border-radius: 12px;
border: 1px solid #334155; padding: 32px; }
        .brand { color: #14b8a6; font-size: 26px; font-weight: bold; text-align: center; }
        .card { background-color: #0f172a; border-radius: 8px; border: 1px solid #334155; padding: 20px;
margin: 24px 0; }
        .btn-container { text-align: center; margin: 32px 0; }
        .btn { background-color: #0d9488; color: #ffffff !important; text-decoration: none; padding: 14px 32px;
border-radius: 8px; font-weight: bold; display: inline-block; }
    </style>
</head>
<body>
    <div class="container">
        <h1 class="brand">NEXUS ERP</h1>
        <h2>Activez votre compte</h2>
        <p>Un administrateur vous a invité à rejoindre la plateforme Nexus ERP.</p>

        <div class="card">
            <p><strong>Identifiant :</strong> <span th:text="{email}"></span></p>
            <p><strong>Rôle :</strong> <span th:text="{role}"></span></p>
            <p><strong>Lien valide pendant :</strong> 24 heures</p>
        </div>

        <div class="btn-container">
            <a th:href="{activationUrl}" class="btn">Créer mon mot de passe</a>

```

</div>

<p style="font-size: 12px; color: #64748b;">Si le bouton ne fonctionne pas, copiez-collez ce lien dans votre navigateur :

</p>

</div>

</body>

</html>

Step 4: Backend Logic for Registration & Password Setting

Create a Request DTO for setting the password:

src/main/java/com/smarterp/security/dto/SetPasswordRequest.java

```
package com.smarterp.security.dto;
```

```
public class SetPasswordRequest {
    private String token;
    private String newPassword;

    public String getToken() { return token; }
    public void setToken(String token) { this.token = token; }
    public String getNewPassword() { return newPassword; }
    public void setNewPassword(String newPassword) { this.newPassword = newPassword; }
}
```

Update AuthService.java:

```
@Transactional
public UserResponse registerUser(RegisterRequest request) {
    // 1. Create user with a random un-usable password placeholder
    User newUser = new User();
    newUser.setEmail(request.getEmail());
    newUser.setRole(request.getRole());
    newUser.setPassword(passwordEncoder.encode(UUID.randomUUID().toString())); // Temporary locked
state
    userRepository.save(newUser);

    // 2. Generate secure token (24h validity)
    String token = UUID.randomUUID().toString();
    PasswordResetToken resetToken = new PasswordResetToken(token, newUser, 24);
    tokenRepository.save(resetToken);

    // 3. Send email with URL pointing to Angular
    String activationUrl = "http://localhost:4200/auth/set-password?token=" + token;
    emailService.sendAccountActivationEmail(request.getEmail(), request.getRole().name(), activationUrl);

    return new UserResponse("Invitation envoyée par email.");
}
```

```

}

@Transactional
public void setPasswordWithToken(SetPasswordRequest request) {
    PasswordResetToken resetToken = tokenRepository.findByToken(request.getToken())
        .orElseThrow(() -> new IllegalArgumentException("Jeton invalide ou expiré."));

    if (resetToken.isExpired()) {
        tokenRepository.delete(resetToken);
        throw new IllegalArgumentException("Ce lien d'activation a expiré.");
    }

    User user = resetToken.getUser();
    user.setPassword(passwordEncoder.encode(request.getNewPassword()));
    userRepository.save(user);

    // Delete token once used
    tokenRepository.delete(resetToken);
}

```

Expose the set-password endpoint in AuthController.java:

```

@PostMapping("/auth/set-password")
public ResponseEntity<String> setPassword(@RequestBody SetPasswordRequest request) {
    authService.setPasswordWithToken(request);
    return ResponseEntity.ok("Mot de passe défini avec succès. Vous pouvez maintenant vous connecter.");
}

```

Ensure this public endpoint is unlocked in SecurityConfig.java:

```

.requestMatchers("/auth/login", "/auth/set-password", "/v3/api-docs/**", "/swagger-ui/**").permitAll()

```

Step 5: Frontend - Angular Set Password Screen

1. Generate the component in Angular:

```
ng g c auth/set-password --standalone
```

2. Add the route to app.routes.ts:

```
{ path: 'auth/set-password', component: SetPasswordComponent }
```

3. Component logic (set-password.component.ts):

```

import { Component, OnInit, inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { ActivatedRoute, Router } from '@angular/router';
import { HttpClient } from '@angular/common/http';

```

```
@Component({
```

```

selector: 'app-set-password',
standalone: true,
imports: [CommonModule, FormsModule],
template: `
  <div class="min-h-screen bg-slate-900 flex items-center justify-center p-4">
    <div class="bg-slate-800 border border-slate-700 p-8 rounded-xl max-w-md w-full shadow-2xl">
      <h2 class="text-2xl font-bold text-teal-400 mb-2">Nexus ERP</h2>
      <p class="text-slate-400 mb-6">Définissez votre nouveau mot de passe pour finaliser l'activation de
votre compte.</p>

      <form (ngSubmit)="onSubmit()" #f="ngForm">
        <div class="mb-4">
          <label class="block text-slate-300 mb-2">Nouveau mot de passe</label>
          <input type="password" [(ngModel)]="password" name="password" required minlength="6"
            class="w-full bg-slate-900 border border-slate-700 rounded-lg p-3 text-white focus:outline-none
focus:border-teal-500">
        </div>

        <div class="mb-6">
          <label class="block text-slate-300 mb-2">Confirmer le mot de passe</label>
          <input type="password" [(ngModel)]="confirmPassword" name="confirmPassword" required
            class="w-full bg-slate-900 border border-slate-700 rounded-lg p-3 text-white focus:outline-none
focus:border-teal-500">
        </div>

        <p *ngIf="errorMessage" class="text-red-400 text-sm mb-4">{{ errorMessage }}</p>
        <p *ngIf="successMessage" class="text-emerald-400 text-sm mb-4">{{ successMessage }}</p>

        <button type="submit" [disabled]="!f.valid || isLoading"
          class="w-full bg-teal-600 hover:bg-teal-500 text-white font-bold py-3 rounded-lg
transition-colors">
          {{ isLoading ? 'Validation...' : 'Valider et continuer' }}
        </button>
      </form>
    </div>
  </div>
  `
})
export class SetPasswordComponent implements OnInit {
  private route = inject(ActivatedRoute);
  private router = inject(Router);
  private http = inject(HttpClient);

  token: string = "";
  password: String = "";
  confirmPassword: String = "";
  errorMessage: string = "";
  successMessage: string = "";
  isLoading: boolean = false;

  ngOnInit(): void {
    this.token = this.route.snapshot.queryParamMap.get('token') || "";
  }
}

```

```

    if (!this.token) {
        this.errorMessage = 'Jeton d\'activation manquant.';
    }
}

onSubmit(): void {
    if (this.password !== this.confirmPassword) {
        this.errorMessage = 'Les mots de passe ne correspondent pas.';
        return;
    }

    this.isLoading = true;
    this.errorMessage = '';

    this.http.post('http://localhost:8080/api/v1/auth/set-password', {
        token: this.token,
        newPassword: this.password
    }, { responseType: 'text' }).subscribe({
        next: (res) => {
            this.isLoading = false;
            this.successMessage = 'Mot de passe configuré ! Redirection vers la page de connexion...';
            setTimeout(() => this.router.navigate(['/auth/login']), 2000);
        },
        error: (err) => {
            this.isLoading = false;
            this.errorMessage = err.error || 'Impossible de configurer le mot de passe. Le jeton est peut-être expiré.';
        }
    });
}

```

User prompt: attempting to save one or more entities that have a non-nullable association with an unsaved transient entity. The unsaved transient entity must be saved in an operation prior to saving these dependent entities. Unsaved transient entity: [com.smarterp.security.domain.User with null id] Dependent entities: [[com.smarterp.security.domain.PasswordResetToken with null id]] Non-nullable associations: [com.smarterp.security.domain.PasswordResetToken.user]

Response: This error happens because Hibernate requires the User to be saved in the database first so it gets a generated ID before PasswordResetToken can reference it.

In your AuthService.java, userRepository.save(newUser) returns the newly saved User entity containing that generated ID. You need to capture that returned object and pass it to PasswordResetToken.

The Fix

Open **AuthService.java** in your registration method and update it:

Change this:

```
userRepository.save(newUser);

// 2. Generate secure token
String token = UUID.randomUUID().toString();
PasswordResetToken resetToken = new PasswordResetToken(token, newUser, 24); // <-- 'newUser' doesn't
have an ID yet!
tokenRepository.save(resetToken);
```

To this:

```
// 1. Capture the saved entity returned by repository
User savedUser = userRepository.save(newUser);

// 2. Pass 'savedUser' (which now has its primary key ID set)
String token = UUID.randomUUID().toString();
PasswordResetToken resetToken = new PasswordResetToken(token, savedUser, 24);
tokenRepository.save(resetToken);
```

Alternative / Additional Safeguard

In **PasswordResetToken.java**, you can also add `CascadeType.MERGE` to the `@OneToOne` association so Hibernate automatically handles object state synchronization:

```
@OneToOne(targetEntity = User.class, fetch = FetchType.EAGER, cascade = CascadeType.MERGE)
@JoinColumn(nullable = false, name = "user_id")
private User user;
```

Rebuild and restart your Spring Boot backend, and the token saving will complete without issue.

```
User prompt: package com.smarterp.security.service; import
com.smarterp.security.domain.PasswordResetToken; import com.smarterp.security.domain.Role; import
com.smarterp.security.domain.RoleName; import com.smarterp.security.domain.User; import
com.smarterp.security.dto.AuthResponse; import com.smarterp.security.dto.LoginRequest; import
com.smarterp.security.dto.RegisterRequest; import com.smarterp.security.dto.SetPasswordRequest; import
com.smarterp.security.dto.UserDTO; import
com.smarterp.security.repository.PasswordResetTokenRepository; import
com.smarterp.security.repository.RoleRepository; import com.smarterp.security.repository.UserRepository;
import com.smarterp.shared.email.EmailService; import
com.smarterp.shared.exception.BadRequestException; import
com.smarterp.shared.exception.ResourceNotFoundException; import
com.smarterp.shared.security.JwtUtils; import com.smarterp.shared.security.SecurityUser; import
java.util.UUID; import org.springframework.security.authentication.AuthenticationManager; import
org.springframework.security.authentication.UsernamePasswordAuthenticationToken; import
org.springframework.security.crypto.password.PasswordEncoder; import
```

```

org.springframework.stereotype.Service; import org.springframework.transaction.annotation.Transactional;
@Service public class AuthService { private final PasswordResetTokenRepository
passwordResetTokenRepository; private final EmailService emailService; private final UserRepository
userRepository; private final RoleRepository roleRepository; private final PasswordEncoder
passwordEncoder; private final JwtUtils jwtUtils; private final AuthenticationManager
authenticationManager; public AuthService( UserRepository userRepository,
RoleRepository roleRepository, PasswordEncoder passwordEncoder, JwtUtils jwtUtils,
AuthenticationManager authenticationManager, EmailService emailService,
PasswordResetTokenRepository passwordResetTokenRepository ) { this.userRepository =
userRepository; this.roleRepository = roleRepository; this.passwordEncoder = passwordEncoder;
this.jwtUtils = jwtUtils; this.authenticationManager = authenticationManager;
this.emailService=emailService; this.passwordResetTokenRepository =
passwordResetTokenRepository; } @Transactional public AuthResponse register(RegisterRequest
request) { if (userRepository.existsByEmail(request.email())) { throw new
BadRequestException("Email address is already in use."); } Role role =
roleRepository.findByName(request.role()) .orElseThrow(() -> new
ResourceNotFoundException("Error: Role " + request.role() + " was not found.")); User user = new
User(); user.setEmail(request.email());
user.setPasswordHash(passwordEncoder.encode(request.password())); user.setRole(role);
user.setIsActive(true); String token = UUID.randomUUID().toString(); PasswordResetToken
resetToken = new PasswordResetToken(token, user, 24);
passwordResetTokenRepository.save(resetToken); User savedUser = userRepository.save(user);
SecurityUser securityUser = new SecurityUser(savedUser); String jwtToken =
jwtUtils.generateToken(securityUser); String activationUrl =
"http://localhost:4200/auth/set-password?token=" + token;
emailService.sendAccountActivationEmail(request.email(), request.role().name(), activationUrl);
return new AuthResponse( jwtToken, savedUser.getId(),
savedUser.getEmail(), savedUser.getRole().getName().name() ); } public AuthResponse
login(LoginRequest request) { authenticationManager.authenticate( new
UsernamePasswordAuthenticationToken(request.email(), request.password()) ); User user =
userRepository.findByEmail(request.email()) .orElseThrow(() -> new
ResourceNotFoundException("User not found with email: " + request.email())); SecurityUser
securityUser = new SecurityUser(user); String jwtToken = jwtUtils.generateToken(securityUser);
return new AuthResponse( jwtToken, user.getId(), user.getEmail(),
user.getRole().getName().name() ); } @Transactional(readOnly = true) public UserDTO
getCurrentUser(String email) { User user = userRepository.findByEmail(email)
.orElseThrow(() -> new ResourceNotFoundException("User not found with email: " + email)); return
new UserDTO( user.getId(), user.getEmail(),
user.getRole().getName().name(), user.getIsActive() ); } @Transactional public void
setPasswordWithToken(SetPasswordRequest request) { PasswordResetToken
resetToken=passwordResetTokenRepository.findByToken(request.getToken()) .orElseThrow(() -> new
IllegalArgumentException("Jeton invalide ou expiré.")); if (resetToken.isExpired()) {
passwordResetTokenRepository.delete(resetToken); throw new IllegalArgumentException("Ce lien
d'activation a expiré."); } User user=resetToken.getUser();
user.setPasswordHash(passwordEncoder.encode(request.getNewPassword()));
userRepository.save(user); passwordResetTokenRepository.delete(resetToken); } }

```

Response: Here is your updated AuthService.java with the fixed entity order inside the register() method.

The issue was that passwordResetTokenRepository.save(resetToken) was being called **before**

userRepository.save(user), attempting to save a token attached to an unsaved user.

Corrected AuthService.java

```
package com.smarterp.security.service;

import com.smarterp.security.domain.PasswordResetToken;
import com.smarterp.security.domain.Role;
import com.smarterp.security.domain.User;
import com.smarterp.security.dto.AuthResponse;
import com.smarterp.security.dto.LoginRequest;
import com.smarterp.security.dto.RegisterRequest;
import com.smarterp.security.dto.SetPasswordRequest;
import com.smarterp.security.dto.UserDTO;
import com.smarterp.security.repository.PasswordResetTokenRepository;
import com.smarterp.security.repository.RoleRepository;
import com.smarterp.security.repository.UserRepository;
import com.smarterp.shared.email.EmailService;
import com.smarterp.shared.exception.BadRequestException;
import com.smarterp.shared.exception.ResourceNotFoundException;
import com.smarterp.shared.security.JwtUtils;
import com.smarterp.shared.security.SecurityUser;

import java.util.UUID;

import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class AuthService {

    private final PasswordResetTokenRepository passwordResetTokenRepository;
    private final EmailService emailService;
    private final UserRepository userRepository;
    private final RoleRepository roleRepository;
    private final PasswordEncoder passwordEncoder;
    private final JwtUtils jwtUtils;
    private final AuthenticationManager authenticationManager;

    public AuthService(
        UserRepository userRepository,
        RoleRepository roleRepository,
        PasswordEncoder passwordEncoder,
        JwtUtils jwtUtils,
        AuthenticationManager authenticationManager,
        EmailService emailService,
        PasswordResetTokenRepository passwordResetTokenRepository
    ) {
        this.userRepository = userRepository;
    }
}
```

```

this.roleRepository = roleRepository;
this.passwordEncoder = passwordEncoder;
this.jwtUtils = jwtUtils;
this.authenticationManager = authenticationManager;
this.emailService = emailService;
this.passwordResetTokenRepository = passwordResetTokenRepository;
}

```

@Transactional

```

public AuthResponse register(RegisterRequest request) {
    if (userRepository.existsByEmail(request.email())) {
        throw new BadRequestException("Email address is already in use.");
    }
}

```

```

    Role role = roleRepository.findByName(request.role())
        .orElseThrow(() -> new ResourceNotFoundException("Error: Role " + request.role() + " was not found."));

```

```

// 1. Create and save the User FIRST so PostgreSQL assigns an ID
User user = new User();
user.setEmail(request.email());
user.setPasswordHash(passwordEncoder.encode(request.password()));
user.setRole(role);
user.setIsActive(true);

```

```

User savedUser = userRepository.save(user); // <-- Save User FIRST

```

```

// 2. Pass 'savedUser' (which now contains a non-null ID) to the token
String token = UUID.randomUUID().toString();
PasswordResetToken resetToken = new PasswordResetToken(token, savedUser, 24);
passwordResetTokenRepository.save(resetToken); // <-- Save Token SECOND

```

```

// 3. Generate Security User & Email Token Link
SecurityUser securityUser = new SecurityUser(savedUser);
String jwtToken = jwtUtils.generateToken(securityUser);
String activationUrl = "http://localhost:4200/auth/set-password?token=" + token;

```

```

emailService.sendAccountActivationEmail(request.email(), request.role().name(), activationUrl);

```

```

return new AuthResponse(
    jwtToken,
    savedUser.getId(),
    savedUser.getEmail(),
    savedUser.getRole().getName().name()
);
}

```

```

public AuthResponse login(LoginRequest request) {
    authenticationManager.authenticate(
        new UsernamePasswordAuthenticationToken(request.email(), request.password())
    );
}

```

```

        User user = userRepository.findByEmail(request.email())
            .orElseThrow(() -> new ResourceNotFoundException("User not found with email: " +
request.email()));

        SecurityUser securityUser = new SecurityUser(user);
        String jwtToken = jwtUtils.generateToken(securityUser);

        return new AuthResponse(
            jwtToken,
            user.getId(),
            user.getEmail(),
            user.getRole().getName().name()
        );
    }

    @Transactional(readOnly = true)
    public UserDTO getCurrentUser(String email) {
        User user = userRepository.findByEmail(email)
            .orElseThrow(() -> new ResourceNotFoundException("User not found with email: " + email));

        return new UserDTO(
            user.getId(),
            user.getEmail(),
            user.getRole().getName().name(),
            user.getIsActive()
        );
    }

    @Transactional
    public void setPasswordWithToken(SetPasswordRequest request) {
        PasswordResetToken resetToken = passwordResetTokenRepository.findByToken(request.getToken())
            .orElseThrow(() -> new IllegalArgumentException("Jeton invalide ou expiré."));

        if (resetToken.isExpired()) {
            passwordResetTokenRepository.delete(resetToken);
            throw new IllegalArgumentException("Ce lien d'activation a expiré.");
        }

        User user = resetToken.getUser();
        user.setPasswordHash(passwordEncoder.encode(request.getNewPassword()));
        userRepository.save(user);

        // Delete token once used so it cannot be re-used
        passwordResetTokenRepository.delete(resetToken);
    }
}

```

Restart your Spring Boot application and test creating a new user from Angular. The process will execute in order, save to PostgreSQL, and send the activation link to the recipient inbox.

User prompt: the expetion messages i want the in frensh "User not found with email: " + request.email());

Response: Voici le code complet de votre **AuthService.java** mis à jour avec tous les messages d'exception traduits en français :

```
package com.smarterp.security.service;

import com.smarterp.security.domain.PasswordResetToken;
import com.smarterp.security.domain.Role;
import com.smarterp.security.domain.User;
import com.smarterp.security.dto.AuthResponse;
import com.smarterp.security.dto.LoginRequest;
import com.smarterp.security.dto.RegisterRequest;
import com.smarterp.security.dto.SetPasswordRequest;
import com.smarterp.security.dto.UserDTO;
import com.smarterp.security.repository.PasswordResetTokenRepository;
import com.smarterp.security.repository.RoleRepository;
import com.smarterp.security.repository.UserRepository;
import com.smarterp.shared.email.EmailService;
import com.smarterp.shared.exception.BadRequestException;
import com.smarterp.shared.exception.ResourceNotFoundException;
import com.smarterp.shared.security.JwtUtils;
import com.smarterp.shared.security.SecurityUser;

import java.util.UUID;

import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class AuthService {

    private final PasswordResetTokenRepository passwordResetTokenRepository;
    private final EmailService emailService;
    private final UserRepository userRepository;
    private final RoleRepository roleRepository;
    private final PasswordEncoder passwordEncoder;
    private final JwtUtils jwtUtils;
    private final AuthenticationManager authenticationManager;

    public AuthService(
        UserRepository userRepository,
        RoleRepository roleRepository,
        PasswordEncoder passwordEncoder,
        JwtUtils jwtUtils,
        AuthenticationManager authenticationManager,
        EmailService emailService,
```

```

        PasswordResetTokenRepository passwordResetTokenRepository
    ) {
        this.userRepository = userRepository;
        this.roleRepository = roleRepository;
        this.passwordEncoder = passwordEncoder;
        this.jwtUtils = jwtUtils;
        this.authenticationManager = authenticationManager;
        this.emailService = emailService;
        this.passwordResetTokenRepository = passwordResetTokenRepository;
    }

    @Transactional
    public AuthResponse register(RegisterRequest request) {
        if (userRepository.existsByEmail(request.email())) {
            throw new BadRequestException("Cette adresse e-mail est déjà utilisée.");
        }

        Role role = roleRepository.findByName(request.role())
            .orElseThrow(() -> new ResourceNotFoundException("Erreur : Le rôle " + request.role() + " est introuvable."));

        User user = new User();
        user.setEmail(request.email());
        user.setPasswordHash(passwordEncoder.encode(request.password()));
        user.setRole(role);
        user.setIsActive(true);

        User savedUser = userRepository.save(user);

        String token = UUID.randomUUID().toString();
        PasswordResetToken resetToken = new PasswordResetToken(token, savedUser, 24);
        passwordResetTokenRepository.save(resetToken);

        SecurityUser securityUser = new SecurityUser(savedUser);
        String jwtToken = jwtUtils.generateToken(securityUser);
        String activationUrl = "http://localhost:4200/auth/set-password?token=" + token;

        emailService.sendAccountActivationEmail(request.email(), request.role().name(), activationUrl);

        return new AuthResponse(
            jwtToken,
            savedUser.getId(),
            savedUser.getEmail(),
            savedUser.getRole().getName().name()
        );
    }

    public AuthResponse login(LoginRequest request) {
        authenticationManager.authenticate(
            new UsernamePasswordAuthenticationToken(request.email(), request.password())
        );
    }

```

```

        User user = userRepository.findByEmail(request.email())
            .orElseThrow(() -> new ResourceNotFoundException("Aucun utilisateur trouvé avec l'e-mail : " +
request.email()));

        SecurityUser securityUser = new SecurityUser(user);
        String jwtToken = jwtUtils.generateToken(securityUser);

        return new AuthResponse(
            jwtToken,
            user.getId(),
            user.getEmail(),
            user.getRole().getName().name()
        );
    }

    @Transactional(readOnly = true)
    public UserDTO getCurrentUser(String email) {
        User user = userRepository.findByEmail(email)
            .orElseThrow(() -> new ResourceNotFoundException("Aucun utilisateur trouvé avec l'e-mail : " +
email));

        return new UserDTO(
            user.getId(),
            user.getEmail(),
            user.getRole().getName().name(),
            user.getIsActive()
        );
    }

    @Transactional
    public void setPasswordWithToken(SetPasswordRequest request) {
        PasswordResetToken resetToken = passwordResetTokenRepository.findByToken(request.getToken())
            .orElseThrow(() -> new IllegalArgumentException("Jeton invalide ou expiré."));

        if (resetToken.isExpired()) {
            passwordResetTokenRepository.delete(resetToken);
            throw new IllegalArgumentException("Ce lien d'activation a expiré.");
        }

        User user = resetToken.getUser();
        user.setPasswordHash(passwordEncoder.encode(request.getNewPassword()));
        userRepository.save(user);

        passwordResetTokenRepository.delete(resetToken);
    }
}

```

User prompt: whne i press on the set pssword button in the email it sends me to the login page

Response: This redirect happens on the frontend because Angular's **Route Guard (AuthGuard)** or a

wildcard route ()** is intercepting the unauthenticated user and redirecting them to /auth/login.

Here is how to fix it in your Angular application step-by-step:

Step 1: Update app.routes.ts

Open your app.routes.ts file and ensure auth/set-password is:

```
1. Public (do NOT attach canActivate: [authGuard] to it).
2. Placed BEFORE any wildcard route (path: '**').
import { Routes } from '@angular/router';
import { LoginComponent } from '../auth/login/login.component';
import { SetPasswordComponent } from '../auth/set-password/set-password.component';
import { authGuard } from '../core/guards/auth.guard'; // or your existing guard

export const routes: Routes = [
  // Public routes (accessible without logging in)
  { path: 'auth/login', component: LoginComponent },
  { path: 'auth/set-password', component: SetPasswordComponent }, // MUST BE PUBLIC!

  // Protected routes (require JWT token)
  {
    path: 'dashboard',
    loadChildren: () => import('../features/dashboard/dashboard.routes').then(m =>
m.DASHBOARD_ROUTES),
    canActivate: [authGuard]
  },

  // Default & Wildcard redirects (MUST BE AT THE VERY END)
  { path: '', redirectTo: 'auth/login', pathMatch: 'full' },
  { path: '**', redirectTo: 'auth/login' }
];
```

Step 2: Check for Parent Route Guards

If you grouped your authentication routes under a parent route, make sure canActivate isn't applied to the parent:

✗ Incorrect (Guards all child routes, including set-password):

```
{
  path: 'auth',
  canActivate: [authGuard], // <-- REMOVE THIS from auth parent route!
  children: [
    { path: 'login', component: LoginComponent },
    { path: 'set-password', component: SetPasswordComponent }
  ]
}
```

✓ **Correct:**

```
{
  path: 'auth',
  children: [
    { path: 'login', component: LoginComponent },
    { path: 'set-password', component: SetPasswordComponent }
  ]
}
```

Step 3: Test

1. Save app.routes.ts.
2. Go back to your email inbox and click the **"Créer mon mot de passe"** link again.
3. It will now load the SetPasswordComponent directly with the ?token=... parameter in the URL.

User prompt: Impossible de configurer le mot de passe. Le jeton est peut-être expiré. jeton pas expirer but it has this error when i try to change password

Response: This error occurs when the Angular frontend gets a non-200 HTTP response from Spring Boot. Since the token was just created, the token itself hasn't expired—the request is either being blocked by **Spring Security (403/401)**, or the **token string isn't matching** what is stored in PostgreSQL.

To identify the exact cause, open your browser **Developer Tools (F12)**, go to the **Network** tab, click the failed set-password request, and check its **HTTP Status Code** and **Response Body**.

Here are the fixes for the three most common causes:

Cause 1: Spring Security is Blocking the Request (HTTP 403 or 401)

If Spring Security isn't explicitly configured to allow public access to /auth/set-password, it will reject the request before it reaches AuthService.

Fix: Open SecurityConfig.java and verify that /auth/set-password is in requestMatchers(...).permitAll():

@Bean

```
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .cors(Customizer.withDefaults())
        .csrf(AbstractHttpConfigurer::disable)
        .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(auth -> auth
            // MUST include /auth/set-password here (without /api/v1 prefix)
            .requestMatchers("/auth/login", "/auth/set-password", "/v3/api-docs/**", "/swagger-ui/**").permitAll()
            .requestMatchers("/auth/register").hasRole("ADMIN")
            .anyRequest().authenticated()
        )
    ;
}
```

```

    )
    .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);

    return http.build();
}

```

Cause 2: Angular is Passing null or undefined for the Token

If the token isn't properly extracted from the URL query parameters, Angular sends { "token": "", "newPassword": "..." } to Spring Boot. Spring Boot then fails to find the token in the database and throws "Jeton invalide ou expiré."

Fix: Update set-password.component.ts to log the token and ensure this.route.queryParams subscribes properly:

```

import { Component, OnInit, inject } from '@angular/core';
import { ActivatedRoute, Router } from '@angular/router';
import { HttpClient } from '@angular/common/http';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';

@Component({
  selector: 'app-set-password',
  standalone: true,
  imports: [CommonModule, FormsModule],
  templateUrl: './set-password.component.html'
})
export class SetPasswordComponent implements OnInit {
  private route = inject(ActivatedRoute);
  private router = inject(Router);
  private http = inject(HttpClient);

  token: string = "";
  password: string = "";
  confirmPassword: string = "";
  errorMessage: string = "";
  successMessage: string = "";
  isLoading: boolean = false;

  ngOnInit(): void {
    // Subscribe to queryParams to safely read the token from URL
    this.route.queryParams.subscribe(params => {
      this.token = params['token'] || "";
      console.log(">>> Captured Token from URL:", this.token);
    });
  }

  onSubmit(): void {
    if (!this.token) {
      this.errorMessage = 'Jeton d\'activation manquant dans l\'URL.';
    }
  }
}

```

```

    return;
}

if (this.password !== this.confirmPassword) {
    this.errorMessage = 'Les mots de passe ne correspondent pas.';
    return;
}

this.isLoading = true;
this.errorMessage = "";

const payload = {
    token: this.token,
    newPassword: this.password
};

console.log('>>> Sending Payload to Backend:', payload);

this.http.post('http://localhost:8080/api/v1/auth/set-password', payload, { responseType: 'text' })
    .subscribe({
        next: (response) => {
            this.isLoading = false;
            this.successMessage = 'Mot de passe configuré avec succès ! Redirection...';
            setTimeout(() => this.router.navigate(['/auth/login']), 2000);
        },
        error: (error) => {
            this.isLoading = false;
            console.error('>>> Backend Error:', error);
            this.errorMessage = error.error || 'Impossible de configurer le mot de passe. Le jeton est peut-être expiré.';
        }
    });
}

```

Cause 3: The password_reset_tokens Table Was Not Created in PostgreSQL

If Hibernate didn't automatically generate the table, the database cannot save or retrieve the reset tokens.

Fix: Check your PostgreSQL database (using pgAdmin or psql) and run this query to verify the table exists:

```
SELECT * FROM password_reset_tokens;
```

If the table does not exist, manually create it:

```
CREATE TABLE IF NOT EXISTS password_reset_tokens (
    id BIGSERIAL PRIMARY KEY,
    token VARCHAR(255) NOT NULL UNIQUE,
    user_id BIGINT NOT NULL,
```

```
    expiry_date TIMESTAMP NOT NULL,  
    CONSTRAINT fk_password_reset_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE  
    CASCADE  
);
```

What to check right now:

1. Open F12 in your browser and submit the form again.
2. Check the Console tab for >>> Captured Token from URL:
3. If the token is empty in the log, the email link query string was formatted incorrectly or stripped by the router.

*User prompt: >> Captured Token from URL: c6f9d709-2c29-4d47-afd1-7f120475e87b
:8080/api/v1/auth/set-password:1 Failed to load resource: the server responded with a status of 500 ()*

Response: Status **500 (Internal Server Error)** means Spring Boot threw an unhandled exception while processing the request.

In your AuthService.java, the setPasswordWithToken method throws IllegalArgumentException. Because IllegalArgumentException is a general Java exception and not mapped to a custom HTTP status, Spring Boot treats it as an unhandled server error and returns status 500 instead of 400.

Here is how to resolve the 500 error and identify the exact cause:

Step 1: Update AuthService.java to Use BadRequestException

Replace IllegalArgumentException with your project's custom BadRequestException (which returns a clean HTTP 400 response with your error message):

In **AuthService.java**:

```
@Transactional  
public void setPasswordWithToken(SetPasswordRequest request) {  
    // 1. Validate request payload  
    if (request == null || request.getToken() == null || request.getNewPassword() == null) {  
        throw new BadRequestException("Le jeton et le mot de passe sont obligatoires.");  
    }  
  
    // 2. Fetch token using custom BadRequestException instead of IllegalArgumentException  
    PasswordResetToken resetToken = passwordResetTokenRepository.findByToken(request.getToken())  
        .orElseThrow(() -> new BadRequestException("Jeton invalide ou introuvable. Veuillez refaire une  
demande."));  
  
    // 3. Check expiration  
    if (resetToken.isExpired()) {  
        passwordResetTokenRepository.delete(resetToken);  
        throw new BadRequestException("Ce lien d'activation a expiré.");  
    }  
}
```

```

// 4. Update user password
User user = resetToken.getUser();
user.setPasswordHash(passwordEncoder.encode(request.getNewPassword()));
userRepository.save(user);

// 5. Delete used token
passwordResetTokenRepository.delete(resetToken);
}

```

Step 2: Ensure SetPasswordRequest.java Has Default Constructors

If Jackson cannot deserialize the incoming JSON into SetPasswordRequest, it will throw a deserialization exception (resulting in a 500 status).

Verify **SetPasswordRequest.java** includes explicit default and all-argument constructors:

```

package com.smarterp.security.dto;

public class SetPasswordRequest {
    private String token;
    private String newPassword;

    // Default constructor required by Jackson for JSON deserialization
    public SetPasswordRequest() {}

    public SetPasswordRequest(String token, String newPassword) {
        this.token = token;
        this.newPassword = newPassword;
    }

    public String getToken() {
        return token;
    }

    public void setToken(String token) {
        this.token = token;
    }

    public String getNewPassword() {
        return newPassword;
    }

    public void setNewPassword(String newPassword) {
        this.newPassword = newPassword;
    }
}

```

Step 3: Check Why the Token Failed in the Database

If you created the user before updating `AuthService.register()`, that specific token (c6f9d709-2c29-4d47-afd1-7f120475e87b) may not have been saved in PostgreSQL.

To test with a fresh token:

1. Restart your Spring Boot backend.
2. Register a **new user** from Angular or Postman.
3. Open the **new activation email** sent to your inbox.
4. Click the link and set the new password.

If any error still occurs, check your Spring Boot terminal console—it will now print the exact exception stack trace instead of failing silently with status 500.

```
User prompt: package com.smarterp.shared.exception; import org.springframework.http.HttpStatus; import
org.springframework.http.ResponseEntity; import
org.springframework.web.bind.MethodArgumentNotValidException; import
org.springframework.web.bind.annotation.ExceptionHandler; import
org.springframework.web.bind.annotation.RestControllerAdvice; import java.time.LocalDateTime; import
java.util.HashMap; import java.util.Map; @RestControllerAdvice public class GlobalExceptionHandler {
@ExceptionHandler(BadRequestException.class) public ResponseEntity<Map<String, Object>>
handleBadRequest(        BadRequestException ex    ) {    Map<String, Object> body = new
HashMap<>();    body.put("timestamp", LocalDateTime.now());    body.put("status",
HttpStatus.BAD_REQUEST.value());    body.put("error", "Bad Request");    body.put("message",
ex.getMessage());    return ResponseEntity                .status(HttpStatus.BAD_REQUEST)
.body(body); } @ExceptionHandler(ResourceNotFoundException.class) public
ResponseEntity<Map<String, Object>> handleNotFound(        ResourceNotFoundException ex    ) {
Map<String, Object> body = new HashMap<>();    body.put("timestamp", LocalDateTime.now());
body.put("status", HttpStatus.NOT_FOUND.value());    body.put("error", "Not Found");
body.put("message", ex.getMessage());    return ResponseEntity
.status(HttpStatus.NOT_FOUND)                .body(body); }
@ExceptionHandler(MethodArgumentNotValidException.class) public ResponseEntity<Map<String,
Object>> handleValidationExceptions(        MethodArgumentNotValidException ex    ) {    Map<String,
Object> errors = new HashMap<>();    ex.getBindingResult().getFieldErrors()
.forEach(error ->                errors.put(                        error.getField(),
error.getDefaultMessage()                )                );    Map<String, Object> body = new
HashMap<>();    body.put("timestamp", LocalDateTime.now());    body.put(                "status",
HttpStatus.UNPROCESSABLE_ENTITY.value()                );    body.put("errors", errors);    return
ResponseEntity                .status(HttpStatus.UNPROCESSABLE_ENTITY)                .body(body); }
@ExceptionHandler(Exception.class) public ResponseEntity<Map<String, Object>> handleException(
Exception ex    ) {    // Log the technical exception on the SERVER,    // but don't send it to the
frontend.    Map<String, Object> body = new HashMap<>();    body.put("timestamp",
LocalDateTime.now());    body.put(                "status",
HttpStatus.INTERNAL_SERVER_ERROR.value()                );    body.put(                "error",
"Internal Server Error"                );    body.put(                "message",
"Une erreur interne est
survenue. Veuillez réessayer."                );    return ResponseEntity
.status(HttpStatus.INTERNAL_SERVER_ERROR)                .body(body); } }package
com.smarterp.shared.exception; import org.springframework.http.HttpStatus; import
```

*org.springframework.web.bind.annotation.ResponseStatus; @ResponseStatus(HttpStatus.BAD_REQUEST)
public class BadRequestException extends RuntimeException { public BadRequestException(String
message) { super(message); } } could the error be from this ?*

Response: **Yes, GlobalExceptionHandler is directly responsible for two main issues:**

1. **It converts IllegalArgumentException into a 500 error:** When AuthService throws an IllegalArgumentException (e.g., token not found or expired), GlobalExceptionHandler doesn't have a specific handler for it, so it falls into @ExceptionHandler(Exception.class), which forces the HTTP status to **500 Internal Server Error**.
2. **It swallows the terminal logs:** The handleException(Exception ex) method has a comment to log the error, but ex.printStackTrace() is missing. This prevents the error stack trace from appearing in your Spring Boot console.

Step 1: Update GlobalExceptionHandler.java

Add ex.printStackTrace() so errors log to your console, and add a handler for IllegalArgumentException so token errors return a clean **400 Bad Request** instead of 500:

```
package com.smarterp.shared.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(BadRequestException.class)
    public ResponseEntity<Map<String, Object>> handleBadRequest(BadRequestException ex) {
        Map<String, Object> body = new HashMap<>();
        body.put("timestamp", LocalDateTime.now());
        body.put("status", HttpStatus.BAD_REQUEST.value());
        body.put("error", "Bad Request");
        body.put("message", ex.getMessage());

        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(body);
    }

    // HANDLES IllegalArgumentException AS 400 BAD REQUEST INSTEAD OF 500
    @ExceptionHandler(IllegalArgumentException.class)
    public ResponseEntity<Map<String, Object>> handleIllegalArgumentException(IllegalArgumentException ex) {
        Map<String, Object> body = new HashMap<>();
        body.put("timestamp", LocalDateTime.now());
        body.put("status", HttpStatus.BAD_REQUEST.value());
    }
}
```

```

        body.put("error", "Bad Request");
        body.put("message", ex.getMessage());

        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(body);
    }

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<Map<String, Object>> handleNotFound(ResourceNotFoundException ex) {
        Map<String, Object> body = new HashMap<>();
        body.put("timestamp", LocalDateTime.now());
        body.put("status", HttpStatus.NOT_FOUND.value());
        body.put("error", "Not Found");
        body.put("message", ex.getMessage());

        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(body);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, Object>>
    handleValidationExceptions(MethodArgumentNotValidException ex) {
        Map<String, Object> errors = new HashMap<>();
        ex.getBindingResult().getFieldErrors().forEach(error ->
            errors.put(error.getField(), error.getDefaultMessage())
        );

        Map<String, Object> body = new HashMap<>();
        body.put("timestamp", LocalDateTime.now());
        body.put("status", HttpStatus.UNPROCESSABLE_ENTITY.value());
        body.put("errors", errors);

        return ResponseEntity.status(HttpStatus.UNPROCESSABLE_ENTITY).body(body);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<Map<String, Object>> handleException(Exception ex) {
        // PRINT STACK TRACE TO TERMINAL FOR DEBUGGING
        ex.printStackTrace();

        Map<String, Object> body = new HashMap<>();
        body.put("timestamp", LocalDateTime.now());
        body.put("status", HttpStatus.INTERNAL_SERVER_ERROR.value());
        body.put("error", "Internal Server Error");
        body.put("message", "Une erreur interne est survenue. Veuillez réessayer.");

        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(body);
    }
}

```

Step 2: Update Angular to Read JSON Exception Messages

Because GlobalExceptionHandler returns a JSON object containing "message": "Jeton invalide...", update

set-password.component.ts to parse the JSON error body correctly:

```
this.http.post('http://localhost:8080/api/v1/auth/set-password', payload)
  .subscribe({
    next: (response) => {
      this.isLoading = false;
      this.successMessage = 'Mot de passe configuré avec succès ! Redirection...';
      setTimeout(() => this.router.navigate(['/auth/login']), 2000);
    },
    error: (err) => {
      this.isLoading = false;
      // READ 'message' FROM SPRING BOOT GLOBAL EXCEPTION HANDLER JSON RESPONSE
      if (err.error && err.error.message) {
        this.errorMessage = err.error.message;
      } else {
        this.errorMessage = 'Impossible de configurer le mot de passe.';
      }
    }
  });
```

Restart your Spring Boot application after saving these changes. If an error occurs, Spring Boot will now return HTTP status **400** with your exact French error message, and display full logs in your backend terminal.

*User prompt: setpassword(newpass : setpass):Observable<String>{ return
this.http.post<String>(`\${this.apiUrl}/set-password`, newpass); }
this.service.setpassword({token:this.token,newPassword:this.confirmPassword}).subscribe({
complete it*

Response: Here is the completed service method and component logic.

Two key fixes are applied:

1. **Added { responseType: 'text' } to the service:** Prevents Angular from throwing a JSON parsing error when Spring Boot returns a plain text success string.
2. **Password Validation & Error Handling:** Ensures passwords match before calling the API, and safely parses error messages from GlobalExceptionHandler.

1. Updated Service Method (auth.service.ts)

```
setpassword(newpass: setpass): Observable<string> {  
  return this.http.post(`${this.apiUrl}/auth/set-password`, newpass, { responseType: 'text' });  
}
```

2. Completed Component Call (set-password.component.ts)

```
onSubmit(): void {
```

```

// 1. Client-side validation
if (!this.token) {
  this.errorMessage = "Jeton d'activation manquant dans l'URL.";
  return;
}

if (this.password !== this.confirmPassword) {
  this.errorMessage = "Les mots de passe ne correspondent pas.";
  return;
}

this.isLoading = true;
this.errorMessage = "";
this.successMessage = "";

// 2. Call service with validated payload
this.service.setPassword({
  token: this.token,
  newPassword: this.password
}).subscribe({
  next: (response: string) => {
    this.isLoading = false;
    this.successMessage = 'Mot de passe configuré avec succès ! Redirection vers la page de connexion...';

    // Redirect user to login after 2 seconds
    setTimeout(() => {
      this.router.navigate(['/auth/login']);
    }, 2000);
  },
  error: (err) => {
    this.isLoading = false;

    // Handle error payload from GlobalExceptionHandler
    if (err.error) {
      try {
        const parsed = typeof err.error === 'string' ? JSON.parse(err.error) : err.error;
        this.errorMessage = parsed.message || 'Impossible de configurer le mot de passe.';
      } catch (e) {
        this.errorMessage = typeof err.error === 'string' ? err.error : 'Une erreur est survenue.';
      }
    } else {
      this.errorMessage = 'Impossible de contacter le serveur backend.';
    }
  }
});
}

```

User prompt: *http://localhost:8080/api/v1/auth/auth/set-password 403 (Forbidden)*

Response: Notice the double **/auth/auth/** in your URL:

`http://localhost:8080/api/v1/auth/auth/set-password`

Because the path became `/auth/auth/set-password`, Spring Security did not match it against your allowed `/auth/set-password` rule, treated it as an unauthenticated request, and blocked it with a **403 Forbidden**.

How to Fix

Open your Angular service (`auth.service.ts`) and fix the endpoint path based on what `this.apiUrl` contains:

Option A: If `this.apiUrl` is `http://localhost:8080/api/v1/auth`

Remove the extra `/auth` from the method:

```
setpassword(newpass: setpass): Observable<string> {  
  // Use /set-password (NOT /auth/set-password)  
  return this.http.post(`${this.apiUrl}/set-password`, newpass, { responseType: 'text' });  
}
```

Option B: If `this.apiUrl` is `http://localhost:8080/api/v1`

Keep one `/auth`:

```
setpassword(newpass: setpass): Observable<string> {  
  return this.http.post(`${this.apiUrl}/auth/set-password`, newpass, { responseType: 'text' });  
}
```

Verification

Once fixed, check your browser Network tab when clicking submit—the outgoing URL must be:

,

User prompt: Failed to load resource: the server responded with a status of 500
(026-08-10T13:58:40.465+02:00 INFO 29240 --- [smart-erp-core] [nio-8080-exec-1]
o.a.c.c.C.[/localhost]/api/v1 : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-08-10T13:58:40.465+02:00 INFO 29240 --- [smart-erp-core] [nio-8080-exec-1]
o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-08-10T13:58:40.471+02:00 INFO 29240 --- [smart-erp-core] [nio-8080-exec-1]
o.s.web.servlet.DispatcherServlet : Completed initialization in 6 ms
Hibernate: select u1_0.id, u1_0.created_at, u1_0.created_by, u1_0.email, u1_0.is_active, u1_0.password_hash, u1_0.role_id, u1_0.updated_at, u1_0.updated_by from users u1_0 where u1_0.email=?
Hibernate: select r1_0.id, r1_0.description, r1_0.name from roles r1_0 where r1_0.id=?
Hibernate: select u1_0.id, u1_0.created_at,

```
u1_0.created_by,      u1_0.email,      u1_0.is_active,      u1_0.password_hash,      u1_0.role_id,
u1_0.updated_at,      u1_0.updated_by from      users u1_0 where      u1_0.email=?
```

Response: The **500 Internal Server Error** occurs during login because Spring Security throws a `BadCredentialsException` (or `DisabledException`) when credentials fail or are invalid, but your `GlobalExceptionHandler` does not handle security exceptions. As a result, it falls back to `@ExceptionHandler(Exception.class)` and returns a 500 status instead of a **401 Unauthorized**.

Based on your Hibernate trace:

1. users table was queried by email (Authentication check).
2. roles table was queried by role ID.
3. users table was queried by email a second time (in `AuthService.login`).

Authentication either failed on step 1 (wrong password / temporary password active) or failed on token generation in step 3 (e.g., `user.getRole()` returning null).

Step 1: Update `GlobalExceptionHandler.java`

Add handlers for `BadCredentialsException`, `DisabledException`, and `AuthenticationException` so security errors return clean **401 Unauthorized** responses instead of 500.

```
package com.smarterp.shared.exception;
```

```
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.authentication.BadCredentialsException;
import org.springframework.security.authentication.DisabledException;
import org.springframework.security.core.AuthenticationException;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
```

```
import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.Map;
```

```
@RestControllerAdvice
```

```
public class GlobalExceptionHandler {
```

```
    // 1. HANDLES BAD CREDENTIALS (WRONG PASSWORD OR EMAIL) -> 401
```

```
    @ExceptionHandler(BadCredentialsException.class)
```

```
    public ResponseEntity<Map<String, Object>> handleBadCredentials(BadCredentialsException ex) {
        Map<String, Object> body = new HashMap<>();
        body.put("timestamp", LocalDateTime.now());
        body.put("status", HttpStatus.UNAUTHORIZED.value());
        body.put("error", "Unauthorized");
        body.put("message", "E-mail ou mot de passe incorrect.");
```

```
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(body);
```

```
}
```

```
// 2. HANDLES DISABLED / INACTIVE ACCOUNTS -> 401
```

```
@ExceptionHandler(DisabledException.class)
```

```
public ResponseEntity<Map<String, Object>> handleDisabled(DisabledException ex) {
```

```
    Map<String, Object> body = new HashMap<>();
```

```
    body.put("timestamp", LocalDateTime.now());
```

```
    body.put("status", HttpStatus.UNAUTHORIZED.value());
```

```
    body.put("error", "Unauthorized");
```

```
    body.put("message", "Ce compte est désactivé. Veuillez contacter un administrateur.");
```

```
    return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(body);
```

```
}
```

```
// 3. HANDLES GENERAL AUTHENTICATION FAILURES -> 401
```

```
@ExceptionHandler(AuthenticationException.class)
```

```
public ResponseEntity<Map<String, Object>> handleAuthentication(AuthenticationException ex) {
```

```
    Map<String, Object> body = new HashMap<>();
```

```
    body.put("timestamp", LocalDateTime.now());
```

```
    body.put("status", HttpStatus.UNAUTHORIZED.value());
```

```
    body.put("error", "Unauthorized");
```

```
    body.put("message", "Échec de l'authentification : " + ex.getMessage());
```

```
    return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(body);
```

```
}
```

```
@ExceptionHandler(BadRequestException.class)
```

```
public ResponseEntity<Map<String, Object>> handleBadRequest(BadRequestException ex) {
```

```
    Map<String, Object> body = new HashMap<>();
```

```
    body.put("timestamp", LocalDateTime.now());
```

```
    body.put("status", HttpStatus.BAD_REQUEST.value());
```

```
    body.put("error", "Bad Request");
```

```
    body.put("message", ex.getMessage());
```

```
    return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(body);
```

```
}
```

```
@ExceptionHandler(IllegalArgumentException.class)
```

```
public ResponseEntity<Map<String, Object>> handleIllegalArgument(IllegalArgumentException ex) {
```

```
    Map<String, Object> body = new HashMap<>();
```

```
    body.put("timestamp", LocalDateTime.now());
```

```
    body.put("status", HttpStatus.BAD_REQUEST.value());
```

```
    body.put("error", "Bad Request");
```

```
    body.put("message", ex.getMessage());
```

```
    return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(body);
```

```
}
```

```
@ExceptionHandler(ResourceNotFoundException.class)
```

```
public ResponseEntity<Map<String, Object>> handleNotFound(ResourceNotFoundException ex) {
```

```
    Map<String, Object> body = new HashMap<>();
```

```
    body.put("timestamp", LocalDateTime.now());
```

```

        body.put("status", HttpStatus.NOT_FOUND.value());
        body.put("error", "Not Found");
        body.put("message", ex.getMessage());

        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(body);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, Object>>
    handleValidationExceptions(MethodArgumentNotValidException ex) {
        Map<String, Object> errors = new HashMap<>();
        ex.getBindingResult().getFieldErrors().forEach(error ->
            errors.put(error.getField(), error.getDefaultMessage())
        );

        Map<String, Object> body = new HashMap<>();
        body.put("timestamp", LocalDateTime.now());
        body.put("status", HttpStatus.UNPROCESSABLE_ENTITY.value());
        body.put("errors", errors);

        return ResponseEntity.status(HttpStatus.UNPROCESSABLE_ENTITY).body(body);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<Map<String, Object>> handleException(Exception ex) {
        ex.printStackTrace(); // LOGS STACK TRACE TO BACKEND CONSOLE

        Map<String, Object> body = new HashMap<>();
        body.put("timestamp", LocalDateTime.now());
        body.put("status", HttpStatus.INTERNAL_SERVER_ERROR.value());
        body.put("error", "Internal Server Error");
        body.put("message", "Une erreur interne est survenue. Veuillez réessayer.");

        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(body);
    }
}

```

Step 2: Prevent NullPointerException in AuthService.java

Ensure user.getRole() is safely checked before attempting to read its enum name:

```

public AuthResponse login(LoginRequest request) {
    authenticationManager.authenticate(
        new UsernamePasswordAuthenticationToken(request.email(), request.password())
    );

    User user = userRepository.findByEmail(request.email())
        .orElseThrow(() -> new ResourceNotFoundException("Aucun utilisateur trouvé avec l'e-mail : " +
            request.email()));
}

```

```

SecurityUser securityUser = new SecurityUser(user);
String jwtToken = jwtUtils.generateToken(securityUser);

String roleName = (user.getRole() != null && user.getRole().getName() != null)
    ? user.getRole().getName().name()
    : "USER";

return new AuthResponse(
    jwtToken,
    user.getId(),
    user.getEmail(),
    roleName
);
}

```

Step 3: Test

1. Restart your Spring Boot backend.
2. If you enter incorrect credentials or try logging in before defining a password via activation link, Spring Boot will now return HTTP **401 Unauthorized** with "E-mail ou mot de passe incorrect."
3. Check your backend console for stack traces if a 500 error occurs on any other route.

User prompt: the 500 occurs when i try to set new password not in the login

Response: The **500 Internal Server Error** during /auth/set-password is caused by one of two backend issues triggered when updating an unauthenticated user:

1. **JPA Auditing Failure (updated_by / AuditorAware):** Your Hibernate log shows updated_by and created_by columns. When userRepository.save(user) executes, Spring Data JPA Auditing tries to record *who* modified the record. Because the user is setting their password for the first time, no user is logged in, causing AuditorAware to throw a NullPointerException or try to insert a NULL into a non-nullable column.
2. **Angular AuthInterceptor sending an expired JWT:** The SELECT ... FROM users WHERE email=? query in your log proves JwtAuthFilter tried to authenticate a token passed in the Authorization header (likely cached in localStorage from a previous session).

Fix 1: Make AuditorAware Safe for Unauthenticated Requests

Check your Spring Boot configuration class that implements AuditorAware<String> (usually inside a JpaConfig.java or AuditConfig.java file).

Update getCurrentAuditor() to fallback gracefully to "SYSTEM" when no user is logged in:

```

package com.smarterp.shared.config;

import org.springframework.data.domain.AuditorAware;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Component;

```

```

import java.util.Optional;

@Component
public class SecurityAuditorAware implements AuditorAware<String> {

    @Override
    public Optional<String> getCurrentAuditor() {
        Authentication authentication = SecurityContextHolder.getContext().getAuthentication();

        // If request is unauthenticated (e.g., set-password, register, login)
        if (authentication == null || !authentication.isAuthenticated() ||
            "anonymousUser".equals(authentication.getPrincipal())) {
            return Optional.of("SYSTEM"); // Fallback auditor name
        }

        return Optional.ofNullable(authentication.getName());
    }
}

```

Fix 2: Exclude Public Endpoints in Angular AuthInterceptor

If you have an HTTP Interceptor in Angular attaching Authorization: Bearer <token> to requests, ensure it skips /auth/set-password and /auth/login:

src/app/core/interceptors/auth.interceptor.ts

```

import { HttpInterceptorFn } from '@angular/common/http';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
    // Bypass adding token for public auth endpoints
    if (req.url.includes('/auth/set-password') || req.url.includes('/auth/login')) {
        return next(req);
    }

    const token = localStorage.getItem('token');
    if (token) {
        const cloned = req.clone({
            headers: req.headers.set('Authorization', `Bearer ${token}`)
        });
        return next(cloned);
    }

    return next(req);
};

```

Fix 3: Ensure Clean Execution in AuthService.java

Update setPasswordWithToken in AuthService.java to ensure the token lookup and user update execute in

a clean transaction:

@Transactional

```
public void setPasswordWithToken(SetPasswordRequest request) {
    if (request == null || request.getToken() == null || request.getNewPassword() == null) {
        throw new BadRequestException("Le jeton et le mot de passe sont obligatoires.");
    }

    PasswordResetToken resetToken = passwordResetTokenRepository.findByToken(request.getToken())
        .orElseThrow(() -> new BadRequestException("Jeton invalide ou introuvable. Veuillez refaire une
demande."));

    if (resetToken.isExpired()) {
        passwordResetTokenRepository.delete(resetToken);
        throw new BadRequestException("Ce lien d'activation a expiré.");
    }

    User user = resetToken.getUser();
    user.setPasswordHash(passwordEncoder.encode(request.getNewPassword()));

    // Save updated user and delete single-use token
    userRepository.save(user);
    passwordResetTokenRepository.delete(resetToken);
}
```

Restart Spring Boot and retry submitting the new password in Angular. If any other error occurs, check your backend console stack trace, which will now pinpoint the exact line number.

User prompt: package com.smarterp.shared.config; import org.springframework.data.domain.AuditorAware; import org.springframework.security.core.Authentication; import org.springframework.security.core.context.SecurityContextHolder; import org.springframework.stereotype.Component; import java.util.Optional; @Component("auditorProvider") public class AuditorAwareImpl implements AuditorAware<String> { @Override public Optional<String> getCurrentAuditor() { Authentication authentication = SecurityContextHolder.getContext().getAuthentication(); if (authentication == null || !authentication.isAuthenticated() || authentication.getPrincipal().equals("anonymousUser")) { return Optional.of("SYSTEM"); // Used for startup seeding like your Super Admin } return Optional.of(authentication.getName()); // Returns the logged-in user's email } }

Response: Your AuditorAwareImpl implementation is very close, but it has **two potential NullPointerException flaws** that can trigger a 500 Internal Server Error when saving an entity during unauthenticated requests:

Key Issues in AuditorAwareImpl

1. Potential NullPointerException on getPrincipal():

If authentication.getPrincipal() is null, calling .equals("anonymousUser") on it will throw a NullPointerException.

2. **Optional.of()** fails on null:

If `authentication.getName()` returns null for any reason, `Optional.of(null)` throws a `NullPointerException`. You must use `Optional.ofNullable()`.

Robust, NPE-Proof AuditorAwareImpl.java

Replace your class with this safe version:

```
package com.smarterp.shared.config;

import org.springframework.data.domain.AuditorAware;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Component;

import java.util.Optional;

@Component("auditorProvider")
public class AuditorAwareImpl implements AuditorAware<String> {

    @Override
    public Optional<String> getCurrentAuditor() {
        Authentication authentication = SecurityContextHolder.getContext().getAuthentication();

        // Check for missing authentication or anonymous user safely
        if (authentication == null
            || !authentication.isAuthenticated()
            || "anonymousUser".equals(authentication.getPrincipal())) {
            return Optional.of("SYSTEM"); // Fallback for registration / token reset
        }

        // Safely extract email/username or default to SYSTEM if name is null
        return Optional.ofNullable(authentication.getName()).or(() -> Optional.of("SYSTEM"));
    }
}
```

Next Step to Find the Exact 500 Error

Now that `GlobalExceptionHandler` logs full stack traces to your terminal via `ex.printStackTrace()`:

1. Restart your Spring Boot application.
2. Submit the **Set Password** form again from Angular.
3. Look at your **Spring Boot IDE console (terminal)**. Scroll to the very top of the red error text to see the exact exception line number (e.g., `NullPointerException`, `DataIntegrityViolationException`, or `NoSuchElementException`).

```
User prompt: u1_0.email=?    fetch    first ? rows only 2026-08-10T14:35:33.448+02:00 INFO 26160
--- [smart-erp-core] [nio-8080-exec-5] o.a.c.c.C.[.[localhost].[/api/v1]    : Initializing Spring
DispatcherServlet 'dispatcherServlet' 2026-08-10T14:35:33.448+02:00 INFO 26160 --- [smart-erp-core]
```

[nio-8080-exec-5] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-08-10T14:35:33.448+02:00 INFO 26160 --- [smart-erp-core] [nio-8080-exec-5]
o.s.web.servlet.DispatcherServlet : Completed initialization in 0 ms
org.springframework.web.servlet.resource.NoResourceFoundException: No static resource
auth/set-password for request '/api/v1/auth/set-password'. at
org.springframework.web.servlet.resource.ResourceHttpRequestHandler.handleRequest(ResourceHttpReq
uestHandler.java:527) at
org.springframework.web.servlet.mvc.HttpRequestHandlerAdapter.handle(HttpRequestHandlerAdapter.java:
50) at org.springframework.web.servlet.DispatcherServlet.doDispatch(DispatcherServlet.java:963)
at org.springframework.web.servlet.DispatcherServlet.doService(DispatcherServlet.java:866) at
org.springframework.web.servlet.FrameworkServlet.processRequest(FrameworkServlet.java:1000) at
org.springframework.web.servlet.FrameworkServlet.doPost(FrameworkServlet.java:903) at
jakarta.servlet.http.HttpServlet.service(HttpServlet.java:649) at
org.springframework.web.servlet.FrameworkServlet.service(FrameworkServlet.java:874) at
jakarta.servlet.http.HttpServlet.service(HttpServlet.java:710) at
org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:128) at
org.apache.tomcat.websocket.server.WsFilter.doFilter(WsFilter.java:53) at
org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:107) at
org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:110) at
org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:107) at
org.springframework.web.filter.CompositeFilter\$VirtualFilterChain.doFilter(CompositeFilter.java:108) at
org.springframework.security.web.FilterChainProxy.lambda\$doFilterInternal\$2(FilterChainProxy.java:235)
at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:376)
at
org.springframework.security.web.access.intercept.AuthorizationFilter.doFilter(AuthorizationFilter.java:101)
at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385)
at
org.springframework.security.web.access.ExceptionTranslationFilter.doFilter(ExceptionTranslationFilter.java:
126) at
org.springframework.security.web.access.ExceptionTranslationFilter.doFilter(ExceptionTranslationFilter.java:
120) at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385)
at
org.springframework.security.web.session.SessionManagementFilter.doFilter(SessionManagementFilter.jav
a:132) at
org.springframework.security.web.session.SessionManagementFilter.doFilter(SessionManagementFilter.jav
a:86) at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385)
at
org.springframework.security.web.authentication.AnonymousAuthenticationFilter.doFilter(AnonymousAuthen
ticationFilter.java:100) at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385)
at
org.springframework.security.web.servletapi.SecurityContextHolderAwareRequestFilter.doFilter(SecurityCon
textHolderAwareRequestFilter.java:181) at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385)
at
org.springframework.security.web.savedrequest.RequestCacheAwareFilter.doFilter(RequestCacheAwareFilt
er.java:63) at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385)

at com.smarterp.shared.security.JwtAuthenticationFilter.doFilterInternal(JwtAuthenticationFilter.java:56)
at org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:116) at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385)
at org.springframework.security.web.authentication.logout.LogoutFilter.doFilter(LogoutFilter.java:110)
at org.springframework.security.web.authentication.logout.LogoutFilter.doFilter(LogoutFilter.java:96) at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385)
at org.springframework.web.filter.CorsFilter.doFilterInternal(CorsFilter.java:91) at
org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:116) at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385)
at
org.springframework.security.web.header.HeaderWriterFilter.doHeadersAfter(HeaderWriterFilter.java:90)
at org.springframework.security.web.header.HeaderWriterFilter.doFilterInternal(HeaderWriterFilter.java:75)
at org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:116)
at org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385)
at
org.springframework.security.web.context.SecurityContextHolderFilter.doFilter(SecurityContextHolderFilter.java:82) at
org.springframework.security.web.context.SecurityContextHolderFilter.doFilter(SecurityContextHolderFilter.java:69) at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385)
at
org.springframework.security.web.context.request.async.WebAsyncManagerIntegrationFilter.doFilterInternal
(WebAsyncManagerIntegrationFilter.java:62) at
org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:116) at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385)
at
org.springframework.security.web.session.DisableEncodeUrlFilter.doFilterInternal(DisableEncodeUrlFilter.java:42) at
org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:116) at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385)
at org.springframework.security.web.FilterChainProxy.doFilterInternal(FilterChainProxy.java:237) at
org.springframework.security.web.FilterChainProxy.doFilter(FilterChainProxy.java:195) at
org.springframework.web.filter.CompositeFilter\$VirtualFilterChain.doFilter(CompositeFilter.java:113) at
org.springframework.web.filter.ServletRequestPathFilter.doFilter(ServletRequestPathFilter.java:52) at
org.springframework.web.filter.CompositeFilter\$VirtualFilterChain.doFilter(CompositeFilter.java:113) at
org.springframework.web.filter.CompositeFilter.doFilter(CompositeFilter.java:74) at
org.springframework.security.config.annotation.web.configuration.WebSecurityConfiguration\$CompositeFilterChainProxy.doFilter(WebSecurityConfiguration.java:317) at
org.springframework.web.filter.DelegatingFilterProxy.invokeDelegate(DelegatingFilterProxy.java:355) at
org.springframework.web.filter.DelegatingFilterProxy.doFilter(DelegatingFilterProxy.java:272) at
org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:107) at
org.springframework.web.filter.RequestContextFilter.doFilterInternal(RequestContextFilter.java:100) at
org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:116) at
org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:107) at
org.springframework.web.filter.FormContentFilter.doFilterInternal(FormContentFilter.java:93) at
org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:116) at
org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:107) at
org.springframework.web.filter.CharacterEncodingFilter.doFilterInternal(CharacterEncodingFilter.java:199)
at org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:116) at
org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:107) at
org.apache.catalina.core.StandardWrapperValve.invoke(StandardWrapperValve.java:165) at
org.apache.catalina.core.StandardContextValve.invoke(StandardContextValve.java:77) at

org.apache.catalina.authenticator.AuthenticatorBase.invoke(AuthenticatorBase.java:492) at
org.apache.catalina.core.StandardHostValve.invoke(StandardHostValve.java:113) at
org.apache.catalina.valves.ErrorReportValve.invoke(ErrorReportValve.java:83) at
org.apache.catalina.core.StandardEngineValve.invoke(StandardEngineValve.java:72) at
org.apache.catalina.connector.CoyoteAdapter.service(CoyoteAdapter.java:341) at
org.apache.coyote.http11.Http11Processor.service(Http11Processor.java:397) at
org.apache.coyote.AbstractProcessorLight.process(AbstractProcessorLight.java:63) at
org.apache.coyote.AbstractProtocol\$ConnectionHandler.process(AbstractProtocol.java:1272) at
org.apache.tomcat.util.net.NioEndpoint\$SocketProcessor.doRun(NioEndpoint.java:1801) at
org.apache.tomcat.util.net.SocketProcessorBase.run(SocketProcessorBase.java:52) at
org.apache.tomcat.util.threads.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:946) at
org.apache.tomcat.util.threads.ThreadPoolExecutor\$Worker.run(ThreadPoolExecutor.java:480) at
org.apache.tomcat.util.threads.TaskThread\$WrappingRunnable.run(TaskThread.java:57) at
java.base/java.lang.Thread.run(Thread.java:1583)
org.springframework.web.servlet.resource.NoResourceFoundException: No static resource
auth/set-password for request '/api/v1/auth/set-password'. at
org.springframework.web.servlet.resource.ResourceHttpRequestHandler.handleRequest(ResourceHttpRequestHandler.java:527) at
org.springframework.web.servlet.mvc.HttpRequestHandlerAdapter.handle(HttpRequestHandlerAdapter.java:50) at org.springframework.web.servlet.DispatcherServlet.doDispatch(DispatcherServlet.java:963)
at org.springframework.web.servlet.DispatcherServlet.doService(DispatcherServlet.java:866) at
org.springframework.web.servlet.FrameworkServlet.processRequest(FrameworkServlet.java:1000) at
org.springframework.web.servlet.FrameworkServlet.doPost(FrameworkServlet.java:903) at
jakarta.servlet.http.HttpServlet.service(HttpServlet.java:649) at
org.springframework.web.servlet.FrameworkServlet.service(FrameworkServlet.java:874) at
jakarta.servlet.http.HttpServlet.service(HttpServlet.java:710) at
org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:128) at
org.apache.tomcat.websocket.server.WsFilter.doFilter(WsFilter.java:53) at
org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:107) at
org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:110) at
org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:107) at
org.springframework.web.filter.CompositeFilter\$VirtualFilterChain.doFilter(CompositeFilter.java:108) at
org.springframework.security.web.FilterChainProxy.lambda\$doFilterInternal\$2(FilterChainProxy.java:235)
at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:376)
at
org.springframework.security.web.access.intercept.AuthorizationFilter.doFilter(AuthorizationFilter.java:101)
at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385)
at
org.springframework.security.web.access.ExceptionTranslationFilter.doFilter(ExceptionTranslationFilter.java:126) at
org.springframework.security.web.access.ExceptionTranslationFilter.doFilter(ExceptionTranslationFilter.java:120) at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385)
at
org.springframework.security.web.session.SessionManagementFilter.doFilter(SessionManagementFilter.java:132) at
org.springframework.security.web.session.SessionManagementFilter.doFilter(SessionManagementFilter.java:86) at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385)
at

org.springframework.security.web.authentication.AnonymousAuthenticationFilter.doFilter(AnonymousAuthenticationFilter.java:100) at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385) at
org.springframework.security.web.servletapi.SecurityContextHolderAwareRequestFilter.doFilter(SecurityContextHolderAwareRequestFilter.java:181) at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385) at
org.springframework.security.web.savedrequest.RequestCacheAwareFilter.doFilter(RequestCacheAwareFilter.java:63) at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385) at com.smarterp.shared.security.JwtAuthenticationFilter.doFilterInternal(JwtAuthenticationFilter.java:56) at
org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:116) at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385) at org.springframework.security.web.authentication.logout.LogoutFilter.doFilter(LogoutFilter.java:110) at
org.springframework.security.web.authentication.logout.LogoutFilter.doFilter(LogoutFilter.java:96) at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385) at org.springframework.web.filter.CorsFilter.doFilterInternal(CorsFilter.java:91) at
org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:116) at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385) at
org.springframework.security.web.header.HeaderWriterFilter.doHeadersAfter(HeaderWriterFilter.java:90) at org.springframework.security.web.header.HeaderWriterFilter.doFilterInternal(HeaderWriterFilter.java:75) at
org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:116) at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385) at
org.springframework.security.web.context.SecurityContextHolderFilter.doFilter(SecurityContextHolderFilter.java:82) at
org.springframework.security.web.context.SecurityContextHolderFilter.doFilter(SecurityContextHolderFilter.java:69) at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385) at
org.springframework.security.web.context.request.async.WebAsyncManagerIntegrationFilter.doFilterInternal(WebAsyncManagerIntegrationFilter.java:62) at
org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:116) at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385) at
org.springframework.security.web.session.DisableEncodeUrlFilter.doFilterInternal(DisableEncodeUrlFilter.java:42) at
org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:116) at
org.springframework.security.web.FilterChainProxy\$VirtualFilterChain.doFilter(FilterChainProxy.java:385) at org.springframework.security.web.FilterChainProxy.doFilterInternal(FilterChainProxy.java:237) at
org.springframework.security.web.FilterChainProxy.doFilter(FilterChainProxy.java:195) at
org.springframework.web.filter.CompositeFilter\$VirtualFilterChain.doFilter(CompositeFilter.java:113) at
org.springframework.web.filter.ServletRequestPathFilter.doFilter(ServletRequestPathFilter.java:52) at
org.springframework.web.filter.CompositeFilter\$VirtualFilterChain.doFilter(CompositeFilter.java:113) at
org.springframework.web.filter.CompositeFilter.doFilter(CompositeFilter.java:74) at
org.springframework.security.config.annotation.web.configuration.WebSecurityConfiguration\$CompositeFilterChainProxy.doFilter(WebSecurityConfiguration.java:317) at
org.springframework.web.filter.DelegatingFilterProxy.invokeDelegate(DelegatingFilterProxy.java:355) at
org.springframework.web.filter.DelegatingFilterProxy.doFilter(DelegatingFilterProxy.java:272) at
org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:107) at

```

org.springframework.web.filter.RequestContextFilter.doFilterInternal(RequestContextFilter.java:100)      at
org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:116)      at
org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:107)      at
org.springframework.web.filter.FormContentFilter.doFilterInternal(FormContentFilter.java:93)      at
org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:116)      at
org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:107)      at
org.springframework.web.filter.CharacterEncodingFilter.doFilterInternal(CharacterEncodingFilter.java:199)
    at org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:116)      at
org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:107)      at
org.apache.catalina.core.StandardWrapperValve.invoke(StandardWrapperValve.java:165)      at
org.apache.catalina.core.StandardContextValve.invoke(StandardContextValve.java:77)      at
org.apache.catalina.authenticator.AuthenticatorBase.invoke(AuthenticatorBase.java:492)      at
org.apache.catalina.core.StandardHostValve.invoke(StandardHostValve.java:113)      at
org.apache.catalina.valves.ErrorReportValve.invoke(ErrorReportValve.java:83)      at
org.apache.catalina.core.StandardEngineValve.invoke(StandardEngineValve.java:72)      at
org.apache.catalina.connector.CoyoteAdapter.service(CoyoteAdapter.java:341)      at
org.apache.coyote.http11.Http11Processor.service(Http11Processor.java:397)      at
org.apache.coyote.AbstractProcessorLight.process(AbstractProcessorLight.java:63)      at
org.apache.coyote.AbstractProtocol$ConnectionHandler.process(AbstractProtocol.java:1272)      at
org.apache.tomcat.util.net.NioEndpoint$SocketProcessor.doRun(NioEndpoint.java:1801)      at
org.apache.tomcat.util.net.SocketProcessorBase.run(SocketProcessorBase.java:52)      at
org.apache.tomcat.util.threads.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:946)      at
org.apache.tomcat.util.threads.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:480)      at
org.apache.tomcat.util.threads.TaskThread$WrappingRunnable.run(TaskThread.java:57)      at
java.base/java.lang.Thread.run(Thread.java:1583)

```

Response: This error occurs because Spring Boot cannot find a `@RestController` method mapped to `POST /api/v1/auth/set-password`. When no controller matches the requested URL, Spring Boot assumes you are requesting a static file, fails to find it, and throws `NoResourceFoundException`.

Step 1: Check AuthController.java

Ensure your `AuthController.java` contains the `@PostMapping("/set-password")` endpoint and has the proper class annotations.

Open **AuthController.java** and verify it matches this structure:

```
package com.smarterp.security.controller;
```

```

import com.smarterp.security.dto.AuthResponse;
import com.smarterp.security.dto.LoginRequest;
import com.smarterp.security.dto.RegisterRequest;
import com.smarterp.security.dto.SetPasswordRequest;
import com.smarterp.security.service.AuthService;
import jakarta.validation.Valid;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

```

```
@RestController
```

```
@RequestMapping("/auth") // If application.properties has server.servlet.context-path=/api/v1
```

```
// OR use @RequestMapping("/api/v1/auth") if you don't have context-path set
```

```

public class AuthController {

    private final AuthService authService;

    public AuthController(AuthService authService) {
        this.authService = authService;
    }

    @PostMapping("/register")
    public ResponseEntity<AuthResponse> register(@Valid @RequestBody RegisterRequest request) {
        return ResponseEntity.ok(authService.register(request));
    }

    @PostMapping("/login")
    public ResponseEntity<AuthResponse> login(@Valid @RequestBody LoginRequest request) {
        return ResponseEntity.ok(authService.login(request));
    }

    // THIS IS THE MISSING ENDPOINT CAUSING NoResourceFoundException
    @PostMapping("/set-password")
    public ResponseEntity<String> setPassword(@RequestBody SetPasswordRequest request) {
        authService.setPasswordWithToken(request);
        return ResponseEntity.ok("Mot de passe configuré avec succès.");
    }
}

```

Step 2: Verify SecurityConfig.java Matching Rule

Make sure /auth/set-password is permitted without authentication in SecurityConfig.java:

```

.authorizeHttpRequests(auth -> auth
    .requestMatchers("/auth/login", "/auth/set-password", "/v3/api-docs/**", "/swagger-ui/**").permitAll()
    .anyRequest().authenticated()
)

```

Step 3: Rebuild and Test

1. Save AuthController.java.
2. Stop and restart your Spring Boot application so Spring registers the new URL mapping.
3. Submit the set-password form from Angular again. Spring Boot will now route the request directly to authService.setPasswordWithToken().

User prompt: `<!DOCTYPE html> <html xmlns:th="http://www.thymeleaf.org"> <head> <meta charset="UTF-8"> <style> body { font-family: 'Segoe UI', Arial, sans-serif; background-color: #0f172a; color: #f8fafc; margin: 0; padding: 20px; } .container { max-width: 600px; margin: 0 auto; background-color: #1e293b; border-radius: 12px; border: 1px solid #334155; padding: 32px; } .brand { color: #14b8a6; font-size: 26px; font-weight: bold; text-align: center; } .card { background-color: #0f172a; border-radius: 8px; border: 1px solid #334155; padding: 20px; margin: 24px 0; } .btn-container`

```
{ text-align: center; margin: 32px 0; } .btn { background-color: #0d9488; color: #ffffff !important;
text-decoration: none; padding: 14px 32px; border-radius: 8px; font-weight: bold; display: inline-block; }
</style> </head> <body> <div class="container"> <h1 class="brand">NEXUS ERP</h1>
<h2>Activez votre compte</h2> <p>Un administrateur vous a invité à rejoindre la plateforme Nexus
ERP.</p> <div class="card"> <p><strong>Identifiant :</strong> <span
th:text="{email}"></span></p> <p><strong>Rôle :</strong> <span th:text="{role}"></span></p>
<p><strong>Lien valide pendant :</strong> 24 heures</p> </div> <div class="btn-container">
<a th:href="{activationUrl}" class="btn">Créer mon mot de passe</a> </div> <p
style="font-size: 12px; color: #64748b;">Si le bouton ne fonctionne pas, copiez-collez ce lien dans votre
navigateur : <br/> <span th:text="{activationUrl}"></span></p> </div> </body> </html> make this
mor professional ad appealing
```

Response: Here is a modern, enterprise-grade redesign of your email template. It incorporates professional visual hierarchy, improved email-client compatibility, better accessibility, and a clean slate/teal UI design suitable for an ERP system.

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Activez votre compte Nexus ERP</title>
<style>
/* Base resets for email clients */
body, table, td, a { -webkit-text-size-adjust: 100%; -ms-text-size-adjust: 100%; }
table, td { mso-table-lspace: 0pt; mso-table-rspace: 0pt; }
img { -ms-interpolation-mode: bicubic; border: 0; height: auto; line-height: 100%; outline: none;
text-decoration: none; }
body { margin: 0 !important; padding: 0 !important; width: 100% !important; background-color: #0f172a;
font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, Helvetica, Arial, sans-serif; }

/* Mobile styling */
@media screen and (max-width: 600px) {
.email-container { width: 100% !important; padding: 12px !important; }
.content-box { padding: 24px 20px !important; }
.btn { width: 100% !important; text-align: center !important; box-sizing: border-box; }
}
</style>
</head>
<body style="background-color: #0f172a; color: #f8f9fa; margin: 0; padding: 32px 0;">

<!-- Main Wrapper Table for maximum email client compatibility -->
<table border="0" cellpadding="0" cellspacing="0" width="100%" role="presentation">
<tr>
<td align="center">

<table border="0" cellpadding="0" cellspacing="0" width="100%" class="email-container"
style="max-width: 580px; margin: 0 auto;">

<!-- Header / Logo -->
<tr>
<td align="center" style="padding-bottom: 28px;">
```

```

<table border="0" cellpadding="0" cellspacing="0" role="presentation">
  <tr>
    <td align="center" style="background-color: #1e293b; border: 1px solid #334155;
border-radius: 12px; padding: 10px 20px;">
      <span style="color: #14b8a6; font-size: 22px; font-weight: 800; letter-spacing:
1.5px; text-transform: uppercase;">NEXUS<span style="color: #f8fafc; font-weight:
400;">ERP</span></span>
    </td>
  </tr>
</table>
</td>
</tr>

<!-- Main Card -->
<tr>
  <td class="content-box" style="background-color: #1e293b; border: 1px solid #334155;
border-radius: 16px; padding: 40px; box-shadow: 0 10px 25px -5px rgba(0, 0, 0, 0.3);">

    <!-- Title & Subtitle -->
    <h1 style="color: #ffffff; font-size: 22px; font-weight: 700; margin: 0 0 12px 0; text-align:
left;">
      Bienvenue sur la plateforme ! 🙌
    </h1>
    <p style="color: #94a3b8; font-size: 15px; line-height: 1.6; margin: 0 0 28px 0;">
      Un administrateur vous a invité à rejoindre votre espace de travail <strong style="color:
#f8fafc;">Nexus ERP</strong>. Veuillez activer votre compte pour configurer vos accès.
    </p>

    <!-- Account Information Summary Box -->
    <table border="0" cellpadding="0" cellspacing="0" width="100%" role="presentation"
style="background-color: #0f172a; border: 1px solid #334155; border-radius: 10px; margin-bottom: 32px;">
      <tr>
        <td style="padding: 20px;">
          <table border="0" cellpadding="0" cellspacing="0" width="100%"
role="presentation">
            <tr>
              <td style="padding-bottom: 10px; font-size: 14px; color: #94a3b8;"
width="110">Identifiant</td>
              <td style="padding-bottom: 10px; font-size: 14px; color: #f8fafc; font-weight:
600;" th:text="{email}">user@company.com</td>
            </tr>
            <tr>
              <td style="padding-bottom: 10px; font-size: 14px; color: #94a3b8;">Rôle
attribué</td>
              <td style="padding-bottom: 10px; font-size: 14px; color: #14b8a6; font-weight:
600;" th:text="{role}">ADMINISTRATEUR</td>
            </tr>
            <tr>
              <td style="font-size: 14px; color: #94a3b8;">Validité</td>
              <td style="font-size: 14px; color: #f8fafc; font-weight: 500;">24 heures</td>
            </tr>
          </table>
        </td>
      </tr>
    </table>

```

```

        </td>
    </tr>
</table>

<!-- Call To Action Button -->
<table border="0" cellpadding="0" cellspacing="0" width="100%" role="presentation">
    <tr>
        <td align="center" style="padding-bottom: 32px;">
            <a th:href="{activationUrl}" class="btn" style="background-color: #0d9488; color:
#ffffff !important; display: inline-block; font-size: 15px; font-weight: 600; text-decoration: none; padding: 14px
36px; border-radius: 10px; box-shadow: 0 4px 12px rgba(13, 148, 136, 0.3);">
                Activer mon compte →
            </a>
        </td>
    </tr>
</table>

<!-- Alternative Plain Link Section -->
<div style="border-top: 1px solid #334155; padding-top: 24px;">
    <p style="color: #64748b; font-size: 12px; line-height: 1.5; margin: 0 0 8px 0;">
        Si le bouton ne fonctionne pas, vous pouvez copier et coller ce lien direct dans votre
navigateur :
    </p>
    <p style="margin: 0; word-break: break-all;">
        <a th:href="{activationUrl}" th:text="{activationUrl}" style="color: #14b8a6; font-size:
12px; text-decoration: underline;">http://localhost:4200/auth/set-password?token=...</a>
    </p>
</div>

</td>
</tr>

<!-- Footer -->
<tr>
    <td style="padding-top: 28px; text-align: center;">
        <p style="color: #64748b; font-size: 12px; margin: 0 0 8px 0;">
            Ce lien d'activation est à usage unique et expirera dans 24h.
        </p>
        <p style="color: #475569; font-size: 12px; margin: 0;">
            © 2026 Nexus ERP. Tous droits réservés.
        </p>
    </td>
</tr>

</table>

</td>
</tr>
</table>

</body>
</html>

```

Key Enhancements Added:

1. **Email Client Resets:** Added explicit MSO table borders, high-DPI scaling rules, and web-safe font stacks to render cleanly across Gmail, Outlook, Apple Mail, and Yahoo.
 2. **Responsive Layout:** Included @media queries and fluid max-width containers (580px) so it automatically resizes smoothly on mobile screens.
 3. **Enterprise Visual Hierarchy:**
 - Clean top brand badge (NEXUS ERP).
 - Structured key-value details table replacing standard bullet points.
 - High-contrast, tactile call-to-action button with fallback text.
 - Distinct, subtle footer containing a security note and copyright metadata.
-

User prompt: befor we continue i want to download my data pool , give me data that is compatable with my project that i can insert into my db

Response: Here is a ready-to-use dataset matching your **Nexus ERP** schema (roles and users tables), including auditing fields and sample account states.

Core SQL Insertion Script (data.sql)

All generated user accounts use the default test password: **Password123!** (pre-hashed with BCrypt).

-- 1. SEED ROLES

INSERT INTO roles (id, name, description) VALUES

(1, 'SUPER_ADMIN', 'Accès complet à la plateforme et à la gestion système'),

(2, 'ADMIN', 'Gestion des utilisateurs, rôles et paramètres d"entreprise'),

(3, 'MANAGER', 'Supervision des modules opérationnels et rapports'),

(4, 'USER', 'Accès standard aux fonctionnalités métier')

ON CONFLICT (id) DO NOTHING;

-- Reset identity sequence for PostgreSQL if using AUTO INCREMENT

SELECT setval(pg_get_serial_sequence('roles', 'id'), (SELECT MAX(id) FROM roles));

-- 2. SEED USERS (Default Password for all: Password123!)

INSERT INTO users (id, email, password_hash, is_active, role_id, created_at, updated_at, created_by, updated_by) VALUES

(1, 'superadmin@nexuserp.com',

'\$2a\$10\$8.UnVuG9HHgffUDAlk8qfOuVGkqRzgVymGe07xd0P1R6sKz19IZ2O', true, 1,

CURRENT_TIMESTAMP, CURRENT_TIMESTAMP, 'SYSTEM', 'SYSTEM'),

(2, 'admin@nexuserp.com', '\$2a\$10\$8.UnVuG9HHgffUDAlk8qfOuVGkqRzgVymGe07xd0P1R6sKz19IZ2O', true, 2, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP, 'SYSTEM', 'SYSTEM'),

(3, 'manager@nexuserp.com',

'\$2a\$10\$8.UnVuG9HHgffUDAlk8qfOuVGkqRzgVymGe07xd0P1R6sKz19IZ2O', true, 3,

CURRENT_TIMESTAMP, CURRENT_TIMESTAMP, 'admin@nexuserp.com', 'admin@nexuserp.com'),

(4, 'employee.one@nexuserp.com',

'\$2a\$10\$8.UnVuG9HHgffUDAlk8qfOuVGkqRzgVymGe07xd0P1R6sKz19IZ2O', true, 4,

CURRENT_TIMESTAMP, CURRENT_TIMESTAMP, 'admin@nexuserp.com', 'admin@nexuserp.com'),

(5, 'inactive.user@nexuserp.com',

```
'$2a$10$8.UnVuG9HHgffUDAlk8qfOuVGkqRzgVymGe07xd0P1R6sKz19IZ2O', false, 4,  
CURRENT_TIMESTAMP, CURRENT_TIMESTAMP, 'admin@nexuserp.com', 'admin@nexuserp.com')  
ON CONFLICT (id) DO NOTHING;
```

```
-- Reset identity sequence for PostgreSQL  
SELECT setval(pg_get_serial_sequence('users', 'id'), (SELECT MAX(id) FROM users));
```

How to Apply This Data

- **Option A (Automatic execution):** Save this code as `data.sql` inside `src/main/resources/` and add this line to your `application.properties`:
`spring.jpa.defer-datasource-initialization=true`
`spring.sql.init.mode=always`
- **Option B (Manual execution):** Copy and run the script directly in **pgAdmin**, **DBeaver**, or your PostgreSQL terminal (`psql -d smart_erp_db -f data.sql`).

Do you have other ERP domain entities (such as clients, products, orders, or invoices) that you would like mock data seeded for as well?

User prompt: isnt there a doawmloadable data from the net i can work with

Response: Yes, several standard open-source ERP datasets are available for PostgreSQL that you can download and restore into your database in minutes.

1. Northwind Database (PostgreSQL Port)

- **Domain:** Lightweight ERP / E-commerce (Customers, Orders, Products, Suppliers, Shippers, Employees).
- **Best for:** Fast setup, testing basic CRUD endpoints, and building simple dashboards.
- **How to install:**
 1. Download the single SQL file from GitHub:
`git clone https://github.com/pthom/northwind_psql.git`
`cd northwind_psql`
 2. Import it into PostgreSQL:
`psql -U postgres -d smart_erp_db -f northwind.sql`

2. AdventureWorks (PostgreSQL Port)

- **Domain:** Enterprise-grade manufacturing and retail ERP (68 tables across Human Resources, Sales, Purchasing, Production, and Inventory).
- **Best for:** Complex relationships, multi-role security testing, and heavy reporting queries.
- **Repository:** Download from GitHub at [lorint/AdventureWorks-for-Postgres](https://github.com/lorint/AdventureWorks-for-Postgres).

3. Mockaroo (Generate Data for Your Exact Schema) If you want data that matches your project's specific

JPA entities (users, roles, etc.) without altering your table structures:

- Go to mockaroo.com.
- Add your table fields (e.g., email, password_hash, is_active, role_id).
- Select **SQL** as the export format and download an INSERT script tailored to your schema.

Would you like help generating a Mockaroo template or mapping one of the Northwind PostgreSQL tables to your Spring Boot JPA entities?

Sources

1. [harryho/db-samples: Northwind sample database for MySql, PostgresQL etc. - GitHub](#)
-

User prompt: Yes. I looked specifically for datasets that fit your actual project, not just generic HR datasets. Your PDF defines the project around an HR ERP with: employee/departement/leave/attendance data, BI dashboards and KPIs, AI recruitment/CV matching, performance analysis, workforce/capacity analysis, and natural-language querying. My recommendation: don't use only one dataset There isn't one public dataset that perfectly covers ERP + BI + recruitment + CVs + attendance + performance. The best approach is to use one main HR dataset and, if you implement the recruitment AI, add a second specialized CV dataset. 1. 🏆 Best main dataset: Employee/HR Dataset (All in One) This is probably the closest match to your project. It is explicitly designed around: Employee records Training Surveys Performance Recruitment Attendance The dataset is synthetic, so you don't have the privacy problems associated with real employee information. About Dataset Employee Data The Synthetic Employee Records Dataset is a simulated dataset created to explore various data analysis and machine learning techniques in the context of human resources and employee management. This synthetic dataset mirrors the structure and characteristics of real employee data, while all the information contained within is entirely fictional and generated for illustrative purposes. Descriptions for each of the columns in the dataset: Employee ID: Unique identifier for each employee in the organization. First Name: The first name of the employee. Last Name: The last name of the employee. Start Date: The date when the employee started working for the organization. Exit Date: The date when the employee left or exited the organization (if applicable). Title: The job title or position of the employee within the organization. Supervisor: The name of the employee's immediate supervisor or manager. Email: The email address associated with the employee's communication within the organization. Business Unit: The specific business unit or department to which the employee belongs. Employee Status: The current employment status of the employee (e.g., Active, On Leave, Terminated). Employee Type: The type of employment the employee has (e.g., Full-time, Part-time, Contract). Pay Zone: The pay zone or salary band to which the employee's compensation falls. Employee Classification Type: The classification type of the employee (e.g., Exempt, Non-exempt). Termination Type: The type of termination if the employee has left the organization (e.g., Resignation, Layoff, Retirement). Termination Description: Additional details or reasons for the employee's termination (if applicable). Department Type: The broader category or type of department the employee's work is associated with. Division Description: The division or branch of the organization where the employee works. DOB (Date of Birth): The date of birth of the employee. State: The state or region where the employee is located. Job Function: A brief description of the employee's primary job function or role. Gender: A code representing the gender of the employee (e.g., M for Male, F for Female, N for Non-binary). Location: A code representing the physical location or office where the employee is based. Race (or) Ethnicity: A description of the employee's racial or ethnic background (if provided). Marital Status: The marital status of the employee (e.g., Single, Married, Divorced). Performance Score: A score indicating the employee's performance level (e.g., Excellent, Satisfactory, Needs Improvement). Current Employee Rating: The current rating or evaluation of the employee's overall performance. Training/Development Data The Training and Development Records Dataset provides a comprehensive overview of training activities and developmental programs within an organization. This dataset is designed

to capture the details of employee participation in various training initiatives, helping to assess the impact of professional development efforts and guide strategic decision-making. Descriptions for each of the columns in the dataset: Employee ID: A unique identifier for each employee who participated in the training program. Training Date: The date on which the training session took place. Training Program Name: The name or title of the training program attended by the employee. Training Type: The categorization of the training, indicating its purpose or focus (e.g., Technical, Soft Skills, Safety). Training Outcome: The observed outcome or result of the training for the employee (e.g., Completed, Partial Completion, Not Completed). Location: The physical or virtual location where the training session was conducted. Trainer: The name of the trainer or instructor who facilitated the training. Training Duration (Days): The duration of the training program in days. Training Cost: The cost associated with organizing and conducting the training program.

Recruitment Data The Recruitment Activity Records Dataset offers a comprehensive view of the recruitment process within an organization, encompassing key data points from initial job posting to candidate selection. This dataset aims to facilitate insights into the efficiency of recruitment efforts, candidate profiles, and the effectiveness of sourcing channels. Descriptions for each of the columns in the dataset: Applicant ID: A unique identifier assigned to each applicant who has submitted their information for a job opportunity. Application Date: The date on which the applicant submitted their application for the job. First Name: The first name of the applicant. Last Name: The last name of the applicant. Gender: The gender of the applicant. Date of Birth: The birthdate of the applicant. Phone Number: The contact phone number of the applicant. Email: The email address of the applicant for communication purposes. Address: The street address where the applicant resides. City: The city where the applicant's address is located. State: The state or province where the applicant's address is situated. Zip Code: The postal or ZIP code associated with the applicant's address. Country: The country where the applicant's address is located. Education Level: The highest level of education attained by the applicant. Years of Experience: The number of years of professional experience the applicant has. Desired Salary: The salary the applicant wishes to receive for the job. Job Title: The title or designation of the job the applicant is applying for. Status: The status of the applicant's application (e.g., Submitted, Under Review, Rejected, Selected).

Employee Engagement Survey Data The Employee Engagement Survey Dataset presents a comprehensive collection of responses obtained through an organization-wide employee engagement survey. This dataset is intended to analyze the levels of employee engagement, satisfaction, and sentiment across various facets of the workplace, facilitating insights into workforce dynamics and guiding strategies for improvement. Descriptions for each of the columns in the dataset: Employee ID: A unique identifier assigned to each employee who participated in the employee engagement survey. Survey Date: The date on which the engagement survey was administered to employees. Engagement Score: A calculated numerical score representing the level of employee engagement based on survey responses. Satisfaction Score: A numerical score indicating employee satisfaction with various aspects of their job and workplace. Work-Life Balance Score: A numerical score reflecting employee perceptions of the balance between work and personal life. this is the data from kaggle . i can use it to fill my db but first the data need cleaning, also its messing some tables like departemnt ans else , also its in .csv i need t find a way to download it into my post gress then modifie it so that it compatable with my code , also there are colomn that are unnecessesary sometimes , also some relation inside the data is not logic like if there is an exist date but the emmployee is still active, my objective now is to clean this data then doawnload it in the postgres db in a way compatable with my project so i can start working on the project

Response: This is an excellent choice for your project. The challenges you identified—flat CSV files missing relational tables (like departments), logical inconsistencies, and unneeded columns—are exactly what happens in real-world data engineering.

To bridge the gap between this raw Kaggle data and your Spring Boot PostgreSQL database, we need to perform an **ETL (Extract, Transform, Load)** process. The safest and most flexible way to do this is using a short Python script (with the pandas library) to clean the data *before* it ever touches your database.

Here is the exact workflow to clean this dataset, fix the logic, extract the missing tables, and load it into PostgreSQL.

1 Set up your Python environment

Prerequisites

You will need Python installed. Open your terminal in the folder where you downloaded the Kaggle CSVs and run:

```
pip install pandas numpy
```

2 Clean the data and fix logic

Data Transformation

Create a file named `clean_data.py`. This script handles the exact issues you mentioned: dropping unnecessary columns, fixing the Active/Exit Date mismatch, and automatically generating the missing departments table.

```
import pandas as pd
import numpy as np
```

```
# 1. Load the raw dataset
df = pd.read_csv('Employee_Data.csv')
```

```
# 2. Drop unnecessary columns (customize this list based on your needs)
columns_to_drop = ['Race (or) Ethnicity', 'Marital Status', 'Pay Zone']
df = df.drop(columns=[col for col in columns_to_drop if col in df.columns])
```

```
# 3. Fix Logical Inconsistencies
# Rule A: If an employee is 'Active', they cannot have an Exit Date
df.loc[df['Employee Status'] == 'Active', 'Exit Date'] = np.nan
```

```
# Rule B: If there is an Exit Date, the status must be 'Terminated'
df.loc[df['Exit Date'].notna(), 'Employee Status'] = 'Terminated'
```

```
# Rule C: Ensure Start Date and Exit Date are actual datetime objects
df['Start Date'] = pd.to_datetime(df['Start Date']).dt.strftime('%Y-%m-%d')
df['Exit Date'] = pd.to_datetime(df['Exit Date']).dt.strftime('%Y-%m-%d')
```

```
# 4. Extract Relational Tables (Normalization)
# Create a unique Departments table from the flat data
departments = df[['Business Unit', 'Department Type']].drop_duplicates().reset_index(drop=True)
departments.index += 1 # Generate IDs starting at 1
```

```

departments.rename_axis('department_id', inplace=True)
departments = departments.reset_index()

# Map the new department_id back to the main employee dataframe
df = df.merge(departments, on=['Business Unit', 'Department Type'], how='left')

# Drop the redundant text columns now that we have a foreign key (department_id)
df = df.drop(columns=['Business Unit', 'Department Type'])

# 5. Export the clean data to new CSVs for PostgreSQL
departments.to_csv('cleaned_departments.csv', index=False)
df.to_csv('cleaned_employees.csv', index=False)

print("Data cleaned and separated successfully!")

```

Run this script: `python clean_data.py`. You will get two perfect relational CSVs.

3 Create the tables in PostgreSQL

Before importing, your database needs to know the structure. Open pgAdmin (or your SQL tool) and run:

```

CREATE TABLE departments (
    id SERIAL PRIMARY KEY,
    business_unit VARCHAR(255),
    department_type VARCHAR(255)
);

CREATE TABLE employees (
    id VARCHAR(50) PRIMARY KEY, -- Using the Kaggle Employee ID
    first_name VARCHAR(100),
    last_name VARCHAR(100),
    email VARCHAR(255) UNIQUE,
    start_date DATE,
    exit_date DATE,
    title VARCHAR(150),
    supervisor VARCHAR(150),
    employee_status VARCHAR(50),
    employee_type VARCHAR(50),
    employee_classification_type VARCHAR(50),
    termination_type VARCHAR(100),
    termination_description TEXT,
    division_description VARCHAR(255),
    dob DATE,
    state VARCHAR(50),
    job_function VARCHAR(255),
    gender VARCHAR(20),

```

```
location VARCHAR(50),
performance_score VARCHAR(50),
current_employee_rating INT,
department_id INT REFERENCES departments(id)
);
```

4 Import the clean CSVs into PostgreSQL

Using pgAdmin or PSQL

Using pgAdmin:

1. Right-click the departments table > **Import/Export Data**.
2. Set slider to **Import**. Select cleaned_departments.csv.
3. Go to the **Columns** tab, ensure header is enabled, and click OK.
4. Repeat for the employees table using cleaned_employees.csv.

Alternative (Using SQL COPY command):

```
COPY departments(id, business_unit, department_type)
FROM '/path/to/cleaned_departments.csv' DELIMITER ',' CSV HEADER;
```

-- Note: Ensure column order in COPY matches your CSV exactly

```
COPY employees(id, first_name, last_name, start_date, exit_date, title, supervisor, email,
employee_status, employee_type, employee_classification_type, termination_type,
termination_description, division_description, dob, state, job_function, gender, location,
performance_score, current_employee_rating, department_id)
FROM '/path/to/cleaned_employees.csv' DELIMITER ',' CSV HEADER;
```

Once this is done, you will have a perfectly structured relational database. You can repeat this exact logic using Pandas for the Training, Recruitment, and Survey datasets to link them all together using the Employee ID.

User prompt: first thing first i want to foccuss on the etl for all my .csv file prepare all the missing tables and all that i need later in my project clean the data fix any error prepare everything then loand everything into my db , i want this step to be compleated so focuss on everything tables devision data cleaning evrything , so in the load stage vrything will be loanded and i can start working on my project main goal

Response: To build a robust ERP, getting the database perfectly clean and normalized right now is the best decision you can make. It prevents endless debugging later when you're writing your Spring Boot APIs.

We are going to build a complete **ETL pipeline**. We will normalize the data (extracting a departments table), fix the logic gaps (active users with exit dates), format all dates for PostgreSQL (YYYY-MM-DD), and link everything together.

Here is your master plan.

1 Run the Master ETL Python Script

Extract, Clean, and Transform

Place all four Kaggle CSV files in the same folder. Create a file named `etl_pipeline.py` in that folder and run it. This script handles all the heavy lifting: cleaning logic, creating the relational departments table, and formatting everything for PostgreSQL.

```
import pandas as pd
import numpy as np

# --- Configuration: Update these if your downloaded files have different names ---
FILE_EMPLOYEE = 'Employee_Data.csv'
FILE_TRAINING = 'Training_and_Development_Data.csv'
FILE_RECRUIT = 'Recruitment_Data.csv'
FILE_SURVEY = 'Employee_Engagement_Survey_Data.csv'

def clean_dates(df, date_columns):
    """Converts dates to standard PostgreSQL YYYY-MM-DD format."""
    for col in date_columns:
        if col in df.columns:
            df[col] = pd.to_datetime(df[col], errors='coerce').dt.strftime('%Y-%m-%d')
    return df

print("1. Processing Employee Data...")
df_emp = pd.read_csv(FILE_EMPLOYEE)

# Drop non-essential / privacy columns
cols_to_drop = ['Race (or) Ethnicity', 'Marital Status', 'Pay Zone']
df_emp = df_emp.drop(columns=[c for c in cols_to_drop if c in df_emp.columns])

# Fix Active / Exit Date logic
df_emp.loc[df_emp['Employee Status'] == 'Active', 'Exit Date'] = np.nan
df_emp.loc[df_emp['Exit Date'].notna(), 'Employee Status'] = 'Terminated'

# Clean Dates
df_emp = clean_dates(df_emp, ['Start Date', 'Exit Date', 'DOB'])

# Normalize: Create Departments Table
dept_cols = ['Business Unit', 'Department Type', 'Division Description']
departments = df_emp[dept_cols].drop_duplicates().reset_index(drop=True)
departments.index += 1
departments.rename_axis('department_id', inplace=True)
departments = departments.reset_index()

# Map department_id back to employees and drop old text columns
```

```

df_emp = df_emp.merge(departments, on=dept_cols, how='left')
df_emp = df_emp.drop(columns=dept_cols)

print("2. Processing Training Data...")
df_train = pd.read_csv(FILE_TRAINING)
df_train = clean_dates(df_train, ['Training Date'])
# Fill missing costs/durations with 0 to prevent DB insertion errors
df_train['Training Cost'] = df_train['Training Cost'].fillna(0)
df_train['Training Duration (Days)'] = df_train['Training Duration (Days)'].fillna(0)

print("3. Processing Recruitment Data...")
df_recruit = pd.read_csv(FILE_RECRUIT)
df_recruit = clean_dates(df_recruit, ['Application Date', 'Date of Birth'])
# Standardize status to uppercase
df_recruit['Status'] = df_recruit['Status'].str.upper()

print("4. Processing Survey Data...")
df_survey = pd.read_csv(FILE_SURVEY)
df_survey = clean_dates(df_survey, ['Survey Date'])

# --- Export Cleaned CSVs ---
print("Exporting Cleaned CSVs for PostgreSQL...")
departments.to_csv('pg_departments.csv', index=False)
df_emp.to_csv('pg_employees.csv', index=False)
df_train.to_csv('pg_trainings.csv', index=False)
df_recruit.to_csv('pg_recruitment.csv', index=False)
df_survey.to_csv('pg_surveys.csv', index=False)

print("ETL Transformation Complete! ✅")

```

2 Create the PostgreSQL Schema

Run in pgAdmin / DBeaver

Now that the data structure is finalized, create the tables in your database. Open your SQL client and execute this script. Notice how employees, trainings, and surveys are relationally linked.

```

-- 1. Create Lookup Table
CREATE TABLE departments (
    id SERIAL PRIMARY KEY,
    business_unit VARCHAR(255),
    department_type VARCHAR(255),
    division_description VARCHAR(255)
);

-- 2. Create Main Employee Table
CREATE TABLE employees (
    employee_id VARCHAR(50) PRIMARY KEY,
    first_name VARCHAR(100),
    last_name VARCHAR(100),

```

```

start_date DATE,
exit_date DATE,
title VARCHAR(150),
supervisor VARCHAR(150),
email VARCHAR(255),
employee_status VARCHAR(50),
employee_type VARCHAR(50),
employee_classification_type VARCHAR(50),
termination_type VARCHAR(100),
termination_description TEXT,
dob DATE,
state VARCHAR(50),
job_function VARCHAR(255),
gender VARCHAR(20),
location VARCHAR(50),
performance_score VARCHAR(50),
current_employee_rating INT,
department_id INT REFERENCES departments(id)
);

-- 3. Create Training Table
CREATE TABLE trainings (
    id SERIAL PRIMARY KEY,
    employee_id VARCHAR(50) REFERENCES employees(employee_id),
    training_date DATE,
    training_program_name VARCHAR(255),
    training_type VARCHAR(100),
    training_outcome VARCHAR(100),
    location VARCHAR(150),
    trainer VARCHAR(150),
    training_duration_days NUMERIC,
    training_cost NUMERIC
);

-- 4. Create Survey Table
CREATE TABLE engagement_surveys (
    id SERIAL PRIMARY KEY,
    employee_id VARCHAR(50) REFERENCES employees(employee_id),
    survey_date DATE,
    engagement_score NUMERIC,
    satisfaction_score NUMERIC,
    work_life_balance_score NUMERIC
);

-- 5. Create Recruitment Table (Standalone until hired)
CREATE TABLE recruitment_applicants (
    applicant_id VARCHAR(50) PRIMARY KEY,
    application_date DATE,
    first_name VARCHAR(100),
    last_name VARCHAR(100),
    gender VARCHAR(20),
    dob DATE,

```

```

phone_number VARCHAR(50),
email VARCHAR(255),
address TEXT,
city VARCHAR(100),
state VARCHAR(100),
zip_code VARCHAR(50),
country VARCHAR(100),
education_level VARCHAR(100),
years_of_experience NUMERIC,
desired_salary NUMERIC,
job_title VARCHAR(150),
status VARCHAR(50)
);

```

3 Load the Data into PostgreSQL

Finally, load the 5 newly generated pg_*.csv files into your database. Order is important here due to Foreign Key constraints.

Option A: Using pgAdmin interface Right-click each table -> Import/Export Data -> Select Import -> Toggle "Header" to ON -> Select the corresponding CSV. *Do them in this exact order: departments, then employees, then the rest.*

Option B: Using SQL COPY (Requires absolute file paths)

```

COPY departments(id, business_unit, department_type, division_description)
FROM '/absolute/path/to/pg_departments.csv' DELIMITER ',' CSV HEADER;

```

```

COPY employees(employee_id, first_name, last_name, start_date, exit_date, title, supervisor, email,
employee_status, employee_type, employee_classification_type, termination_type,
termination_description, dob, state, job_function, gender, location, performance_score,
current_employee_rating, department_id)
FROM '/absolute/path/to/pg_employees.csv' DELIMITER ',' CSV HEADER;

```

```

COPY trainings(employee_id, training_date, training_program_name, training_type, training_outcome,
location, trainer, training_duration_days, training_cost)
FROM '/absolute/path/to/pg_trainings.csv' DELIMITER ',' CSV HEADER;

```

```

COPY engagement_surveys(employee_id, survey_date, engagement_score, satisfaction_score,
work_life_balance_score)
FROM '/absolute/path/to/pg_surveys.csv' DELIMITER ',' CSV HEADER;

```

```

COPY recruitment_applicants(applicant_id, application_date, first_name, last_name, gender, dob,
phone_number, email, address, city, state, zip_code, country, education_level, years_of_experience,
desired_salary, job_title, status)
FROM '/absolute/path/to/pg_recruitment.csv' DELIMITER ',' CSV HEADER;

```

User prompt: isnt there smthng messing from the table like departement cv per recutes, even if it nit in the .csv if its needed in the project used it also dont forget that i have more 0that just employee i have managers , and i have user table that employee and admin and manager late use , focust on the traitement part to prepare evrything like creating users coerent with same number of employe also departemnet that exist in employe create them in there own table , iawant pipline that take this data and make it usable with and comfrotable for my project . even empl ai if neccessary

Response: You are absolutely right. A real Spring Boot ERP is not just a collection of flat HR records. It requires an **Authentication Identity (Users/Roles)**, a clear **Corporate Hierarchy (Managers vs. Employees)**, and the necessary infrastructure for your **AI Recruitment system (CV texts, skills, AI match scores)**.

Since the Kaggle CSVs don't have these, we must **synthesize them during the ETL process**. We will write an advanced Python script that reads the flat CSVs, cleans the data, and artificially generates the Users, Roles, and CV records required to run your complete project.

1Run the Advanced ETL Script

Data Generation & Cleaning

Save this as `advanced_etl.py` in the same folder as your CSVs and run it. This script creates the Users table, assigns Roles based on job titles, extracts Departments, and generates synthetic CV text for the recruitment AI.

```
import pandas as pd
import numpy as np
import uuid
import json

# --- 1. Load Data ---
df_emp = pd.read_csv('Employee_Data.csv')
df_recruit = pd.read_csv('Recruitment_Data.csv')
df_train = pd.read_csv('Training_and_Development_Data.csv')
df_survey = pd.read_csv('Employee_Engagement_Survey_Data.csv')

def clean_dates(df, date_columns):
    for col in date_columns:
        if col in df.columns:
            df[col] = pd.to_datetime(df[col], errors='coerce').dt.strftime('%Y-%m-%d')
    return df

# --- 2. Clean Employee Data & Fix Logic ---
df_emp.loc[df_emp['Employee Status'] == 'Active', 'Exit Date'] = np.nan
df_emp.loc[df_emp['Exit Date'].notna(), 'Employee Status'] = 'Terminated'
df_emp = clean_dates(df_emp, ['Start Date', 'Exit Date', 'DOB'])
```

```

# --- 3. Generate Roles Table ---
roles_data = [
    {'id': 1, 'name': 'SUPER_ADMIN', 'description': 'Full System Access'},
    {'id': 2, 'name': 'ADMIN', 'description': 'HR Admin'},
    {'id': 3, 'name': 'MANAGER', 'description': 'Department Manager'},
    {'id': 4, 'name': 'USER', 'description': 'Standard Employee'}
]
roles_df = pd.DataFrame(roles_data)

# --- 4. Generate Users Table & Map Roles ---
users_data = []
# Using the standard Spring Boot BCrypt hash for 'Password123!'
default_hash = '$2a$10$8.UnVuG9HHgffUDAlk8qfOuVGkqRzgVymGe07xd0P1R6sKz19lZ2O'

for index, row in df_emp.iterrows():
    title = str(row['Title']).upper()
    # Determine Role based on Job Title
    if 'CEO' in title or 'PRESIDENT' in title:
        role_id = 1 # SUPER_ADMIN
    elif 'DIRECTOR' in title or 'VP' in title or 'HR' in title:
        role_id = 2 # ADMIN
    elif 'MANAGER' in title or 'SUPERVISOR' in title or 'LEAD' in title:
        role_id = 3 # MANAGER
    else:
        role_id = 4 # USER

    user_id = str(uuid.uuid4()) # Generate a UUID for the User
    df_emp.at[index, 'user_id'] = user_id

    users_data.append({
        'id': user_id,
        'email': row['Email'],
        'password_hash': default_hash,
        'is_active': True if row['Employee Status'] == 'Active' else False,
        'role_id': role_id
    })

users_df = pd.DataFrame(users_data)

# --- 5. Extract Departments ---
dept_cols = ['Business Unit', 'Department Type', 'Division Description']
departments = df_emp[dept_cols].drop_duplicates().reset_index(drop=True)
departments.index += 1
departments.rename_axis('id', inplace=True)
departments = departments.reset_index()

df_emp = df_emp.merge(departments, on=dept_cols, how='left')
df_emp = df_emp.drop(columns=dept_cols)
df_emp = df_emp.rename(columns={'id': 'department_id'})

# --- 6. Generate AI-Ready CVs for Recruitment ---
df_recruit = clean_dates(df_recruit, ['Application Date', 'Date of Birth'])

```

```

cv_data = []

for index, row in df_recruit.iterrows():
    # Generate synthetic CV metadata for your AI matching system
    skills = [row['Job Title'].split()[0], 'Communication', 'Problem Solving']
    cv_text = f"{row['First Name']} is a professional with {row['Years of Experience']} years of experience applying for {row['Job Title']}."

    cv_data.append({
        'id': str(uuid.uuid4()),
        'applicant_id': row['Applicant ID'],
        'file_url': f"/storage/cvs/{row['Applicant ID']}_cv.pdf",
        'parsed_text': cv_text,
        'extracted_skills_json': json.dumps(skills),
        'ai_match_score': np.random.randint(40, 99) # Mock AI score
    })

cvs_df = pd.DataFrame(cv_data)

# --- 7. Clean remaining tables ---
df_train = clean_dates(df_train, ['Training Date'])
df_survey = clean_dates(df_survey, ['Survey Date'])

# --- 8. Export All to CSV ---
roles_df.to_csv('pg_roles.csv', index=False)
users_df.to_csv('pg_users.csv', index=False)
departments.to_csv('pg_departments.csv', index=False)
df_emp.to_csv('pg_employees.csv', index=False)
df_train.to_csv('pg_trainings.csv', index=False)
df_survey.to_csv('pg_surveys.csv', index=False)
df_recruit.to_csv('pg_recruitment.csv', index=False)
cvs_df.to_csv('pg_cvs.csv', index=False)

print("Data engineered! Users, Roles, and CVs successfully generated. ✅")

```

2 Create the Complete PostgreSQL Schema

Enterprise Architecture

Execute this SQL in pgAdmin. This schema is now structurally identical to what Spring Boot JPA expects for an ERP.

```

-- 1. Security & Auth Tables
CREATE TABLE roles (
    id INT PRIMARY KEY,
    name VARCHAR(50) UNIQUE,
    description VARCHAR(255)
);

```

```

CREATE TABLE users (

```

```
id UUID PRIMARY KEY,  
email VARCHAR(255) UNIQUE,  
password_hash VARCHAR(255),  
is_active BOOLEAN,  
role_id INT REFERENCES roles(id)  
);
```

-- 2. Organization Tables

```
CREATE TABLE departments (  
id SERIAL PRIMARY KEY,  
business_unit VARCHAR(255),  
department_type VARCHAR(255),  
division_description VARCHAR(255)  
);
```

-- 3. Core HR Tables

```
CREATE TABLE employees (  
employee_id VARCHAR(50) PRIMARY KEY,  
user_id UUID REFERENCES users(id) UNIQUE, -- 1:1 mapping to User account  
department_id INT REFERENCES departments(id),  
first_name VARCHAR(100),  
last_name VARCHAR(100),  
start_date DATE,  
exit_date DATE,  
title VARCHAR(150),  
supervisor VARCHAR(150),  
email VARCHAR(255),  
employee_status VARCHAR(50),  
employee_type VARCHAR(50),  
employee_classification_type VARCHAR(50),  
termination_type VARCHAR(100),  
termination_description TEXT,  
dob DATE,  
state VARCHAR(50),  
job_function VARCHAR(255),  
gender VARCHAR(20),  
location VARCHAR(50),  
performance_score VARCHAR(50),  
current_employee_rating INT  
);
```

```
CREATE TABLE trainings (  
id SERIAL PRIMARY KEY,  
employee_id VARCHAR(50) REFERENCES employees(employee_id),  
training_date DATE,  
training_program_name VARCHAR(255),  
training_type VARCHAR(100),  
training_outcome VARCHAR(100),  
location VARCHAR(150),  
trainer VARCHAR(150),  
training_duration_days NUMERIC,  
training_cost NUMERIC
```

```
);
```

```
CREATE TABLE engagement_surveys (  
  id SERIAL PRIMARY KEY,  
  employee_id VARCHAR(50) REFERENCES employees(employee_id),  
  survey_date DATE,  
  engagement_score NUMERIC,  
  satisfaction_score NUMERIC,  
  work_life_balance_score NUMERIC  
);
```

-- 4. AI Recruitment Tables

```
CREATE TABLE recruitment_applicants (  
  applicant_id VARCHAR(50) PRIMARY KEY,  
  application_date DATE,  
  first_name VARCHAR(100),  
  last_name VARCHAR(100),  
  gender VARCHAR(20),  
  dob DATE,  
  phone_number VARCHAR(50),  
  email VARCHAR(255),  
  address TEXT,  
  city VARCHAR(100),  
  state VARCHAR(100),  
  zip_code VARCHAR(50),  
  country VARCHAR(100),  
  education_level VARCHAR(100),  
  years_of_experience NUMERIC,  
  desired_salary NUMERIC,  
  job_title VARCHAR(150),  
  status VARCHAR(50)  
);
```

```
CREATE TABLE applicant_cvs (  
  id UUID PRIMARY KEY,  
  applicant_id VARCHAR(50) REFERENCES recruitment_applicants(applicant_id) UNIQUE,  
  file_url VARCHAR(255),  
  parsed_text TEXT,  
  extracted_skills_json JSONB, -- Stored as JSON for easy parsing by Spring Boot / AI  
  ai_match_score INT  
);
```

3 Load the Data

Import the generated CSV files into PostgreSQL. Because we have strictly defined foreign keys, you **must** import them in this exact order to prevent dependency errors:

1. pg_roles.csv

2. pg_users.csv
 3. pg_departments.csv
 4. pg_employees.csv
 5. pg_trainings.csv
 6. pg_surveys.csv
 7. pg_recruitment.csv
 8. pg_cvs.csv
-